free study notes

Phase 6 Advanced & Specialized Systems

73 lessons · notes, diagrams and 508 interview questions

1 · Watch
Scan the QR code on a lesson to open its video.

2 · Read
Notes go deeper than the video: worked examples, trade-offs, mistakes.

3 · Practise
Answer the interview questions out loud before reading the answers.

Storage

1 · Object Storage
2 · Block vs File Storage
3 · Blob Storage Patterns
4 · File Upload Architecture
5 · Pre-Signed URLs
6 · Resumable Uploads
7 · File Metadata Storage
8 · Chunking and Deduplication
9 · Image Processing
10 · Video Streaming
11 · CDN for Private Content
12 · Data Lifecycle
13 · Backup and Archival

Search

14 · Full-Text Search
15 · Inverted Indexes
16 · Elasticsearch Architecture
17 · Search Ranking
18 · Filtering and Faceting
19 · Indexing Pipelines
20 · Search Consistency
21 · Autocomplete
22 · Vector Search
23 · Vector Index Tuning
24 · Hybrid Search
25 · Search Scaling

Real-time and data

26 · Real-Time Messaging
27 · Presence
28 · Notification Systems
29 · Feed Generation
30 · Fan-Out on Write vs Read
31 · Batch Processing
32 · Stream Processing
33 · Lambda vs Kappa
34 · Data Pipelines
35 · Time-Series Data
36 · Geospatial Indexing
37 · Leaderboards
38 · Counting at Scale
39 · Probabilistic Data Structures
40 · Top-K Heavy Hitters

Multi-tenant SaaS

41 · Tenant Isolation
42 · Shared vs Dedicated Databases
43 · Tenant-Aware Authorization
44 · Noisy Neighbour
45 · Usage Metering
46 · Data Residency
47 · Enterprise SSO
48 · Enterprise Audit Trails
49 · Tenant Configuration
50 · Deployment Stamps

Cloud and platform

51 · Cloud Service Models
52 · Regions and Availability Zones
53 · Cloud Networking
54 · Well-Architected Frameworks
55 · Containers and Docker
56 · Kubernetes Core Objects
57 · Kubernetes Scheduling
58 · Kubernetes Autoscaling
59 · Stateful Workloads on Kubernetes
60 · Kubernetes Networking
61 · Infrastructure as Code
62 · GitOps
63 · Immutable Infrastructure
64 · Cloud Cost Optimization
65 · Cost-Aware Architecture
66 · Data Consistency Across Systems
67 · Privacy Engineering
68 · Data Governance and Lineage
69 · Edge Computing
70 · Platform Engineering
71 · Workflow Engines
72 · Distributed Job Scheduling
73 · Green Software

Free to share. Videos: youtube.com/@tapankumarbarik9454 · Notes and practice: backendroadmap.com

Object Storage

buckets, keys, whole objects, eleven nines

Watch the 4-min video

In one line: object storage keeps immutable blobs under keys in a flat namespace, served over HTTP, for a few cents per GB-month with eleven nines of durability; keep the bytes there and the facts about them (owner, name, status) in your database.

1 The database that became a file server

Profile photos and videos go into a `bytea` column in the main Postgres database. It works on day one. A year later the database is mostly video: backups take all night, every read replica copies terabytes of files, a restore takes hours, and expensive database disks (provisioned IOPS, replicated) are storing bytes that are read once and never updated. Large binary data has a completely different access pattern from rows: it is big, written once, read whole and streamed. That is exactly what object storage is built for.

2 Buckets, keys and objects

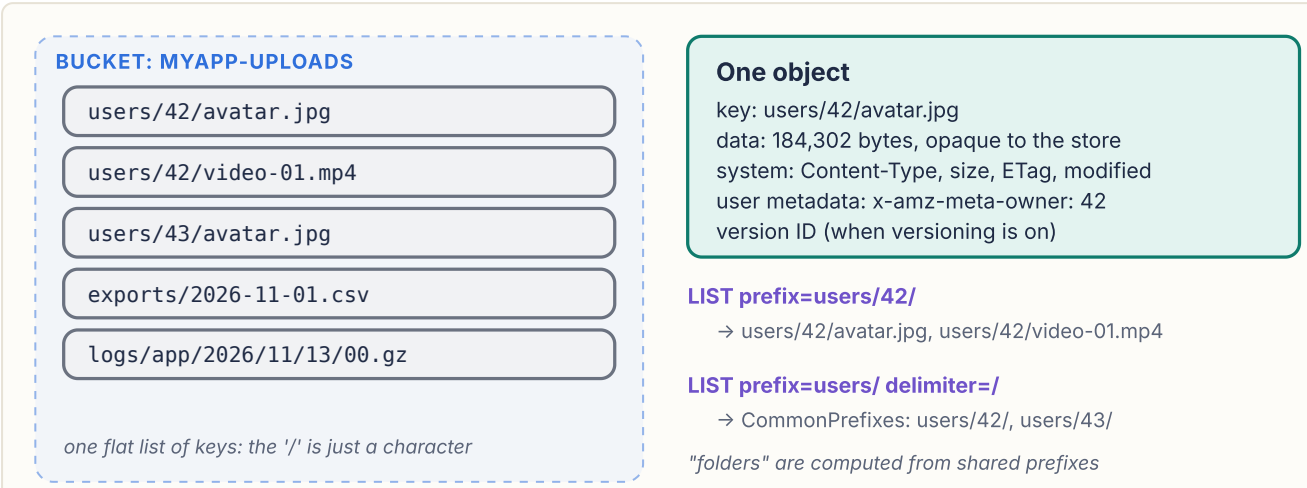


Figure 1. A bucket holds objects under keys. The namespace is flat: `LIST` with a **prefix** and a **delimiter** makes keys look like folders.

- **Bucket** (Azure: *container* inside a *storage account*): a named namespace with its own access policy, region and settings (versioning, lifecycle rules, encryption). S3 bucket names are globally unique.
- **Key**: the object's name, up to 1,024 bytes in S3. `users/42/avatar.jpg` is one string; there is no folder `users/42`. Renaming a "folder" therefore means copying and deleting every object under it.
- **Object**: the data plus metadata. System metadata (Content-Type, Content-Length, ETag, Last-Modified) and a small amount of user metadata (S3: 2 KB of `x-amz-meta-*` headers). Metadata can't be queried: there is no "find all objects owned by user 42".
- **API**: plain HTTP. `PUT`, `GET` (with `Range` for partial reads), `HEAD`, `DELETE`, `LIST` (paged, 1,000 keys per page in S3). Amazon S3, Azure Blob Storage, Google Cloud Storage, and self-hosted MinIO or Ceph RGW all follow this model; many speak the S3 API.

3 Objects are written whole

You can't change byte 40 of an object or append to it (Azure's *append blobs* are a special case). You `PUT` a complete new object, which atomically replaces the old one: a reader sees either the whole old object or the whole new one, never a mix. This immutability is what lets the service replicate and cache objects freely and scale to trillions of them. Reads can still be partial: an HTTP `Range` request fetches any byte range, which is how video players seek and how parallel downloads work.

- **Versioning** keeps every previous version. An overwrite creates a new version; a delete adds a *delete marker* on top, so both are undoable (lesson 3). Old versions are billed, so pair versioning with a lifecycle rule (lesson 12).
- **Concurrent writers**: two PUTs to the same key race and the last one wins; there are no locks. Use unique keys (e.g. a UUID per upload) or conditional requests: S3 supports `If-None-Match: *` (create only if absent) and `If-Match: <ETag>` on writes, and Azure Blob supports ETag conditions and leases.
- **Large objects**: S3 accepts up to 5 GB in a single PUT; bigger files (and anything over ~100 MB, in practice) use multipart upload (lesson 6).

4 Consistency: no longer a gotcha

Until December 2020, S3 was *eventually* consistent for overwrites and listings: you could PUT an object and briefly read the old one, or not see it in a LIST. Systems built workarounds (e.g. a consistent metadata layer). Since then S3 gives **strong read-after-write consistency** for PUT, DELETE and LIST in all regions at no extra cost: once a write returns success, every subsequent read or list sees it. Azure Blob Storage and Google Cloud Storage are also strongly consistent for object operations. What is still *not* consistent: a CDN in front of the bucket (cached copies live until they expire, lesson 11) and cross-region replicas (copied asynchronously, lesson 13).

5 Durability, availability and storage classes

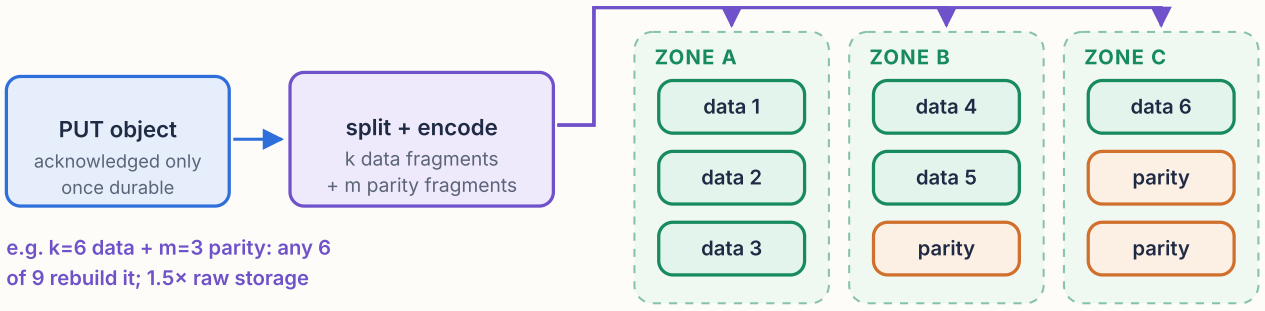


Figure 2. How durability is achieved (conceptually): data is split and encoded, and fragments are spread across failure domains, so losing a disk, a rack or even a zone loses nothing.

Durability is the chance of not losing data; **availability** is the chance a request succeeds right now. S3 Standard is designed for **99.999999999%** (eleven nines) durability by storing data redundantly across at least three Availability Zones, while its availability design target is 99.99%. Eleven nines means that if you store 10 million objects you can expect to lose one about every 10,000 years. Large object stores typically use **erasure coding** rather than full copies: data is split into *k* fragments plus *m* parity fragments, and any *k* rebuild it, so the overhead is far below the 3× of triple replication. Background scrubbing re-checks checksums and repairs fragments.

Durability against hardware is not protection against *you*: a buggy script that deletes objects is executed with eleven-nines reliability. Versioning, Object Lock and replication to another account (lessons 3, 12, 13) cover that.

S3 storage class	First byte	Minimum duration	≈ $/GB-month*	Use for
Standard	milliseconds	none	0.023	active data, user uploads
Standard-IA	milliseconds	30 days, 128 KB min size	0.0125 + per-GB retrieval	backups, older media read occasionally
One Zone-IA	milliseconds	30 days	0.01	re-creatable data (one zone only)
Glacier Instant Retrieval	milliseconds	90 days	0.004	archives read a few times a year
Glacier Flexible Retrieval	minutes to hours (restore first)	90 days	0.0036	archives, compliance copies
Glacier Deep Archive	within 12 h (bulk 48 h)	180 days	0.00099	keep-for-years records
Intelligent-Tiering	milliseconds (default tiers)	none	by tier + small monitoring fee	unknown or changing access patterns

*Approximate us-east-1 list prices for the first 50 TB; they vary by region and change over time. Azure Blob has the equivalent Hot, Cool, Cold and Archive tiers (lesson 3). Requests are billed too: PUT/LIST cost more than GET, and colder tiers charge per GB retrieved.

Lifecycle rules move objects between classes by age or delete them automatically (lesson 12). For comparison, an SSD block volume (lesson 2) costs about 3–4× more per GB than S3 Standard and is tied to one machine; a managed database's storage is usually more expensive still, and then multiplied by every replica.

6 Why not a database, and what you give up

	Relational database	Object storage
Good at	small rows, indexes, queries, transactions	large immutable blobs, streaming reads
Scale	one primary's disk; replicas copy everything	practically unlimited; no capacity planning
Serving files	bytes flow through DB and app servers	clients download directly or via a CDN
Updates	in place, row locks	whole-object replace, no locks
Queries	SQL over any column	GET by key, LIST by prefix only
Transactions	ACID across rows and tables	none across objects

The standard design uses both: **bytes in the bucket, metadata in the database**. A `files` row holds the object key, owner, original file name, size, content type, checksum and status; the bucket holds the content (lesson 7). Because there is no transaction across the two, write the object first and the "ready" row second, and periodically clean up objects that no row points to.

7 Example: user uploads

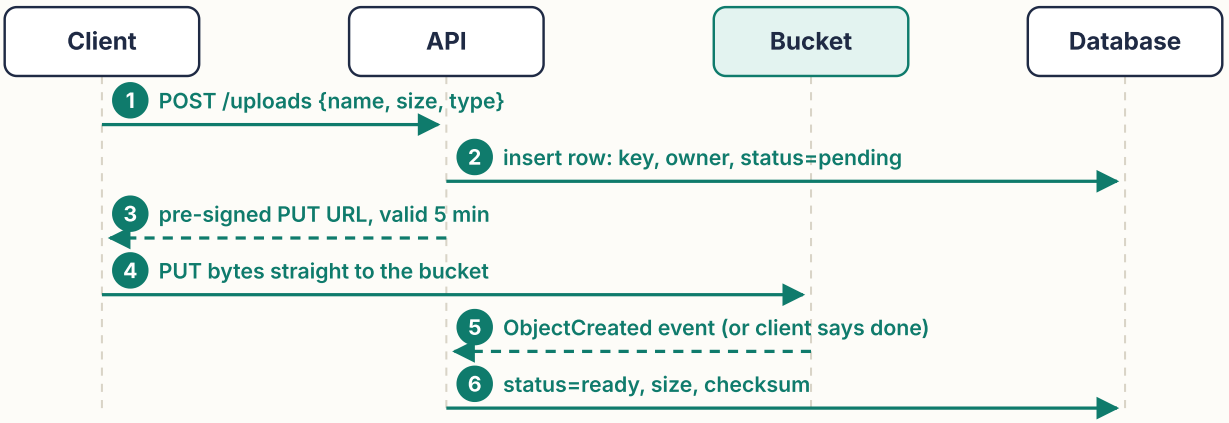


Figure 3. The API only issues permission and records facts; the bytes go straight from the client to the bucket (lessons 4–5).

WORKED EXAMPLE: STORING USER UPLOADS IN S3 OR AZURE BLOB

The client asks the API for an upload slot. The API authenticates the user, checks the declared size against their quota, chooses the key itself (`uploads/user-42/7f3c9e…`, never the client's file name), writes a *pending* row, and returns a pre-signed PUT URL valid for 5 minutes. The client uploads directly to the bucket, so a 200 MB video never touches the API servers. An `ObjectCreated` event (or a "done" call from the client, verified with a `HEAD`) marks the row *ready* with the real size and checksum.

Downloads mirror this: public files go through a CDN with long cache times; private files get a short-lived signed GET URL (lessons 5, 11).

TRADE-OFFS

- Cheap and unlimited, but no in-place edits, appends or locks: workloads that modify files need block or file storage (lesson 2).
- Higher per-request latency (tens of milliseconds to first byte) than a local disk; great for throughput, poor for many tiny random reads.
- Requests cost money: billions of tiny objects can cost more in PUT/GET fees than in storage. Batch small items into larger objects.
- No queries over metadata: you need a database (or an inventory report) to answer "which files belong to whom".

COMMON MISTAKES

- Making a bucket public to "just serve files"; keep it private (S3 Block Public Access) and use a CDN or signed URLs.
- Using client-supplied file names as keys: collisions, overwrites and path tricks. Generate keys on the server.
- Treating eleven nines as a backup: deletes and overwrites by your own code are durable too. Turn on versioning.
- Designing "rename folder" or "list everything owned by X" around LIST calls instead of the metadata database.

8 Key terms

Bucket / container: namespace for objects with its own policy and settings

Key: the object's full name; the namespace is flat

Object: immutable data + metadata, replaced as a whole

Read-after-write consistency: a successful write is visible to every later read and list

Durability: probability data is not lost (S3: eleven nines)

Storage class / tier: price-versus-access option for an object

9 Interview questions

Q1. Why use object storage instead of a database for files? BASIC

It's much cheaper per GB, scales almost without limit, is extremely durable (eleven nines on S3), and serves large files directly or via a CDN without loading your database or app servers.

Q2. What does object storage give up? BASIC

In-place edits, appends, locks and transactions across objects, and any querying beyond key and prefix; so keep the metadata (owner, file name, key, size) in your database.

Q3. Is S3 eventually consistent? BASIC

Not any more: since December 2020 S3 offers strong read-after-write consistency for PUT, DELETE and LIST. Caches and cross-region replicas in front of or behind it are still eventually consistent.

Q4. Users want to rename a folder containing a million files. How do you design for that? SCENARIO

Don't encode folders in object keys: store a folder tree in the database and use opaque keys, so a rename updates one row. With key-based folders it is a million COPY + DELETE calls.

Q5. Two clients upload to the same key at the same moment. What happens? SCENARIO

The last write to complete wins and the other is silently replaced. Use unique keys per upload, or conditional writes (If-None-Match / If-Match on S3, ETag conditions or leases on Azure).

Q6. Your app stores millions of 2 KB JSON events in S3 and the bill is mostly requests. What would you change? SCENARIO

Batch events into larger objects (e.g. one compressed file per minute per partition), since per-request fees and minimum billable sizes dominate for tiny objects.

Q7. Eleven nines durability: so we don't need backups? TRICKY

Durability protects against hardware loss, not against deletes, overwrites, ransomware or a compromised account; you still need versioning, Object Lock and copies in another account or region.

Q8. Durability vs availability: S3 Standard is 11 nines and 99.99%. Why so different? TRICKY

Durability is about never losing bytes (redundant fragments across zones); availability is whether a request succeeds right now, which also depends on networks and front ends and can dip without any data loss.

Next: SD-O-002 · Block vs File Storage backendroadmap.com

Block vs File Storage

a disk for one machine, a folder for many

Watch the 4-min video

In one line: block storage is a raw, low-latency disk attached to one machine (databases, brokers); file storage is a shared file system that many machines mount over NFS or SMB; object storage is whole objects over HTTP. Size block volumes by **IOPS** for random I/O and file or object storage by **throughput** for big sequential reads.

1 Two problems object storage won't solve

A Postgres database runs on cheap network disks and every query crawls: `iostat` shows the disk at 100% busy with single-digit-MB/s reads. Meanwhile ten GPU servers each download the same 40 GB model from a bucket every time they start, so scaling out takes minutes. A database needs a **disk** it can read and write 8 KB at a time at random offsets with `fsync`; an object store only replaces whole objects. The model servers need a **shared folder**, not 10 private copies. Different access patterns, different storage.

2 The three models

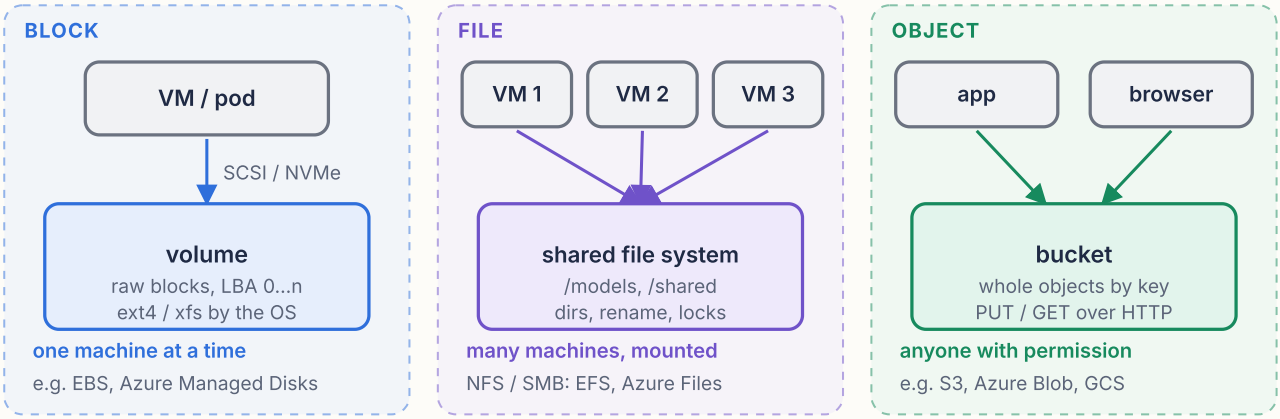


Figure 1. Block storage is a device; file storage is a network file system; object storage is an HTTP service. The interface decides what software can use it.

Block storage: a raw disk

The volume is an array of fixed-size blocks addressed by number. The operating system puts a file system on it (ext4, xfs, NTFS) and owns everything above the blocks: directories, caching, locking. Because only one OS manages that file system, a volume is normally attached to **one machine at a time**. Cloud examples: **Amazon EBS**, **Azure Managed Disks**, Google Persistent Disk; in Kubernetes a `ReadWriteOnce` PersistentVolume backed by a CSI driver.

- **Network-attached** (EBS, Managed Disks): survives instance failure, can be detached and re-attached, but lives in one Availability Zone (Azure also offers zone-redundant SSD disks). Snapshots are incremental and stored durably (EBS snapshots live in S3).
- **Local NVMe / instance store**: far lower latency and higher IOPS, but **ephemeral**: the data is gone when the instance stops or fails. Fine for caches, scratch space or replicated systems (Cassandra,

Kafka with replication) that can rebuild a node.

- **Multi-attach** exists (EBS io1/io2 to up to 16 instances in one AZ, Azure shared disks) but only with a cluster-aware file system or application; two ordinary ext4 mounts of one disk will corrupt it.

File storage: a shared file system

A server (or managed service) exposes directories over **NFS** (v3, v4.1; Linux) or **SMB** (3.x; Windows, also Linux). Many clients mount the same tree and get ordinary file semantics: open, seek, append, rename, permissions and locks. Managed versions: **Amazon EFS** (NFSv4.1, elastic capacity, multi-AZ), **Azure Files** (SMB, plus NFS on premium shares), **Google Filestore**; for high-throughput ML and HPC, **FSx for Lustre** or Azure Managed Lustre. In Kubernetes: `ReadWriteMany` volumes.

The cost is per-operation latency: every `open`, `stat` or directory listing is a network round trip, so workloads with millions of tiny files (a `node_modules` tree, a `git checkout`) are much slower than on a local disk.

Tools like **Mountpoint for Amazon S3** or **BlobFuse** mount a bucket as a folder. Good for reading large files, but not full POSIX: no in-place edits, limited renames, no locks.

3 IOPS, throughput and latency

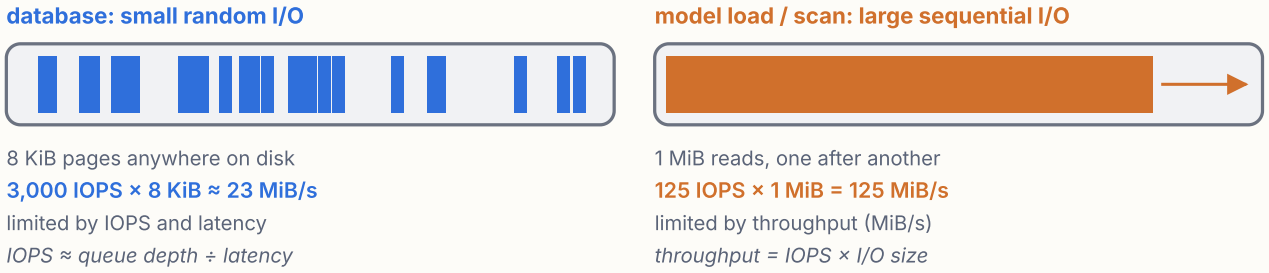


Figure 2. The same disk can be the bottleneck in two different ways. Work out which number your workload needs before choosing a tier.

- **IOPS**: I/O operations per second. Databases read and write 8 KiB (Postgres) or 16 KiB (InnoDB) pages at random places, so they are IOPS-bound.
- **Throughput**: MB per second. Loading model weights, scanning Parquet files or streaming video uses large sequential reads, so it's throughput-bound. **Throughput = IOPS × I/O size.**
- **Latency**: time per operation. With one request in flight, 1 ms latency caps you at 1,000 IOPS; by Little's law, IOPS ≈ outstanding requests (queue depth) ÷ latency. That's why a single-threaded workload can't use the IOPS you paid for, and why commit latency (`fsync` of the write-ahead log) matters more than peak IOPS for a busy OLTP database.

Volume type (examples)	Medium	Performance model	Good for
EBS gp3	SSD	3,000 IOPS + 125 MiB/s baseline at any size; buy more of each independently	most databases and boot volumes
EBS io2 Block Express	SSD	provisioned IOPS, up to 256,000; sub-millisecond latency	large, latency-critical databases
EBS st1 / sc1	HDD	throughput credits; poor at random I/O	big sequential data: logs, cold data
Azure Premium SSD (v1)	SSD	performance tied to size (e.g. P30, 1 TiB: 5,000 IOPS, 200 MB/s)	databases; buy size for IOPS
Azure Premium SSD v2 / Ultra Disk	SSD	IOPS and throughput set independently of size	databases needing tuning
Local NVMe (instance store)	SSD	hundreds of thousands of IOPS, ~100 µs; ephemeral	caches, replicated stores, scratch

Two more limits catch people out: the **instance** has its own maximum storage bandwidth and IOPS (a small VM can't drive a big io2 volume), and network file systems often scale throughput with stored size or a provisioned/elastic mode, so a small share may be slow.

4 Side by side

	Block	File	Object
Interface	device (SCSI/NVMe); OS file system on top	NFS / SMB mount, POSIX-like	HTTP API: PUT, GET, LIST
Sharing	one machine (normally)	many machines at once	any number of clients
Updates	any block, in place	in place, append, rename, locks	replace whole object
Latency	lowest: sub-ms to a few ms	ms per operation (network round trip)	tens of ms to first byte
Scale	volume size limit (EBS: 16–64 TiB)	elastic up to petabytes	practically unlimited
≈ price per GB-month*	$0.08 (gp3)	$0.30 (EFS Standard)	$0.023 (S3 Standard)
Typical use	databases, brokers, boot disks	shared content, home dirs, ML datasets, legacy apps	media, backups, data lakes, artifacts

*Approximate us-east-1 list prices; extra IOPS/throughput and requests are billed on top.

5 How to choose

1. **Does the software expect a local disk** (a database, Kafka, Elasticsearch, a VM boot volume)? Use **block**. Size it by IOPS and latency, not just capacity.
2. **Do several machines need the same files through normal file operations** (shared uploads in a legacy app, home directories, a model or dataset read by many pods)? Use **file**.
3. **Everything else**, especially large amounts of data written once and read by key (media, backups, logs, data lakes): use **object**. It's the cheapest and the easiest to scale and share.

Real systems use all three at once. A common pattern: object storage is the durable source of truth, and block or file storage is a fast working copy (a database's volume backed up to a bucket; a file share or local NVMe cache filled from a bucket).

6 Example: Postgres + shared model weights

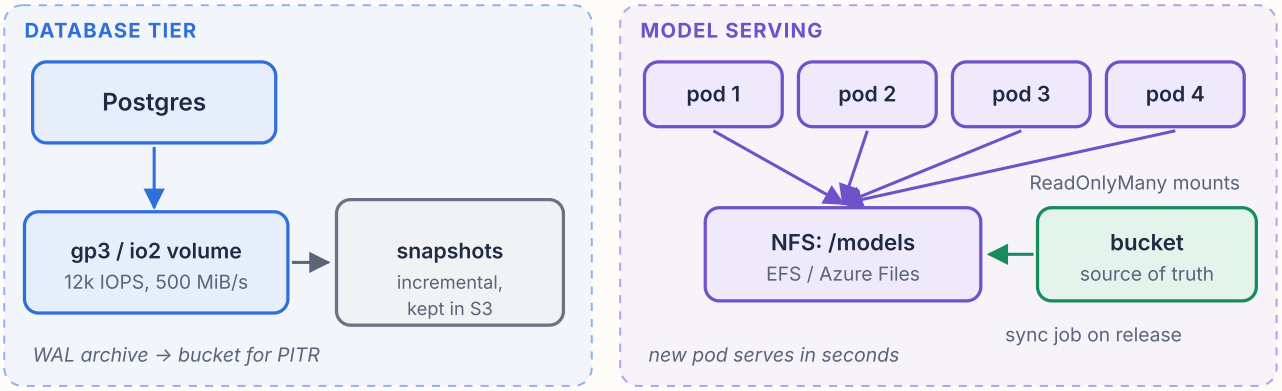


Figure 3. Each workload gets the storage its access pattern needs; object storage stays the durable source for both.

WORKED EXAMPLE: ONE PLATFORM, THREE KINDS OF STORAGE

Postgres (500 GB, peaks of ~10,000 random IOPS) runs on an SSD block volume: gp3 provisioned at 12,000 IOPS and 500 MiB/s, or io2 if commit latency must stay sub-millisecond. Volume snapshots give quick restores, and continuous WAL archiving to a bucket gives point-in-time recovery (snapshots alone are only crash-consistent).

Model serving: the 40 GB weights live in a bucket. On each release a job copies them to an NFS share (EFS / Azure Files), and every pod mounts `/models` read-only. Before: 10 pods × 40 GB = 400 GB downloaded per rollout; at 125 MiB/s one download takes 40 × 1024 ÷ 125 ≈ 330 s, about 5½ minutes per pod. After: a new pod mounts the share and starts loading at once, and the operating system's page cache keeps hot files in memory. Check the share's throughput mode: ten pods loading at once share its bandwidth.

TRADE-OFFS

- Block: fastest and simplest for one machine, but tied to one zone and one host; HA needs replication at the application level.
- File: easy sharing with familiar semantics, but per-operation latency, higher price per GB and lock/consistency subtleties (NFS close-to-open consistency).
- Object: cheapest and most durable, but no in-place edits and higher latency; apps must use its API or a limited mount.
- Local NVMe: unbeatable speed, zero durability; only for data you can rebuild.

COMMON MISTAKES

- Running a database on NFS or a bucket mount: slow commits, and fragile locking and `fsync` semantics.
- Sizing a volume by gigabytes and ignoring IOPS, latency and the instance's own storage bandwidth limit.
- Keeping irreplaceable data on ephemeral instance storage.
- Treating a crash-consistent volume snapshot as a complete database backup strategy (no point-in-time recovery, no restore test).

7 Key terms

Block storage: raw volume of numbered blocks, formatted by one OS

File storage: shared file system mounted over NFS or SMB

IOPS: I/O operations per second; what random-access workloads need

Throughput: MB/s; what large sequential reads need

Queue depth: outstanding I/Os; IOPS ≈ queue depth ÷ latency

ReadWriteOnce / ReadWriteMany: Kubernetes access modes for block vs shared volumes

8 Interview questions

Q1. When do you need block storage vs object storage? BASIC

Block storage when software needs a low-latency disk with random reads and writes, like a database or broker; object storage for large amounts of files accessed by key, cheaply and durably.

Q2. What's the difference between IOPS and throughput? BASIC

IOPS is operations per second (small random access, what databases need); throughput is MB per second (large sequential reads). Throughput = IOPS × I/O size.

Q3. Why can't two servers mount the same EBS volume and share files? BASIC

Each OS caches and manages the file system as if it owned the disk, so two writers corrupt it; use a shared file system (NFS/SMB) or a cluster-aware file system with multi-attach.

Q4. Your Postgres p99 latency spikes and the volume shows 3,000 IOPS flat. What's happening and what do you do? SCENARIO

It's hitting the gp3 baseline of 3,000 IOPS. Provision more IOPS (or move to io2), check the instance's EBS limit, and reduce I/O with more RAM for cache or better indexes.

Q5. Ten inference pods each download a 40 GB model at start-up. How would you speed this up? SCENARIO

Keep the model in a bucket as the source, but serve it from a shared read-only file system or a node-local cache, so new pods read it instead of each downloading 40 GB.

Q6. You provisioned 64,000 IOPS but a single-threaded job gets 1,500. Why? TRICKY

With one I/O in flight at ~0.6 ms latency, it can do at most ~1,600 IOPS (Little's law); increase concurrency (queue depth) or batch I/O.

Q7. Can you run a database on S3 via a FUSE mount? TRICKY

Not safely: bucket mounts lack in-place writes, locks and real fsync semantics, at tens of ms per operation. Databases that tier to object storage do it inside their own engine.

Next: SD-O-003 · Blob Storage Patterns backendroadmap.com

Blob Storage Patterns

tier by access, protect by policy

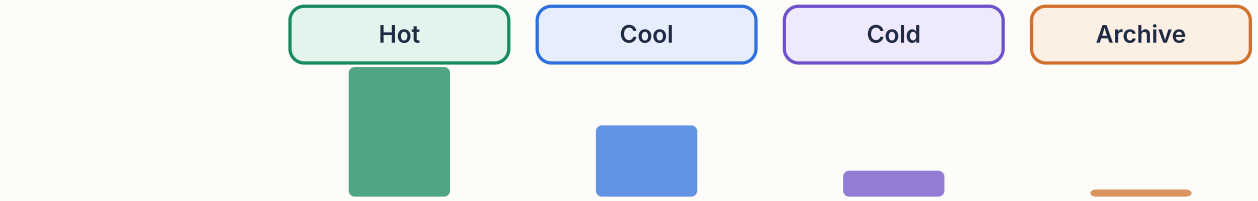
Watch the 4-min video

In one line: put each blob in the tier that matches how often it is *read* (hot, cool, cold, archive) and let lifecycle rules move it there by age; protect it with versioning and soft delete against mistakes, and with immutability (WORM) and legal holds against tampering.

1 The bill for data nobody reads

Every log file, export and backup goes into blob storage. The first month costs almost nothing. Two years later blob storage is one of the biggest lines on the cloud bill, and access logs show that almost nothing older than 30 days is ever read. Everything still sits in the hot tier, priced for data that is read every day. Typical data has a steep access curve: read heavily for days, occasionally for weeks, then almost never. Tiers and lifecycle rules make the price follow that curve.

2 Access tiers



	Hot	Cool	Cold	Archive
storage $/GB-month	**≈ 0.018**	**≈ 0.010**	**≈ 0.0036**	**≈ 0.001**
first byte	ms	ms	ms	hours
minimum duration	none	30 days	90 days	180 days
per-GB read cost	lowest	higher	higher still	rehydrate first

colder → cheaper to keep, dearer and slower to read

Figure 1. Azure Blob tiers (approximate East US LRS list prices). Storage gets cheaper as you go colder; reads get slower and dearer, and each tier has a minimum billable duration.

- **Hot**: highest storage price, cheapest reads and writes. Data in active use.
- **Cool** (≥ 30 days) and **Cold** (≥ 90 days): still **online** (millisecond reads) but each read and write costs more, plus a per-GB retrieval charge. Moving or deleting a blob earlier triggers an **early-deletion** charge for the remaining days.
- **Archive** (≥ 180 days): **offline**. You can't read the blob until you **rehydrate** it by changing its tier (or copying it) to an online tier: up to 15 hours at standard priority, often under an hour at high priority for blobs under 10 GB, at a higher price. Archive is only available on LRS/GRS-type accounts, not zone-redundant ones.

Amazon S3 has the same ladder as storage classes: Standard → Standard-IA (30 days) → Glacier Instant Retrieval (90) → Glacier Flexible Retrieval (90, restore in minutes to hours) → Glacier Deep Archive (180, restore within 12 hours). Google Cloud Storage: Standard, Nearline (30), Coldline (90), Archive (365).

3 The trade-off in numbers

The right tier depends on **how often data is read**, not on how important it is. A compliance archive read once a year belongs in archive even though it matters a lot; a small config file read a million times a day belongs in hot.

```
cost per month = GB stored × price + operations + GB read × retrieval
```

Moving 1 GB from hot to cool saves about $0.008 per month, while reading it from cool costs about $0.01 in retrieval. So if data is read roughly **once a month or more**, cool costs *more* than hot. Two more traps: tiering many **tiny objects** (each transition is a billed write operation, and S3's IA classes bill a minimum of 128 KB per object), and the **minimum durations**: data deleted after 40 days should never go to archive. When access patterns are unknown, let the platform decide: S3 **Intelligent-Tiering** moves objects between tiers based on observed access (small per-object monitoring fee), and Azure lifecycle rules can use `daysAfterLastAccessTimeGreaterThan` once last-access tracking is enabled.

4 Protect against mistakes: versioning and soft delete

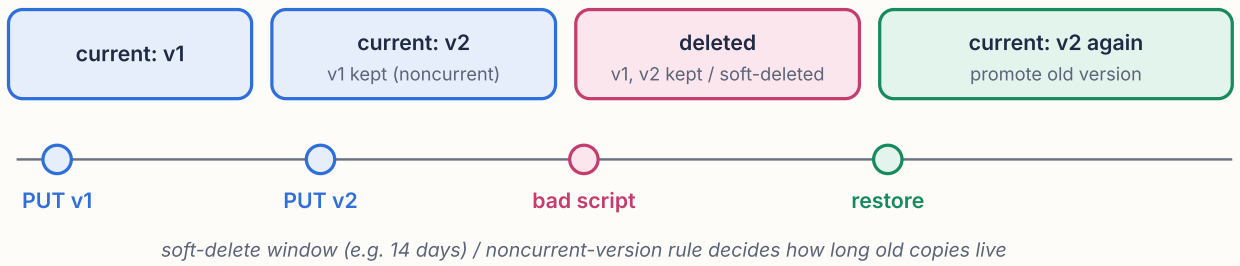


Figure 2. With versioning, an overwrite or delete never destroys data; the old version is promoted back. Old versions are billed until a rule removes them.

- **Versioning** (Azure Blob versioning, S3 Versioning) keeps every previous version automatically. In S3 a delete adds a *delete marker*; removing the marker brings the object back.
- **Soft delete** (Azure) keeps deleted blobs, and with container soft delete whole containers, for a retention window of 1–365 days. Add a resource lock so nobody can delete the storage account itself.
- Both multiply storage: a file overwritten daily keeps 30 versions a month. Pair them with a lifecycle rule that deletes noncurrent versions after N days (lesson 12).
- They don't stop a **privileged** attacker, who can disable soft delete or purge versions. That's what immutability is for.

5 Data that must never change: WORM and legal holds

Immutability (write once, read many) locks blobs so they can't be overwritten or deleted until a retention period ends, not even by an administrator or the account owner.

	Azure immutable storage	S3 Object Lock
Time-based retention	policy on container or version; *unlocked* for testing, *locked* = can only be extended	**governance** mode: users with a special permission can bypass; **compliance** mode: nobody can shorten or delete, not even root
Legal hold	tag-based hold, no expiry, until cleared	on/off flag per object version, independent of retention
Prerequisite	version-level WORM needs versioning	versioning must be on
Compliance	both have been assessed for SEC 17a-4(f), FINRA and CFTC record-keeping rules	

Uses: audit logs, financial records, and **backups that ransomware can't encrypt or delete** (lesson 13). Test with governance mode or an unlocked policy first: a locked compliance-mode retention of 7 years on the wrong data can't be undone, and you pay for it for 7 years.

6 Naming keys to avoid hot partitions

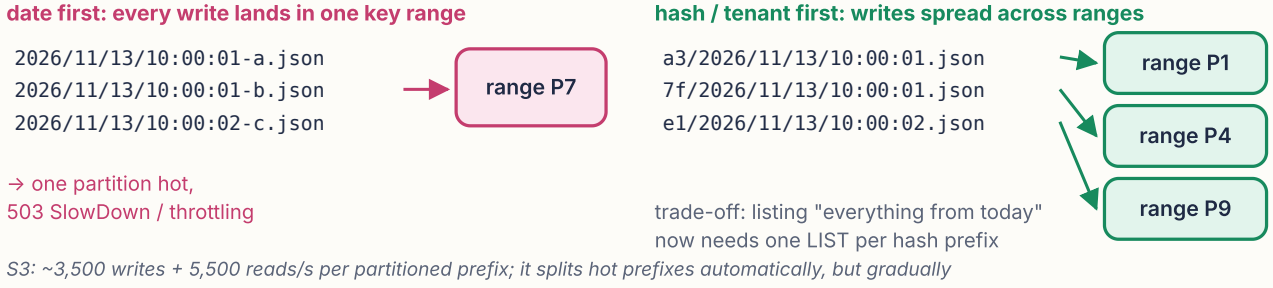


Figure 3. Object stores partition the key space by name ranges. Keys that start with a timestamp send all new writes to one range.

S3 supports at least 3,500 PUT/COPY/POST/DELETE and 5,500 GET/HEAD requests per second **per partitioned prefix**, and splits busy prefixes automatically; during the ramp you may see `503 Slow Down`, so clients must retry with exponential backoff. Azure partitions by account + container + blob name, also range-based, and documents per-blob (≈ 500 requests/s) and per-account request targets. Most workloads never notice. At very high request rates, put a short hash or tenant ID *before* the date (`7f/2026/11/13/…`) so writes spread, or spread load across several prefixes or accounts.

7 Example: lifecycle for logs

```
{ "rules": [ { "name": "logs-tiering", "enabled": true, "type": "Lifecycle",
    "definition": {
      "filters": { "blobTypes": ["blockBlob"],
                   "prefixMatch": ["appdata/logs/"] },   # container/prefix
      "actions": { "baseBlob": {
        "tierToCool":    { "daysAfterModificationGreaterThan": 30 },
        "tierToArchive": { "daysAfterModificationGreaterThan": 180 },
        "delete":        { "daysAfterModificationGreaterThan": 365 } } } } ] }
```

Figure 4. Azure Blob lifecycle policy. `prefixMatch` starts with the container name; the policy runs about once a day.

WORKED EXAMPLE: LOGS TO COOL AFTER 30 DAYS, ARCHIVE AFTER 180, DELETE AFTER 365

A platform writes **10 TB of logs a month** and keeps them for a year, so about 120 TB is stored at any time. All in hot at ≈ $0.018/GB: 120,000 GB × 0.018 ≈ **$2,160/month**. With the policy: the newest 10 TB in hot ($180), 50 TB aged 30–180 days in cool (× 0.010 = $500), 60 TB aged 180–365 days in archive (× 0.001 ≈ $60): **≈ $740/month**, about 65% less, plus small transition charges.

The minimum durations fit: blobs spend 150 days in cool (≥ 30) and 185 days in archive (≥ 180). Had the policy deleted logs at day 300, every blob would pay an early-deletion charge for its last 60 archive days. And if incident investigations sometimes need 3-month-old logs, keep them online in cool or cold rather than archive: a 15-hour rehydration during an outage is not acceptable.

TRADE-OFFS

- Colder tiers: lower storage price, higher per-read cost, minimum durations, slower (archive: hours) access.
- Versioning and soft delete: cheap insurance against mistakes, but they grow storage until a rule prunes old versions.
- WORM: protects against insiders and ransomware, but also against you; legitimate deletion (e.g. a GDPR request) waits for retention to end.
- Hashed key prefixes: spread load, but make time-range listing and browsing harder.

COMMON MISTAKES

- Choosing tiers by importance instead of read frequency.
- Archiving data you may need quickly (incident logs, restores with a tight RTO).
- Tiering millions of tiny objects, where transition and minimum-size charges exceed the savings.
- Enabling versioning without a noncurrent-version rule, then wondering why storage doubled.

8 Key terms

Access tier / storage class: price-vs-access option: hot, cool, cold, archive

Early deletion charge: fee for leaving a tier before its minimum duration

WORM / immutability: no overwrite or delete until retention ends

Rehydration: bringing an archived blob back online before reading it

Soft delete: deleted data kept for a retention window, restorable

Legal hold: open-ended lock for litigation or investigation

9 Interview questions

Q1. How do blob tiers reduce cost? BASIC

Rarely read data moves to cooler, cheaper tiers (cool, cold, archive), automatically by age via lifecycle rules, trading slower and costlier access for much lower storage prices.

Q2. How do you protect blobs from mistakes and tampering? BASIC

Versioning and soft delete undo accidental overwrites and deletes; immutability policies (WORM) and legal holds lock data that must never change, even for admins.

Q3. A team asks to move all data older than 7 days to archive to save money. What do you check first? SCENARIO

How often old data is read and how fast it's needed (archive takes hours to rehydrate), object sizes and counts (transition costs), and how long it's kept (180-day minimum).

Q4. Ransomware gets admin credentials to your storage account. What would have saved your backups? SCENARIO

Immutable (locked WORM / compliance-mode Object Lock) copies, ideally replicated to a separate account; soft delete and versioning alone can be disabled by an admin.

Q5. Your service writes 20,000 objects/s with keys starting with a timestamp and sees 503 errors. Fix? SCENARIO

Retry with backoff, and change the key scheme to put a hash or tenant prefix first so writes spread across partitions; keep a time index elsewhere for listing.

Q6. When is the cool tier more expensive than hot? TRICKY

When data is read often, roughly once a month or more per byte, since retrieval and operation charges outweigh the storage saving; also when data is deleted before 30 days.

Q7. Governance vs compliance mode in S3 Object Lock? TRICKY

Governance can be bypassed by users granted a special permission (useful for testing); compliance can't be shortened or removed by anyone, including the root account, until retention ends.

Next: SD-O-004 · File Upload Architecture backendroadmap.com

File Upload Architecture

permission from the API, bytes to storage

Watch the 4-min video

In one line: don't stream uploads through your API: it issues a short-lived signed URL and the client sends bytes straight to object storage; then storage events drive validation, a malware scan in quarantine and processing, and a status field in the database tells the app when the file is ready.

1 Your API servers as byte pumps

A user uploads a 2 GB video through `POST /videos`. On a 5 Mbit/s mobile uplink that takes 2 × 8,000 ÷ 5 ≈ **53 minutes**, and for all of it an API worker holds an open connection, buffers or streams the bytes, and forwards them to storage. A thousand such uploads at once means a thousand busy workers and twice the bandwidth (in and out), and API servers now need scaling for uploads rather than requests. Managed front doors also refuse it: API Gateway caps request bodies at 10 MB and Lambda's synchronous payload at 6 MB.

2 Direct upload

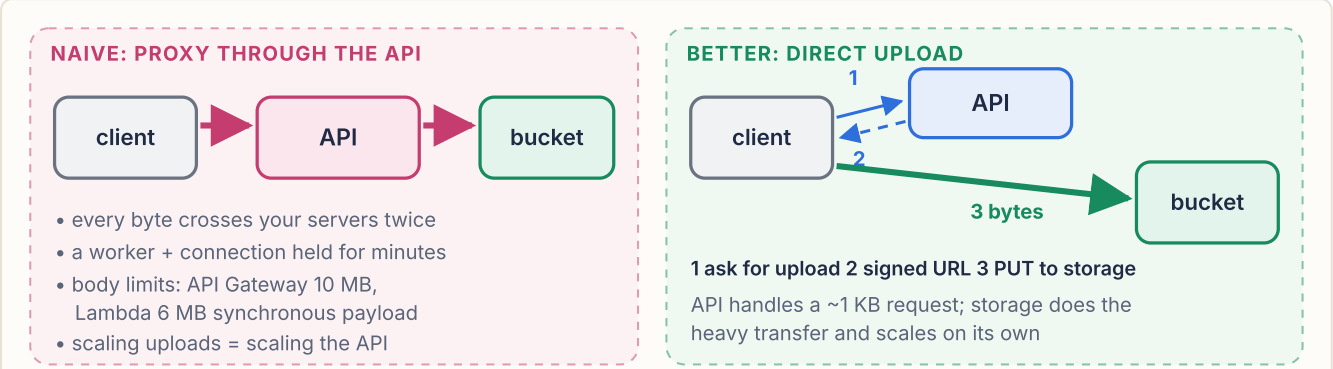


Figure 1. Proxying makes the API a byte pump. A direct upload keeps the API in the control path only.

1. **Initiate**: `POST /uploads {fileName, size, contentType}`. The API authenticates the user, checks quota, file size and allowed types, creates a `files` row with `status=pending`, and picks the object key itself (`quarantine/7f3c…`).
2. **Sign**: it returns a pre-signed URL (lesson 5) that allows exactly one PUT to that key for a few minutes. With an S3 pre-signed **POST** the policy can also enforce `content-length-range` and a `Content-Type`; Azure uses a SAS token with write-only permission.
3. **Upload**: the client sends bytes straight to storage. Files over ~100 MB use multipart or resumable uploads (lesson 6).
4. **Complete**: storage emits an `ObjectCreated` event; the client may also call `POST /uploads/{id}/complete` for a fast UI, but the server verifies with a `HEAD` rather than trusting the client.

CORS must be configured on the bucket for browser uploads (allowed origin, `PUT`, the headers you sign).

Approach	When	Watch out for
Proxy through API	tiny files (avatars < 1 MB) where you must inspect bytes inline	body limits, held workers
Pre-signed PUT	single files up to a few hundred MB	size not enforced unless signed; no resume
Pre-signed POST (policy)	browser forms needing size/type limits	S3-specific; single request
Multipart / resumable	large files, mobile networks	abandoned parts cost money (lesson 12)

3 Validate it

An upload isn't done when the bytes land. Treat every file as hostile until checked, following the OWASP File Upload Cheat Sheet:

- **Size**: enforce at signing (signed length or POST policy) and re-check the real size from the event or a `HEAD`.
- **Real type**: read the first bytes ("magic numbers"), not the extension or the client's `Content-Type`. JPEG starts `FF D8 FF`, PNG `89 50 4E 47`, PDF `%PDF-`. Allowlist types; reject everything else.
- **Deep checks**: limit image pixel counts and archive expansion (decompression bombs); re-encode images, which also strips hidden payloads and metadata.
- **Safe serving**: keep the generated key, store the user's file name only as metadata, serve downloads with `Content-Disposition: attachment` and `X-Content-Type-Options: nosniff`, ideally from a separate domain, so an uploaded HTML or SVG file can't run script on your main site.

4 Scan it in quarantine

Uploads land in a **quarantine** bucket or prefix that no user and no CDN can read. A scanner checks each object; clean files are copied (or tagged) to the clean location, infected ones are deleted or moved to an isolated bucket, and the row is marked `rejected`. Options: **ClamAV** in a container or function, **Amazon GuardDuty Malware Protection for S3**, or **Microsoft Defender for Storage** on-upload scanning; the managed ones can tag the object with the verdict. Limits to know: signature scanners miss new malware, password-protected archives can't be scanned (reject them or flag them), and very large files may exceed a scanner's size limit.

5 Driven by events

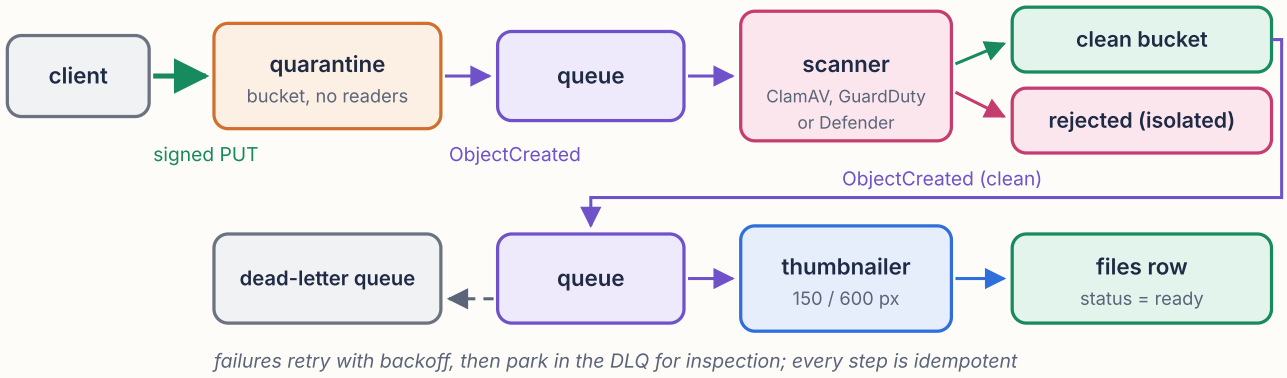


Figure 2. The worked example as a pipeline: each step is triggered by a storage event through a queue, retries on its own and scales on its own.

- **Sources**: S3 Event Notifications (to SQS, SNS, Lambda or EventBridge); Azure Event Grid `BlobCreated`; GCS Pub/Sub notifications.
- **At-least-once**: events can be duplicated and arrive out of order, so consumers must be **idempotent**: key work by object key + version (or ETag), and use conditional updates such as `UPDATE files SET status='clean' WHERE id=? AND status='scanning'`.
- **Queues between steps**: put a queue in front of each worker so a slow scanner doesn't drop events; retry with backoff and send poison messages to a dead-letter queue.
- **Reconcile**: events can be lost by misconfiguration; a periodic sweep finds rows stuck in `uploaded` or `scanning` and objects with no row, and a lifecycle rule deletes stale quarantine objects (lesson 12).

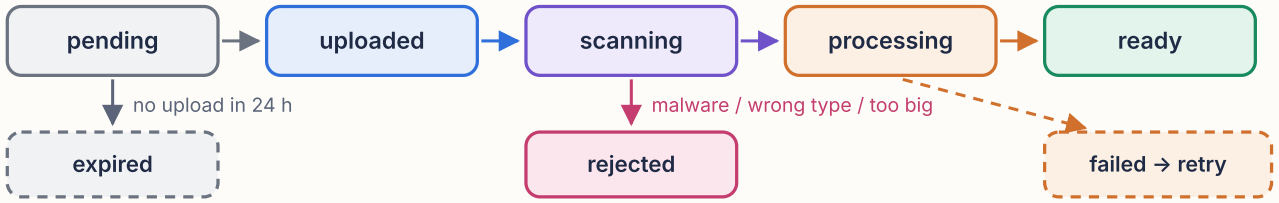


Figure 3. The upload as a state machine in the database. The app shows the file only in **ready**; every transition is a conditional update.

6 Example: upload → event → scan → thumbnail

WORKED EXAMPLE: A PHOTO UPLOAD

The app calls `POST /uploads`; the API creates row 81 (`pending`) and returns a 5-minute signed PUT URL for `quarantine/81-7f3c.jpg`, limited to 20 MB. The phone uploads the 4 MB photo directly. The bucket emits `ObjectCreated` onto a queue; the row becomes `scanning`. The scanner reads the magic bytes (JPEG), checks the pixel count, runs the malware scan, copies the object to `media/81.jpg` in the clean bucket and deletes the quarantine copy. That write emits another event; the thumbnail worker creates 150 px and 600 px versions (lesson 9) and sets the row to `ready` with the variant keys. The app, polling or notified over a WebSocket, shows the photo about 3–5 seconds after the upload finished. If the scanner had found malware, the row would read `rejected` and the user would see why.

TRADE-OFFS

- Direct uploads offload bandwidth, but you lose inline inspection; all checks become asynchronous.
- Asynchronous pipelines scale and retry well, but the file isn't usable instantly; the UI needs a "processing" state.
- Copying from quarantine to clean doubles writes briefly; tagging in place is cheaper but relies on every reader checking the tag.
- Managed scanners are simple but priced per GB scanned; self-hosted ClamAV is cheap but needs signature updates and scaling.

COMMON MISTAKES

- Trusting the extension or `Content-Type` sent by the client.
- Letting users read files before the scan finishes (e.g. serving the quarantine bucket via the CDN).
- Non-idempotent consumers: a duplicate event creates two thumbnails or charges quota twice.
- Serving user uploads from the main domain inline, turning an uploaded SVG/HTML file into stored XSS.

7 Key terms

Direct upload: client sends bytes to storage with a signed URL

Magic bytes: file signature in the first bytes; reveals the real type

Idempotent consumer: processing the same event twice has no extra effect

Quarantine: unreadable landing area until checks pass

Event notification: storage emits an event when an object is created or deleted

Dead-letter queue: where messages go after repeated failures

8 Interview questions

Q1. Why avoid streaming large uploads through your API servers? BASIC

They hold connections, memory and bandwidth for minutes, hit request-size limits, and scaling uploads means scaling the whole API; direct uploads with signed URLs send bytes straight to storage.

Q2. What happens after the upload? BASIC

Validation (size, real type), a malware scan in quarantine and processing steps such as thumbnails, triggered by storage events through queues, with a status row that flips to ready.

Q3. How do you stop a user uploading a 50 GB file through a signed URL meant for avatars? SCENARIO

Sign the size: an S3 pre-signed POST policy with content-length-range (or a signed Content-Length), check quota before signing, and re-check the real size from the event and delete oversize objects.

Q4. The thumbnail worker sometimes runs twice for the same photo. Why, and is it a problem? SCENARIO

Storage events are at-least-once, and queue redelivery adds more; make the worker idempotent (deterministic output keys, conditional status updates) and duplicates are harmless.

Q5. How does the client know when its file is ready? SCENARIO

The upload returns an ID; the client polls GET /uploads/{id} or gets a WebSocket/push notification when the pipeline sets the row to ready or rejected.

Q6. A user uploads an SVG profile picture. What can go wrong? TRICKY

SVG can contain script; served inline from your domain it's stored XSS. Rasterise or sanitise it, or serve user files as attachments from a separate domain with nosniff.

Q7. Why not trust the client's 'upload complete' call? TRICKY

It may be missing, early, faked or duplicated; verify with storage (HEAD or the ObjectCreated event) before moving the row forward.

Next: SD-O-005 · Pre-Signed URLs

Pre-Signed URLs

one permission, one object, a short time

Watch the 4-min video

In one line: a pre-signed URL is a normal URL plus a signature, computed locally by your server, that grants exactly one operation (method + object + expiry) to whoever holds it; storage re-checks the signature on every request. It's a bearer token, so keep the scope narrow and the expiry short.

1 Private bucket, real users

The bucket must stay private, yet one user needs to upload a video into it and another needs to download a report from it. Handing out cloud credentials is out of the question, and proxying the bytes through your API makes it a byte pump (lesson 4). A pre-signed URL lets the client talk to storage directly, with a permission your server chose and signed.

2 A URL with a signature

```
https://myapp-uploads.s3.eu-west-1.amazonaws.com/uploads/u42/7f3c.mp4
?X-Amz-Algorithm=AWS4-HMAC-SHA256                          ── signature version 4
&X-Amz-Credential=AKIA…/20261113/eu-west-1/s3/aws4_request  ── who signed + scope
&X-Amz-Date=20261113T101500Z                               ── signed at
&X-Amz-Expires=900                                         ── valid 15 min (max 7 days)
&X-Amz-SignedHeaders=content-type;host                     ── headers the client must send
&X-Amz-Signature=5d672d79c15b13162d9279b0855cfba6…         ── HMAC over all of the above + method
note: the HTTP method (PUT) is part of what is signed; using the URL for GET or DELETE fails
```

Figure 1. An S3 pre-signed URL (Signature Version 4). Everything that defines the permission is in the query string and covered by the signature.

Your server, which holds credentials, signs a specific request: *this method, on this object, with these headers, until this time*. Change any part (another key, another method, a later expiry) and the signature no longer matches. The URL also carries the signer's identity, and S3 checks that identity's permissions **at the time of the request**: if the signing role loses `s3:PutObject`, every URL it signed stops working.

3 Signing is local and cheap

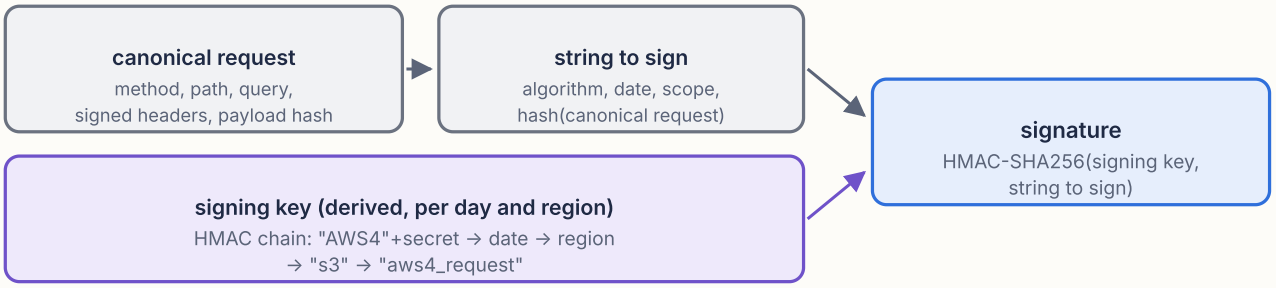


Figure 2. SigV4: the secret never leaves your server. Storage, which knows the same secret, recomputes the HMAC and compares.

Generating a URL makes **no network call**: the SDK computes an HMAC-SHA256 from the secret key, date, region, service, method, key, query and signed headers. That takes microseconds, so you can sign a fresh URL for every request and never cache them. Two practical details:

- **Lifetime**: SigV4 allows up to 7 days, but a URL signed with *temporary* credentials (an IAM role in Lambda, ECS or EKS) dies when those credentials expire, possibly within an hour, whatever `X-Amz-Expires` says.
- **Expiry is checked when the request starts**: an upload or download that begins before expiry completes even if it runs past it.

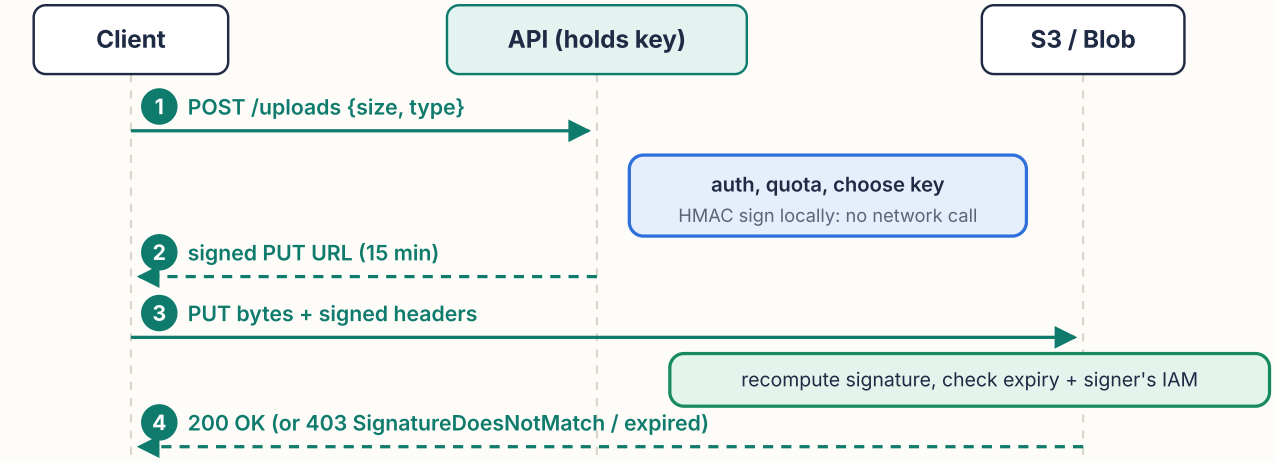


Figure 3. The API is in the control path only. Storage validates signature, expiry and the signer's current permissions on each request.

4 Azure SAS tokens and friends

Azure's equivalent is the **shared access signature** (SAS): query parameters such as `sp=cw` (permissions: create, write), `st` / `se` (start, expiry), `sip` (allowed IP range), `spr=https`, `sr=b` (one blob) and `sig`. Three kinds:

	Signed with	How to revoke early
S3 pre-signed URL	IAM user or role credentials (SigV4)	remove the signer's permission, deactivate its key, or a bucket-policy deny (e.g. on `s3:signatureAge` or source IP)
Azure service / account SAS	storage account key	ad hoc: only by rotating the account key (breaks every SAS); service SAS tied to a **stored access policy**: edit or delete the policy
Azure **user delegation SAS**	a user delegation key obtained with Microsoft Entra ID credentials (valid up to 7 days)	revoke the delegation keys or remove the principal's RBAC role; recommended by Microsoft
GCS signed URL (V4)	service account key or IAM signBlob	remove the service account's permission or its key; max 7 days

User delegation SAS is the safer default because no account key is involved; you can then disable shared-key authorization on the account entirely. Clock skew between machines can make a fresh SAS look "not yet valid": omit the start time or set it a few minutes in the past.

5 Scope is everything

- **One object, one method**: sign `PUT uploads/u42/7f3c.mp4`, never a container-wide or list permission for a client.
- **Server-chosen keys**: generate the key (UUID) on the server, so a client can't overwrite another user's file or pick a path like `../`.
- **Short expiry**: minutes for uploads to start, minutes to hours for downloads; days only for rare, deliberate sharing.
- **Constrain content**: a pre-signed PUT can't express a size range. Use a pre-signed **POST** policy with `content-length-range` and a `Content-Type` condition, or sign the exact `Content-Type` header. You can even sign a checksum header (`Content-MD5` or `x-amz-checksum-sha256`) declared at initiation, so only that exact file is accepted.
- **Least-privilege signer**: a dedicated role that can only `PutObject` on `uploads/*` limits damage if URLs or the role leak.
- **Downloads**: add `response-content-disposition=attachment; filename="report.pdf"` to the signed GET so the browser downloads with a friendly name instead of rendering the file.

```
post = s3.generate_presigned_post(
    Bucket="myapp-uploads", Key=f"uploads/{user_id}/{uuid4()}",
    Conditions=[["content-length-range", 1, 2_000_000_000],   # ≤ 2 GB
                ["starts-with", "$Content-Type", "video/"]],
    ExpiresIn=900)                                            # 15 minutes
# returns {url, fields}: the client POSTs the fields + file as multipart/form-data
```

Figure 4. boto3: a pre-signed POST that enforces size and type. The policy is part of what is signed, so the client can't change it.

6 The risks

A pre-signed URL is a **bearer token**: whoever holds it can use it. It leaks through access and proxy logs (query strings are logged), browser history, analytics tools that record URLs, and links pasted into chats or tickets. Most can't be revoked one by one, so short expiry is the main defence; add IP conditions

where clients have stable addresses (rarely true on mobile). Unique, short-lived URLs also defeat CDN caching: for content served to many users, use CDN signed URLs or cookies instead (lesson 11).

7 Example: a 2 GB video straight to S3

WORKED EXAMPLE: UPLOADING A 2 GB VIDEO

The app calls `POST /videos` with size 2,000,000,000 and type `video/mp4`. The API checks the user's remaining quota, creates a `videos` row (`pending`), picks the key `uploads/u42/7f3c.mp4` and signs a PUT URL that expires in **15 minutes** with `content-type: video/mp4` among the signed headers, using a role that may only write under `uploads/`. The client starts the upload at minute 1; it runs for 40 minutes on a slow link and still succeeds because expiry is checked at the start. S3 emits `ObjectCreated`, a consumer verifies the size and sets the row to `uploaded`, and the API never touched the 2 GB.

2 GB is under S3's 5 GB single-PUT limit, but one dropped connection restarts it from zero. In production you'd use multipart with a pre-signed URL per part (lesson 6).

TRADE-OFFS

- No credentials on clients and no bytes through your servers, but authorization happens once, at signing time.
- Short expiry limits leaks but can break slow or queued clients; re-sign on demand rather than lengthening expiry.
- Account-key SAS is simple but revocation means rotating the key for everyone; user delegation SAS is safer but needs Entra ID setup.
- Per-user URLs are uncacheable; CDN signing (lesson 11) suits widely read private content.

COMMON MISTAKES

- Signing with long-lived admin keys, or for a whole bucket or container.
- Letting the client choose the object key.
- 7-day URLs "to be safe", which then show up in logs and chat history.
- Logging full URLs (including the signature) in application or proxy logs.

8 Key terms

Pre-signed URL: URL carrying a signature that grants one operation until expiry

SAS token: Azure shared access signature: permissions, expiry, IP, signature

Stored access policy: server-side policy a service SAS can reference, editable to revoke

SigV4: AWS request-signing algorithm based on HMAC-SHA256

User delegation SAS: SAS signed with an Entra ID–derived key, not the account key

Bearer token: a credential that works for whoever holds it

9 Interview questions

Q1. How do pre-signed URLs work? BASIC

The server signs one permission (object, method, expiry, optional headers) with its key; anyone holding the URL can do exactly that without credentials, and storage recomputes and checks the signature.

Q2. What are their risks? BASIC

They're bearer tokens that can leak via logs, history or chats and usually can't be revoked individually; keep scope narrow, expiry short, and enforce size and content type.

Q3. A pre-signed download URL was posted publicly. How do you kill it before it expires? SCENARIO

Remove the signing identity's permission or rotate/deactivate its key (S3), or revoke the stored access policy / delegation keys (Azure); then move to shorter expiries and a dedicated signer.

Q4. How do you stop clients uploading 10 GB files with an upload URL? SCENARIO

Use a pre-signed POST policy with content-length-range (or a signed exact size/checksum), check quota before signing, and verify the real size after upload.

Q5. Your Lambda signs URLs with a 7-day expiry, but users report them failing after a few hours. Why? SCENARIO

The URL was signed with the function role's temporary credentials; it becomes invalid when those session credentials expire. Sign with a longer-lived identity if long URLs are really needed.

Q6. Does generating a pre-signed URL check that the object exists or that the caller is allowed? TRICKY

No; signing is purely local. Existence and permissions (of the signer) are checked by storage when the URL is used.

Q7. A 15-minute upload URL is used for an upload that takes an hour. Does it fail? TRICKY

No: expiry is checked when the request starts, so a transfer that begins in time completes. Multipart uploads need a valid URL per part, so later parts need fresh URLs.

Next: SD-O-006 · Resumable Uploads backendroadmap.com

Resumable Uploads

upload in parts, resume what's missing

Watch the 4-min video

In one line: split large uploads into numbered parts under one upload ID, send parts in parallel, retry only failed parts, and after any interruption ask the server what it already has and send only the rest (S3 multipart, Azure block blobs, GCS resumable uploads or the open tus protocol); then let a lifecycle rule clean up uploads nobody finished.

1 Back to zero at 90%

A user uploads a 5 GB video over hotel Wi-Fi at 20 Mbit/s, which takes about 36 minutes. At 90% the connection drops. With a single `PUT`, those 4.5 GB are gone and the next attempt starts from byte zero; on a flaky network the chance of 36 failure-free minutes is low, so the user gives up. The fix is to make progress **durable in small pieces**.

2 Multipart upload

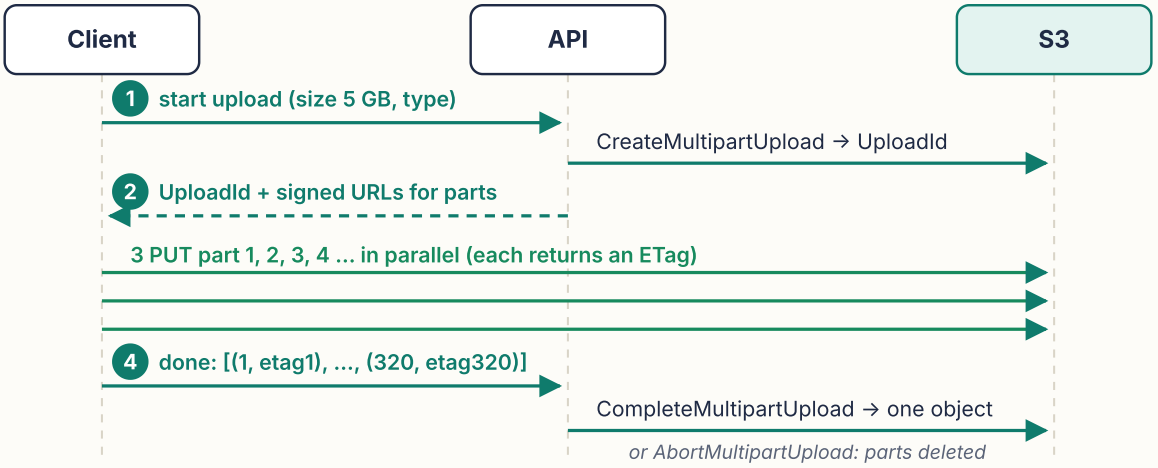


Figure 1. S3 multipart: one UploadId ties numbered parts together. Parts are invisible until Complete stitches them into one object.

1. **Create**: `CreateMultipartUpload` returns an `UploadId`.
2. **Upload parts**: `UploadPart` with `PartNumber` 1–10,000. Each part is 5 MiB to 5 GiB (only the last may be smaller) and returns an ETag. Parts can be sent in any order, in parallel, and re-sent (the last upload of a part number wins).
3. **Complete**: `CompleteMultipartUpload` with the list of (part number, ETag) in ascending order. S3 assembles the object atomically; until then nothing is visible under the key.
4. **Or abort**: `AbortMultipartUpload` deletes the parts. `ListParts` (1,000 per page) shows what has arrived.

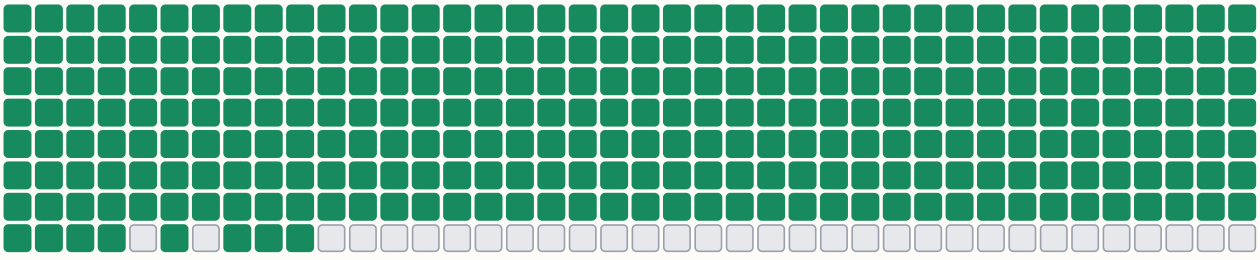
The browser never needs AWS credentials: the API makes the Create/Complete calls and pre-signs one URL per `UploadPart` (lesson 5). Choose the part size from the file size: with a 10,000-part cap, 16 MiB parts allow files up to ~160 GB; bigger files need bigger parts.

3 Two big wins: parallelism and small retries

- **Parallelism**: one TCP stream is often limited by latency and congestion control, not by link speed. Four to eight parts in flight usually fill a fast link; SDK transfer managers (boto3 `TransferConfig`, AWS CLI, Azure `max_concurrency`) do this for you.
- **Small retries**: a failed 16 MiB part is retried alone, with exponential backoff. The worst case after a drop is losing the parts in flight, not the file.
- **Integrity per part**: send a checksum with each part (`Content-MD5` or `x-amz-checksum-crc32` / `-sha256`); S3 rejects a corrupted part instead of storing it. Note the final ETag of a multipart object is *not* the MD5 of the file: it's a hash of the part hashes with a `-320` suffix.

Smaller parts mean less rework per failure but more requests (each `UploadPart` is a billed PUT) and more URLs to sign; 8–64 MiB is a common range.

4 Resumable: survive a crash or restart



4 parts were in flight when Wi-Fi dropped; parallel workers finish out of order, so gaps are normal

Figure 2. After reconnecting, the client asks what arrived and sends only the gaps; order doesn't matter until Complete.

Resuming needs state that outlives the connection. The **server** keeps the parts (keyed by UploadId); the **client** persists `{uploadId, key, partSize, file fingerprint}` in IndexedDB or the app's database. After a drop, a crash or even a reboot, the app reopens the same file, checks the fingerprint (name, size, modified time), asks the API for `ListParts`, gets fresh signed URLs for the missing part numbers (old ones may have expired) and continues. On mobile, OS background transfer services (e.g. iOS background `URLSession`) let parts keep uploading while the app is suspended.

Mechanism	Pieces	Ask what's there	Unfinished uploads
S3 multipart	parts 5 MiB–5 GiB, ≤ 10,000	`ListParts`	kept and billed until aborted; use a lifecycle rule
Azure block blob	`Put Block` with a block ID; ≤ 50,000 blocks; commit with `Put Block List`	`Get Block List` (uncommitted)	uncommitted blocks discarded after about a week
GCS resumable	byte ranges, chunks in multiples of 256 KiB	empty PUT with `Content-Range: bytes */total` → 308 + `Range`	session URI valid for one week
tus 1.0	byte offset; parallel via *concatenation* extension	`HEAD` → `Upload-Offset`	*expiration* extension; server deletes

5 The tus protocol

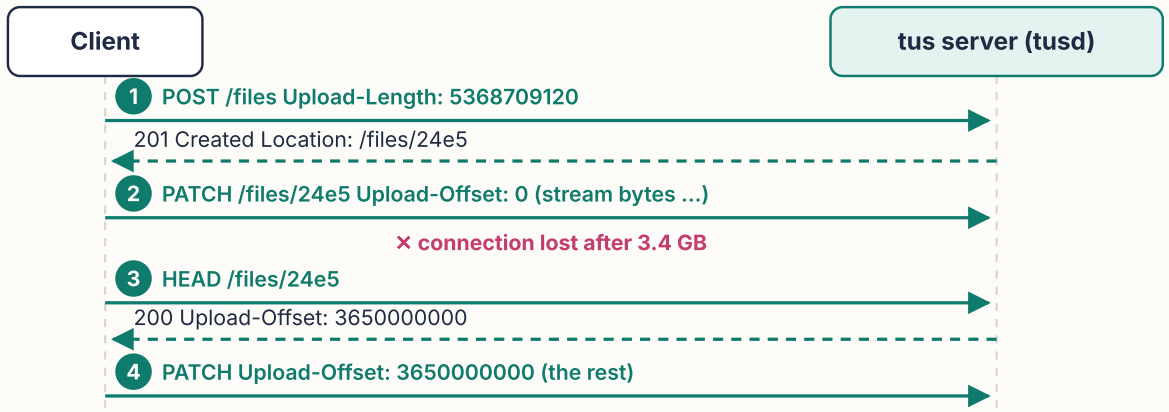


Figure 3. tus: create, PATCH bytes at an offset, and after a failure HEAD tells you where to continue. Every request carries `Tus-Resumable: 1.0.0`.

tus is an open HTTP protocol for resumable uploads. The core is two requests: `HEAD` returns the current `Upload-Offset`, and `PATCH` (body type `application/offset+octet-stream`) appends bytes starting at a given offset; the server rejects a PATCH whose offset doesn't match (409). Extensions add *creation* (POST with `Upload-Length`), *checksum*, *expiration*, *termination* (DELETE) and *concatenation* (upload pieces in parallel, then join). Clients: tus-js-client, Uppy, iOS and Android libraries; servers: tusd (which can store into S3, Azure or GCS), tus-node-server. Use tus when you want one protocol across web and mobile and your own server in the path for auth and hooks; use native multipart when you want bytes to go straight to the cloud provider.

6 Clean up abandoned uploads

An abandoned multipart upload keeps its parts: invisible in normal listings, but **billed**. A busy upload service can quietly accumulate terabytes of them. Add a lifecycle rule `AbortIncompleteMultipartUpload` with `DaysAfterInitiation: 3` (lesson 12), and set business limits: a maximum file size, a maximum number of parts and of concurrent uploads per user, and count in-progress uploads against quota, so nobody fills your storage on purpose.

7 Example: resuming a 5 GB upload

WORKED EXAMPLE: 320 PARTS, WI-FI DROPS AT PART 280

The file is 5,120 MiB; with 16 MiB parts that's **320 parts**. The app asks the API to start; the API calls `CreateMultipartUpload`, stores the UploadId against the `videos` row and returns signed URLs in batches of 50. The app uploads 4 parts at a time and saves progress locally. Around part 280 the Wi-Fi drops. Back online, the app calls `GET /uploads/{id}/parts`; the API's `ListParts` shows **288 of 320** present (in-flight parts finished out of order). The app requests fresh URLs for the 32 missing parts and sends them: 512 MiB, about 3½ minutes at 20 Mbit/s instead of 36 minutes from zero. Then it sends the 320 (part, ETag) pairs; the API calls `CompleteMultipartUpload` and the video appears as one object, triggering the processing pipeline (lesson 10).

TRADE-OFFS

- Multipart adds round trips and state; below ~100 MB a single PUT with a retry is simpler.
- Smaller parts: cheaper failures and finer progress, but more requests, signatures and per-request cost.
- Native multipart sends bytes straight to the cloud; tus is portable and uniform but puts your server (or tusd) in the data path.
- More parallelism helps fast links but hurts slow, shared ones (and phones on metered data).

COMMON MISTAKES

- No lifecycle rule for incomplete uploads: invisible parts pile up on the bill.
- Keeping the UploadId only in memory, so an app restart loses the ability to resume.
- Picking a part size that hits the 10,000-part cap for large files.
- Treating the multipart ETag as a file MD5 when verifying integrity.

8 Key terms

Multipart upload: object uploaded as numbered parts, then assembled

ListParts: call that returns the parts already received

tus: open HTTP protocol: POST to create, PATCH at offset, HEAD to resume

UploadId: identifier tying parts of one multipart upload together

Resumable upload: upload that can continue from its last durable point

AbortIncompleteMultipartUpload: lifecycle action that deletes unfinished uploads

9 Interview questions

Q1. How do you support resumable uploads? BASIC

Split the file into numbered parts under one upload ID, upload parts in parallel, retry only failed parts, and after a failure ask which parts exist and send only the rest, via multipart uploads or the tus protocol.

Q2. What about abandoned uploads? BASIC

A lifecycle rule aborts incomplete multipart uploads after a few days so their parts stop costing money; also cap file size and concurrent uploads per user.

Q3. Users upload 200 GB scientific files. What part size do you choose? SCENARIO

At least 200 GB ÷ 10,000 ≈ 20 MiB, so pick something like 32–64 MiB; that stays under the part cap and keeps the request count reasonable.

Q4. The browser tab is closed mid-upload. How does the user continue tomorrow? SCENARIO

Persist the upload ID, key, part size and a file fingerprint client-side; when the user re-selects the file, match the fingerprint, ListParts via the API, and upload only the missing parts with fresh URLs.

Q5. Upload throughput is poor on a 1 Gbit office link. What do you tune? SCENARIO

Raise concurrency (more parts in flight) and use reasonably large parts; one TCP stream rarely fills a long-latency fast link.

Q6. Why doesn't the ETag of a multipart object match the file's MD5? TRICKY

S3 computes it from the MD5s of the parts plus a part-count suffix; use per-part checksums or the additional checksum algorithms (e.g. SHA-256, CRC32C) for end-to-end verification.

Q7. Do parts have to be uploaded in order? TRICKY

No: any order, in parallel, and a part number can be re-uploaded; only Complete needs the part list in ascending order.

Next: SD-O-007 · File Metadata Storage

File Metadata Storage

facts in a database, bytes in a bucket

Watch the 4-min video

In one line: store file **metadata** (name, folder, owner, permissions, size, hash, versions) in a database and the **content** in object storage, linked by an object key; a content hash catches corruption and, used as the key, deduplicates identical content, while versions become rows that point at immutable bytes.

1 Everything in one place is slow

A file service keeps bytes, names, folders, owners and sharing settings together, say as objects whose keys are paths, with facts in object metadata. Listing a folder means a `LIST` over a prefix; renaming a folder means copying and deleting every object under it; checking whether a user may see a file means fetching its metadata; "files I modified last week" can't be answered at all. Bytes and facts have opposite needs: bytes are big, immutable and read sequentially; facts are tiny, change often and must be queried, indexed and updated atomically.

2 Separate metadata from content

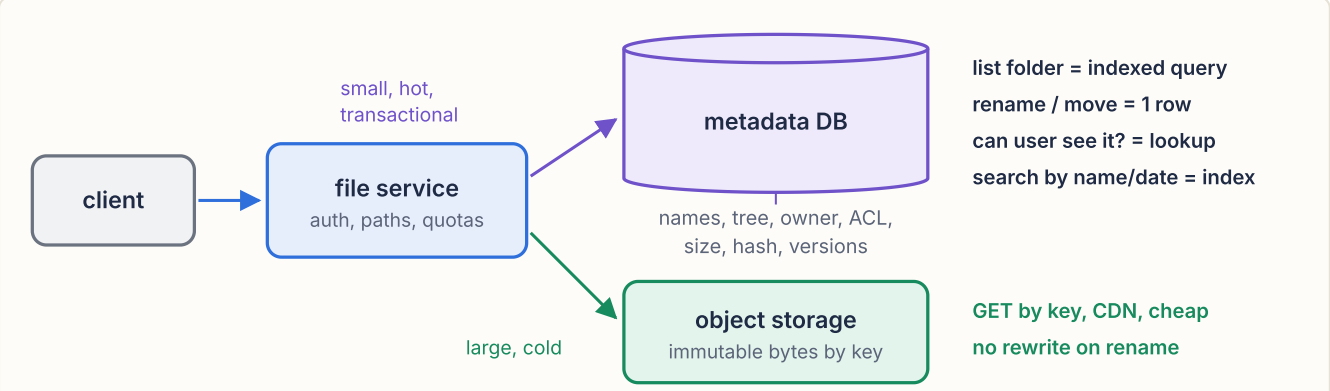


Figure 1. The metadata path is small and transactional; the content path is large and immutable. Each gets the store built for it.

Content goes to object storage under a key the system generates (a UUID or a hash, never the user's path). Metadata goes to a relational or key-value database. Every metadata row points at its content by that key. The split pays off immediately:

- **Listing** a folder is an index range scan on `(parent_id, name)`, with sorting and paging.
- **Rename or move** updates one row; the bytes never move.
- **Permission checks** are database lookups (with inheritance from parent folders), not storage calls.
- **Search** by name, size, type or date uses indexes; a search engine can be fed from the same rows.
- **Transactions**: a move between folders, a quota update and an audit entry commit together.

3 Modelling the tree

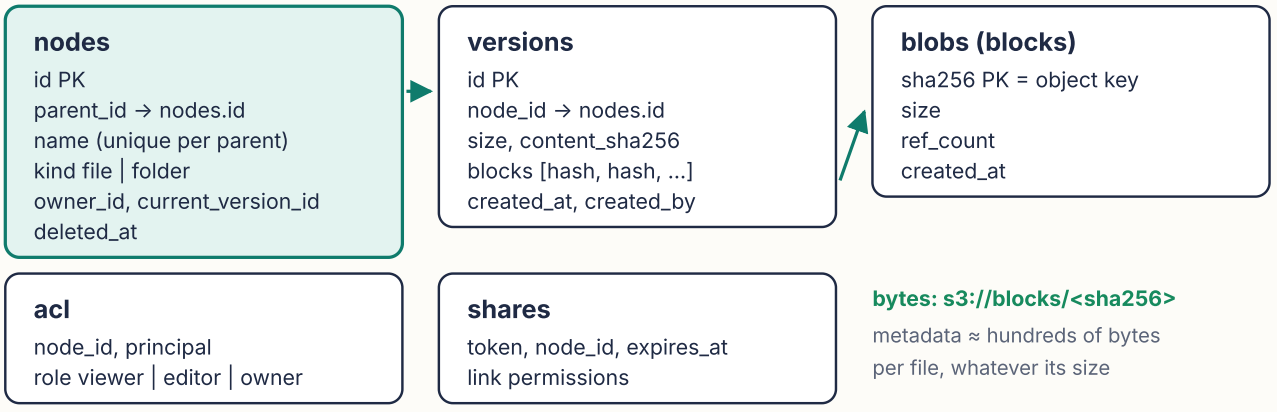


Figure 2. A typical schema: a node tree with parent pointers, immutable versions, content-addressed blobs with reference counts, and ACLs.

The **adjacency list** (`parent_id, name`, unique together) makes rename and move a single-row update, which is what users do most. Resolving a full path walks up the tree, so cache paths or use a recursive query. A **materialized path** column (`/a/b/c`) makes subtree queries trivial but turns moving a folder into rewriting every descendant; a **closure table** answers "all descendants" fast at the cost of extra rows. Permissions are usually stored on folders and inherited, with explicit grants and links (`acl`, `shares`) checked on access.

At scale, shard metadata by owner or namespace (account, shared folder) so a folder listing hits one shard. Dropbox, for example, keeps file metadata in MySQL-backed systems (Edgestore and successors) and the 4 MB blocks in its own block store, Magic Pocket. Metadata grows with the *number* of files, not their size: a billion files at ~500 bytes each is ~0.5 TB before indexes, easily held by a sharded database.

4 Keeping DB and storage consistent

There's no transaction across a database and a bucket, so order the writes so failures leave only *garbage*, never *dangling pointers*:

1. Write the bytes first (immutable, under a new key).
2. Then commit the metadata row that references them, in one database transaction.
3. Garbage-collect objects that no row references, after a grace period (e.g. 24 hours) so an upload that's about to commit isn't collected.

Deletion runs the other way: soft-delete the row (trash), and only when nothing references the content does a background job delete the object.

5 Store a checksum

Store a strong content hash such as **SHA-256** with every file. The client computes it before uploading; the server (or the storage service) computes it on arrival; a mismatch means corruption in transit and the upload is rejected. S3 can verify checksums on upload (`Content-MD5`, or additional algorithms such as CRC32C and SHA-256) and stores them with the object. Later, the stored hash detects silent corruption during migrations, restores or periodic scrubbing. Use SHA-256 rather than MD5 or SHA-1 when the hash also identifies content: practical collisions have been demonstrated for both of those.

6 The hash as the address

In **content-addressed storage** (CAS) the object key *is* the hash: `blocks/9f86d081…`. Identical content produces the same key, so it is stored once and every file row that contains it points to the same object: deduplication for free. Git, Docker/OCI image layers and many backup tools work this way. Bonuses: objects are immutable, so they can be cached forever; uploads are idempotent; integrity is self-checking (re-hash and compare with the key).

Shared content needs **reference counting** (or periodic mark-and-sweep): increment when a version references a blob, decrement when it's deleted, and delete the object only at zero, after the grace period. Before linking a user to an existing blob just because they sent its hash, make sure they actually have the content, or a leaked hash becomes a way to read someone else's file (lesson 8).

7 Versioning

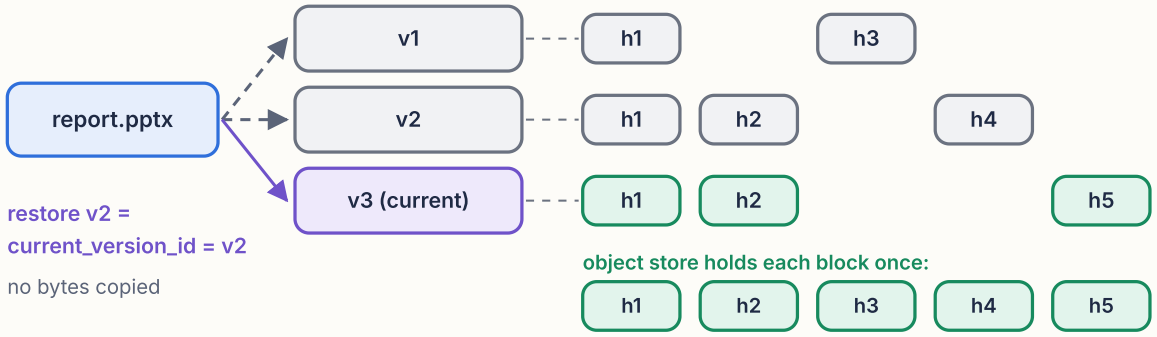


Figure 3. Each version is a row listing its blocks. Unchanged blocks are shared; restoring a version only moves a pointer.

Each save creates a new `versions` row pointing at new content; the file's `current_version_id` moves forward. Old versions keep their pointers, so history costs only the *changed* content when files are stored as blocks. Restore is a pointer update, not a copy. A retention rule ("keep versions for 30 days" or "the last 100") deletes old rows, decrements ref counts, and lets GC free blocks nobody uses.

8 Example: Dropbox-style metadata

WORKED EXAMPLE: TWO USERS UPLOAD THE SAME INSTALLER

A file row holds `id, owner, path, size` and its block list: the SHA-256 of each 4 MB block. A 300 MB installer is 75 blocks. User A uploads it: the client hashes the 75 blocks and asks the server which it lacks; all 75 are new, so it uploads 300 MB, each block stored as `blocks/<sha256>`, then commits one `nodes` row and one `versions` row (ref counts +1). User B saves the same installer: the server already has all 75 hashes, so after an ownership check the "upload" finishes in a second; only a new metadata row is written and ref counts go to 2. B renames it: one row changes. A deletes theirs: ref counts drop to 1 and the blocks stay for B.

TRADE-OFFS

- Two stores mean no shared transaction: you need careful write ordering and garbage collection.
- Content addressing deduplicates and self-verifies, but needs ref counting and complicates per-user deletion and encryption.
- Adjacency lists make moves cheap and path lookups slower; materialized paths are the reverse.
- Block-level versions save space but turn one file into many objects (more requests, more metadata).

COMMON MISTAKES

- Using the user's path as the object key, so renames rewrite bytes.
- Committing metadata before the bytes exist, creating rows that point at nothing.
- Deleting a shared content object when one of its owners deletes their file.
- Trusting a client-supplied hash without verifying it on the server.

9 Key terms

Metadata: facts about a file: name, owner, folder, size, hash, ACL

Checksum: hash used to detect corruption (e.g. SHA-256, CRC32C)

Adjacency list: tree stored as (parent_id, name) rows

Content-addressed storage: objects keyed by the hash of their content

Reference count: number of versions pointing at a blob; delete at zero

Garbage collection: deleting unreferenced objects after a grace period

10 Interview questions

Q1. Where do you store file metadata, and why separately? BASIC

In a database, with the bytes in object storage: metadata is small, structured and changes often (needs indexes, queries, transactions), while content is large and immutable.

Q2. Why store a content hash? BASIC

It catches corruption in transit and at rest, and when the hash is the object key, identical content is stored only once (with a reference count).

Q3. How do you implement 'move folder with 100,000 files'? SCENARIO

With an adjacency-list tree, update the folder's parent_id in one transaction; no object moves. With materialized paths you'd rewrite 100,000 rows, so avoid that model if moves are common.

Q4. The upload succeeded but the metadata insert failed. What's the state, and how do you clean up? SCENARIO

An orphan object with no row; harmless if objects are written first. A GC job deletes unreferenced objects older than a grace period.

Q5. How do you shard the metadata of a billion-file service? SCENARIO

By owner or namespace (account, shared folder), so listings and permission checks stay on one shard; keep a directory from namespace to shard.

Q6. With dedup by hash, how do you safely delete a user's file? TRICKY

Delete their metadata rows and decrement ref counts; only when a blob reaches zero (and after a grace period, to avoid racing with new references) delete the object.

Q7. Why not just use S3 object metadata and tags instead of a database? TRICKY

They can't be queried or indexed, are limited in size, and can't be updated transactionally with other records; you'd need a LIST plus a HEAD per object to answer simple questions.

Next: SD-O-008 · Chunking and Deduplication backendroadmap.com

Chunking and Deduplication

think in chunks, not files

Watch the 4-min video

In one line: split files into chunks, hash each one, and upload only the chunks the server doesn't already have; content-defined chunking (a rolling hash picks the boundaries) keeps an insert from shifting every chunk, so a new version costs only its changed chunks: that's delta sync and deduplication.

1 500 MB for a one-slide edit

You change one slide in a 500 MB presentation and the sync client re-uploads all 500 MB: ten minutes on a slow uplink for a few kilobytes of real change. Across millions of users this is mostly wasted bandwidth and storage, because most of each new version is identical to the previous one, and many files (installers, shared videos, photos forwarded around) are identical across users. Sync and backup systems stop treating files as the unit of transfer and storage; they use **chunks**.

2 Chunk, hash, ask

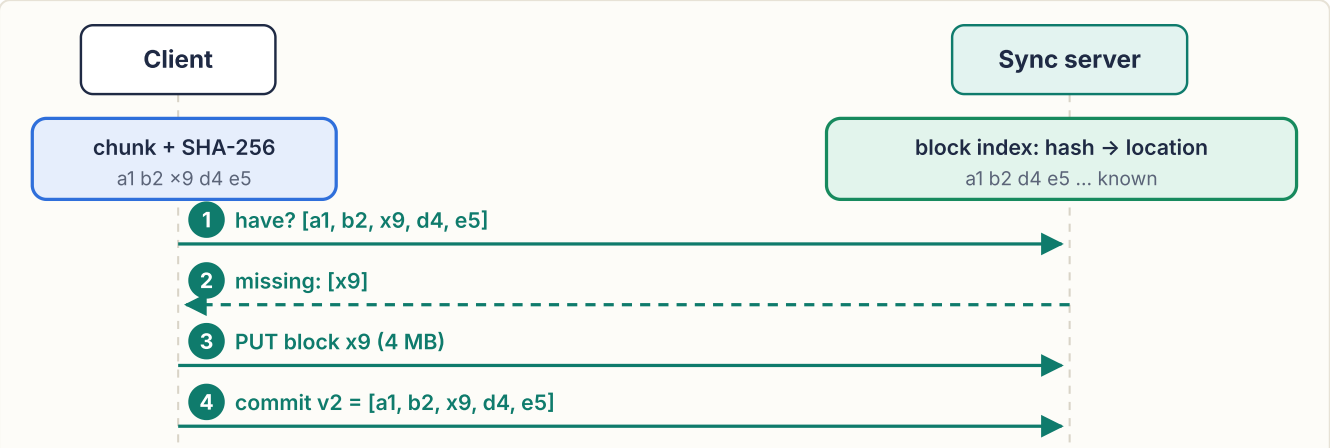


Figure 1. Block-level deduplication: the client sends hashes first, and the server says which chunks it has never seen.

1. The client splits the file into chunks and computes a strong hash (SHA-256) of each.
2. It sends the list of hashes; the server checks its block index and replies with the ones it lacks.
3. Only missing chunks are uploaded (each stored once, content-addressed, lesson 7).
4. The client commits the new version as an ordered list of hashes. Unchanged chunks, from this file, an older version or another file, are referenced again.

Downloads work the same way in reverse: the client fetches only the chunks it doesn't hold locally.

3 Fixed-size chunking and the insert problem

Fixed-size chunking cuts every N bytes (Dropbox uses 4 MB blocks). It's simple and fast, and it works well when edits overwrite bytes in place, as in databases, VM images or many media formats. But insert or delete even one byte near the start of a file and every later boundary shifts: each chunk now holds different bytes, every hash changes, and the whole file looks new.

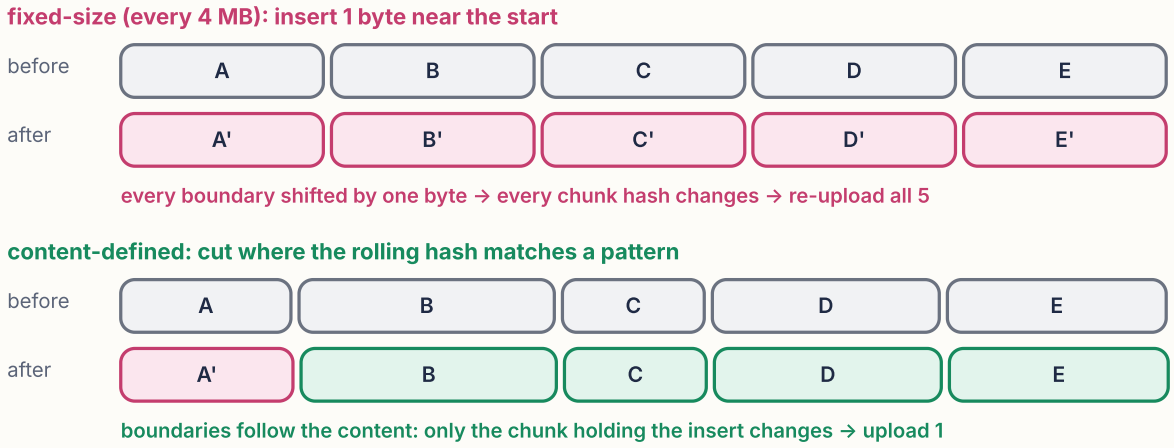


Figure 2. Fixed boundaries are positions; content-defined boundaries are patterns in the data, so they move with the data.

4 Content-defined chunking (CDC)

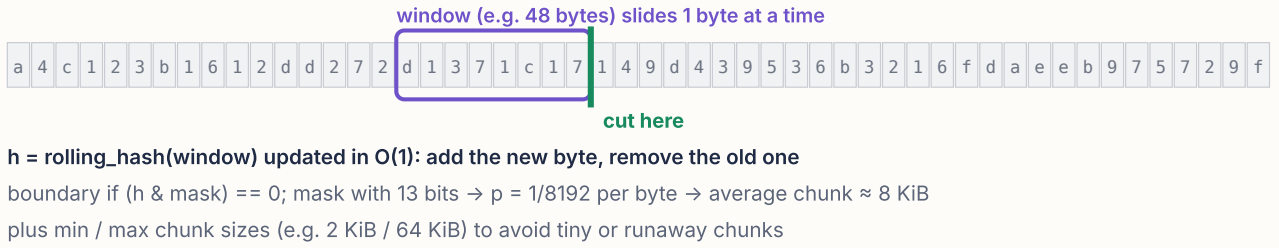


Figure 3. A rolling hash over a small sliding window decides where chunks end. The same content produces the same cut points wherever it sits in the file.

Slide a small window over the data and compute a **rolling hash** of it at every byte; the hash updates in constant time as one byte enters and one leaves. Declare a boundary wherever the hash has a chosen bit pattern, e.g. its low 13 bits are zero. Because the decision depends only on the bytes in the window, an insert changes the boundaries only near the edit; after it, the same content produces the same cut points, so the following chunks keep their hashes. The expected chunk size is about 2^k bytes for a k-bit mask, bounded by minimum and maximum sizes.

System	Chunking	Typical sizes
rsync (Tridgell, 1996)	receiver's file in fixed blocks; sender's rolling weak checksum finds them at any offset, a strong hash confirms	block size scales with file size
LBFS (SOSP 2001)	CDC with Rabin fingerprints over a 48-byte window	average 8 KB, min 2 KB, max 64 KB
Dropbox	fixed-size blocks, SHA-256	4 MB
restic / Borg	CDC (Rabin / buzhash) for backups	about 1–2 MiB average (512 KiB–8 MiB)
FastCDC (USENIX ATC 2016)	CDC with a cheaper "Gear" hash and normalised sizes	configurable; several times faster than Rabin CDC

5 Delta sync

With chunking, a new version is just a new list of hashes, most of them already stored. **Delta sync** moves only the changed chunks in both directions; unchanged ones are referenced again. The same mechanism deduplicates **across** files and users: a popular installer is stored once no matter how many people have it. Two related ideas: rsync computes a delta between two specific files, without a global chunk store; and some services add **compression** per chunk and **streaming** (upload chunks while the file is still being written).

6 Trade-offs: chunk size and privacy

Chunk size. Smaller chunks find more duplicates and make edits cheaper, but cost more hashes, index entries and requests. Rough numbers for 1 PB of unique data, with ~40 bytes per index entry (32-byte hash + location): 8 KiB chunks → ~1.2×10^{11} chunks and ~5 TB of index; 4 MiB chunks → ~2.4×10^{8} chunks and ~10 GB of index. Object stores also charge per request, so tiny chunks are often packed into larger objects ("pack files").

Privacy. Cross-user dedup is a side channel: if uploading a file finishes instantly, someone else already has it. Attackers can confirm that a known file exists, or guess a secret inside a templated document by trying variants. And if the server links a user to content just because they sent its hash, a leaked hash becomes a download token (2011's "Dropship" did this with Dropbox hashes). Mitigations: dedup only within one account, dedup server-side only (client always uploads; saves storage, not bandwidth), or ask the client to prove possession (e.g. hashes of random byte ranges). End-to-end encryption with per-user keys makes identical files look different, which disables cross-user dedup unless you use *convergent encryption* (key derived from the content), which reintroduces the equality leak.

TRADE-OFFS

- Fixed-size: fast and simple, perfect for in-place edits; fails on inserts and deletes.
- CDC: robust to shifts; costs hashing CPU and gives variable chunk sizes.
- Small chunks: better dedup, more metadata and requests. Large chunks: the reverse.
- Cross-user dedup: big savings, but a privacy side channel; per-account dedup is safer.

7 Example: only the changed 4 MB blocks

WORKED EXAMPLE: A 2 GB VIDEO PROJECT, A FEW SECONDS CHANGED

The file is 2,000 MB, so with fixed 4 MB blocks it's **500 blocks**. The user re-renders a few seconds in the middle; the project format overwrites bytes in place, so the file length and all other offsets stay the same. The client re-chunks and hashes the file (SHA-256 runs at hundreds of MB/s per core, so a few seconds), sends 500 hashes (16 KB of hashes), and the server replies that it lacks 2. The client uploads **2 blocks = 8 MB** instead of 2 GB, and the server records version 2 as the new block list; version 1 keeps its list, so restoring it is free.

Had the edit *inserted* data (adding a frame to a format that isn't rewritten in place), fixed blocks after the insert would all shift and ~half the file would re-upload. With CDC only the chunks around the insert would change.

COMMON MISTAKES

- Using fixed-size chunks for data that is edited by insertion (text, archives rebuilt from scratch).
- Using a weak or non-cryptographic hash (CRC, MD5) as the chunk identity.
- Linking content to a user on hash alone, without proof of possession.
- Ignoring index and request overhead when choosing a tiny average chunk size.

8 Key terms

Chunk / block: piece of a file, identified by the hash of its bytes

Content-defined chunking: boundaries chosen by content, not position

Delta sync: transfer only the chunks that changed between versions

Block-level dedup: store and send each distinct chunk only once

Rolling hash: hash of a sliding window updated in O(1) per byte (Rabin, buzhash, Gear)

Convergent encryption: key derived from content so equal files encrypt equally

9 Interview questions

Q1. How does a file sync service avoid re-uploading unchanged data? BASIC

It splits files into chunks, hashes them, and uploads only chunks the server doesn't have; content-defined chunking with a rolling hash keeps an insert from shifting every boundary.

Q2. What is delta sync? BASIC

Sending only the changed chunks of a new version, while unchanged chunks are referenced again by hash.

Q3. Users edit large text-based CSV exports by inserting rows near the top. Fixed or CDC? SCENARIO

Content-defined chunking: inserts shift every fixed boundary, while CDC boundaries follow the content and only nearby chunks change.

Q4. Design dedup for a backup product storing many VM images. SCENARIO

CDC with ~1–8 MiB average chunks, SHA-256 identities, chunks packed into larger objects, a chunk index, and per-snapshot chunk lists with ref counting or mark-and-sweep GC.

Q5. How do you pick the average chunk size? SCENARIO

Balance dedup ratio against index size and request counts: estimate chunks = data ÷ chunk size and the memory/DB cost of the index; MB-scale for cloud sync and backups, KB-scale only with packing.

Q6. Why can cross-user deduplication leak information? TRICKY

An instant upload reveals the server already has that content, so an attacker can test whether a file (or a guessed variant of a document) exists; per-account dedup or proof of possession limits this.

Q7. Why does CDC need minimum and maximum chunk sizes? TRICKY

Without a minimum, unlucky data produces tiny chunks (index blow-up); without a maximum, data that never matches the pattern (e.g. long runs of zeros) produces huge chunks that dedup poorly.

Next: SD-O-009 · Image Processing backendroadmap.com

Image Processing

resize, convert, cache

Watch the 4-min video

In one line: never serve the original upload: create smaller variants in modern formats (WebP, AVIF, with JPEG fallback), either pre-generated by an asynchronous worker or transformed on demand and cached at the CDN, with guard rails on sizes, pixel counts and decoders.

1 An 8 MB thumbnail

A user uploads a 12-megapixel phone photo (4032 × 3024, about 8 MB). The product page shows it at 300 × 225 CSS pixels. Serve the original and every visitor downloads 8 MB for an image that needs roughly 30–60 KB, pages load slowly, Largest Contentful Paint suffers, and mobile users pay for the data. Phones also upload formats browsers can't all show (HEIC from iPhones), rotated via EXIF tags, in wide-gamut colour, with GPS coordinates embedded. Images need processing before they're served.

2 Asynchronous resizing after upload

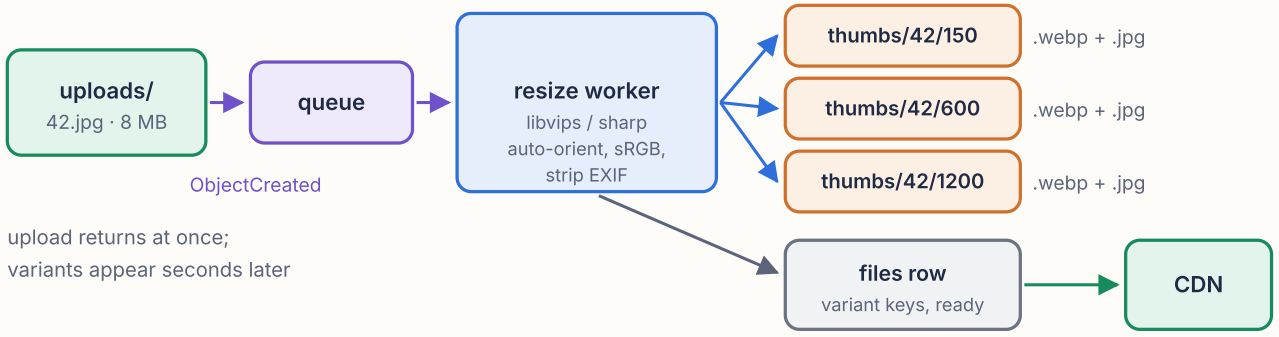


Figure 1. Pre-generating: an upload event triggers a worker that writes every variant the app needs under predictable keys.

The upload goes straight to the bucket (lesson 4); an `ObjectCreated` event puts a job on a queue; a worker downloads the original, creates each size and format, writes them as separate objects and records them in the database. The upload request returns immediately and the UI shows a placeholder until the row is `ready`. Processing steps, in order:

1. **Decode safely**: check the declared dimensions before decoding (see section 5).
2. **Auto-orient**: apply the EXIF Orientation tag to the pixels, *then* strip metadata, or photos appear sideways.
3. **Colour**: convert to sRGB (or keep the ICC profile); dropping a Display P3 profile without converting makes colours look washed out.
4. **Resize** with a good filter (Lanczos) to each target width, keeping the aspect ratio; crop only where a fixed box is needed (avatars).
5. **Encode** each format at a tuned quality, and **strip metadata**: GPS location and camera serials are a privacy leak, and EXIF can be tens of KB.

Use **libvips** (sharp, pyvips, imgproxy): it streams through the image and is typically much faster and lighter on memory than ImageMagick.

3 Formats matter as much as size

Format	Compared with JPEG	Browser support	Notes
JPEG	baseline	universal	fallback; use progressive / mozjpeg encoding
WebP	typically 25–35% smaller at similar quality	all modern browsers (Safari 14+)	fast to encode; supports alpha and animation
AVIF	often around 50% smaller	Chrome, Firefox, Safari 16+	slow and CPU-heavy to encode; great for large hero images
PNG	much larger for photos	universal	only for graphics needing lossless or sharp edges

Serve the best format each browser supports. Either the HTML chooses (`<picture>` with `<source type>`), or the server negotiates on the request's `Accept` header (`image/avif,image/webp,…`) and replies with `Vary: Accept`. With negotiation, make the CDN cache key include the *normalised* format (avif / webp / jpeg), not the raw Accept string, or the hit rate collapses (lesson 11).

```
<picture>
  <source type="image/avif" srcset="/t/42/600.avif 600w, /t/42/1200.avif 1200w">
  <source type="image/webp" srcset="/t/42/600.webp 600w, /t/42/1200.webp 1200w">
  <img src="/t/42/600.jpg" srcset="/t/42/150.jpg 150w, /t/42/600.jpg 600w,
       /t/42/1200.jpg 1200w" sizes="(max-width: 600px) 100vw, 600px"
       width="600" height="450" loading="lazy" alt="…">
</picture>
```

Figure 2. Responsive images: the browser picks the first format it supports and, via `srcset` / `sizes`, the smallest width that fills the slot at its pixel ratio (600w for a 300 px slot on a 2× screen). Explicit width/height prevent layout shift.

4 On-demand transformation

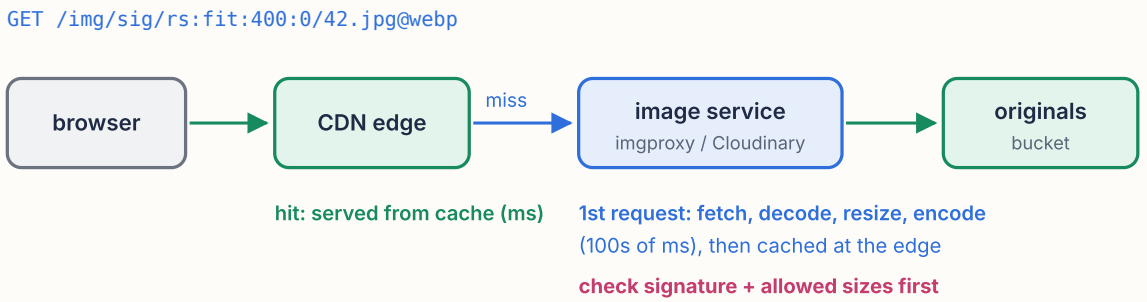


Figure 3. On demand: the URL describes the variant; the image service makes it on the first request and the CDN caches it.

The URL itself describes the variant, e.g. `/img/<sig>/rs:fit:400:0/42.jpg@webp` (imgproxy) or `/w_400,f_auto/42.jpg` (Cloudinary-style). On a CDN miss, an image service fetches the original, transforms it and returns it with long cache headers; every later request is an edge hit. Options: self-hosted **imgproxy** or **thumbor**; managed **Cloudinary**, **imgix**, **Cloudflare Images**; or AWS's Dynamic Image Transformation for CloudFront solution (Lambda + sharp).

	Pre-generated	On demand
Latency	every request is a cache or storage hit	first request per variant per region is slow (100s of ms)
Storage	all variants for all images, used or not	only variants actually requested (in cache)
New size or format	backfill job over every original	change the URL; nothing to reprocess
Risk	simple, predictable cost	abuse: arbitrary sizes force endless transforms
Best when	few, known sizes (avatars, product grids)	many sizes, formats, art direction, frequent redesigns

A backfill is not trivial: adding one size for 50 million images at ~100 ms each is 5 million CPU-seconds, about 58 CPU-days, plus the new storage. That's the usual reason teams move to on-demand transforms.

5 Caching and guard rails

- **Cache forever, change the URL**: variants get `Cache-Control: public, max-age=31536000, immutable`, and the URL includes a version or content hash, so an edited image gets a new URL instead of a stale cache (no purges needed, lesson 11).
- **Allowlist variants**: accept only known widths (e.g. 150, 300, 600, 1200; round requests up) and formats, or **sign** transform URLs (imgproxy uses an HMAC key and salt) so only your app can mint them. Otherwise anyone can request `w=1..5000` and burn CPU and cache.
- **Decompression bombs**: a 50 KB PNG can declare 50,000 × 50,000 pixels, which is 10 GB of RAM at 4 bytes per pixel. Check dimensions from the header and cap pixels (Pillow refuses images over about 179 megapixels by default; set your own lower limit) and input bytes.
- **Isolate decoders**: image parsers are a classic attack surface (ImageTragick, CVE-2016-3714, gave remote code execution via ImageMagick). Run them in sandboxed, low-privilege workers with timeouts and memory limits, and keep libraries patched.
- **Origin shield** in front of the image service collapses simultaneous first requests for the same variant into one transform.

TRADE-OFFS

- Pre-generate: fast and predictable, but storage for unused variants and costly backfills.
- On demand: flexible and lean, but slow first hits, more moving parts and an abuse surface.
- AVIF: smallest files, but slow encoding; often worth it only for large or popular images.
- Higher compression saves bytes but can show artefacts; tune quality per format with visual metrics (SSIM, Butteraugli).

6 Example: three thumbnail sizes after upload

WORKED EXAMPLE: 150, 600 AND 1200 PX IN WEBP AND JPEG

Photo 42 lands as `uploads/42.jpg` (8 MB) and an event goes onto a queue. A worker downloads it, checks its 12 MP size against a 50 MP cap, auto-orients, converts to sRGB, and produces widths **150, 600 and 1200** in **WebP and JPEG**: six objects under predictable keys such as `thumbs/42/600.webp`, together around 0.3–0.5 MB instead of 8 MB per view. It writes the variant keys to the photo's row and marks it `ready`; the page uses a `<picture>` element pointing at the CDN, which caches each variant for a year. If the worker crashes, the queue redelivers and the job simply overwrites the same keys: it's idempotent.

COMMON MISTAKES

- Stripping EXIF before applying orientation: sideways photos.
- Leaving GPS metadata in public images.
- Unbounded on-demand sizes, letting anyone trigger expensive transforms.
- Decoding untrusted images in the API process without pixel limits or sandboxing.

7 Key terms

Variant / rendition: resized or re-encoded copy of an original image

srcset / sizes: HTML attributes letting the browser pick the right width

Decompression bomb: tiny file that expands to huge memory when decoded

WebP / AVIF: modern formats, typically 25–50% smaller than JPEG

On-demand transform: variant created on first request from URL parameters, then cached

Auto-orient: apply the EXIF rotation to pixels before stripping metadata

8 Interview questions

Q1. On-demand vs pre-generated image variants? BASIC

Pre-generate when sizes are few and known: simple, and every request is fast. On demand suits many sizes or formats, at the cost of a slower first request and an abuse surface.

Q2. When do on-demand transforms win, and what must you add? BASIC

When you need many sizes or formats or change them often; cache each variant at the CDN, accept a slower first request, and limit or sign the allowed sizes so attackers can't force endless transforms.

Q3. How do you serve WebP or AVIF to browsers that support them and JPEG to others? SCENARIO

Use with typed sources, or negotiate on the Accept header with Vary: Accept and a cache key that includes the normalised format.

Q4. Product wants a new 800 px size for 50 million existing images. Options? SCENARIO

Backfill with a batch job (tens of CPU-days plus storage), generate lazily on first request and store, or switch to an on-demand transform service with CDN caching.

Q5. Your image workers keep running out of memory on some uploads. What's happening? SCENARIO

Likely huge-dimension images or decompression bombs; read dimensions from the header, enforce a pixel cap before decoding, use a streaming library like libvips, and set memory limits.

Q6. Why might thumbnails appear rotated or with dull colours? TRICKY

EXIF orientation was stripped before being applied, and a wide-gamut ICC profile was dropped without converting pixels to sRGB.

Q7. Why put a version or hash in image URLs instead of purging the CDN? TRICKY

A new URL is a guaranteed cache miss everywhere instantly, so you can cache for a year; purges are slow to propagate, rate-limited and miss browser caches.

Next: SD-O-010 · Video Streaming backendroadmap.com

Video Streaming

renditions, segments, a player that adapts

Watch the 4-min video

In one line: transcode each upload into a ladder of renditions, cut each into short keyframe-aligned segments listed in an HLS or DASH manifest, and let the player pick the rendition for each next segment from measured bandwidth and buffer; segments are ordinary immutable files, so a CDN serves almost all of the traffic.

1 One file can't fit everyone

A creator uploads a 2 GB 4K video. A viewer on a train with a weak signal presses play. Send the original (say 40 Mbit/s) and it buffers forever; send one small 480p file to everyone and viewers on fibre and big TVs get a blurry picture. Networks also change minute to minute, and devices differ in codecs and screen sizes. Video needs **several versions** and the ability to **switch mid-stream** without stopping.

2 Transcoding into renditions

The original is decoded and re-encoded into a **bitrate ladder**: the same video at several resolutions and bitrates. An illustrative H.264 ladder:

Rendition	Resolution	Video bitrate (example)	10 minutes ≈
240p	426 × 240	0.4 Mbit/s	30 MB
360p	640 × 360	0.8 Mbit/s	60 MB
480p	854 × 480	1.4 Mbit/s	105 MB
720p	1280 × 720	2.8 Mbit/s	210 MB
1080p	1920 × 1080	5 Mbit/s	375 MB

All five together are about 10.6 Mbit/s, so a 10-minute video costs ~800 MB of renditions on top of the original. **Codecs**: H.264/AVC plays everywhere; HEVC, VP9 and AV1 typically need 30–50% less bitrate for the same quality but cost much more CPU to encode and aren't supported on every device, so platforms encode popular videos in newer codecs too and let the player choose. **Per-title encoding** (Netflix, 2015) tunes the ladder to each video's complexity: a cartoon needs far fewer bits than a football match.

3 Segments and manifests: HLS and DASH

```
# master.m3u8  (one line per rendition)
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=5300000,RESOLUTION=1920x1080,CODECS="avc1.640028,mp4a.40.2"
1080p/index.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=3000000,RESOLUTION=1280x720,CODECS="avc1.64001f,mp4a.40.2"
720p/index.m3u8
# 720p/index.m3u8  (media playlist: the segments, in order)
#EXTM3U
#EXT-X-TARGETDURATION:6
#EXTINF:6.0,
seg_00001.m4s
#EXTINF:6.0,
seg_00002.m4s
#EXT-X-ENDLIST
```

Figure 1. HLS: a master playlist lists renditions (bandwidth, resolution, codecs); each media playlist lists that rendition's segments in order.

Each rendition is cut into **segments** of typically 2–6 seconds (Apple's HLS guidance uses 6 s). For seamless switching, every segment must start with a keyframe (IDR) and keyframes must be **aligned across all renditions** (fixed GOP, closed GOPs), so segment 37 covers exactly the same time in every rendition. A **manifest** describes it all:

	HLS	MPEG-DASH
Origin	Apple (RFC 8216)	ISO/IEC 23009-1 standard
Manifest	`.m3u8` playlists (master + media)	`.mpd` XML
Segments	MPEG-TS or fragmented MP4	fragmented MP4 (or WebM)
Playback	native on Apple devices; hls.js elsewhere	dash.js / Shaka Player via Media Source Extensions; Android ExoPlayer

CMAF standardises fragmented-MP4 segments that both HLS and DASH can reference, so you store the media once and publish two small manifests.

4 How adaptive bitrate switching works

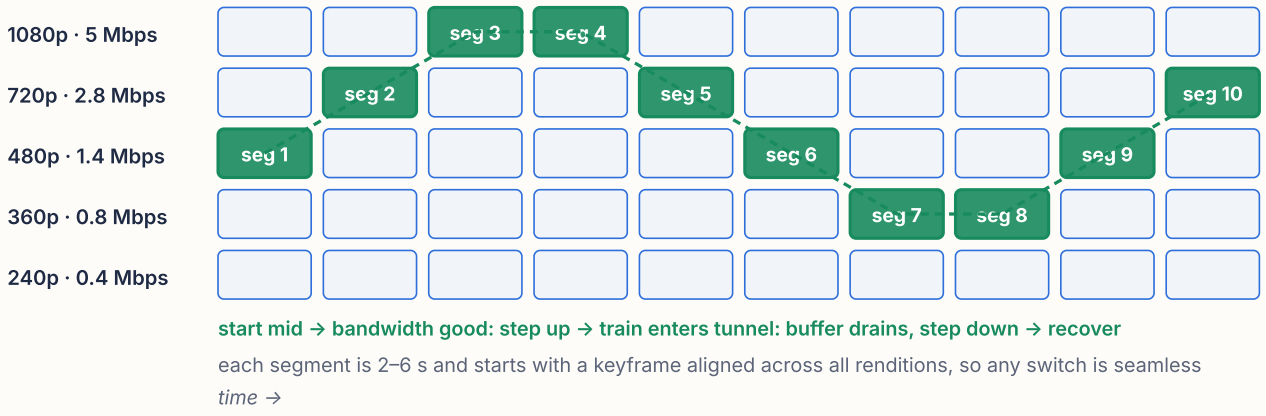


Figure 2. The player chooses a rendition per segment. Here it starts at 480p, climbs to 1080p, drops to 360p as the buffer drains, then recovers.

1. **Start**: fetch the master manifest, pick a modest rendition so the first frame appears quickly (start-up time matters more than initial sharpness), and fill a buffer.

2. **Measure**: after each segment, estimate throughput (segment bytes ÷ download time, smoothed) and check the buffer level.
3. **Decide** for the next segment: *throughput-based* rules pick the highest bitrate below a safety fraction (e.g. 70–80%) of the estimate; *buffer-based* rules (BBA, Netflix/Stanford, SIGCOMM 2014; BOLA in dash.js) map buffer level to bitrate, stepping down as the buffer drains; hybrids combine both. Switching up is cautious, switching down is fast, to avoid stalls.
4. **Switch at a boundary**: the next segment simply comes from another media playlist. Playback never stops, and a rebuffer only happens if the buffer empties completely.

The quality metrics that platforms optimise: start-up time, rebuffer ratio, average bitrate, and the number of switches.

5 Processing as a DAG

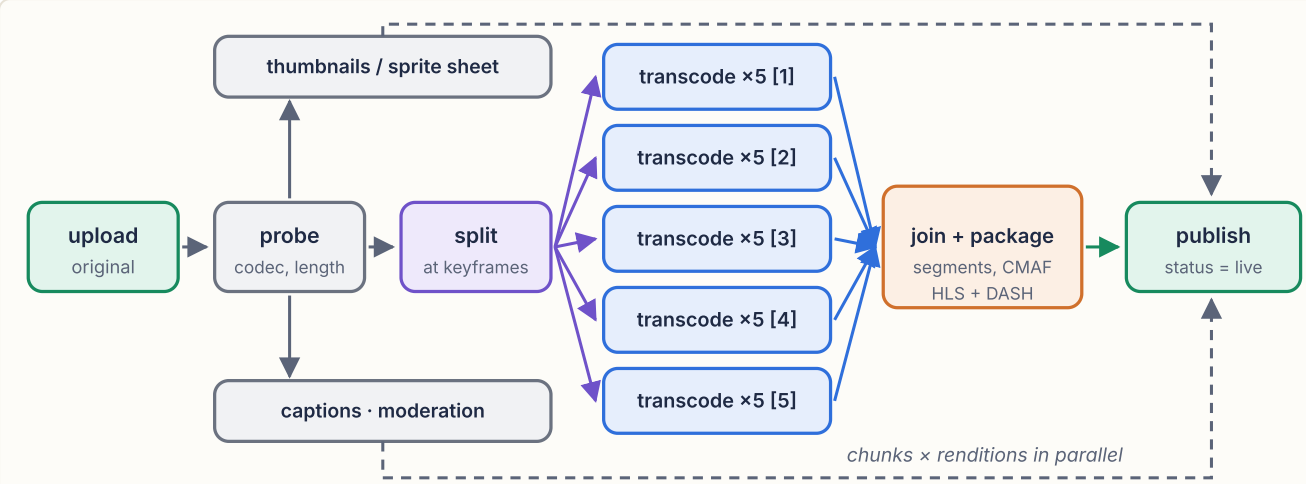


Figure 3. A YouTube-style processing DAG: split, transcode chunks in parallel, join and package, with side branches. Each node retries independently.

Transcoding a 2-hour 4K film on one machine can take hours. Pipelines split the work: **probe** the input (codec, duration, rotation), **split** it into chunks at keyframes (e.g. 30–60 s), **transcode** every chunk × rendition as an independent task on a worker pool, then **join** and **package** into segments and manifests. A 10-minute video in 20 chunks × 5 renditions is 100 parallel tasks and finishes in minutes. Thumbnails, captions (speech-to-text) and content moderation run as separate branches. An orchestrator (Step Functions, Temporal, Airflow, or a home-grown DAG engine) tracks dependencies, retries failed tasks only, and marks the video **live** when the required nodes finish; high-resolution or new-codec renditions can be added later, after the video is already watchable.

6 Delivery via CDN

Segments are ordinary files that never change once written, so they get long cache lifetimes; VOD manifests are cacheable too (live manifests only for about one segment duration). A popular video is served almost entirely from edge caches; the origin sees roughly one request per segment per region (fewer with an origin shield, lesson 11). The hard part is the **long tail**: millions of rarely watched videos miss the cache and hit storage. Large platforms push caches into ISPs: Netflix's Open Connect appliances sit inside ISP networks and are filled overnight with what is predicted to be popular. Paid content adds signed cookies (lesson 11) and DRM (Widevine, FairPlay, PlayReady, via Common Encryption). Live streaming adds ingest (RTMP or SRT), real-time transcoding and constantly updated manifests; Low-Latency HLS and chunked CMAF bring latency down to a few seconds.

7 Example: YouTube-style upload → transcode → CDN

WORKED EXAMPLE: A 10-MINUTE 4K UPLOAD

The creator uploads a 2 GB original to object storage with a resumable multipart upload (lesson 6). The `ObjectCreated` event starts the DAG: probe (3840 × 2160, H.264, 10:00), split into 20 thirty-second chunks, transcode 20 × 5 = 100 tasks in parallel into 240p–1080p H.264, join, cut into 6-second segments (100 per rendition, 500 in total) as CMAF, write HLS and DASH manifests. A side branch extracts thumbnails. When the required nodes finish, the video row becomes `live`, about 800 MB of renditions sit next to the original, and viewers stream from the CDN. The train passenger starts at 360p and moves between 240p and 720p as the signal changes; the fibre viewer settles on 1080p within a few segments.

TRADE-OFFS

- More renditions: smoother adaptation, but more encoding CPU and storage.
- Short segments: faster switching and lower live latency, but more requests and slightly worse compression (more keyframes).
- Newer codecs: big bandwidth savings, but costly encoding and partial device support; often worth it only for popular titles.
- Encoding everything up front vs on first view: instant availability vs saving CPU on videos nobody watches.

COMMON MISTAKES

- Keyframes not aligned across renditions, so switches glitch or stall.
- Transcoding a long video as one serial job: hours of latency and a full restart on failure.
- Short CDN TTLs on immutable segments, sending traffic back to the origin.
- Starting playback at the highest rendition, making start-up slow.

8 Key terms

Rendition / bitrate ladder: the same video at several resolutions and bitrates

Manifest: HLS .m3u8 or DASH .mpd listing renditions and segments

CMAF: common fMP4 segment format usable by both HLS and DASH

Segment: short media file (2–6 s) starting with a keyframe

ABR: adaptive bitrate: the player picks a rendition per segment

Per-title encoding: a ladder tuned to each video's complexity

9 Interview questions

Q1. How does adaptive bitrate streaming work? BASIC

The video is transcoded into renditions, each cut into short segments listed in a manifest; the player measures bandwidth and buffer and picks the best rendition for each next segment, switching without stopping.

Q2. Why cut video into segments? BASIC

Switches happen at segment boundaries, playback can start after one segment, and segments are ordinary immutable files any CDN can cache.

Q3. Transcoding a 2-hour upload takes 3 hours. How do you speed it up? SCENARIO
Split at keyframes into chunks and transcode chunks × renditions in parallel across workers, then join; publish lower renditions first and add the rest later.

Q4. Viewers complain of frequent quality flips. What would you look at? SCENARIO
The ABR rules: smoothing of the throughput estimate, safety margin, buffer thresholds and hysteresis for switching up; also whether segments are too short.

Q5. How would you estimate storage for 1 million hours of video? SCENARIO
Ladder sum × duration: ~10.6 Mbit/s ≈ 4.8 GB per hour of renditions, so ~4.8 PB, plus originals and any extra codecs.

Q6. Why must keyframes be aligned across renditions? TRICKY
So that segment N covers the same time span and starts decodable in every rendition; otherwise switching mid-stream shows glitches or needs extra decoding.

Q7. The CDN hit ratio is 98% but origin load is still high. Why? TRICKY
The long tail: many rarely watched videos and their many segments miss at each edge; add an origin shield, tiered caching, and pre-positioning of predicted-popular content.

Next: SD-O-011 · CDN for Private Content backendroadmap.com

CDN for Private Content

fast for everyone, readable only by payers

Watch the 4-min video

In one line: keep the origin private so only the CDN can read it, have your app issue short-lived signed URLs or signed cookies after checking access, and let the CDN validate them at the edge; design cache keys so everyone shares cached copies, prefer versioned URLs to purges, and protect the origin with a shield layer.

1 Fast, and leaked

An online course platform serves its videos through a CDN and they load fast worldwide. Then someone posts a lesson's URL on a forum and thousands of people who never paid start watching, at your bandwidth cost. Or someone finds the bucket URL behind the CDN and downloads everything directly. A CDN makes content *fast*; making private content fast *and* private needs three things: a locked origin, proof of access checked at the edge, and caching that still works when every user has different credentials.

2 Lock down the origin

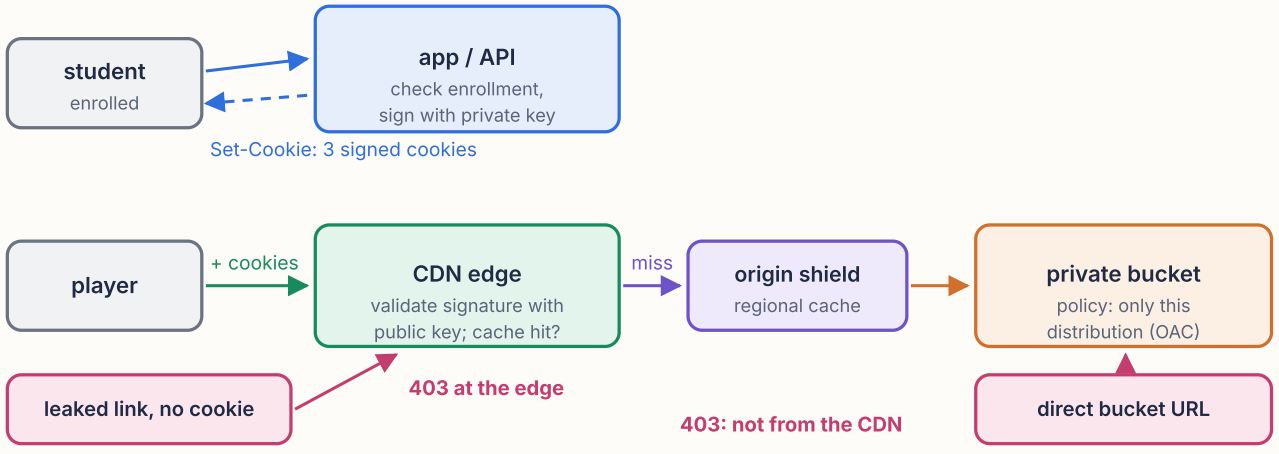


Figure 1. The app proves access by signing; the edge verifies without calling the app; the bucket accepts only the CDN.

The CDN fetches from an **origin**, usually a bucket. If the bucket is reachable directly, any access check at the edge can be skipped. On AWS, block all public access and use CloudFront **Origin Access Control** (OAC): CloudFront signs its requests to S3 with SigV4, and the bucket policy allows the `cloudfront.amazonaws.com` service principal only when `AWS:SourceArn` is your distribution (OAC replaces the older origin access identity and also supports SSE-KMS). On Azure, Front Door Premium can reach a storage origin over **Private Link** so the account needs no public endpoint. For a custom origin, accept only the CDN's IP ranges plus a secret header the CDN adds, or use mutual TLS.

3 Signed URLs and signed cookies

The CDN, not your app, checks every request. Your app signs a **policy** (which resource, until when, optionally from which IP) with a **private key**; the CDN holds the matching **public key** (CloudFront: a *trusted key group*) and verifies the signature locally, so validation adds no round trip to your servers. A

canned policy covers one exact URL and an expiry; a **custom** policy allows wildcards (`/courses/sd/*`), a start time and an IP range.

	Signed URL	Signed cookies
Carries	query string: `Expires` or `Policy`, `Signature`, `Key-Pair-Id`	cookies: `CloudFront-Policy` (or `-Expires`), `CloudFront-Signature`, `CloudFront-Key-Pair-Id`
Covers	one file (or a wildcard, with a custom policy)	many files at once, e.g. every manifest and segment of a course
Best for	a single download, an email link, clients that can't hold cookies	HLS/DASH streaming, galleries, whole private sections
Needs	nothing; works across domains	CDN on a subdomain of your app (cookie domain `.example.com`), Secure + HttpOnly
Leak risk	URL ends up in logs, chats, history	cookies can be exported, but don't appear in links

For streaming, signed URLs are awkward: the player requests hundreds of segment URLs listed in manifests you would have to rewrite per user. Signed cookies cover them all. (If a request carries both, CloudFront uses the signed URL.) Other CDNs offer the same idea under other names: Cloudflare token authentication (HMAC checked by a WAF rule), Akamai EdgeAuth tokens. These CDN signatures are different from storage pre-signed URLs (lesson 5): they're validated at the edge, so the content stays cacheable.

4 Cache keys

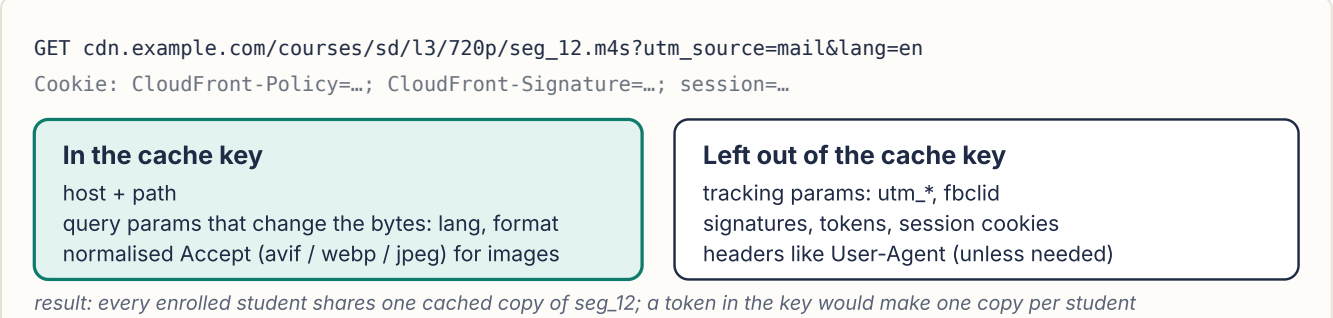


Figure 2. The cache key decides what counts as "the same object". Keep per-user and tracking data out; keep content-changing inputs in.

By default the key is roughly host + path. Two opposite failure modes: **too much** in the key (tracking parameters, tokens, cookies, `User-Agent`) and the same bytes are cached thousands of times with a terrible hit ratio; **too little** (ignoring a `lang` or format parameter that changes the response) and users get the wrong version. On CloudFront this is a *cache policy* (which headers, cookies and query strings are in the key) separate from an *origin request policy* (what is forwarded to the origin). Signed cookies are validated separately and don't need to be in the key, so all students share one cached segment.

5 Purge or version?

- **Purge / invalidation**: remove paths (or tags, on some CDNs) from every edge. Useful for emergencies (a leaked file, a legal takedown), but it takes time to propagate, is rate-limited and may be billed (CloudFront: first 1,000 paths a month free), and it doesn't clear browser caches.
- **Versioned URLs** (the default): put a version or content hash in the name, `app.3f9a2c.js`, `/v7/l3/seg_12.m4s`. Changed content gets a new URL, so it is a guaranteed miss everywhere at once;

old objects just expire. Cache versioned files for a year (`max-age=31536000, immutable`) and give only the few entry points (HTML pages, VOD master manifests if they change) short TTLs.

6 Origin shield and multi-CDN

A CDN has hundreds of edge locations; without help, a new video's first segment can be fetched from the origin once per edge. An **origin shield** (CloudFront Origin Shield, Cloudflare Tiered Cache, Akamai tiered distribution) routes all edge misses through one regional cache layer, and request collapsing turns simultaneous misses into one origin fetch, so the origin sees one request instead of hundreds. **Multi-CDN** spreads traffic over two or more providers for resilience and price: DNS-based steering (weighted, latency- or health-based), or player-side switching on errors (HLS and DASH both define *content steering*). The cost is keeping config in sync: cache keys, TLS certificates, purge APIs, logging, and especially token formats, since each CDN validates its own signatures; your app must issue a token per CDN.

7 Example: paid course videos

WORKED EXAMPLE: SERVING PAID COURSE VIDEOS WITH SIGNED COOKIES

The app runs at `example.com`, the CDN at `cdn.example.com`, the bucket is private with OAC. A student opens lesson 3; the API checks enrollment and sets three CloudFront cookies (`Domain=.example.com; Secure; HttpOnly`) signed over a custom policy: resource `https://cdn.example.com/courses/sd/*`, expiry now + 2 hours. The player loads `/courses/sd/l3/master.m3u8` and its segments; each request carries the cookies, the edge validates them, and the cache key is just the path, so the second student's requests are edge hits. Every hour the player calls `/refresh-cdn-cookies`, which re-checks enrollment, so a refund cuts access within an hour.

The forum link fails with 403 (no cookies); the raw bucket URL fails with 403 (only CloudFront may read). A determined user can still copy their own cookies for up to 2 hours; if that matters, add DRM (Widevine / FairPlay, with per-user licences and concurrent-stream limits) and forensic watermarking.

TRADE-OFFS

- Short expiry limits leaks but needs a refresh flow; long expiry is simple but leaks last longer.
- IP-restricted policies stop link sharing but break mobile users who change networks.
- Signed cookies need the CDN on your domain; signed URLs work anywhere but bloat manifests and leak through links.
- Multi-CDN improves resilience and pricing leverage at the cost of duplicated config and token logic.

COMMON MISTAKES

- Leaving the bucket readable, so the CDN's checks can be bypassed.
- Putting tokens, session cookies or tracking parameters in the cache key: near-zero hit ratio.
- Relying on purges for every deploy instead of versioned file names.
- Storing the CDN signing private key in the client or in a public repo.

8 Key terms

Origin Access Control: only the CloudFront distribution may read the bucket

Canned vs custom policy: one URL + expiry vs wildcards, start time, IP

Versioned URL: new content gets a new URL, so no purge is needed

Signed URL / signed cookie: CDN-validated proof of access with an expiry

Cache key: inputs that identify one cached object

Origin shield: extra cache tier that funnels edge misses to the origin

9 Interview questions

Q1. How do you serve private content via a CDN? BASIC

Keep the origin private (reachable only from the CDN) and have the CDN validate short-lived signed URLs or signed cookies that your app issues after checking access.

Q2. Signed URLs or signed cookies? BASIC

Signed URLs suit single files and clients without cookies; signed cookies cover many files at once, like video segments, without rewriting manifests.

Q3. Your CDN hit ratio for a course platform is 5%. What do you check? SCENARIO

The cache key: per-user tokens, session cookies or tracking parameters in it create one copy per user; key on path (plus real variant inputs) and validate tokens separately.

Q4. A file was published by mistake and must disappear now. What do you do? SCENARIO

Remove it at the origin, invalidate the path on every CDN, and revoke access tokens if needed; for routine updates use versioned URLs instead of purges.

Q5. A new video launches and the origin is hammered on release. Fix? SCENARIO

Add an origin shield / tiered cache with request collapsing, and optionally pre-warm popular segments, so each object is fetched from the origin once.

Q6. Why not use S3 pre-signed URLs for every video segment? TRICKY

Each viewer gets unique URLs, so the CDN can't share cached copies and every manifest must be rewritten per user; CDN-validated cookies keep URLs identical and cacheable.

Q7. What breaks when you add a second CDN? TRICKY

Token formats and signing keys, cache-key rules, purge APIs, TLS certificates and logging all differ per provider, so access control and invalidation must be implemented for each.

Next: SD-O-012 · Data Lifecycle backendroadmap.com

Data Lifecycle

decide once, as rules; let storage apply them

Watch the 4-min video

In one line: lifecycle rules, filtered by prefix, tag or size, move data to cheaper tiers by age and delete expired objects, old versions and abandoned multipart uploads automatically; retention policies and legal holds override them for data that must be kept, so classify data when it's written and test rules before trusting them.

1 The bill that only grows

Storage grows every month: half-finished uploads, exports from three years ago, logs, test files, and old versions kept by versioning. Nobody deletes anything because nobody is sure what's safe to delete, and a cleanup script written once is forgotten or fails silently. **Data lifecycle management** turns those decisions into declarative rules attached to the bucket, which the storage service applies every day, forever, at no compute cost to you.

2 Lifecycle rules

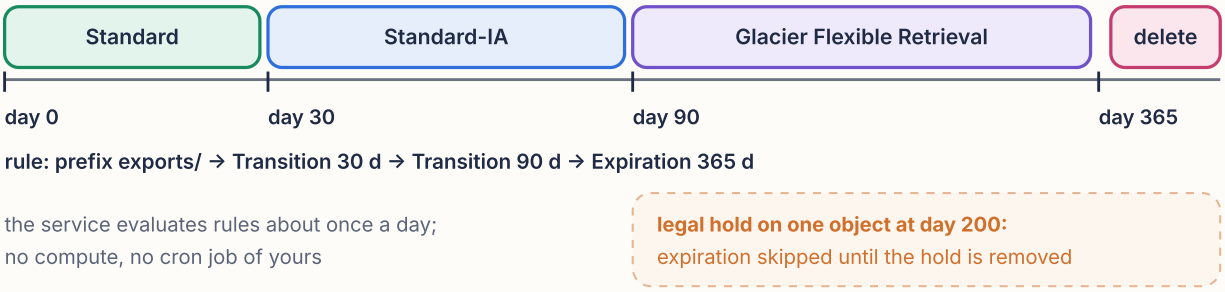


Figure 1. One rule, applied to every object under a prefix, by age. A legal hold on a single object overrides the expiration.

A rule = **filter** + **actions by age**.

- **Filters**: key prefix (`logs/`), object tags (`retention=90d`), object size (S3 `ObjectSizeGreaterThan`), or a combination. Azure filters on blob type, prefix (starting with the container name) and blob index tags.
- **Transition** to a cooler tier after N days (lesson 3). Transitions only go colder; going back needs a copy or rehydration.
- **Expiration / delete** after N days, or on a date.
- **Age**: S3 counts days from creation (rounded to the next midnight UTC); Azure can use modification time, creation time, last access time (with access tracking on) or time since the last tier change.

Execution is asynchronous: S3 evaluates rules about once a day and queues eligible objects, and you stop paying storage for an expired object once it is eligible, even if removal lags. A new or changed Azure policy can take up to 24 hours to take effect. S3 allows up to 1,000 rules per bucket. Constraints worth knowing: S3 won't transition to Standard-IA before an object is 30 days old, by default doesn't transition objects under 128 KB, and charges per transition request, which can make tiering millions of tiny objects a loss.

3 Old versions and incomplete uploads

Two rules alone often remove a surprising share of a bill:

- **Noncurrent versions**: with versioning on, every overwrite and delete keeps the previous version (lesson 3). A file rewritten daily keeps 365 versions a year. `NoncurrentVersionExpiration` (with `NoncurrentDays`, optionally keeping the newest N via `NewerNoncurrentVersions`) deletes them after a window; Azure has equivalent actions on `version` and `snapshot`.
- **Incomplete multipart uploads**: parts of abandoned uploads are invisible in listings but billed (lesson 6). `AbortIncompleteMultipartUpload` after a few days removes them. (Azure discards uncommitted blocks automatically after about a week.)
- **Expired delete markers**: after old versions are gone, a delete marker with nothing behind it is clutter that slows listings; `ExpiredObjectDeleteMarker` removes it.

4 Retention policies

Deletion is half the job; the other half is **not** deleting what must be kept. Financial records are typically kept for several years (the exact period depends on the law and country), audit logs as long as regulations require. A **retention policy** makes data undeletable and unmodifiable until a date, even by administrators, so a mistake, a rogue script or an attacker can't erase records early:

	Retention	Legal hold
Amazon S3 Object Lock	per version or bucket default; governance or compliance mode	per-version on/off flag, no expiry
Azure immutable storage	time-based policy on container or version; locked policies can only be extended	tag-based hold on container or version
Google Cloud Storage	bucket retention policy, lockable (Bucket Lock)	object holds: *temporary* and *event-based*

Retention that starts at an **event** rather than at creation ("keep 7 years after the account closes") needs either an event-based hold (GCS) or your application setting a retain-until date on the objects when the event happens.

5 Legal hold

When data is relevant to a lawsuit, audit or investigation, a **legal hold** freezes it with no end date, overriding every lifecycle rule and retention expiry until it is explicitly removed, typically by the legal team through a separately permissioned role (S3: `s3:PutObjectLegalHold`). Your rules don't need special cases: the platform skips held objects.

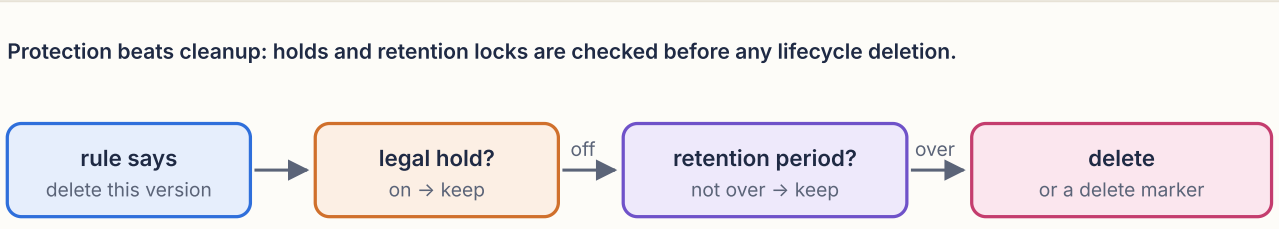


Figure 2. Order of precedence when a lifecycle rule wants to delete a version.

Watch the conflict with privacy law: a GDPR erasure request can't remove data under a legal retention obligation (Article 17(3)(b) allows keeping it), but you must then restrict its use and delete it when the obligation ends. Lifecycle rules are how that "when it ends" actually happens.

6 Know your data first

- **Classify at write time**: put data type and retention class in the key prefix (`temp/`, `exports/`, `records/`) or in tags (S3 allows 10 tags per object; Azure blob index tags are also queryable). Rules can only act on what they can match; classifying billions of existing objects later is expensive.
- **Measure**: S3 Storage Lens and Inventory, Storage Class Analysis, Azure blob inventory: find the biggest, oldest prefixes, noncurrent bytes and incomplete uploads.
- **Test safely**: a wrong delete rule is a very efficient way to lose data. Apply it to a test prefix first, use an inventory report to count what a rule would match, roll out transitions before deletions, keep versioning on with a noncurrent window as a safety net, and review rules like code (infrastructure as code, peer review).

7 Example: delete temp uploads after 7 days

```
{ "Rules": [
  { "ID": "expire-temp",  "Status": "Enabled", "Filter": { "Prefix": "temp/" },
    "Expiration": { "Days": 7 } },
  { "ID": "abort-mpu",    "Status": "Enabled", "Filter": {},
    "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 3 } },
  { "ID": "old-versions", "Status": "Enabled", "Filter": {},
    "NoncurrentVersionExpiration": { "NoncurrentDays": 30 },
    "Expiration": { "ExpiredObjectDeleteMarker": true } } ] }
```

Figure 3. S3 lifecycle configuration for the example. `Filter: {}` applies a rule to the whole bucket.

WORKED EXAMPLE: TEMP UPLOADS, ABANDONED UPLOADS, OLD VERSIONS

Uploads land under `temp/`; when processing succeeds the app copies the file to `media/` and deletes the temp copy (lessons 4, 9). Anything still in `temp/` after 7 days is an abandoned or failed upload. Rule 1 expires `temp/` objects after 7 days. Rule 2 aborts multipart uploads not completed within 3 days. Because the bucket is versioned, rule 1 only adds delete markers, so rule 3 removes noncurrent versions after 30 days (a month to recover from mistakes) and cleans up the leftover markers. If 2,000 uploads a day are abandoned at 50 MB each, that's 100 GB a day, ~36 TB a year that would otherwise pile up; with the rules, temp data stays bounded at about a week of abandoned uploads (plus the 30-day version window). No cleanup job to maintain.

TRADE-OFFS

- Aggressive expiry saves money but risks deleting data someone needed; retention windows and versioning buy recovery time at a cost.
- Prefix-based rules are simple; tag-based rules are flexible but need tags written reliably at upload.
- Locked retention gives strong compliance guarantees and removes your own ability to fix mistakes.
- Daily asynchronous execution is free and hands-off, but nothing happens at an exact time.

COMMON MISTAKES

- Versioning on, no noncurrent-version rule: storage silently doubles.
- No rule for incomplete multipart uploads.
- Expecting an expiration rule to free space in a versioned bucket (it only adds delete markers).
- Shipping a broad delete rule without testing its filter on real inventory.

8 Key terms

Lifecycle rule: filter + age-based transition or deletion actions

Delete marker: placeholder that hides an object in a versioned bucket

Legal hold: open-ended freeze that overrides everything until removed

Noncurrent version: an older version kept by versioning

Retention policy: no delete or overwrite until a date (WORM)

Storage inventory: scheduled report listing objects and their properties

9 Interview questions

Q1. How do you automate storage cost control? BASIC

Lifecycle rules filtered by prefix or tag: move data to cheaper tiers by age, and delete expired data, old versions and incomplete multipart uploads.

Q2. What can override lifecycle rules? BASIC

Retention policies and legal holds, which protect data that must be kept, whatever its age; the platform skips locked or held versions.

Q3. Storage doubled after enabling versioning. Why, and what do you add? SCENARIO

Every overwrite and delete keeps old versions; add a NoncurrentVersionExpiration rule (e.g. 30 days, keep the newest few) and clean up expired delete markers.

Q4. Records must be kept 7 years after a customer closes their account. How? SCENARIO

Retention starting at an event: an event-based hold (GCS) or the app setting a retain-until date when the account closes, plus a rule that deletes once retention ends.

Q5. How would you introduce a delete rule on a 2 PB bucket safely? SCENARIO

Use inventory to measure exactly what the filter matches, apply it to a test prefix, start with transitions, keep versioning with a noncurrent window, and review the rule as code.

Q6. An expiration rule ran but the bucket size didn't drop. Why? TRICKY

The bucket is versioned: expiration added delete markers and turned current versions into noncurrent ones; only a noncurrent-version rule removes the bytes.

Q7. A user exercises the GDPR right to erasure, but the data is under a 7-year legal retention. What happens? TRICKY

The legal obligation takes precedence (GDPR Art. 17(3)(b)); restrict processing now and delete when retention ends, which the lifecycle rules enforce.

Backup and Archival

restore the system, keep the record

Watch the 4-min video

In one line: backups restore a working system quickly after failure; archives keep rarely needed records for years. Design archives with a searchable catalog, an archival tier matched to how fast you'll need data back, WORM retention, copies in another region *and* account, deletion when retention ends, and restore tests.

1 The missing invoice

An auditor asks for an invoice from six years ago. Old rows were deleted from the production database long ago to keep it fast. The backup system keeps 90 days. The document is gone, and the fine is real. The team had backups, but no **archive**: the two sound similar, solve different problems, and are designed differently.

2 Backup vs archive

Backup: restore a working system
copies of live data, taken often (minutes to daily)
kept weeks to months, rotated
measured by RPO / RTO: restore fast
e.g. DB snapshots + PITR, volume snapshots

Archive: keep the record
data moved out of live systems: rarely read
kept years, often by law, then deleted
measured by findability + integrity
e.g. invoices, contracts, audit logs, old media

a 90-day backup is not an archive, and a 7-year archive won't get your database back by lunchtime

Figure 1. Same storage technology, different goals, retention and success measures.

- **RPO** (recovery point objective): how much data you can afford to lose, which sets backup frequency. Continuous WAL/binlog archiving gives point-in-time recovery to within seconds; nightly snapshots give up to 24 hours of loss.
- **RTO** (recovery time objective): how long a restore may take. It decides where backups live: a multi-terabyte restore from a deep archive tier can't meet a 1-hour RTO.
- **3-2-1**: at least 3 copies, on 2 different media or services, 1 off-site. The modern **3-2-1-1-0** adds 1 immutable or offline copy and 0 errors in restore verification.
- **Replication is not a backup**: it faithfully copies deletes and corruption. Backups are point-in-time copies you can go back to.
- Backup retention usually rotates (e.g. 7 daily, 4 weekly, 12 monthly); archive retention is driven by law or policy and is per record.

3 Archival tiers and retrieval times

For Glacier Flexible and Deep Archive, reading is a two-step process: `RestoreObject` creates a **temporary copy** for the number of days you ask for, and you `GET` that copy. Every retrieval is billed per GB and per request, bulk being cheapest. These two classes also add about **40 KB of overhead per object** (8 KB billed at Standard for the name and metadata, 32 KB at the archive rate), so millions of tiny

files should be bundled (e.g. one tar per day) before archiving, and the bundle's contents indexed in the catalog.

Tier	Retrieval	Minimum duration	≈ $/GB-month*
S3 Glacier Instant Retrieval	milliseconds (no restore step)	90 days	0.004
S3 Glacier Flexible Retrieval	expedited 1–5 min · standard 3–5 h · bulk 5–12 h	90 days	0.0036
S3 Glacier Deep Archive	standard within 12 h · bulk within 48 h	180 days	0.00099
Azure Blob Archive	rehydrate: standard up to 15 h · high priority often < 1 h (< 10 GB)	180 days	≈ 0.001
GCS Archive	milliseconds, high retrieval fee	365 days	≈ 0.0012

*Approximate list prices in a major US region; retrieval and request fees come on top. Check current pricing.

4 Match the tier to how you'll retrieve

Choose by the retrieval promise, not by importance. Records an auditor may want within a day can sit in Glacier Flexible (standard restore in hours) or Deep Archive (within 12 h). Records you might need in seconds (customer self-service downloads of old statements) belong in Glacier Instant Retrieval or a cool tier. Records you'll almost certainly never read but must keep go to the deepest tier. And always build a small, searchable **catalog** (a database table) of what's archived: record ID, business keys (customer, date, type), object key, checksum, retention date. Without it, finding six invoices means restoring and opening millions of objects. The catalog itself needs backups.

Store archives in open, long-lived formats (PDF/A, CSV, Parquet, JSON) with their schema, so they're readable in ten years without today's software.

5 Cross-region, cross-account copies

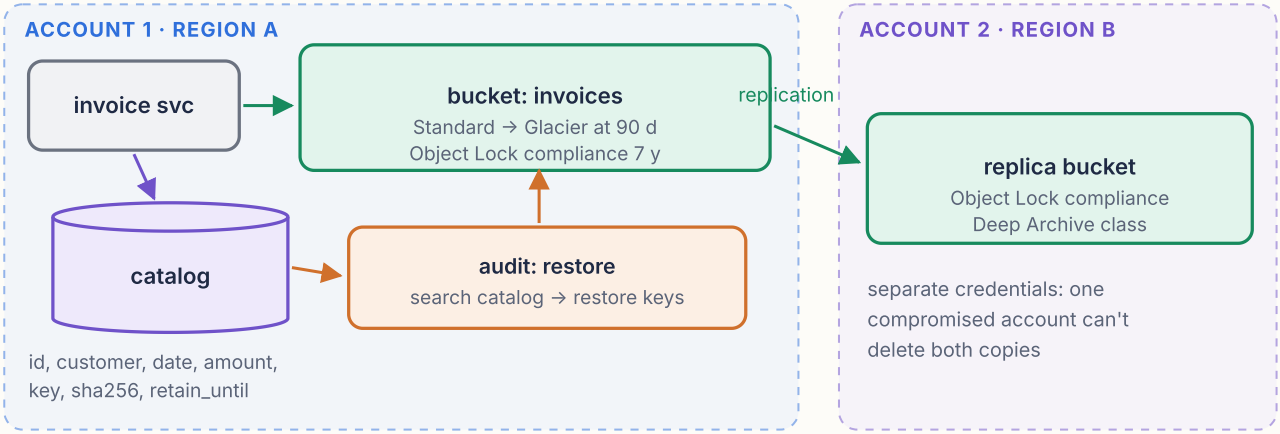


Figure 2. The 7-year archive: locked primary copy plus a locked replica in another region and another account, and a catalog to find records.

One region is one failure domain; one account is one blast radius (a leaked admin key, a malicious insider, a bad automation run). S3 **Cross-Region Replication** copies new objects asynchronously to a bucket in another region; both buckets need versioning, existing objects need S3 Batch Replication, **Replication**

Time Control adds an SLA of 99.99% of objects within 15 minutes, and the replica can land directly in a Glacier class. Replicating into **another account** with ownership transferred to it means credentials for account 1 can't delete the replica. By default S3 replication does not replicate deletions of specific versions, so a malicious version delete in the source doesn't propagate. Azure offers geo-redundant storage (GRS/RA-GRS) and object replication; AWS Backup and Azure Backup can copy backups across regions and accounts.

6 Tamper-proof, and tested

- **WORM**: S3 Object Lock in **compliance** mode (nobody, including root, can shorten retention or delete a locked version), Azure immutable storage with a locked time-based policy, AWS Backup Vault Lock, Azure Backup immutable vaults. This protects records from mistakes, insiders and ransomware, which now routinely tries to delete backups first. Enable it on the replica too.
- **Integrity**: store checksums in the catalog and verify them on restore; periodically restore a random sample and re-hash it.
- **Restore tests**: a backup nobody has restored is only a hope. Schedule restore drills (AWS Backup has a restore-testing feature), measure the real RTO, and test the runbook, permissions and people, not just the bytes.
- **Deletion**: when retention ends, a lifecycle rule deletes the record (lesson 12), unless a legal hold says otherwise. Keeping data longer than required is a liability and a cost.

7 Example: a 7-year archive of financial documents

WORKED EXAMPLE: INVOICES KEPT FOR 7 YEARS

Each invoice PDF (~200 KB) is written to `s3://invoices/2026/11/13/INV-88412.pdf` with Object Lock **compliance** retention until creation + 7 years, and a catalog row (customer, date, amount, key, SHA-256, retain_until). After 90 days a lifecycle rule moves it to Glacier Flexible Retrieval; replication sends a copy to region B, account 2, stored in Deep Archive with the same lock. At 1 million invoices a year, the archive reaches 7 million objects and ~1.4 TB: roughly $5 a month in Glacier Flexible, plus about 280 GB of per-object overhead (7 M × 40 KB), a 20% surcharge worth knowing about. At this size the tier hardly matters in dollars; the retrieval promise decides.

An auditor asks for ACME's 2020 invoices: the catalog returns 42 keys, the API issues 42 standard restores, and 3–5 hours later the PDFs are delivered with verified checksums, without restoring anything else. After 7 years the lock expires and a lifecycle rule deletes the record from both copies.

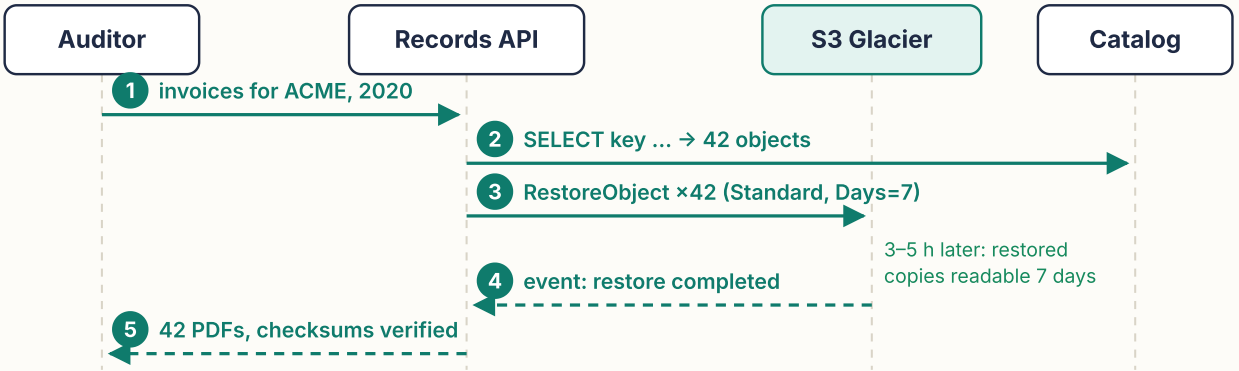


Figure 3. An audit request: query the catalog, restore only the matching objects, verify checksums, deliver.

TRADE-OFFS

- Deeper tiers cost less to keep and more, and longer, to read; minimum durations penalise early deletes.
- Compliance-mode WORM is the strongest protection and is irreversible: a wrong retention period is paid for years.
- Cross-account replication limits blast radius but doubles storage and adds key and permission management.
- Bundling small files cuts per-object overhead but makes single-record retrieval coarser.

COMMON MISTAKES

- Treating backups as the archive (90-day rotation deletes the records you must keep).
- Archiving without a catalog, so finding one record means restoring everything.
- Keeping every copy in one account, where one compromised credential can delete them all.
- Never testing a restore, then discovering missing permissions or a 2-day retrieval during an incident.

8 Key terms

RPO / RTO: acceptable data loss / acceptable restore time

Restore / rehydrate: making an archived object readable again, for a fee

Cross-region replication: asynchronous copy of objects to another region

Archive tier: cheap, slow-to-read storage (Glacier, Azure Archive)

3-2-1(-1-0): 3 copies, 2 media, 1 off-site (+1 immutable, 0 errors)

Compliance-mode Object Lock: WORM retention no one can shorten or remove

9 Interview questions

Q1. What's the difference between a backup and an archive? BASIC

A backup restores a working system after a failure, so it's frequent, short-lived and fast to restore; an archive keeps rarely used records for the long term, findable and intact.

Q2. What are RPO and RTO? BASIC

RPO is how much recent data you can lose (sets backup frequency); RTO is how long recovery may take (sets where and how backups are stored).

Q3. Design an archival strategy for compliance documents. SCENARIO

Object storage plus a searchable catalog, lifecycle moves to an archival tier by age, WORM retention (Object Lock compliance mode), replication to another region and account, restore tests, and deletion when retention ends.

Q4. Ransomware encrypted production and tried to delete backups. What design makes recovery possible? SCENARIO

Immutable backups (Vault Lock / Object Lock / immutable vaults) in a separate account with separate credentials, plus regularly tested restores within the RTO.

Q5. You must archive 500 million small log files. What changes? SCENARIO

Bundle them into large compressed files before archiving to avoid per-object overhead and request costs, and index file-to-bundle locations in a catalog.

Q6. Why isn't cross-region replication alone a backup? TRICKY

It replicates deletes and corruption too, and has no history to go back to; you need versioning, point-in-time copies and immutability.

Q7. A team wants their 5 TB database backups in Deep Archive to save money. Problem? TRICKY

Restores take up to 12–48 hours plus retrieval costs, which breaks any normal RTO; keep recent backups in a fast tier and only age older ones down.

Next: SD-P-001 · Full-Text Search

Full-Text Search

analyse the text once, then search terms not strings

Watch the 4-min video

> **In one line:** full-text search runs every document and every query through the same **analyzer** (tokenize, then normalise: lowercase, fold accents, stem, maybe synonyms), stores the resulting **terms** in an inverted index, and matches terms instead of raw strings, so it is fast, forgiving about word forms, and can rank by relevance.

1 Why LIKE stops working

A shop's search box starts as `SELECT * FROM products WHERE name LIKE '%running shoes%'`. With a hundred products it is fine. With 10 million, it has three separate problems:

- **Speed.** A B-tree index is sorted by the column's first characters, so it can serve `LIKE 'run%'` but not a **leading wildcard**. `'%shoes%'` becomes a full table scan on every keystroke-driven query.
- **Recall.** It compares characters. 'Running Shoes' does not contain 'running shoe', 'Runner's shoe' or 'trainers'. Case is often handled by collation, but plurals, word order and synonyms are not.
- **Ranking.** A row either matches or not. A product whose title is 'Running Shoes' ranks the same as one that mentions shoes once in a 2,000-word description.

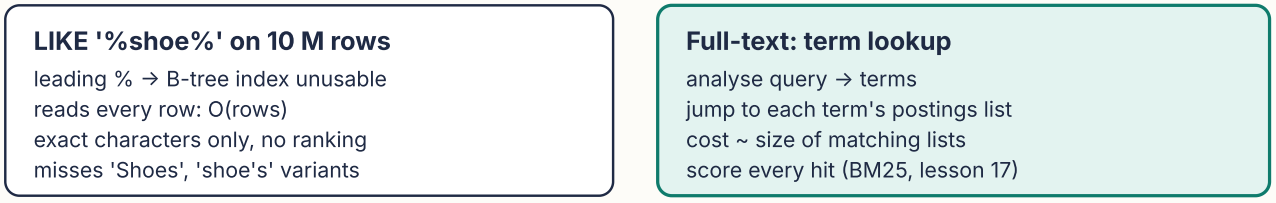


Figure 1. LIKE does work proportional to the table; a full-text engine does work proportional to the matching postings lists.

Databases have partial fixes. PostgreSQL's `pg_trgm` extension builds a GIN index of three-letter fragments that *can* speed up `LIKE '%…%'`, and its built-in full-text search (`to_tsvector('english', name)` plus a GIN index, ranked with `ts_rank`) is enough for many apps. A dedicated engine (Elasticsearch, OpenSearch, Solr, all built on Apache Lucene) adds per-field analyzers, BM25 ranking, facets and horizontal scaling.

2 Analysis: tokenizer plus token filters

An **analyzer** is a pipeline with three stages. **Character filters** clean the raw string (strip HTML, map '&' to 'and'). A **tokenizer** splits it into tokens and records each token's **position**, which phrase queries need later (lesson 15). **Token filters** then change, remove or add tokens. Elasticsearch's default `standard` analyzer is just the Unicode word tokenizer plus lowercase; the built-in `english` analyzer adds a possessive filter ('men's' → 'men'), English stop words and the Porter stemmer.

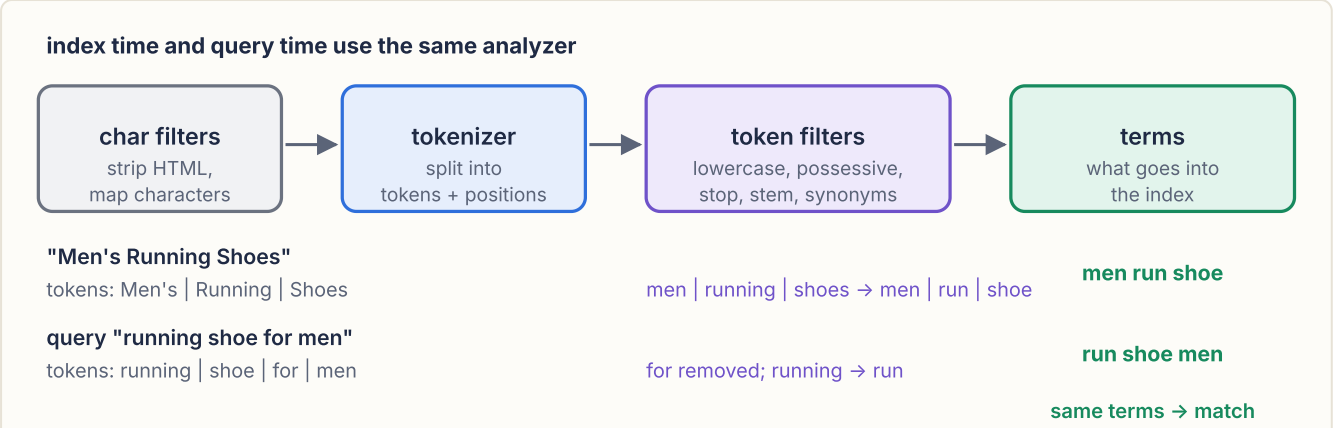


Figure 2. The analyzer runs at index time on the document and at query time on the search text. Only the final terms are compared.

Step	What it does	Example	Watch out for
Lowercase	case-insensitive match	Shoes → shoes	almost always wanted on text fields
ASCII / accent folding	removes diacritics	café → cafe	can merge words that differ in meaning in some languages
Stop words	drops very common words	'shoes for men' → shoes, men	breaks phrases like 'to be or not to be'; BM25 already down-weights them
Stemming (algorithmic)	chops suffixes by rules	running → run, shoes → shoe	Porter leaves 'runner' and 'men' unchanged; can over-merge (university/universe → univers)
Lemmatising / dictionary stemmer	maps to the dictionary form	men → man, ran → run	slower, needs a dictionary (e.g. Hunspell)
Synonyms	adds equivalent terms	sneakers ⇄ trainers	apply at query time so you can change them without reindexing

3 The golden rule: analyse both sides the same way

The index only contains terms. If documents are stemmed but queries are not, the query term 'running' looks for a term that was never stored, because the document's 'running' became 'run'. So each `text` field has one analyzer used at index time and, by default, the same one at search time. The deliberate exceptions are documented ones: **synonyms** are usually added only at search time, and **edge n-grams** only at index time (lesson 21).

Exact values are a different job. Product codes, email addresses, status flags and tags should be `keyword` fields: stored as one un-analysed term, used for filters, sorting and facets (lesson 18). A common pattern is a **multi-field**: `name` as analysed text for search, `name.raw` as keyword for sorting.

4 Forgiving matches: n-grams, fuzziness and synonyms

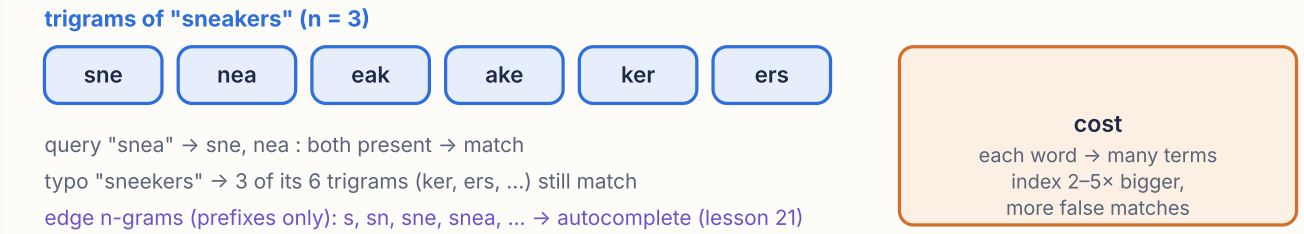


Figure 3. N-grams index fragments so partial words and small typos still share terms. The price is a much bigger index.

- **N-grams** split words into overlapping fragments. They catch partial words and typos but multiply the number of terms, so use them on short fields (names, titles), not on long descriptions.
- **Fuzzy queries** are the cheaper alternative for typos: the engine expands the query term to indexed terms within a small **edit distance** (Levenshtein, max 2 in Lucene). Elasticsearch's `fuzziness: AUTO` allows 0 edits for 1–2 character terms, 1 for 3–5 and 2 for longer ones.
- **Synonyms** fix vocabulary gaps that stemming can't: sneakers/trainers, tv/television, 'nyc'/'new york'. Multi-word synonyms need the graph-aware `synonym_graph` filter at search time.
- **Per-language analyzers.** Stemming rules are language-specific; CJK text has no spaces and needs its own tokenizers (e.g. bigrams or a morphological analyzer like kuromoji for Japanese).

WORKED EXAMPLE: E-COMMERCE PRODUCT SEARCH

Products stay in the database; name and description are copied into a search index whose `name` field uses the `english` analyzer plus a search-time synonym list (`trainers, sneakers, running shoes`). The shopper types **'Running Shoes for Men'**. The query becomes the terms **[run, shoe, men]**: 'for' is a stop word, 'running' and 'shoes' are stemmed. (The video writes 'man'; the Porter stemmer keeps 'men', and you would need a lemmatiser to get 'man'.)

'Men's Runner Shoe' is indexed as [men, runner, shoe], so it matches two of three terms; Porter does not reduce 'runner' to 'run', which is why a `runner → run` synonym or a lemmatiser helps. 'Running Trainers' is indexed as [run, trainer] and matches 'run' directly and 'trainer' through the synonym expansion of 'running shoes'. Both come back in milliseconds, ranked by how many terms matched and where (lesson 17), while a `LIKE` query would have returned neither.

TRADE-OFFS

- **Aggressive stemming** raises recall but lowers precision (Porter turns 'organization' into 'organ').
- **N-grams** catch partial words and typos but bloat the index and add noisy matches.
- **Stop-word removal** saves little space today and breaks phrase queries; many teams keep stop words.
- **A separate engine** gives better search, but it is a second system to keep in sync (lessons 19–20).

COMMON MISTAKES

- Different analyzers at index and query time by accident, so obvious matches never happen.
- Mapping IDs, SKUs or emails as analysed text: 'XB-2017' is split into 'xb' and '2017' and matches too much.
- Changing an analyzer on a live index: existing terms don't change, so you must reindex (lesson 19).
- Never testing the analyzer: use the `_analyze` API to see exactly which terms a string produces.

5 Key terms

Analyzer: char filters + tokenizer + token filters, run on text

Token / term: a unit after tokenizing / after all filters

Stemming: rule-based suffix stripping: running → run

Lemmatisation: dictionary-based base form: men → man

N-gram: overlapping character fragments of a word

keyword field: un-analysed exact value for filters and sorting

6 Interview questions

Q1. Why not use SQL LIKE for search? BASIC

A leading wildcard can't use a B-tree index, so it scans every row; it matches exact characters, missing plurals and variants; and it can't rank results by relevance.

Q2. What does an analyzer do? BASIC

It tokenizes text and normalises the tokens (lowercase, accent folding, stemming, stop words, synonyms), and the same analysis is applied to documents and queries so their terms line up.

Q3. Users search 'iphone15' and find nothing, though 'iPhone 15' exists. What do you change? SCENARIO

Inspect both with the _analyze API; add a word-delimiter or n-gram sub-field (or a synonym) so 'iphone15' and 'iphone 15' produce shared terms, then reindex.

Q4. You need typo tolerance on product names. N-grams or fuzzy queries? SCENARIO

Fuzzy queries first: no index growth, fuzziness AUTO handles 1–2 edits. Use n-grams when you also need partial-word matching, and only on short fields.

Q5. Marketing wants to add synonyms every week. How do you avoid reindexing? SCENARIO

Apply synonyms in a search-time analyzer (synonym_graph) and reload it; index-time synonyms are baked into the terms and need a reindex.

Q6. Why might stemming make results worse? TRICKY

It merges words with different meanings ('universe'/'university' both stem to 'univers'), lowering precision; use a lighter stemmer or keep an unstemmed sub-field and boost exact matches.

Q7. Isn't PostgreSQL full-text search enough? TRICKY

Often yes for one database and modest scale: tsvector + GIN with ts_rank. A dedicated engine wins when you need custom analyzers per field, BM25 tuning, facets at scale or to scale search separately from the database.

Next: SD-P-002 · Inverted Indexes backendroadmap.com

Inverted Indexes

map every word to the documents that contain it

Watch the 4-min video

In one line: an inverted index maps each **term** to a sorted **postings list** of the documents (and positions) where it appears, so a query becomes a few dictionary lookups plus list intersections and unions; the index is compressed and written as **immutable segments** that are merged in the background.

1 Never read documents at query time

You have 10 million documents and someone searches **distributed cache**. Scanning every document's text, even at a few GB/s, takes seconds to minutes. Search engines answer in milliseconds because the expensive work happened at index time: they read a precomputed **inverted index**, which is the same idea as the index at the back of a book.

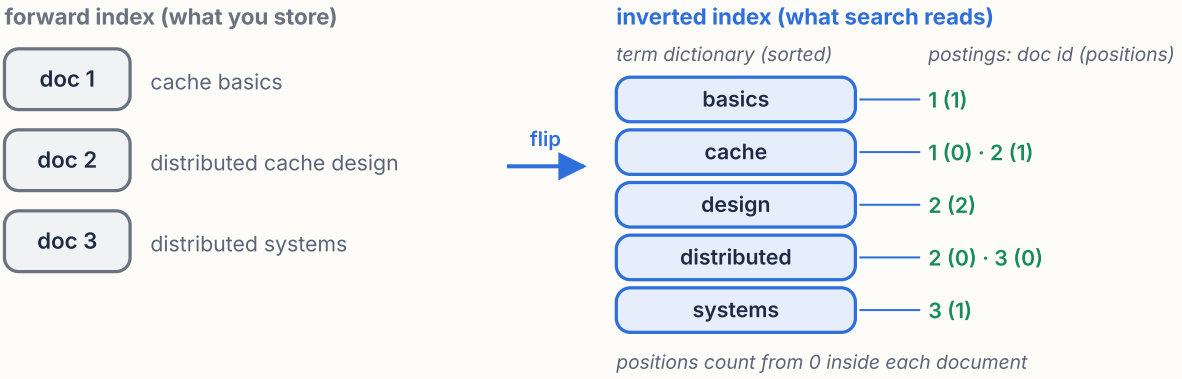


Figure 1. The three documents from the video. A forward index maps doc → words; the inverted index maps term → postings.

The index has two parts. The **term dictionary** holds every distinct term, sorted, so lookups and prefix scans are fast (Lucene stores it as blocks on disk with a small in-memory index built as an FST, a compact automaton over the term bytes). Each term points to its **postings**: the doc IDs that contain it, usually with the **term frequency** (for scoring, lesson 17) and the **positions** (for phrases). Doc IDs are small integers assigned per segment, not your database IDs.

2 Queries are list operations

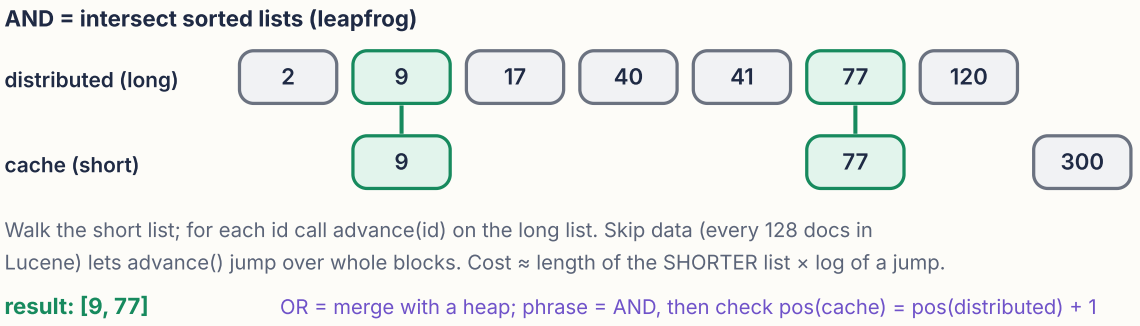


Figure 2. Intersection walks the shorter list and jumps ahead in the longer one, so rare terms make AND queries cheap.

- **AND**: intersect the postings lists. Start with the rarest term; because lists are sorted by doc ID, the engine leapfrogs: ask the next list to `advance()` to the current candidate, using skip data to jump over blocks.
- **OR**: union with a priority queue. For top-k results, Lucene's **block-max WAND** skips blocks whose best possible score can't reach the current top k. This is why Elasticsearch counts hits exactly only up to 10,000 by default (`track_total_hits`): an exact count would force it to visit every match.
- **NOT**: subtract one list from another.
- **Phrase** 'distributed cache': intersect first, then check positions: cache must be at position p + 1 where distributed is at p. **Proximity** ('within 5 words', `slop`) relaxes that check.
- **Prefix / wildcard / fuzzy**: first expand the pattern against the sorted term dictionary into a set of terms, then OR their postings. A pattern that expands to thousands of terms is slow, which is why leading wildcards are discouraged even here.

3 Compression: why indexes fit in memory

Postings are stored as **gaps** between consecutive doc IDs instead of the IDs themselves. Sorted IDs make the gaps small, and small numbers need few bits: after the first, these gaps fit in 6 bits each instead of 32.

```
IDs 1000, 1003, 1010, 1011, 1050 → gaps 1000, 3, 7, 1, 39
```

Lucene packs gaps in blocks of 128 using the bit width of the largest gap in the block (frame-of-reference bit-packing), with variable-byte encoding for the tail. Common terms have dense lists and tiny gaps, so they compress best. Compressed lists mean fewer bytes from disk and more of the index in the OS page cache; decoding a packed block is very fast on modern CPUs, often faster than reading the uncompressed data.

4 Updates: immutable segments plus merging

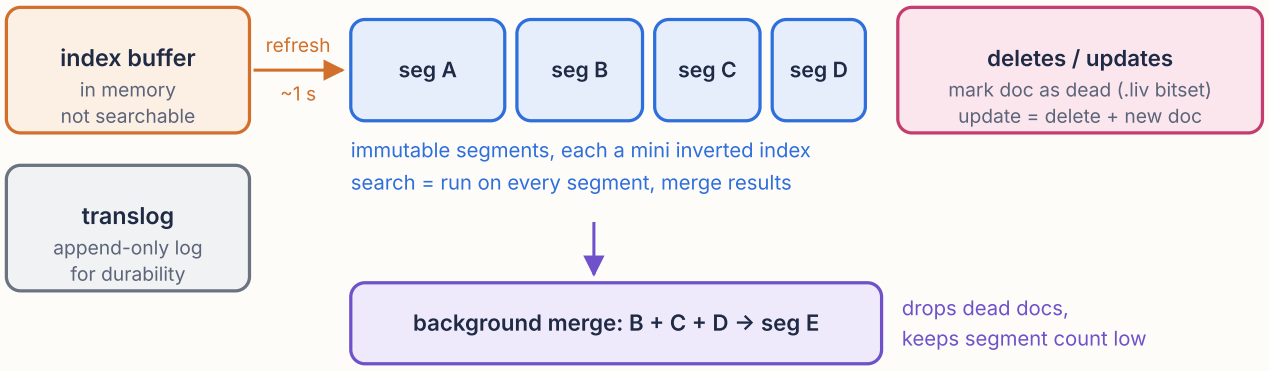


Figure 3. Lucene's write path. New documents become new segments on refresh; deletes are marks; merges rewrite segments in the background.

Rewriting a sorted, compressed postings list for every new document would be very slow, so Lucene never edits a segment in place. New documents collect in an in-memory buffer; a **refresh** (every 1 s by default in Elasticsearch, lesson 20) turns the buffer into a new small, searchable segment. A **delete** just sets a bit in the segment's live-docs bitset; an **update** is a delete plus a new document. Durability comes from a separate **translog** that is replayed after a crash; a **flush** does a full Lucene commit (fsync) and trims the translog.

Many small segments slow every query, because each query visits each segment. A **merge policy** (Lucene's TieredMergePolicy by default) merges similar-sized segments into bigger ones, physically removing deleted documents. Merges cost disk I/O and temporary disk space, which is why heavily-updated indexes need headroom, and why 'force merge to one segment' is only for indexes that will never be written again (e.g. yesterday's logs).

WORKED EXAMPLE: THREE DOCUMENTS

Doc 1 'cache basics', doc 2 'distributed cache design', doc 3 'distributed systems'. After analysis the dictionary is basics, cache, design, distributed, systems. `cache → [1, 2]`, `distributed → [2, 3]`. The query **distributed AND cache** intersects [2, 3] with [1, 2]: the leapfrog starts at 2 in both lists → match; next 3 vs end → done. Result: **doc 2**. For the phrase "distributed cache", doc 2 has distributed at position 0 and cache at position 1, so it also matches as a phrase. No document text was read. Now delete doc 2: its bit in the live-docs set flips to 0, the postings stay until the next merge, and every query simply skips it.

Stored per term or doc	Used for	Cost if you don't need it
Doc IDs	matching	always needed
Term frequencies	BM25 scoring	small; disable only for pure filter fields
Positions	phrase and proximity queries	often the biggest part of the index; keyword fields skip them
Offsets	fast highlighting	extra space; off by default
Doc values (columnar)	sorting, facets (lesson 18)	a separate structure, not part of the inverted index

TRADE-OFFS

- **Read speed vs write cost:** everything is precomputed, so writes do analysis, buffering and later merging.
- **Positions** enable phrases but can double the index size.
- **Immutability** makes caching and concurrency simple but means deletes cost space until merged.
- **Frequent refresh** means fresh results but many tiny segments and more merging.

COMMON MISTAKES

- Thinking an update modifies a document in place: it rewrites the whole document and leaves a deleted copy behind.
- Updating one hot document (a counter) thousands of times a second: each update creates a deleted doc and churns segments.
- Force-merging an index that is still being written to: it creates huge segments that the normal merge policy then avoids.
- Treating hit counts as exact by default: past 10,000 Elasticsearch reports 'gte 10000' unless you ask for exact totals.

5 Key terms

Term dictionary: sorted set of all terms, points to postings

Skip data: pointers that let intersection jump ahead in a list

Segment: immutable mini-index; a shard is many segments

Postings list: sorted doc IDs (+ freq, positions) for one term

Delta / gap encoding: store differences between sorted IDs

Merge: combine segments, purge deleted docs

6 Interview questions

Q1. How does an inverted index work? BASIC

It maps every term to a sorted postings list of the documents containing it, with positions for phrases; a query looks up each term and intersects or unions the lists.

Q2. How do engines keep indexes fast to update? BASIC

They write new immutable segments, mark deletions in a bitset instead of rewriting, and merge segments in the background.

Q3. How is a phrase query answered? BASIC

Intersect the terms' postings, then use stored positions to keep only documents where the terms appear at consecutive positions.

Q4. An AND query with one very common and one very rare term: which list drives? SCENARIO

The rare term's short list: iterate it and advance into the long list with skip data, so the cost tracks the short list.

Q5. Index size doubled after enabling a feature on a text field. What likely happened? SCENARIO

Positions or offsets (or n-grams) were enabled on a large field; check whether you really need phrase queries or fast highlighting on it.

Q6. Why are deletes 'cheap' but not free? TRICKY

A delete only flips a bit, but the document still occupies space and is still visited and skipped by queries until a merge rewrites the segment.

Q7. Why does compression make search faster, not slower? TRICKY

Gap-encoded, bit-packed postings are far smaller, so more fits in RAM and CPU caches, and decoding packed blocks is cheaper than reading extra bytes from memory or disk.

Q8. Why is a wildcard like *cache slow even in a search engine? TRICKY

The term dictionary is sorted by prefix; a leading wildcard must scan every term to find matches and then OR many postings lists. Index a reversed or n-gram sub-field instead.

Next: SD-P-003 · Elasticsearch Architecture backendroadmap.com

Elasticsearch Architecture

shards for size, replicas for safety and reads

Watch the 4-min video

In one line: an Elasticsearch/OpenSearch **cluster** stores each **index** as several **shards** (each a full Lucene index) spread over nodes, keeps **replica** copies for failover and read throughput, makes new documents searchable after a **refresh**, and stays healthy only with explicit mappings and sensibly sized shards.

1 The one-node trap

The team ships logs to Elasticsearch on one node with one big index. A year later a single shard holds 2 TB, searches time out, recovery after a restart takes hours, and losing the node means losing search. Elasticsearch and OpenSearch (a fork of Elasticsearch 7.10 made in 2021; same core architecture) scale horizontally, but only if indices, shards and nodes are planned.

2 Cluster, index, shard, replica

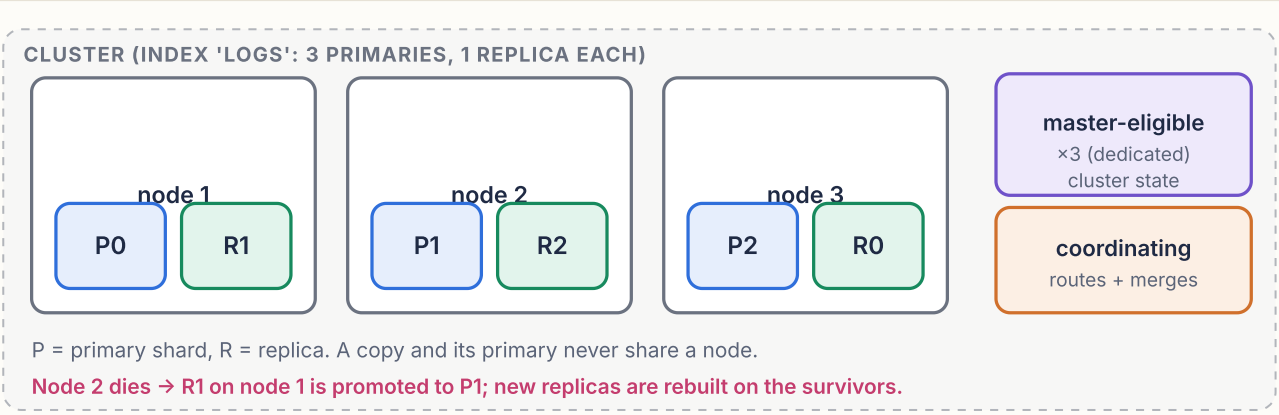


Figure 1. One index split into three primary shards, each with one replica on a different node.

- **Index**: a logical collection of documents with one mapping, roughly a table.
- **Shard**: one partition of an index and a complete Lucene index (lesson 15). A document goes to shard `hash(_routing) % number_of_primary_shards`, where `_routing` defaults to the document `_id`. Because the formula uses the primary count, that count is fixed at creation; changing it means `_split` / `_shrink` (by factors only) or a reindex.
- **Replica**: a full copy of a shard on another node. It gives failover (a replica is promoted if the primary's node dies) and read capacity (searches can use any copy). Replicas can be changed at any time. A single-node cluster can't place replicas, so health stays **yellow**.
- **Health**: green = all copies assigned; yellow = all primaries but some replicas unassigned; red = some primary unassigned, so some data is unavailable.

3 Write path and near-real-time search

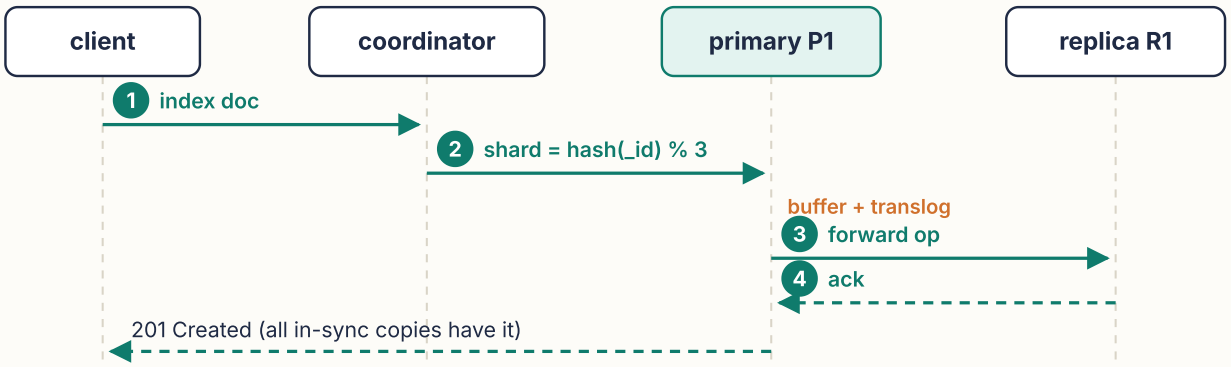


Figure 2. A write is routed to its primary, made durable in the translog, copied to every in-sync replica, then acknowledged.

The primary validates and indexes the document into its in-memory buffer and appends it to the **translog** (fsynced per request by default, so an acknowledged write survives a crash). It forwards the operation to the in-sync replicas in parallel and acknowledges when they have it. The document is **not yet searchable**: a **refresh** writes the buffer into a new segment, every `1s` by default, which is why search is called near real time (lesson 20). Elasticsearch also skips scheduled refreshes on shards that have not been searched for 30 s ('search idle'), so the first search after a quiet period may wait for one. For bulk loads, `refresh_interval: 30s` (or `-1` during a one-off load) makes indexing much cheaper: fewer, bigger segments and less merging.

4 Mappings: the schema you must own

A mapping says how each field is indexed. `text` is analysed for full-text search (lesson 14); `keyword` is one exact term, used for filters, sorting and aggregations (lesson 18); plus numbers, `date`, `ip`, `geo_point`, `dense_vector` (lesson 22). **Dynamic mapping** guesses types from the first document it sees, and by default maps strings as text with a keyword sub-field. In production that causes **mapping explosion**: logs that put user IDs or request headers into field names create thousands of fields, each costing cluster-state memory. The default `index.mapping.total_fields.limit` is 1,000. Set `dynamic: strict` (or `false`), and use the `flattened` type for arbitrary key-value bags. You can add new fields later, but changing an existing field's type or analyzer needs a reindex (lesson 19).

```
PUT _index_template/logs
{ "index_patterns": ["logs-*"],
  "template": { "settings": { "number_of_shards": 4, "number_of_replicas": 1,
      "refresh_interval": "30s", "index.lifecycle.name": "logs-30d" },
    "mappings": { "dynamic": "strict", "properties": {
      "@timestamp": { "type": "date" },
      "service":    { "type": "keyword" },
      "message":    { "type": "text" },
      "status":     { "type": "short" } } } } }
```

Figure 3. An index template: explicit mappings, primary/replica counts, a slower refresh for log ingestion and an ILM policy.

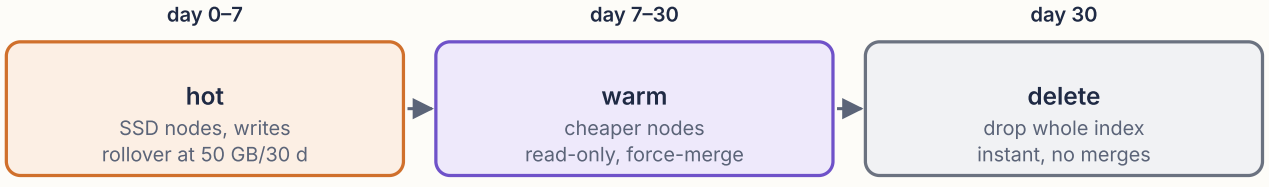
5 Sizing shards and nodes

Decision	Guideline	Why
Shard size	about 10–50 GB (Elastic's guidance); under ~200 M docs	bigger shards recover and rebalance slowly; tiny ones waste overhead
Shard count	avoid thousands of small shards per node	each shard costs heap, file handles, threads and cluster-state work
Primaries	enough to spread writes; fixed after creation	use rollover / time-based indices for growing data
Replicas	1 for safety; more only for read throughput	each replica re-indexes every write (lesson 25)
JVM heap	≤ 50% of RAM and below ~31 GB	the rest is OS page cache for segments; ~31 GB keeps compressed pointers
Master nodes	3 dedicated master-eligible nodes	a majority (2 of 3) is needed to elect a master; avoids split brain

Node roles separate work: master-eligible nodes keep the cluster state (indices, mappings, shard locations); data nodes hold shards and do the heavy indexing and searching, optionally tiered as `data_hot`, `data_warm`, `data_cold`; ingest nodes run pipelines; coordinating-only nodes just route and merge.

WORKED EXAMPLE: DAILY LOG INDICES WITH LIFECYCLE MANAGEMENT

Suppose services produce about 150 GB of logs a day. Each day gets a new index such as `logs-2026-10-31`, created from the template in Figure 3 with explicit mappings. Four primaries put each shard near 37 GB, inside the 10–50 GB band; with one replica that is 8 shards and 300 GB of disk per day. An ILM policy keeps an index on SSD **hot** nodes for 7 days, moves it to cheaper **warm** nodes (read-only, force-merged), and deletes it at 30 days. Steady state: 30 indices, 240 shards, about 9 TB including replicas. Deleting a day is **dropping a whole index**: instant, unlike a delete-by-query over millions of documents, which writes tombstones and triggers merges. Queries for 'the last hour' hit only today's index (lesson 25).



index template + ILM policy are applied automatically to every new logs- index (or data-stream backing index)*

Figure 4. Index lifecycle management (OpenSearch calls it Index State Management). Rollover can also cut indices by size instead of by day.

TRADE-OFFS

- **More primaries**: more write parallelism and smaller shards, but more per-query fan-out and overhead.
- **More replicas**: more read capacity and safety, but every write is done N+1 times and disk use multiplies.
- **Shorter refresh**: fresher results, but tiny segments and lower indexing throughput.
- **Strict mappings**: a stable cluster, but producers must coordinate schema changes.

COMMON MISTAKES

- Leaving dynamic mapping on for logs until a field explosion slows the whole cluster.
- One index per customer or per day at tiny volume: thousands of 50 MB shards ('oversharding').
- Only two master-eligible nodes: losing one leaves no majority, so the cluster can't elect a master.
- Deleting old data with delete-by-query instead of dropping time-based indices.

6 Key terms

Shard: a partition of an index; a full Lucene index

Refresh: makes buffered docs searchable (default 1 s)

Mapping: field types and analyzers: the index schema

Replica: copy of a shard on another node

Translog: per-shard write-ahead log for durability

ILM / ISM: automates rollover, tier moves and deletion

7 Interview questions

Q1. How do you size shards in Elasticsearch? BASIC

Aim for roughly 10–50 GB per shard, avoid too many shards per node, choose the primary count up front, and use time-based indices with rollover for data that keeps growing.

Q2. What do replicas give you? BASIC

Failover, since a replica is promoted when a primary's node dies, and more read throughput, since any copy can serve searches.

Q3. An index was created with 1 primary and is now 400 GB. What do you do? SCENARIO

Reindex into a new index with more primaries behind an alias (or use the _split API), then use rollover so it never grows unbounded.

Q4. Cluster is yellow after adding replicas. Why? SCENARIO

Some replicas can't be allocated, typically because there aren't enough nodes (a replica never shares a node with its primary) or disk watermarks are exceeded.

Q5. Bulk-loading 2 billion documents once. Which settings help? SCENARIO

Replicas 0 and refresh_interval -1 during the load plus bulk requests; restore both afterwards.

Q6. Why can't you just change number_of_shards on a live index? TRICKY

Documents are placed by hash(routing) % primaries, so a new count would move most documents; that needs a reindex, or split/shrink by exact factors.

Q7. A document was indexed successfully but a search right after doesn't find it. Bug? TRICKY

No: search is near real time, so it appears after the next refresh; GET by ID is real-time (lesson 20).

Next: SD-P-004 · Search Ranking backendroadmap.com

Search Ranking

matching is easy; the best result first is the job

Watch the 4-min video

In one line: engines score text matches with **BM25** (rare terms count more, repeated terms saturate, long documents are normalised), then shape the score with **field boosts** and **business signals**, optionally re-rank the top results with a **learning-to-rank** model, and prove every change with offline metrics (NDCG, MRR) and A/B tests.

1 Same products, different results

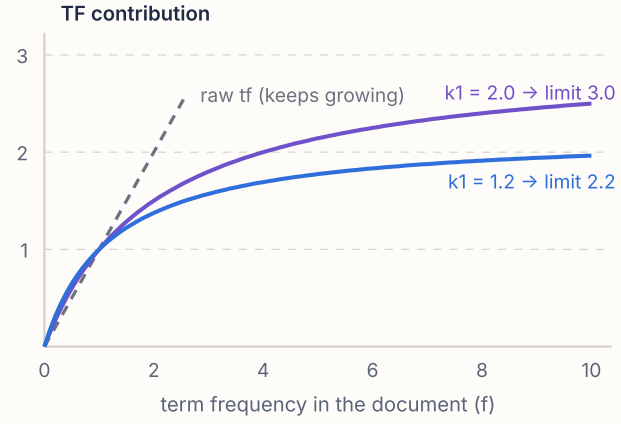
Two shops sell the same catalogue on the same engine. Typing **wireless headphones** in one shows the best-selling in-stock pair first; the other shows a charging cable and a sold-out item. Both found the right documents. Ranking decides whether the user sees them: most clicks go to the first few results, so positions 1–5 are what relevance work is about.

2 TF-IDF, then BM25

TF-IDF multiplies **term frequency** (a document that says 'headphones' often is probably about headphones) by **inverse document frequency** (a term that appears in few documents is more informative). Raw TF has two flaws: the score grows without limit, so keyword stuffing wins; and long documents mention everything, so they win too. **BM25** (Okapi BM25, the default similarity in Lucene since Elasticsearch 5.0) fixes both.

$$\text{score} = \Sigma_t \ \text{IDF}(t) \cdot f(k1+1) \ / \ (f + k1(1 - b + b \cdot |D|/\text{avgdl}))$$

Here f is how often term t appears in document D, |D| is the field length and avgdl the average length of that field. Lucene uses IDF = ln(1 + (N − n + 0.5)/(n + 0.5)), where n of N documents contain the term. Scores are computed **per field**, and the statistics are per shard by default, so tiny indices with uneven shards can rank slightly inconsistently (`search_type=dfs_query_then_fetch` gathers global statistics first, at the cost of an extra round trip).



Length normalisation (b)

k1 is multiplied by (1 − b + b · |D| / avgdl)
|D| short (title) → smaller → higher score
|D| long (manual) → bigger → lower score
b = 0: ignore length; b = 1: full effect
defaults: k1 = 1.2, b = 0.75

IDF: rare terms weigh more

N = 1 M docs: 'the' in 900 k → IDF ≈ 0.1
'wireless' in 5 k → IDF ≈ 5.3
'headphones' in 2 k → IDF ≈ 6.2

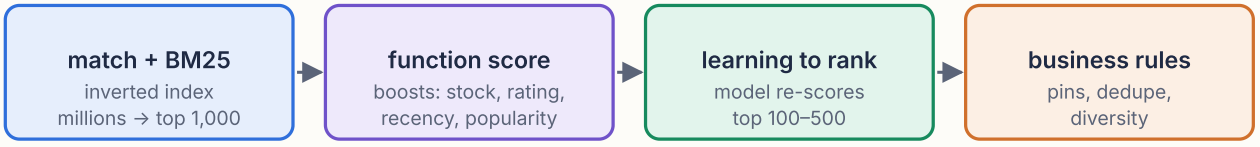
Figure 1. Left: with BM25 the 10th occurrence adds little; the curve flattens toward k1 + 1. Right: what b and IDF do.

Occurrences f	1	2	3	5	10	limit
BM25 TF part (k1 = 1.2, average length)	1.00	1.38	1.57	1.77	1.96	2.20
raw TF	1	2	3	5	10	∞

3 Shaping the score

- **Field boosts**: a match in `title^3` counts more than one in `description`. `multi_match` types matter: `best_fields` takes the best single field, `most_fields` adds them up, `cross_fields` treats fields as one big field (good for names split into first/last).
- **Phrase and exact boosts**: add an optional `match_phrase` clause so documents with the words together rank higher.
- **Function score**: multiply or add business signals. `field_value_factor` on popularity or rating with a `log1p` / `log2p` modifier (note: these are $\log_{10}$ in Elasticsearch) so 10,000 sales doesn't beat 100 by 100×; **decay functions** (`gauss`, `exp`, `linear`) for recency or distance; filter-based `weight`s for flags like in-stock.
- **Combine with care**: `boost_mode: multiply` keeps relevance in charge; `sum` or `replace` can let popularity override what the user typed.

4 Learning to rank: two-stage ranking



cheap → expensive per document; each stage sees fewer documents

Elasticsearch: query + function_score, then a rescore (window_size) or the LTR rescorer on the top hits

Figure 2. A ranking funnel. Expensive scoring only runs on the few hundred candidates that survive the cheap stage.

For big catalogues, hand-tuned boosts plateau. **Learning to rank (LTR)** trains a model, typically gradient-boosted trees such as LambdaMART (XGBoost, LightGBM), on judgement lists built from clicks, add-to-carts and purchases, or from human graders. Features include BM25 per field, phrase match, price, rating, sales, freshness, and user or query context. Because the model is slow compared with BM25, it only **re-scores the top N** (a few hundred) from the first stage. Recent Elasticsearch 8.x versions have a built-in LTR rescorer; OpenSearch and older Elasticsearch use the LTR plugin. Training on clicks needs care: users click higher positions anyway (**position bias**), so raw click-through over-rewards what you already rank first.

5 Measuring relevance

Build a **judgement set**: a few hundred real queries (head and long tail), each with results graded e.g. 0–3. Then compute:

- **NDCG@k**: DCG@k = $\Sigma\ (2^{rel_i} - 1) / \log_2(i + 1)$ over the top k positions, divided by the DCG of the ideal ordering, so 1.0 is perfect. It rewards relevant results near the top and handles graded relevance.
- **MRR**: the average of 1/rank of the first relevant result. First good result at position 1 → 1.0; at position 4 → 0.25. Ideal for 'find one thing' queries such as navigation.
- **Precision@k / recall@k**: share of the top k that is relevant / share of all relevant docs that made the top k.

- **Online**: click-through rate, zero-result rate, search-to-purchase conversion, time to first click. Compare rankings with **A/B tests** or the faster **interleaving** (mix both rankings in one list and see which side's results get clicked).

Example NDCG@3: results graded [2, 0, 1] give DCG = 3/1 + 0/1.58 + 1/2 = 3.5; the ideal order [2, 1, 0] gives 3 + 1/1.58 + 0 = 3.63; NDCG@3 = 3.5 / 3.63 ≈ **0.96**. Swapping the irrelevant item out of position 2 is worth 0.04.

WORKED EXAMPLE: BOOSTING IN-STOCK PRODUCTS

Query **wireless headphones**. A `function_score` wraps the BM25 query with three functions, combined by `score_mode: multiply` and applied with `boost_mode: multiply`: weight **2** if `in_stock: true`, weight **0.5** if false, and `field_value_factor(rating, modifier: log2p)` = $\log_{10}(2 + \text{rating})$. (The video says log1p; log1p of a 0 rating is 0, which would wipe out an unrated product's score in multiply mode, so log2p or `missing` is safer.)

Product	BM25	stock	rating factor	final
Sony wireless headphones	9.0	×2	4.7 → 0.83	**14.9**
Wireless headphones charging cable	9.5	×2	3.2 → 0.72	13.6
Bose wireless headphones (sold out)	9.8	×0.5	4.8 → 0.83	4.1

The available, well-rated headphones now lead; the sold-out pair is still findable, just lower. The cable still ranks second because its text matches well: a category boost ('headphones' query → Headphones category) or an LTR model fixes that. The change ships only after the A/B test shows higher click-through and conversion.

TRADE-OFFS

- **Business boosts** lift revenue but can bury the most relevant item: watch zero-click and refinement rates.
- **LTR** beats hand-tuning at scale but needs judgement data, feature logging and model ops.
- **Higher k1** rewards repetition more; **lower b** stops penalising long documents (good for short, uniform fields).
- **Global statistics (DFS)** give consistent scores across shards at the cost of an extra round trip.

COMMON MISTAKES

- Tuning relevance by eyeballing a few favourite queries instead of a judgement set.
- Adding popularity with `boost_mode: sum`, so bestsellers appear for unrelated queries.
- Training on raw clicks without correcting for position bias.
- Comparing BM25 scores across different queries or indices: they are only meaningful within one result list.

6 Key terms

IDF: inverse document frequency: rare terms weigh more

BM25: TF-IDF with TF saturation (k1) and length normalisation (b)

Function score: mixes business signals into the text score

Learning to rank: ML model re-scores the top candidates

NDCG: graded, position-discounted ranking quality, 0–1 **MRR**: mean of 1/rank of the first relevant result

7 Interview questions

Q1. How does BM25 score documents? BASIC

For each query term it combines saturating term frequency, inverse document frequency (rare terms count more) and document-length normalisation, then sums across terms.

Q2. How do you measure relevance? BASIC

Offline with judged queries and metrics like NDCG and MRR; online with click-through, conversions, zero-result rate and A/B tests or interleaving.

Q3. What do k1 and b control? BASIC

k1 sets how quickly term frequency saturates (default 1.2); b sets how strongly long fields are penalised, from 0 (none) to 1 (full), default 0.75.

Q4. Sold-out items rank first. Fix it without hiding them. SCENARIO

Wrap the query in function_score with a weight > 1 for in-stock and < 1 for out-of-stock, multiply mode, and verify with an A/B test.

Q5. You have millions of products and good click logs. How do you improve ranking further? SCENARIO

Two-stage ranking: BM25 plus boosts retrieves the top few hundred, an LTR model trained on debiased clicks and purchases re-scores them; measure with NDCG offline and A/B online.

Q6. A query returns zero results 8% of the time. What do you look at? SCENARIO

Log and cluster the zero-result queries: missing synonyms, typos (fuzziness), over-strict AND matching (use minimum_should_match), or analyzer gaps; then fall back to a looser query.

Q7. Why might a product rank differently on two identical queries? TRICKY

Requests can hit different shard copies, and BM25 statistics are per shard and include not-yet-merged deleted docs; use a preference key or DFS for consistency.

Q8. The team says the new ranking 'feels better' but the A/B test is flat or negative. Ship it? TRICKY

No: user behaviour is the ground truth; check segment effects and the judgement set, but don't ship on intuition.

Next: SD-P-005 · Filtering and Faceting backendroadmap.com

Filtering and Faceting

filters say yes or no; aggregations count

Watch the 4-min video

In one line: put every non-scoring condition in **filter context** so it skips scoring and can be **cached as a bit set**; compute facet counts with **aggregations** over the current matches, read from columnar **doc values**; and use a **post filter** so a selected facet doesn't hide its own alternatives.

1 The shopping sidebar

A shopper searches **laptops** and gets 12,000 results. The sidebar shows Brand (Lenovo 240, Dell 310...), Price (under $1,000: 3,000), Screen size and Rating, each with live counts that change as the shopper clicks. On a busy site that is thousands of searches a second, and every one needs both the top 20 results and every count, in tens of milliseconds. That works because engines treat filters and counts very differently from text relevance.

2 Queries score, filters don't

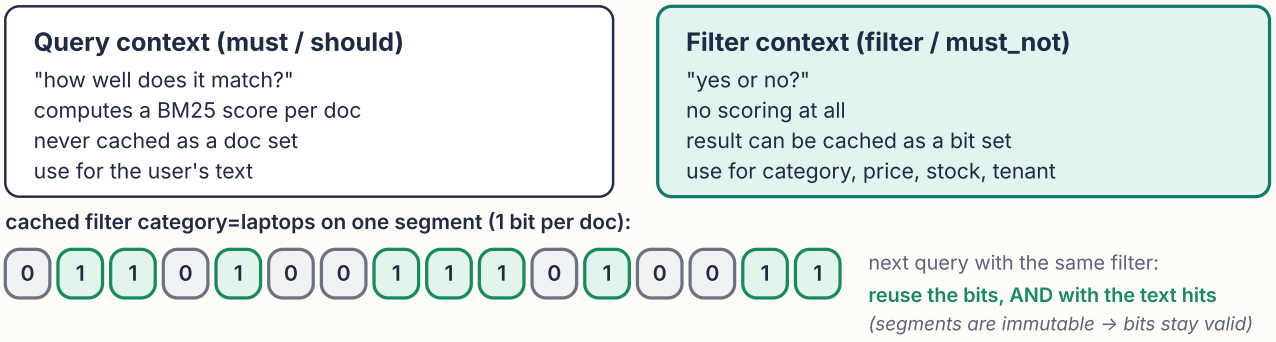


Figure 1. In a bool query, must/should clauses score; filter/must_not clauses only include or exclude, so their results can be cached.

A `bool` query has four clause types: `must` (match and score), `should` (optional, adds score), `filter` (must match, no score) and `must_not` (must not match, no score). Brand, price range, stock, category, tenant ID and permissions are yes/no questions, so they belong in `filter`. That skips BM25 work and lets the engine cache the matching documents.

How the cache works. Elasticsearch's node query cache stores, per segment, a bit set of documents matching a filter. Because segments are immutable (lesson 15), the bits stay valid until the segment is merged away. It is LRU, sized at 10% of heap by default, and Lucene only caches filters that are used repeatedly and only on larger segments (10,000+ docs), since tiny segments are cheap anyway. A separate **shard request cache** stores whole results of `size: 0` requests (aggregation-only, like a facet refresh) and is invalidated whenever the shard refreshes.

Cache-friendly filters. `now` changes every millisecond, so `range: {created: {gte: "now-1h"}}` is never the same filter twice; rounding to `now-1h/m` makes it reusable for a minute.

3 Facet counts are aggregations

```
GET products/_search
{ "size": 20,
  "query": { "bool": {
      "must":   [ { "match": { "name": "laptop" } } ],
      "filter": [ { "term": { "category": "laptops" } },
                  { "term": { "in_stock": true } } ] } },
  "aggs": {
    "brands": { "terms": { "field": "brand", "size": 10 } },
    "prices": { "range": { "field": "price", "ranges": [
        { "to": 500 }, { "from": 500, "to": 1000 }, { "from": 1000 } ] } } } }
```

Figure 2. One request: scored text query, cached filters, a terms facet and a range facet. Range 'from' is inclusive and 'to' exclusive.

- **terms**: groups matching docs by a keyword field and returns the top N buckets with counts (brand, colour, seller).
- **range** / **histogram**: counts per numeric bucket (price bands, screen sizes); **date_histogram** for time.
- **filter** / **filters**: counts for arbitrary conditions ('rating ≥ 4', 'free delivery').
- **Metrics**: min/max price for a slider, avg rating, and **cardinality** (distinct count, approximated with HyperLogLog++).

Aggregations run over exactly the documents that match the query and filters, so counts always reflect the current search. On many shards, `terms` counts can be approximate: each shard returns only its own top `shard_size` buckets (default size × 1.5 + 10), so a brand that is 11th on every shard can be under-counted. The response reports `doc_count_error_upper_bound`; raise `shard_size` if exact top-N matters. For a sidebar, approximate is usually fine.

4 Doc values: columns for counting and sorting



Figure 3. Matching needs term → docs. Counting, sorting and scripting need doc → value, which is what doc values store.

An aggregation needs the reverse of the inverted index: for each matching doc, its brand and price. **Doc values** are an on-disk, column-oriented copy of each field's values, written at index time and read through the OS page cache. They are enabled by default for keyword, numeric, date, boolean and geo fields. Analysed `text` fields have no doc values: aggregating them needs `fielddata`, which builds the column on the JVM heap and is off by default for good reason. So facet on `keyword` or numeric fields (e.g. `brand.keyword`), never on the analysed name. Sorting by price uses the same columns.

5 The facet UX trap: post filters

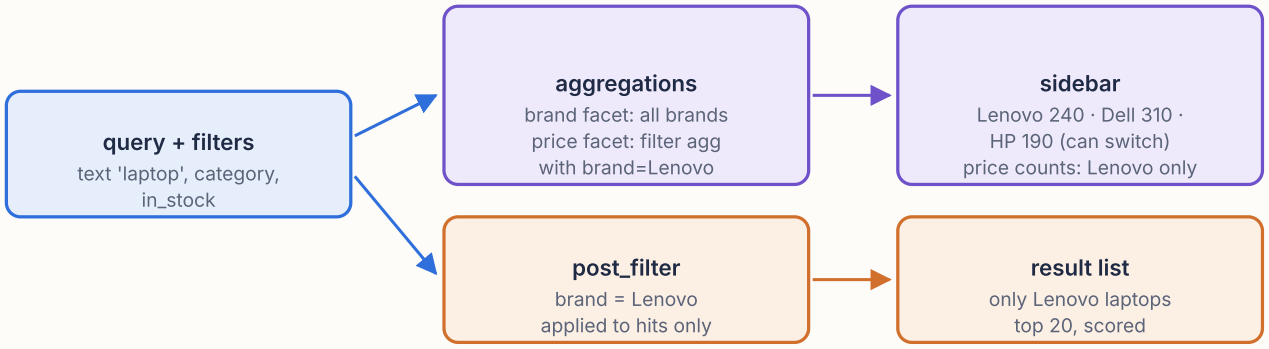


Figure 4. Multi-select facets: the brand filter goes in post_filter so the brand facet still shows other brands, while other facets respect it.

If the brand filter goes into the main query, the brand facet now shows only 'Lenovo 240', and the shopper can't see or add Dell. `post_filter` is applied *after* aggregations are computed, so it narrows the hits but not the facets. Other facets (price, screen) should still respect the chosen brand, so wrap them in a `filter` aggregation with the same brand condition. The general rule: each facet is computed with every selected filter **except its own**. Post filters are slower than normal filters (they can't narrow the work done by aggregations), so use them only for multi-select facets.

WORKED EXAMPLE: CATEGORY AND PRICE FACETS IN ONE ROUND TRIP

The request in Figure 2 carries the text query 'laptop', filters `category=laptops` and `in_stock=true`, a terms aggregation on `brand` and a range aggregation with buckets <500, 500–1000 and ≥1000. Each shard scores its hits, keeps its local top 20, and builds partial brand and price buckets from doc values; the coordinating node merges them (lesson 25). The response holds 20 results plus e.g. Lenovo 240 · Dell 310 · HP 190 and <$500: 1,900 · $500–1,000: 3,100 · ≥$1,000: 7,000, in tens of milliseconds. The two filters are cached after their first few uses, so the next shopper browsing laptops pays almost nothing for them. When she ticks Lenovo, brand moves into `post_filter` and the price aggregation gets a brand filter.

TRADE-OFFS

- **Exact vs fast counts**: bigger `shard_size` means more accurate terms counts and more work per shard.
- **Many facets** make every query heavier; compute rarely-used facets only when the user opens them.
- **High-cardinality facets** (seller ID across millions) are expensive; precompute or limit them.
- **Post filters** give natural multi-select UX but can't speed up aggregations.

COMMON MISTAKES

- Putting yes/no conditions in `must`: they get scored and are never cached.
- Faceting on an analysed text field ('lenovo thinkpad' split into words) or turning on fielddata to make it work.
- Using `now` without rounding in filters, so nothing is ever cached.
- Applying the selected brand to the main query, so the brand facet collapses to one option.

6 Key terms

Filter context: yes/no clause, no scoring, cacheable

Aggregation: computes buckets or metrics over matching docs

post_filter: filters hits after aggregations are computed

Bit set: one bit per doc per segment: matches or not

Doc values: columnar on-disk field values for aggs and sorting

shard_size: buckets each shard returns before merging

7 Interview questions

Q1. How are facet counts computed? BASIC

With aggregations over the documents matching the current query and filters, e.g. terms for brands and range for prices, read from column-oriented doc values.

Q2. Why use filters instead of queries for conditions? BASIC

Filters skip scoring and their results can be cached as per-segment bit sets, so they are faster, especially for repeated conditions.

Q3. Brand facet shows only the selected brand after the user clicks it. Fix? SCENARIO

Move the brand condition into post_filter so the brand aggregation ignores it, and add it as a filter aggregation around the other facets.

Q4. Facet counts on a 30-shard index don't add up exactly. Why? SCENARIO

terms aggregations take each shard's local top shard_size buckets, so a value outside some shards' top lists is under-counted; raise shard_size or accept the documented error bound.

Q5. A 'last 24 hours' filter is slow despite the filter cache. Why? SCENARIO

A filter using raw now differs on every request, so it is never cached; round it (now-24h/h) so the same filter repeats.

Q6. Why can't you aggregate on a text field by default? TRICKY

Text fields have no doc values; aggregating needs fielddata built on the heap from the inverted index, which is memory-hungry and would count analysed tokens, not the original values.

Q7. Do filters change the ranking? TRICKY

Not the scores of the remaining documents, but they change which documents are present; and a filter wrapped in a should clause or constant_score can contribute a fixed score if you ask it to.

Next: SD-P-006 · Indexing Pipelines backendroadmap.com

Indexing Pipelines

the database is the truth; the index follows it

Watch the 4-min video

In one line: keep the database as the source of truth and stream its changes to the search index asynchronously, with **change data capture** into a queue and an **idempotent, versioned, bulk** indexer; rebuild indexes behind an **alias** and swap it atomically for zero-downtime reindexing.

1 The database says $79, search says $99

Products live in Postgres; search runs on Elasticsearch. A price changes but search shows the old one; a deleted product still appears. Querying is the easy part of search. The hard part is keeping a **derived copy** in step with the source of truth, through crashes, retries, deploys and schema changes. Search documents are also usually **denormalised**: one product document joins the product row, its brand name, current price, stock level and category path from several tables.

2 Why synchronous dual writes drift

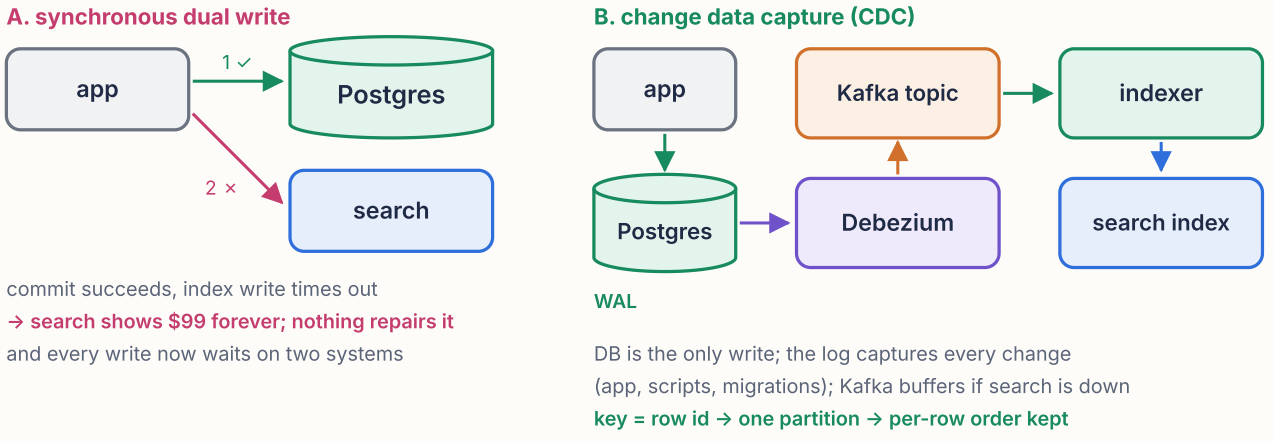


Figure 1. Dual writes have no transaction across two systems. CDC makes the database's own log the single stream of changes.

Writing to the database and then the index from the request handler looks simple, but there is no atomic commit across the two. If the index write fails after the commit (timeout, deploy, crash), they diverge with nothing to repair it. Two concurrent requests can also reach the index in the opposite order from the database. And the user's write latency and availability now depend on the search cluster too.

3 Change data capture into a queue

CDC reads the database's own replication log instead of trusting the application. **Debezium** on Postgres uses logical decoding (the `pgoutput` plugin) through a **replication slot**, turning every committed insert, update and delete into an event with before/after values and the log position (LSN). On MySQL it reads the binlog. Events go to a Kafka topic per table, keyed by primary key, so all changes to one row land in one partition, in order. Changes made by scripts, admin tools or other services are captured too, and nothing is published for a transaction that rolled back.

The alternative when you can't use CDC is the **transactional outbox**: the app writes the row and an 'event' row in the same database transaction, and a relay publishes outbox rows to the queue. Either way the database commit is the single source of events.

Operational catch: a Postgres replication slot makes the database keep WAL until the consumer confirms it. If Debezium is down for a day, WAL piles up and can fill the disk, so monitor slot lag (and on recent Postgres, cap it with `max_slot_wal_keep_size`).

4 The indexer: bulk, idempotent, versioned

- **Bulk API**: send hundreds to a few thousand documents (a few MB) per `_bulk` request, not one HTTP call per document. A bulk response can partly fail: check `errors` and per-item status, retry 429 (rejected, queue full) with backoff, and send poison documents (mapping errors) to a dead-letter topic.
- **Idempotent**: use the database primary key as `_id`, so a replayed event overwrites the same document instead of creating a duplicate. At-least-once delivery from Kafka is then safe.
- **Versioned**: index with `version_type=external` and the event's LSN or `updated_at` as the version. Elasticsearch rejects any write whose version is not higher than the stored one, so a late retry can't overwrite newer data.
- **Fan-out**: when a brand is renamed, every product document embedding it must be re-indexed; the indexer needs a lookup from brand to product IDs, or the brand should be a separate filterable field fetched at query time.
- **Backpressure**: if search is slow, the consumer lags and Kafka holds the events; alert on **consumer lag**, which is exactly your search staleness (lesson 20).



Figure 2. External versioning makes the index converge to the newest state whatever order events arrive in.

5 Rebuilding with aliases

Some changes can't be applied in place: a new analyzer, a changed field type, more primary shards (lesson 16). So the app never talks to a real index; it searches and writes an **alias** such as `products`. To rebuild:

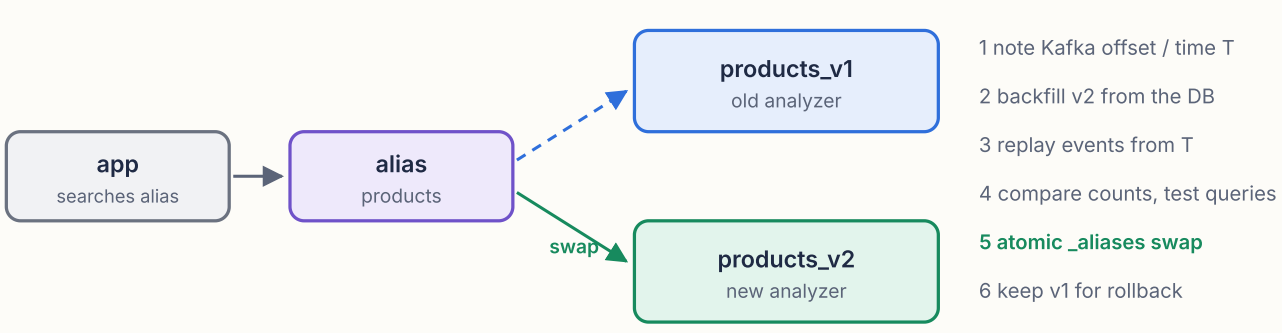


Figure 3. Zero-downtime reindex. The alias moves in one cluster-state update, so no search ever sees a half-built index.

Record the current Kafka offset (or a timestamp) *before* starting, backfill `products_v2` from a database snapshot or with the `_reindex` API, then replay events from the recorded offset. Overlap is harmless because writes are idempotent and versioned. Speed the backfill with `refresh_interval: -1` and zero replicas, then restore them. Verify counts and a set of test queries, then swap:

```
POST _aliases
{ "actions": [
  { "remove": { "index": "products_v1", "alias": "products" } },
  { "add":    { "index": "products_v2", "alias": "products", "is_write_index": true } } ] }
```

Figure 4. Both actions apply atomically. Rolling back is the same call with the indices exchanged.

WORKED EXAMPLE: POSTGRES → DEBEZIUM → KAFKA → ELASTICSEARCH

A seller changes a price from $99 to $79. Postgres commits it; Debezium reads the WAL entry and publishes `{op: "u", after: {id: 42, price: 79}, lsn: …}` to `shop.public.products`, keyed by 42. The indexer consumes a batch of 500 events, loads brand and stock for each product, and sends one `_bulk` request with `_id = 42` and `version = lsn`. After the next refresh (≤ 1 s) search shows $79; end-to-end lag is typically a second or two. A delete event becomes a bulk `delete` for `_id 42`. When the team adds a new analyzer, they build `products_v2`, catch it up from Kafka and swap the alias without users noticing.

TRADE-OFFS

- **Async pipeline**: reliable and decoupled, but search is eventually consistent (lesson 20).
- **CDC** captures everything without app changes but couples you to the DB log format and needs Kafka/Connect ops.
- **Outbox** gives explicit domain events but every writer must use it.
- **Full rebuilds** are safe with aliases but need double disk space while both indices exist.

COMMON MISTAKES

- Letting Elasticsearch auto-generate IDs, so retries create duplicate documents.
- Ignoring per-item errors in bulk responses: some documents silently never get indexed.
- Starting the backfill and only then noting the replay position, leaving a gap of lost changes.
- Pointing the application at the concrete index name instead of an alias.

6 Key terms

CDC: stream committed DB changes from its log

Transactional outbox: event row written in the same transaction

External versioning: only accept writes with a higher version

Replication slot: Postgres bookmark that retains WAL for a consumer

Bulk API: many index/delete ops in one request

Index alias: stable name that points to a real index

7 Interview questions

Q1. How do you reindex with zero downtime? BASIC

Search through an alias, build the new index in the background, catch it up from the change stream, swap the alias atomically, and keep the old index for rollback.

Q2. How do you keep the index in sync with the database? BASIC
Asynchronously, with change data capture (or an outbox) into a queue, and idempotent, versioned bulk writes keyed by the database ID.

Q3. The indexer was down for 2 hours. What happens and what do you check? SCENARIO
Kafka retained the events, so it resumes from its committed offset and catches up; check consumer lag, Postgres replication-slot WAL growth and bulk rejections while it catches up.

Q4. Search shows a product that was deleted yesterday. Where do you look? SCENARIO
Was the delete event produced (CDC), consumed (lag, DLQ), and applied (bulk item errors, version conflicts)? Then run reconciliation to find other strays (lesson 20).

Q5. Why key Kafka messages by product ID? SCENARIO
All events for one product go to one partition, so they are consumed in commit order; different products are processed in parallel.

Q6. Why isn't 'write DB, then write index' in the same request good enough? TRICKY
There is no atomic commit across them; a failure after the DB commit leaves them inconsistent with no repair path, and concurrent writes can reach the index out of order.

Q7. With external versioning, how can a deleted document come back? TRICKY
Delete tombstones are kept only for index.gc_deletes (60 s by default); an older update arriving later recreates it. Use soft deletes (a deleted flag) or ensure ordering per key.

Next: SD-P-007 · Search Consistency backendroadmap.com

Search Consistency

saved is not the same as searchable

Watch the 4-min video

In one line: search is **near real time** and, behind a pipeline, **eventually consistent** with the database; force visibility with `refresh=wait_for` only where you can afford it, hide the gap in the product with **read-your-writes**, never trust the index for permissions or stock, and **reconcile** drift continuously.

1 The duplicated listing

A seller creates a listing, is redirected to their shop page (built from search), and the listing isn't there. They create it again; now there are two. The first save worked perfectly. It just wasn't **searchable yet**. Users only notice the gap when they look for something they just did, so that is where the fix belongs.

2 Where the gap comes from

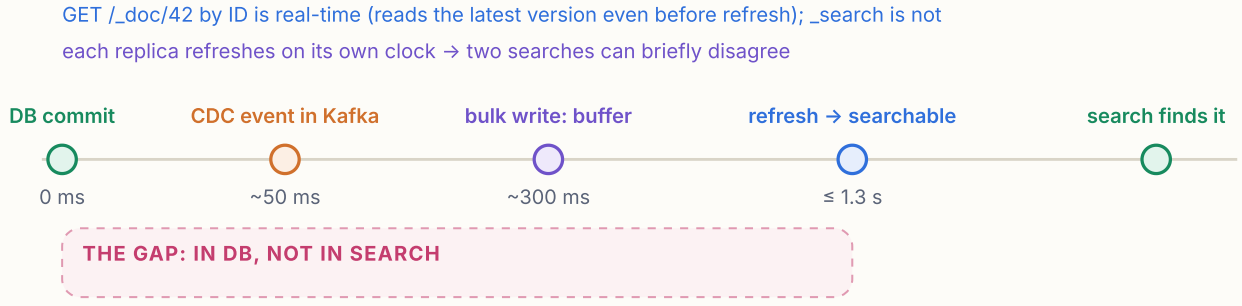


Figure 1. Typical timeline for a CDC pipeline (illustrative numbers). Most of the gap is pipeline lag plus the refresh interval.

Inside the engine, an indexed document sits in an in-memory buffer until a **refresh** (default every 1 s) writes a new searchable segment (lesson 15). With a CDC pipeline in front (lesson 19), add capture, queue and batching delay: usually a second or two, minutes if the indexer is lagging. Three subtleties catch people out:

- **Get vs search**: `GET index/_doc/42` is real-time by default, so 'it's there when I fetch it by ID' doesn't prove a search will find it.
- **Replicas refresh independently**, so consecutive searches routed to different copies can return different results. A `preference` value (e.g. the session ID) sends one user to the same copies for a consistent view.
- **Search-idle shards** skip scheduled refreshes when nobody searches them for 30 s; the first search afterwards waits for a refresh.

3 Forcing visibility, and what it costs

Option	Behaviour	Cost	Use when
`refresh=false` (default)	returns at once; visible after next refresh	none	almost always
`refresh=wait_for`	returns only after a refresh has made it visible	request waits up to the refresh interval	low-volume, user-facing writes
`refresh=true`	forces an immediate refresh of the affected shards	tiny segments, more merging, lower throughput	tests; rarely in production
shorter `refresh_interval`	whole index fresher	indexing throughput drops for every write	small, read-heavy indexes

`wait_for` doesn't create segments itself; it parks the request until the scheduled refresh. If too many requests wait (`index.max_refresh_listeners`, default 1,000) Elasticsearch forces a refresh. It also only helps when the client writes to the engine directly; with an async pipeline the app's request has already returned, so you need the product-side fixes below.

4 Read your own writes in the product

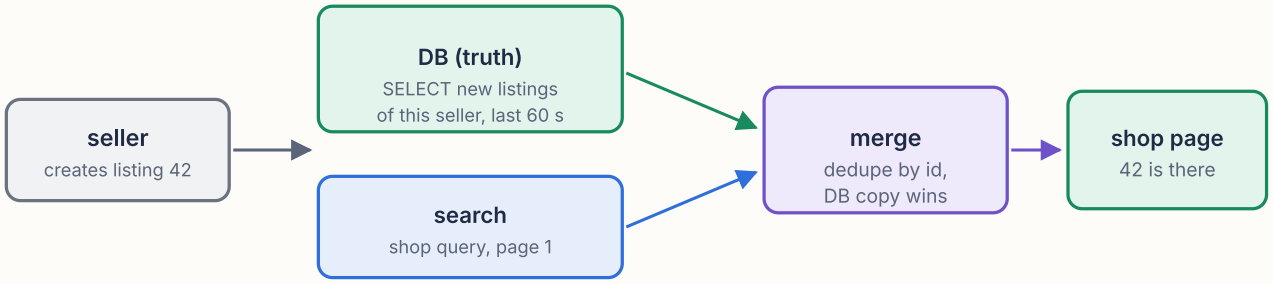


Figure 2. Merge the user's own recent writes from the database into search results until the index catches up.

- **Confirmation from the record**: after 'Create', show a page built from the saved row or the API response, not a search.
- **Merge recent writes**: the shop page asks the DB (or a short-lived cache keyed by user) for that seller's items created in the last minute and merges them into the search results, deduplicating by ID.
- **Optimistic UI**: insert the new item into the client-side list immediately.
- **Idempotency key** on the create endpoint, so a nervous double-submit can't create two listings anyway.

5 Deletes and authority

A deleted document is only marked deleted and disappears from results at the next refresh; behind an async pipeline it can linger for seconds or longer. So treat search results as **candidates**: on click, load the item from the database and show 'no longer available' if it's gone. Never use the index alone for **permissions** (a revoked share could still be visible), **stock** or **price at checkout**; re-check against the source of truth. For document-level security, filter by the user's tenant/ACL in the query (lesson 43) *and* verify on fetch. One more trap: if updates can arrive after a delete, a late update can recreate the document (see lesson 19); soft deletes (`deleted: true` plus a filter) avoid this.

6 Reconciling drift

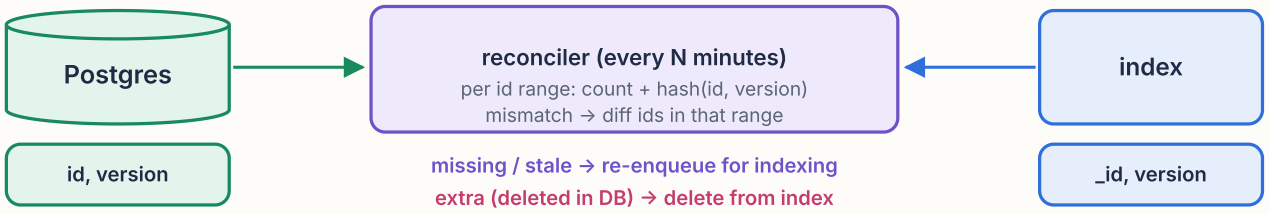


Figure 3. A reconciliation job turns silent drift into a bounded, self-healing delay.

Even a good pipeline drifts: an event dropped by a bug, a document rejected by a mapping error, a manual fix to the DB that bypassed CDC. Run a **reconciler**: split the ID space into ranges, compare per range a count and a hash of (id, version) from the DB and from the index; only for mismatched ranges, list IDs and diff them. Re-enqueue missing or stale IDs and delete extras. Track the **drift rate** as a metric: a rise points to a pipeline bug. Periodic full rebuilds behind an alias (lesson 19) are the heavy-handed version of the same idea.

WORKED EXAMPLE: THE MISSING NEW LISTING

The seller saves listing 42. The API commits it to Postgres and returns 201 with the saved record; the CDC pipeline indexes it about a second later. Instead of redirecting to a search-based page, the app shows a confirmation built from the record. The shop page runs the normal search *plus* `SELECT … WHERE seller_id = ? AND created_at > now() - interval '60 seconds'`, merges by ID and shows 42 at the top. A unique idempotency key on 'create listing' blocks duplicates. By the time the seller searches again, the index has caught up and the merge adds nothing. Cost: one indexed DB query on one page, instead of slowing every write with forced refreshes.

TRADE-OFFS

- **wait_for**: simple and correct for the writer, but adds up to a second of latency per write.
- **Read-your-writes merging**: no engine cost, but extra code and a DB query on specific pages.
- **Faster refresh**: fresher for everyone, but lower indexing throughput and more merging.
- **Frequent reconciliation** shortens drift but adds load on both the DB and the cluster.

COMMON MISTAKES

- Using `refresh=true` on every write to 'fix' consistency, then wondering why indexing collapsed.
- Redirecting to a search-backed page straight after a create.
- Trusting search results for access control or stock.
- Having no reconciliation, so a single lost event stays wrong forever.

7 Key terms

Near real time: searchable after the next refresh, not instantly

refresh=wait_for: write returns once it is searchable

Read-your-writes: a user always sees their own changes

Real-time GET: fetch by ID sees unrefreshed writes

preference: routes a user to the same shard copies

Reconciliation: periodic DB vs index diff and repair

8 Interview questions

Q1. How do you handle a user not seeing their newly created item in search? BASIC

Use refresh=wait_for where the write path can afford it, or better, read your own writes: show the item from the database or merge it into results until the index catches up.

Q2. How do you keep search and the database consistent? BASIC

Accept eventual consistency, make the pipeline idempotent and versioned, handle deletes carefully, and reconcile drift with periodic checks.

Q3. A user refreshes the results page and an item appears, disappears, then appears. Why? SCENARIO

Requests hit different replicas that refreshed at different moments; use a preference key (e.g. session ID) so one user sticks to the same copies.

Q4. A permission was revoked but the document still appears in another user's search. What's the design fix? SCENARIO

Search results are not authoritative: filter by ACL in the query and re-check permissions in the DB when the item is opened.

Q5. Why not just set refresh=true on all writes? TRICKY

Each forced refresh creates a tiny segment, which multiplies merge work and slashes indexing throughput; it only fixes visibility for direct writers anyway.

Q6. GET by ID returns the document but search doesn't. Contradiction? TRICKY

No: GET is real-time (it can read the latest version before a refresh), while search only sees refreshed segments.

Next: SD-P-008 · Autocomplete backendroadmap.com

Autocomplete

answer every keystroke in a few milliseconds

Watch the 4-min video

In one line: precompute the best completions for every prefix offline (a **trie** with the **top-k stored at each node**, or an engine's **completion suggester** / **edge n-grams**), serve them from memory, and cut traffic with **edge caching** of popular prefixes and **client debouncing**.

1 Search under extreme time pressure

The user types i, p, h, and the box already suggests 'iphone 15 case'. Every keystroke is a request: a million people searching per minute, about 10 characters each, is roughly 10 million suggestion requests a minute (~170,000/s). Each answer must arrive in well under 100 ms end to end, or the suggestion shows up after the user has typed past it, so the server budget is a few milliseconds. Normal full-text search per keystroke would be too slow and too expensive.

2 The trie, with answers stored in it

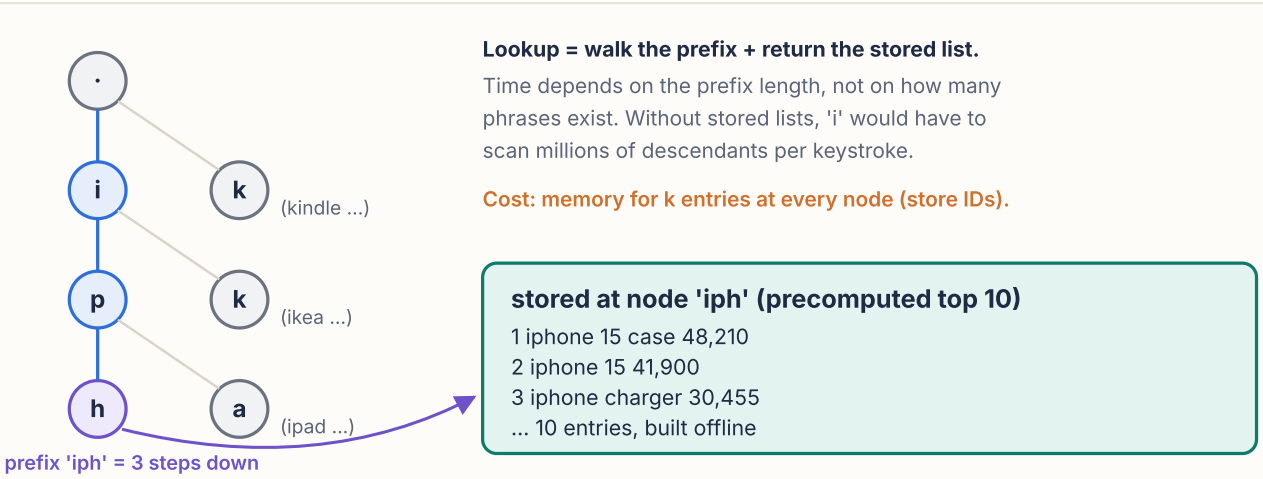


Figure 1. A trie (prefix tree). Each node is one character; each node also stores its top completions, so no subtree search is needed.

In a **trie**, each path from the root spells a prefix and every phrase below a node shares that prefix. Walking 'i' → 'p' → 'h' takes three steps whatever the dictionary size. But a short prefix like 'i' has millions of descendants, and ranking them per keystroke is far too slow. So the builder stores the **top k completions at every node**, computed bottom-up: a node's list is the best k among its own phrase and its children's lists. A lookup becomes 'walk the prefix, return the list'. Store phrase IDs, not strings, to keep memory down; compressed variants (radix trees, FSTs) merge single-child chains.

To scale, the whole structure is usually small enough to **replicate** to every server (millions of phrases → a few GB at most). If not, **shard by prefix** (a–f, g–m, ...), watching for hot first letters. Rebuild offline and ship an immutable snapshot that servers swap atomically, instead of updating a live trie under load.

3 Inside a search engine: edge n-grams or completion suggester

	Edge n-grams on a text field	Completion suggester	search_as_you_type field
How	index i, ip, iph, ipho... of each word; query the prefix as a normal term	in-memory FST per segment, built from a `completion` field	Elasticsearch field type with shingle and edge-n-gram sub-fields
Speed	a normal query: fast, but full query machinery	fastest: a pure prefix walk	normal query
Flexibility	any filters, scoring, mid-phrase words ('case' finds 'iphone case')	prefix of the input only; filters limited to predefined contexts (category, geo)	matches words in any position, phrase-prefix
Cost	bigger index (many terms per word)	heap memory; weights set at index time	several extra sub-fields

One rule for edge n-grams: use the n-gram analyzer at **index time only** and a plain analyzer at **search time**. Otherwise the query 'iph' is itself split into i, ip, iph and matches everything starting with 'i'.

4 Ranking suggestions

- **Popularity**: how often the phrase was searched (and, better, led to a click or purchase) over a window such as 30 days.
- **Recency**: time-decayed counts, plus a small streaming 'trending' layer (counts from the last hour) merged at query time, so breaking topics appear before the nightly rebuild.
- **Personalisation**: the user's own recent searches, merged on the client or in the service; this part can't be cached at the edge.
- **Hygiene**: blocklist offensive phrases, require a minimum count (so one person can't seed a suggestion), drop out-of-stock or zero-result phrases. Doing this offline keeps lookups cheap and policy reviewable.
- **Typos**: fuzzy prefix matching (edit distance 1 after the first couple of characters) rescues 'iphnoe'.

5 Making it cheap to serve

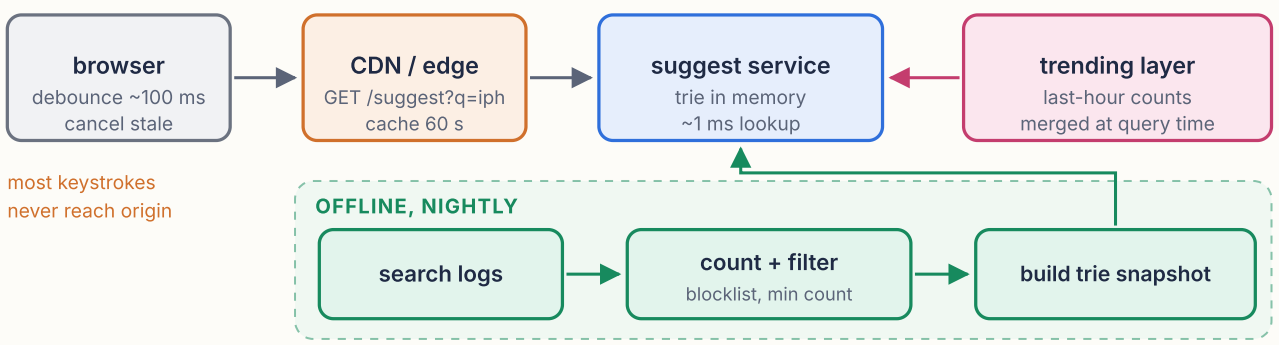


Figure 2. Serving path and offline build. The service only answers what the browser and the CDN didn't.

Prefix traffic is extremely skewed: everyone types 'i' and 'ip'. There are only 26 single-letter and 676 two-letter Latin prefixes, so those can be precomputed and cached at the **CDN** with a short TTL (e.g. 60 s), keyed by prefix and locale. On the client, **debounce** (send after ~100 ms without a keystroke), **cancel** in-

flight requests for prefixes the user typed past, and reuse the previous list when the new prefix just extends it and the list is still valid. Together these cut origin traffic by an order of magnitude or more.

WORKED EXAMPLE: SEARCH-BAR SUGGESTIONS

A nightly job reads 30 days of search logs, normalises phrases (lowercase, trim), counts them, drops phrases searched by fewer than 20 users or on the blocklist, and keeps the top few million. It builds a trie with the top 10 completions at every node and writes a snapshot. Each suggest server loads the snapshot into memory and answers `/suggest?q=iph` in about a millisecond. The CDN caches responses for a minute, so 'i', 'ip' and 'iph' hit the origin once a minute per edge location. The client waits until the user pauses about 100 ms. Result: suggestions feel instant, and the service handles a small fraction of the raw 170,000 keystrokes a second.

TRADE-OFFS

- **Precomputed top-k**: microsecond lookups, but more memory and only as fresh as the last build.
- **Edge n-grams**: filters and mid-word matches, but a larger index and slower than a dedicated structure.
- **CDN caching**: huge savings on short prefixes, but no personalisation in cached responses.
- **Longer debounce**: fewer requests, but suggestions feel laggy.

COMMON MISTAKES

- Running a full search query on every keystroke without debounce or caching.
- Applying the edge n-gram analyzer at search time too.
- Ranking by raw counts with no blocklist or minimum, so abuse becomes a suggestion.
- Showing results from a slow, stale request after a newer one already returned.

6 Key terms

Trie: prefix tree: one node per character

Edge n-gram: prefixes of a word indexed as terms

Debounce: wait for a pause before sending

Top-k per node: precomputed best completions for each prefix

Completion suggester: in-memory FST for fast prefix lookups

Trending layer: short-window counts merged at query time

7 Interview questions

Q1. How would you implement autocomplete at scale? BASIC

A trie or completion suggester with the top suggestions precomputed per prefix, ranked offline by popularity, held in memory, with caching for popular prefixes and debouncing on the client.

Q2. Edge n-grams or a completion suggester? BASIC

Edge n-grams are flexible and work with filters and mid-phrase words; completion suggesters are faster but limited to prefixes and predefined contexts.

Q3. A news event happens and the suggestion doesn't appear until tomorrow. Fix? SCENARIO

Add a streaming trending layer (counts over the last hour) merged with the nightly trie at query time, with its own hygiene filters.

Q4. How do you size the servers for 170,000 suggest requests/s? SCENARIO

Most short prefixes are served by the CDN and debounced clients; the remaining origin requests are ~1 ms in-memory lookups, so load-test one server, then divide the measured origin rate (after CDN hit rate) by its capacity with headroom.

Q5. Users complain that 'case' doesn't suggest 'iphone 15 case'. Why? SCENARIO

A prefix trie or completion suggester only matches from the start of the phrase; index word-level prefixes (edge n-grams or search_as_you_type) for infix matches.

Q6. Why store the top-k at every node instead of computing it? TRICKY

Short prefixes have millions of descendants; storing k IDs per node trades memory for constant-time lookups on exactly the prefixes that get the most traffic.

Q7. Can you cache personalised suggestions at the CDN? TRICKY

Not per user; cache the global list per prefix and locale at the edge, then merge the user's own history on the client or in a small follow-up call.

Next: SD-P-009 · Vector Search backendroadmap.com

Vector Search

search by meaning, not by shared words

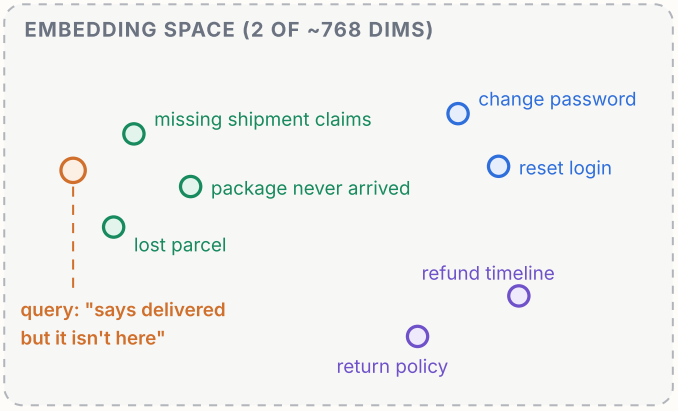
Watch the 4-min video

In one line: an **embedding model** turns text into vectors so that similar meanings land close together; search returns the **nearest neighbours** of the query vector, and at scale uses an **approximate** index such as **HNSW** or **IVF** that trades a little recall for orders of magnitude less work.

1 No shared words, same meaning

A customer types **my package says delivered but it isn't here**. The best help article is titled **Missing shipment claims**. BM25 (lesson 17) sees no common terms and returns nothing useful; a person sees immediately that they mean the same thing. Vector search compares meanings instead of words, which is why it powers semantic search, recommendations, deduplication and the retrieval step of RAG.

2 Embeddings and similarity



Similarity metrics

cosine = a·b / (|a||b|): angle only
dot product = a·b: angle and length
L2 = |a − b|: straight-line distance
unit vectors: cosine = dot,
$L2^2 = 2 - 2 \cdot \text{cosine}$ → same ranking

Rule: use the metric the model was trained with; normalise if it says so.

Same model for documents and queries, or the two spaces don't line up.

Figure 1. Texts with related meaning cluster together, even without shared words. The query lands next to the shipment articles.

An **embedding** is a fixed-length vector of floats produced by a neural model: e.g. 384 dimensions for `all-MiniLM-L6-v2`, 1,536 for OpenAI's `text-embedding-3-small`, 3,072 for `text-embedding-3-large`. The model is trained so that related texts get vectors pointing in similar directions. Key practicalities:

- **Chunking**: models have input limits and one vector per long document blurs its topics, so documents are split into passages (often a few hundred tokens, with some overlap) and each passage is embedded.
- **Same model, same version** for documents and queries. Changing model means **re-embedding everything**: vectors from different models are not comparable. Some models also expect prefixes such as 'query:' and 'passage:'.
- **Metric**: cosine, dot product or L2, as the model card says. For unit-length vectors all three rank identically, so many systems normalise and use dot product, the cheapest.

3 Exact vs approximate nearest neighbours

Exact kNN (brute force) compares the query with every vector and keeps the k closest. It is perfectly accurate and fine for tens of thousands of vectors, or after a filter has already narrowed the set. But 10 million 768-dimension vectors is about 7.7 billion multiply-adds per query, plus reading ~30 GB of floats. **Approximate nearest neighbour (ANN)** indexes visit a tiny fraction of the vectors and return most of the true neighbours. Quality is measured as **recall@k**: the share of the true top k (from exact search) that the ANN result contains. Teams commonly target around 95%, which users rarely distinguish from exact search.

Approach	Work per query	Recall	Memory	Notes
Flat / exact	all N vectors	100%	vectors only	baseline for measuring recall
HNSW	~ a few thousand distance computations, ~log N hops	high (tunable)	vectors + graph links	default in Lucene/Elasticsearch, OpenSearch, pgvector, Qdrant, Weaviate
IVF (IVFFlat)	nprobe lists of ~N/nlist vectors	tunable via nprobe	vectors + centroids	needs training (k-means); often paired with PQ (lesson 23)
DiskANN-style	graph walk, vectors on SSD	high	mostly on disk	for billion-scale on one machine

4 HNSW: a layered graph

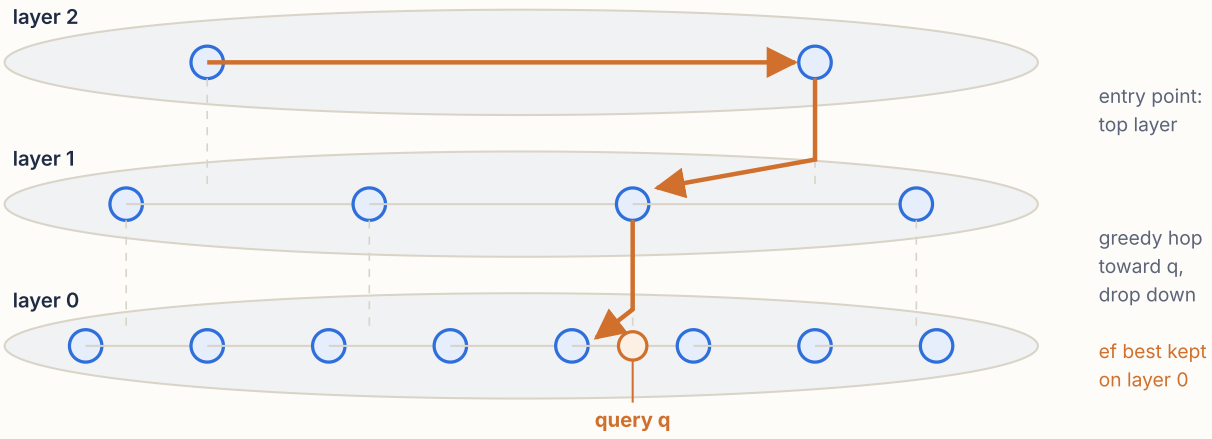
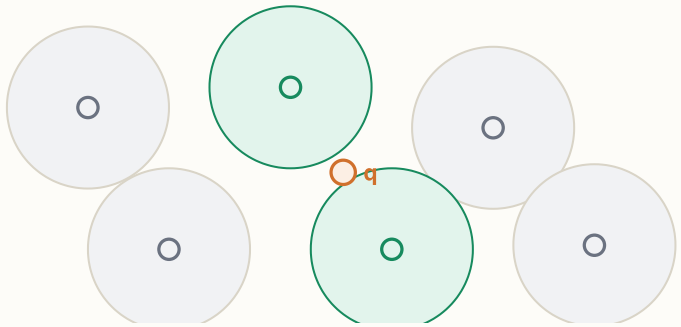


Figure 2. HNSW search: start at the top layer's entry point, hop greedily toward q, drop a layer, repeat; on layer 0 keep the ef best candidates.

Hierarchical Navigable Small World (Malkov and Yashunin, 2016) links each vector to about M of its near neighbours. Each node also gets a random maximum layer, with the probability falling exponentially per layer, so upper layers are sparse 'express lanes' and layer 0 contains everything. A search starts at a fixed entry point on the top layer, repeatedly moves to whichever neighbour is closer to the query, and when no neighbour is closer, drops a layer. On layer 0 it does a best-first search keeping a candidate list of size **ef** and returns the best k. Inserts use the same search to find each new node's neighbours, so HNSW supports incremental adds; deletes are usually tombstones, and heavy churn degrades the graph until it is rebuilt. Its costs: the whole graph and vectors want to live in RAM, and builds are slow. Tuning M, ef_construction and ef_search is lesson 23.

5 IVF: only search nearby clusters



1. offline: k-means → nlist centroids
each vector stored in its nearest list

2. query: find nprobe nearest centroids
(here 2), scan only those lists

3. rule of thumb: nlist ≈ √N to 16√N;
raise nprobe for recall

a true neighbour in an unprobed cell is missed

Figure 3. IVF partitions the space with k-means. Searching 2 of 6 cells does ~1/3 of the work; more probes, more recall.

The **inverted file** index clusters vectors into `nlist` cells at training time. A query computes distances to the centroids, picks the `nprobe` closest cells and scans only their vectors. It is simpler and lighter on memory than HNSW and combines naturally with compression, but needs representative training data, and recall drops for queries near cell borders unless nprobe is raised.

WORKED EXAMPLE: SEMANTIC SEARCH OVER SUPPORT ARTICLES

2,000 help articles are split into ~15,000 passages of a few hundred tokens. Each passage is embedded once with the same model (say 768 dimensions, normalised) and stored with its article ID in an HNSW index; at this size the vectors are only ~46 MB. A customer's question is embedded with the same model (~10–50 ms on a hosted API, less locally), and the index returns the top 5 passages by dot product in a few milliseconds. 'My package says delivered but it isn't here' now retrieves the 'Missing shipment claims' passage without sharing a keyword. The page groups passages by article and shows the top 3 articles. When the team later upgrades the embedding model, they re-embed all passages into a new index and switch over behind an alias (lesson 19).

TRADE-OFFS

- **Meaning vs precision**: great for paraphrases; weak for exact codes, names and rare jargon (lesson 24 combines both).
- **Exact vs ANN**: perfect recall vs orders-of-magnitude speed.
- **HNSW vs IVF**: higher recall at low latency vs less memory and simpler builds.
- **Bigger models** embed better but cost more per query, per vector and in re-embedding time.

COMMON MISTAKES

- Embedding whole long documents as one vector, so specific passages can't be found.
- Mixing vectors from two model versions in one index.
- Using a metric the model wasn't trained for (L2 on unnormalised cosine-trained vectors).
- Never measuring recall against exact search, so a misconfigured index goes unnoticed.

6 Key terms

Embedding: vector representation of meaning from a model

kNN / ANN: exact / approximate k nearest neighbours

HNSW: multi-layer proximity graph, greedy search

Cosine similarity: angle between vectors, ignores length

Recall@k: share of the true top k that ANN returns

IVF: k-means cells; search only nprobe nearest cells

7 Interview questions

Q1. How does HNSW work at a high level? BASIC

It builds a layered graph of vectors linked to their neighbours; a search starts at the sparse top layer, moves greedily toward the query and drops down to denser layers, keeping ef candidates at the bottom.

Q2. Exact vs approximate search? BASIC

Exact compares every vector and is perfectly accurate but slow at scale; approximate indexes visit a small fraction and trade a little recall for huge speed.

Q3. Cosine, dot product or L2? BASIC

Whichever the embedding model was trained for; on normalised vectors they produce the same ranking, so dot product is typical.

Q4. You switch to a better embedding model. What has to happen? SCENARIO

Re-embed every document with the new model into a new index, because old and new vectors live in different spaces, then cut over (e.g. via an alias) and re-tune.

Q5. Only 20,000 vectors. Do you need an ANN index? SCENARIO

Probably not: brute force over 20,000 vectors takes milliseconds and gives exact results; ANN pays off at hundreds of thousands to millions.

Q6. Recall is 95%. Does that mean 5% of answers are wrong? TRICKY

No: 5% of the true top-k neighbours are replaced by slightly farther ones, which are usually still relevant; measure end-task quality too.

Q7. Why does vector search fail on 'XB-2017'? TRICKY

Embeddings capture meaning, and near-identical codes like XB-2017 and XB-2019 map to nearly the same vector; exact tokens need keyword search or hybrid (lesson 24).

Next: SD-P-010 · Vector Index Tuning backendroadmap.com

Vector Index Tuning

recall, latency and memory: pick two, tune the third

Watch the 4-min video

In one line: HNSW's **M** and **ef_construction** fix graph quality at build time and **ef_search** trades recall for latency per query; memory is usually the real limit, so **quantise** (int8, binary, PQ) and **re-score** the top candidates with full vectors; and use engines that **filter during the graph search**.

1 From demo to 50 million vectors

Semantic search works in the demo. Then you load 50 million 1,000-dimension embeddings: the index needs over 200 GB of RAM, queries take 200 ms, and some obvious matches go missing. Vector indexes don't tune themselves. A handful of parameters decide **recall**, **latency** and **cost**, and the only reliable way to set them is to measure on your own data.

2 HNSW's three knobs



Changing M or ef_construction means rebuilding the index; ef_search can change per request.

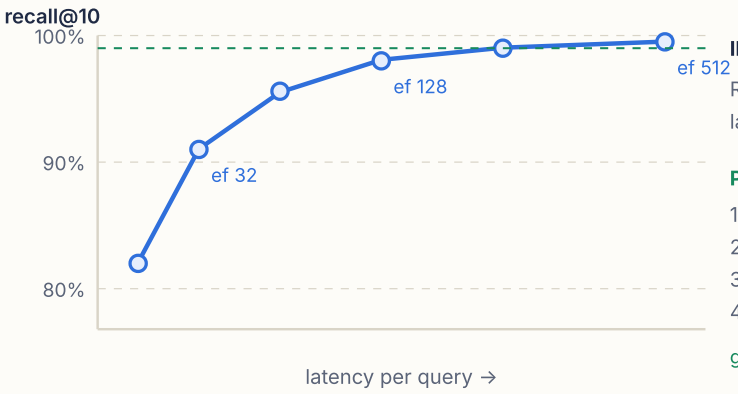
Figure 1. Build-time knobs shape the graph; the runtime knob decides how much of it each query explores.

- **M** is how many neighbours each node links to (layer 0 usually allows 2·M). More links give a better-connected graph and higher recall, especially for high-dimensional or clustered data, but more memory and slower builds and inserts.
- **ef_construction** is the candidate-list size used while choosing each new node's links. Higher values find better neighbours and a higher recall ceiling, paid once at build time. Worth raising if builds aren't on a critical path.
- **ef_search** (called `num_candidates` per shard in Elasticsearch kNN queries, `hnsw.ef_search` in pgvector) is the candidate-list size during a query. It must be at least k. It is the knob you tune first, because it needs no rebuild.

Engine	M default	ef_construction default	query-time knob (default)
hnswlib	16	200	`ef` (10)
Lucene / Elasticsearch	16 (`m`)	100	`num_candidates` per query
pgvector	16 (`m`)	64	`hnsw.ef_search` (40)

Quote these as starting points only; in an interview it is stronger to say *'I'd benchmark M and ef on our data against exact search'* than to quote magic numbers.

3 Trading recall for latency



Illustrative shape, not a benchmark.
Recall rises fast, then flattens;
latency keeps growing roughly linearly.

Procedure:

1. sample ~1,000 real queries
2. exact top-10 by brute force
3. sweep ef_search, measure recall, p99
4. pick the lowest ef that hits target

green line = recall target (e.g. 95%)

Figure 2. The recall–latency curve for ef_search. Choose the knee that meets your recall target, not the maximum.

Recall is always measured against **exact search on a sample**: run real queries brute force to get the true top 10, then compare. Watch **p99** latency, not just the mean, and repeat after big data changes. Sharding also matters: each shard returns its own top candidates, so more shards mean more total work per query but smaller graphs; searching fewer, larger segments is faster than many small ones (in Lucene each segment has its own graph, so force-merging read-only indexes helps).

4 Memory: the real limit

```
RAM ≈ N × dims × bytes per dim + N × ~2M × 4 B (links)
```

A 1,000-dim float32 vector is 4 KB, so 50 M of them are 200 GB before the graph. HNSW is fast only when its working set is in RAM (random access hops), so memory sets the hardware bill. Ways to shrink it:

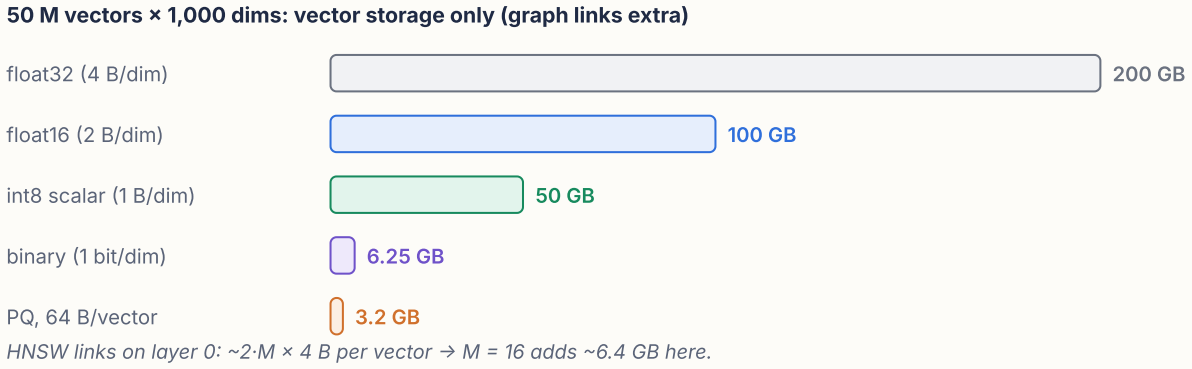


Figure 3. Same 50 M vectors under different encodings. Compression ratios are exact; the recall cost depends on the data and the model.

- **Scalar quantisation (int8)**: map each float dimension to 8 bits using per-dimension or per-segment min/max. 4× smaller, usually a small recall loss. Elasticsearch's `int8_hnsw` index option and OpenSearch/Faiss support it.
- **Binary quantisation**: 1 bit per dimension, 32× smaller, distances via fast bit operations; needs re-scoring and works best with high-dimensional models.
- **Product quantisation (PQ)**: split the vector into m sub-vectors (e.g. 64 chunks of ~16 dims), learn 256 centroids per chunk, store one byte per chunk: 64 bytes instead of 4,000, about 60×. Lossier; usually combined with IVF (IVF-PQ in Faiss).

- **Fewer dimensions**: some models (e.g. OpenAI's text-embedding-3) let you request shorter vectors; a smaller model may be enough.
- **Re-scoring**: search the compressed index for, say, 100 candidates, then recompute exact distances with the full float vectors kept on disk and return the best 10. This recovers most of the lost recall for a little extra I/O.

5 Filtered vector search

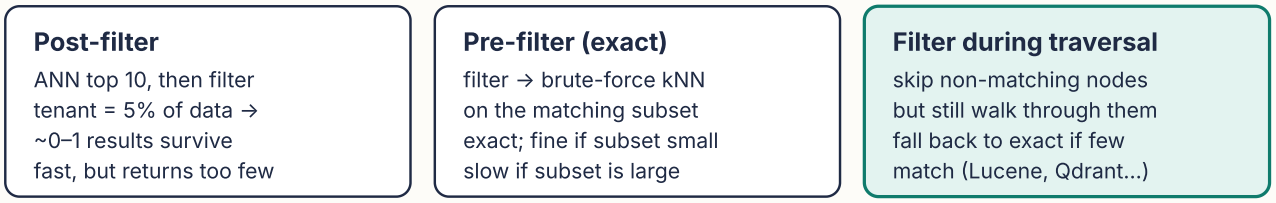


Figure 4. Three ways to combine 'similar to X' with 'tenant = 42 and language = en'.

Real queries have filters: this tenant, this language, in stock. **Post-filtering** the ANN result can leave nothing when the filter is selective; asking for more candidates only partly helps. **Pre-filtering** then brute-forcing is exact and cheap only if the subset is small. Modern engines filter **during** the graph walk: non-matching nodes are traversed but not returned, and when the filter matches only a few documents, the engine switches to exact search over them (Lucene does this based on the filter's cardinality). For multi-tenant systems with very selective tenant filters, a separate index or partition per large tenant (lesson 41) is often the cleanest answer.

WORKED EXAMPLE: 4× LESS MEMORY WITH INT8

A team stores 40 M product embeddings at 1,000 dims: 40 M × 4 KB = **160 GB** of floats, plus ~5 GB of links at M = 16. They switch to int8 scalar quantisation and keep the float vectors on disk: in-memory vectors drop to **40 GB**. Measured recall@10 dips slightly against exact search. So each query fetches the top **100** candidates from the int8 index and re-scores them with the full vectors from disk (100 × 4 KB = 400 KB of reads, mostly from page cache), returning the best 10. Recall@10 is back to **98%**, p99 latency barely moves, and the cluster runs on roughly a quarter of the RAM.

TRADE-OFFS

- **Higher M**: better recall on hard data, but more memory per vector and slower inserts.
- **Higher ef_search**: more recall, linearly more latency and CPU.
- **Quantisation**: 4–60× less memory, some recall loss; re-scoring buys most of it back.
- **In-RAM HNSW vs disk-based indexes** (DiskANN-style): latency vs cost at billion scale.

COMMON MISTAKES

- Tuning without a ground-truth recall measurement against exact search.
- Raising ef_search to fix bad results that are really caused by the embedding model or chunking.
- Post-filtering a selective tenant filter and returning empty pages.
- Sizing RAM for the vectors but forgetting graph links, replicas and headroom for merges/rebuilds.

6 Key terms

M: max links per node in the HNSW graph

ef_construction: candidate list size while building

ef_search: candidate list size per query (≥ k)

Product quantisation: sub-vectors coded by learned centroids

Scalar quantisation: float → int8 per dimension, 4× smaller

Re-scoring: exact distances on top candidates

7 Interview questions

Q1. How do you trade recall for latency in a vector index? BASIC

M and ef_construction shape the graph at build time; ef_search sets how much each query explores. Measure recall against exact search and pick the cheapest setting that meets the target.

Q2. How do you cut vector memory? BASIC

Quantise: int8 scalar for about 4×, binary or product quantisation for much more, and re-score the top candidates with full vectors.

Q3. Recall is 88% and the target is 95%. What do you try, in order? SCENARIO

Raise ef_search (no rebuild) and check the latency; if it plateaus below target, rebuild with higher M or ef_construction; also check quantisation and re-score more candidates.

Q4. Estimate RAM for 100 M × 768-dim float vectors with HNSW M = 16. SCENARIO

Vectors: 100 M × 768 × 4 B ≈ 307 GB; links ≈ 100 M × 32 × 4 B ≈ 13 GB; so ~320 GB per copy before replicas and headroom, or ~90 GB with int8.

Q5. Tenant-filtered queries return 2 results instead of 10. Why? SCENARIO

Post-filtering: the ANN top candidates mostly belong to other tenants. Filter during traversal, raise candidates, or use per-tenant indexes/partitions for large tenants.

Q6. Why can a higher ef_search not fix poor results? TRICKY

It only brings ANN closer to exact search; if exact search itself returns poor matches, the problem is the embeddings, chunking or query, not the index.

Q7. Why does quantisation plus re-scoring work so well? TRICKY

Quantised distances are good enough to get the true neighbours into a top-100 shortlist; exact distances then fix their order, so you pay full-precision cost only for 100 vectors.

Next: SD-P-011 · Hybrid Search backendroadmap.com

Hybrid Search

keywords for precision, vectors for meaning

Watch the 4-min video

In one line: run **BM25** and **vector** search in parallel and merge the two ranked lists, usually with **Reciprocal Rank Fusion** (score = Σ 1/(k + rank), k ≈ 60), which needs no score normalisation; optionally re-rank the top results with a **cross-encoder**. Each method covers the other's blind spot.

1 Each search fails where the other shines

A shopper searches for the exact product code **XB-2017**; vector search returns semantically similar products (XB-2019, XB-2018) but not that one. Another types **something to keep my coffee hot on the go**; keyword search finds nothing, because no product title uses those words. A production search system has to handle both kinds of query, often without knowing in advance which one it got.

Keyword (BM25) wins
codes, SKUs, part numbers, error messages
names, rare or brand-new jargon
short precise queries; legal/medical terms
explainable: you can see what matched

Vector wins
long natural-language questions
paraphrases, synonyms, vague intent
cross-language queries (multilingual models)
no exact word overlap with the answer

Figure 1. Strengths of each retriever. Hybrid search is the default answer when queries mix both kinds.

Why vectors miss codes: an embedding compresses a whole text into a few hundred numbers that capture meaning; 'XB-2017' and 'XB-2019' differ by one character and mean almost the same thing to the model, so their vectors nearly coincide. BM25 (lesson 17) treats `xb-2017` (kept intact by a keyword or code-aware analyzer) as a rare term with a high IDF, so the exact product wins by a mile. Conversely, BM25 can't connect 'coffee hot' to 'insulated travel mug' unless someone wrote that synonym.

2 Architecture

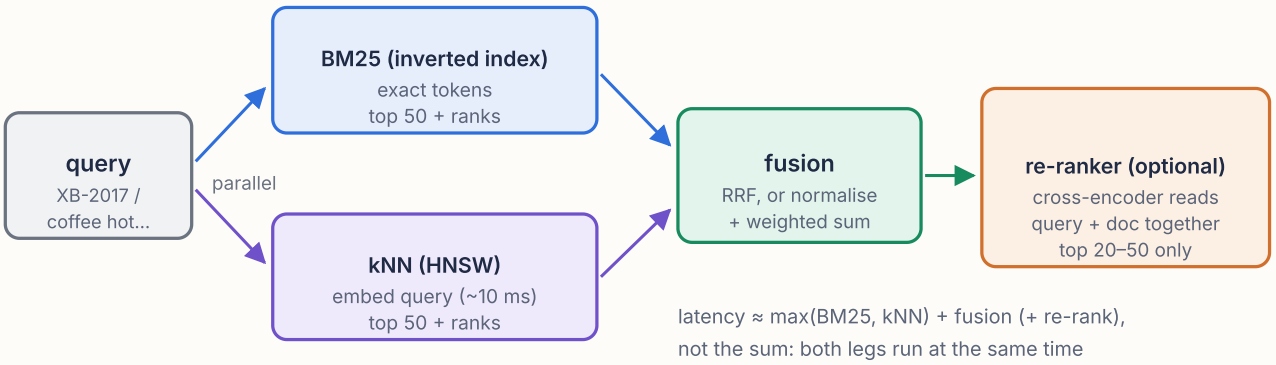


Figure 2. Hybrid query: two retrievers in parallel, a fusion step, and an optional re-ranker on the short list.

Both legs retrieve a candidate window (e.g. top 50–100 each), with the same filters applied to both (tenant, language, stock). Many engines run this as one request: Elasticsearch's `rrf` retriever,

OpenSearch's `hybrid` query with a normalisation search pipeline, and built-in hybrid modes in vector databases such as Weaviate, Qdrant and Vespa. A further option on the keyword side is **learned sparse retrieval** (SPLADE, Elastic's ELSER): a model expands text into weighted terms, adding some semantic matching while keeping an inverted index.

3 Why you can't just add the scores

BM25 scores are unbounded and depend on the query (12.5 for one query, 41 for another). Cosine similarity sits in a narrow band (e.g. 0.79–0.83 for the top 10). Adding them lets BM25 dominate completely. **Weighted (convex) fusion** first normalises each list, then mixes, typically with min–max normalisation norm(x) = (x − min)/(max − min) per result list:

$$s = \alpha \cdot \mathrm{norm}(s_{vec}) + (1 - \alpha) \cdot \mathrm{norm}(s_{BM25})$$

With α = 0.7 the mix is 70% semantic, 30% keyword. It can beat RRF when tuned on a judgement set (lesson 17), because it keeps information about *how much* better one result is. But min–max normalisation is fragile: it stretches the tiny cosine differences 0.79–0.83 across the full 0–1 range, and one outlier score squashes everything else. α also depends on the query type, so it needs tuning and re-tuning when models change.

4 Reciprocal Rank Fusion

$$\mathrm{RRF}(d) = \Sigma_{lists}\ 1 \,/\, (k + \mathrm{rank}(d)),\ k = 60$$

RRF (Cormack, Clarke and Büttcher, 2009) ignores raw scores and uses only ranks, so the two lists never need to be comparable; a document missing from a list simply adds 0 for it. k damps the advantage of the very top positions: with k = 60, rank 1 is worth 0.0164 and rank 10 still 0.0143, so appearing in **both** lists matters more than being first in one. Being ranked 1st and 5th (1/61 + 1/65 = 0.0318) is almost the same as 3rd and 3rd (2/63 = 0.0317). There is nothing to train and it is robust when you change models, which is why it is the common default; if you have judged queries, test whether tuned weighted fusion does better on your data.

Product (query 'XB-2017')	BM25 rank	vector rank	RRF score	final
XB-2017 travel mug	1	3	1/61 + 1/63 = 0.0323	**1**
XB-2017 replacement lid	2	6	1/62 + 1/66 = 0.0313	2
XB-2019 travel mug	—	1	1/61 = 0.0164	3
XB-2018 travel mug	—	2	1/62 = 0.0161	4

5 Re-ranking the top results

Both retrievers are **bi-encoders** or term matchers: documents are encoded independently of the query, which is what makes them indexable. A **cross-encoder** re-ranker feeds the query and one candidate through a transformer *together*, so it sees how every query word relates to every document word. It is far more accurate and far slower (a model call per candidate), so it runs only on the fused top 20–50, often adding tens of milliseconds on a GPU or via a hosted re-rank API. This retrieve → fuse → re-rank funnel is also the standard retrieval stack for RAG.

WORKED EXAMPLE: EXACT CODES AND FUZZY INTENT IN ONE SEARCH BOX

Every query runs BM25 (on a `name` text field plus a `sku` keyword field) and kNN (on 768-dim product embeddings) in parallel, each returning its top 50 with the same in-stock filter. RRF with k = 60 merges them and the page shows the top 20. **'XB-2017'**: BM25 ranks the exact product 1st and its lid 2nd; the vector side ranks XB-2019 first, but the exact product, present in both lists, wins (table above). **'something to keep my coffee hot on the go'**: BM25 matches loosely on 'coffee' (coffee beans, coffee grinders), while the vector side ranks insulated travel mugs and thermos flasks at the top; mugs appear in both lists at moderate ranks and rise to the top after fusion. Latency is the slower of the two legs plus a millisecond of fusion.

TRADE-OFFS

- **RRF**: no tuning, robust; discards score magnitude, so a clear BM25 winner gets no extra credit.
- **Weighted fusion**: can be more accurate when tuned; fragile normalisation, α per query type.
- **Cross-encoder re-ranking**: big quality gains; costs latency and GPU or API spend per query.
- **Two indexes**: more storage and two pipelines to keep in sync (lesson 19).

COMMON MISTAKES

- Adding raw BM25 and cosine scores.
- Applying filters to only one leg, so fused results include forbidden or out-of-stock items.
- Analysing product codes with the standard analyzer, which splits 'XB-2017' into 'xb' and '2017'.
- Choosing hybrid weights by gut feel instead of judged queries and A/B tests.

6 Key terms

Hybrid search: keyword and vector retrieval, fused

Weighted fusion: normalise scores, mix with weight α

Cross-encoder: re-ranker reading query + doc together

RRF: Σ 1/(k + rank) across lists, k ≈ 60

Min–max normalisation: rescale a list's scores to 0–1

Learned sparse: model-weighted terms in an inverted index

7 Interview questions

Q1. Why combine keyword and vector search? BASIC

Keyword search is precise for exact terms like codes and names, vector search understands meaning and paraphrases; together they cover each other's blind spots.

Q2. Explain RRF. BASIC

Each document scores the sum of 1/(k + rank) over the result lists it appears in (k ≈ 60), so rankings merge without comparing incompatible scores.

Q3. Users of an internal docs search complain that error codes don't find the right page after you moved to embeddings. Fix? SCENARIO

Add a BM25 leg with a keyword or code-preserving analyzer and fuse with RRF; exact tokens like error codes are where vectors are weakest.

Q4. You have 500 judged queries. RRF or weighted fusion? SCENARIO

Evaluate both offline with NDCG; weighted fusion may win when tuned, but keep RRF if the gain is small, because it's robust to model and data changes.

Q5. Hybrid quality is good but you want more. Next step? SCENARIO

Add a cross-encoder re-ranker on the fused top 20–50, measure NDCG and latency, and cache or batch re-rank calls.

Q6. Why is k = 60 and not 1? TRICKY

With k = 1, rank 1 (0.5) dwarfs rank 2 (0.33), so one list's top hit dominates; k = 60 flattens the curve so agreement between lists matters most. 60 is the value from the original paper and works well broadly.

Q7. Doesn't hybrid search double latency? TRICKY

No: the legs run in parallel, so latency is roughly the slower leg plus fusion; cost (CPU, storage) is what roughly doubles.

Next: SD-P-012 · Search Scaling

Search Scaling

reads scale with copies, writes with shards

Watch the 4-min video

In one line: every search is **scatter-gather** across shards, so the slowest shard sets latency; scale **reads** with replicas, routing, partitioning and caching, and scale **writes** with more primaries, bulk requests and longer refresh intervals, because every write lands on a primary and all of its replicas.

1 The campaign spike

A cluster serves 500 queries per second at 40 ms. Marketing launches a campaign; traffic jumps to 10,000 QPS and latency climbs to 2 seconds, while a nightly bulk import is hammering the same nodes. Search reads and writes compete for the same CPU, disk and page cache, and they scale with different levers. Knowing which lever helps which side is the whole lesson.

2 How a query runs: scatter-gather

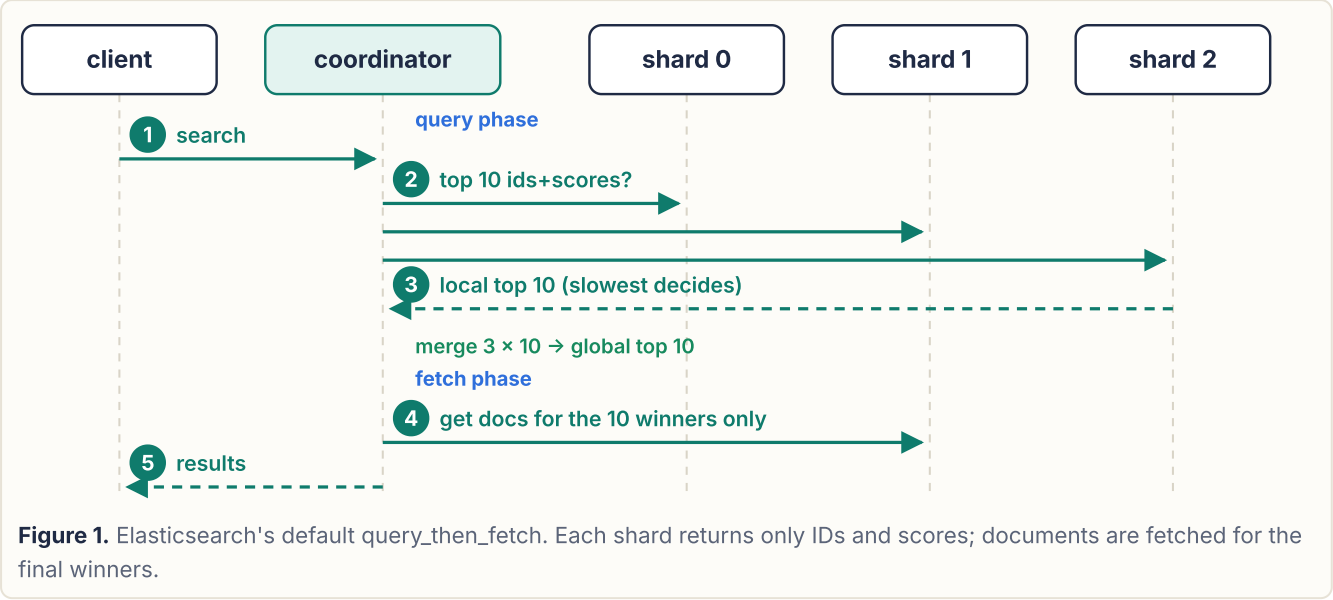


Figure 1. Elasticsearch's default query_then_fetch. Each shard returns only IDs and scores; documents are fetched for the final winners.

The coordinating node sends the query to **one copy of every shard** of the target indices. Each shard scores locally and returns its top `from + size` IDs and scores; the coordinator merges them, then fetches the full documents for the winners only. Three consequences:

- **The slowest shard decides.** A query is as slow as its slowest shard copy: a GC pause, a merge or a hot node on any one of them delays the whole response.
- **Fan-out amplifies tails.** If 1% of shard requests are slow, a query touching 20 shards is slow 18% of the time.
- **Deep pagination is expensive.** Page 1,000 (`from = 10,000`) makes every shard return 10,010 hits and the coordinator sort 20 × 10,010. Elasticsearch caps `from + size` at 10,000 (`index.max_result_window`); use `search_after` with a point-in-time for deep scrolling.

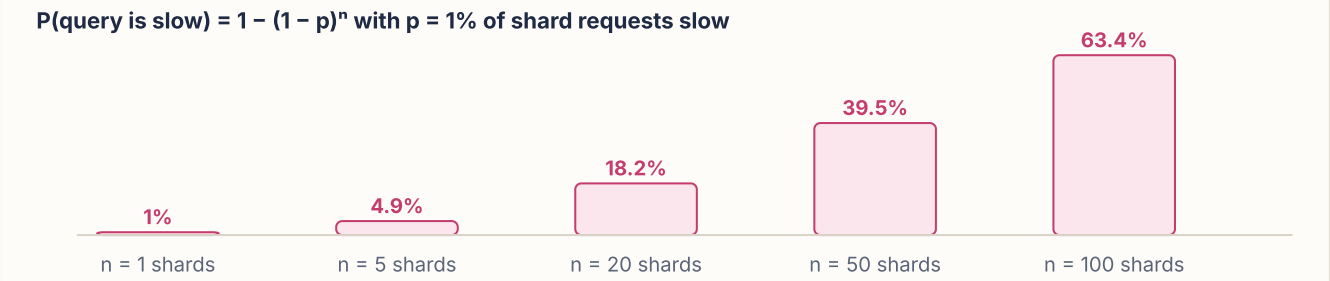


Figure 2. Tail amplification with fan-out ('The Tail at Scale', Dean and Barroso). Fewer shards per query means fewer chances to hit a slow one.

3 Scaling reads

- **Replicas**: each replica is a full copy that can answer queries, so 1 primary + 3 replicas gives roughly 4× the query capacity *if* the extra copies sit on extra nodes. Replicas on the same overloaded nodes add nothing.
- **Adaptive replica selection** (on by default since Elasticsearch 7) sends each shard request to the copy with the best recent response time and queue size, steering around slow nodes.
- **Fewer shards per query**: partitioning and routing (below). Also avoid oversharding: 100 tiny shards per query is 100 tasks of fixed overhead.
- **Memory for the page cache**: Lucene reads segments through the OS cache, so keep JVM heap ≤ 50% of RAM (lesson 16) and size nodes so the hot data fits in what's left.
- **Cheaper queries**: filters instead of scoring clauses (lesson 18), no leading wildcards, sensible aggregation sizes, `track_total_hits` left at its default.

4 Scaling writes is different

Every document is indexed on its primary and then **re-indexed on every replica** (Elasticsearch replicates documents, not files), so adding replicas makes writes **more** expensive. Write levers:

- **More primary shards** (on more nodes) spread indexing; choose at creation or via rollover (lesson 16).
- **Bulk requests**: hundreds to thousands of docs per `_bulk` call cut per-request overhead dramatically (lesson 19).
- **Longer refresh interval** (e.g. 30 s, or -1 during a one-off load): fewer tiny segments, less merging.
- **Replicas to 0 during a one-off load**, then back to 1+: replicas then copy finished segments instead of re-indexing each document.
- **Separate the work**: run imports off-peak, or on separate hot nodes. OpenSearch's **segment replication** copies segments to replicas instead of re-indexing, cutting replica CPU; Elastic's serverless offering separates indexing and search tiers entirely.

Lever	Reads	Writes
Add replicas (on new nodes)	↑ capacity, ↑ availability	↓ every write done N+1 times
Add primaries	more fan-out per query	↑ parallel indexing
Routing / partitioning	↑ fewer shards per query	neutral (watch hot shards)
Longer refresh interval	results less fresh	↑ fewer segments, less merging
Caching	↑ repeated queries skip the engine	refreshes invalidate shard caches

5 Partitioning and routing

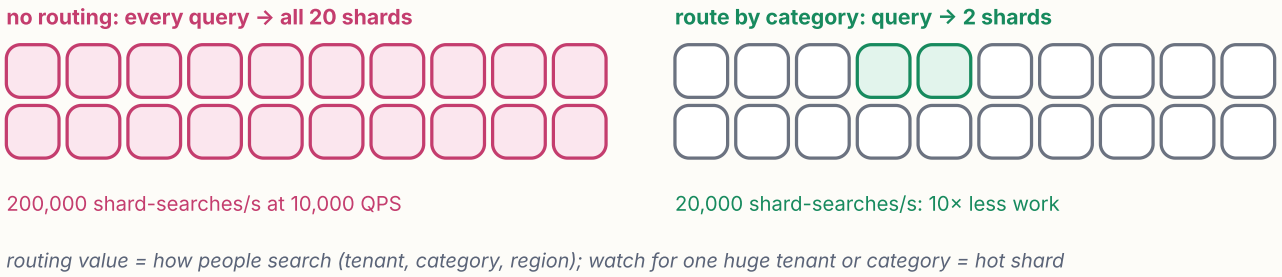


Figure 3. Routing by a key that matches how people search turns a 20-shard fan-out into a 2-shard lookup.

Time-based indices (lesson 16): a query for the last 24 hours touches today's index, not a year of data. **Custom routing**: index each document with `routing = tenant_id` (or category) and pass the same value at query time; the query goes only to the shard(s) holding that key. `index.routing_partition_size` spreads one key over several shards to soften hot spots. **Per-tenant indices** isolate big tenants entirely (lesson 41) but multiply shard counts if overused. Routing only pays off when most queries carry the routing key; a query without it still has to visit every shard.

6 Caching

Inside the engine: the **node query cache** for filters, the **shard request cache** for aggregation-only results, and the OS page cache for segments (lesson 18). In front of it: an application cache (Redis) or CDN for the most popular queries, keyed by the normalised query, filters and page. Search traffic is usually skewed, so a small set of head queries can be a large share of requests, but measure the hit rate: personalised or long-tail traffic may hit rarely, and a 5% hit rate only adds complexity and staleness.

WORKED EXAMPLE: 10,000 QPS AT ABOUT 50 MS

The product index has 20 primaries. Before: 1 replica, every query touches all 20 shards, so 10,000 QPS means **200,000 shard-searches/s**. Changes: (1) **routing by category**, since almost every product search is within a category; most queries now touch 2 shards, so 20,000 shard-searches/s. (2) **Replicas 1 → 3** on twice as many nodes, giving 4 copies to spread that load. (3) The **top 1,000 queries cached for 60 s** in Redis; if, say, 30% of traffic hits, the engine sees ~7,000 QPS. (4) **Bulk imports** move to night with `refresh_interval: 30s`, so daytime nodes only serve searches. Load tests at 10,000 QPS now show p50 around 50 ms with headroom; the remaining p99 spikes trace to merges, which the night-time imports also reduce.

TRADE-OFFS

- **More replicas**: read capacity and resilience vs write cost and disk.
- **More shards**: write parallelism vs per-query fan-out and tail latency.
- **Routing**: far less work per query vs hot shards from skewed keys.
- **Result caching**: big savings for head queries vs staleness and invalidation.

COMMON MISTAKES

- Adding replicas to fix slow indexing, which makes it slower.
- Adding replicas without adding nodes, so the same hardware just holds more copies.
- Hundreds of tiny shards per query: fan-out and overhead, not parallelism.
- Allowing unlimited deep pagination instead of search_after.

7 Key terms

Scatter-gather: fan the query out to every shard, merge results

Adaptive replica selection: route to the fastest copy

Tail amplification: more fan-out, more chance of a slow part

query_then_fetch: IDs and scores first, documents for winners

Custom routing: send a key's docs and queries to one shard

search_after: cursor-style deep pagination

8 Interview questions

Q1. How do you scale search reads versus writes? BASIC

Reads scale with replicas on more nodes, routing and caching, because any copy can answer; writes scale with more primaries, bulk indexing and longer refresh intervals, because every write goes to the primary and all replicas.

Q2. Why does partitioning help? BASIC

Queries touch fewer shards, so there is less scatter-gather work per request and fewer chances of hitting a slow shard.

Q3. p50 is fine but p99 is terrible on a 50-shard index. What do you look at? SCENARIO

Fan-out tail amplification: slow nodes (GC, merges, hot shards), adaptive replica selection, and reducing shards per query via routing, time-based indices or fewer, larger shards.

Q4. Indexing throughput dropped after you went from 1 to 3 replicas. Why? SCENARIO

Each document is now indexed on four copies instead of two; for bulk loads drop replicas temporarily, or consider segment replication.

Q5. Users want to page to result 50,000. How? SCENARIO

Not with from/size (capped at 10,000 and expensive); use search_after with a point-in-time, or redesign the UX with filters.

Q6. Would adding more shards make a slow query faster? TRICKY

Only if shards are huge and CPU is idle; otherwise more shards mean more fan-out, more overhead and worse tails. Measure per-shard query time first.

Q7. When does custom routing hurt? TRICKY

When keys are skewed (one giant tenant makes a hot shard) or many queries lack the key and must still hit every shard.

Next: SD-Q-001 · Real-Time Messaging backendroadmap.com

Real-Time Messaging

gateways hold sockets, the backplane crosses servers

Watch the 4-min video

In one line: clients keep WebSocket connections to a fleet of thin **gateways**; a chat service **stores** each message first, looks up the recipient's gateway in a **session registry**, and publishes to that gateway over a **pub/sub backplane**. Sequence numbers, acks and sync-on-reconnect make delivery reliable even when the recipient is offline.

1 Two servers, one chat

Asha is connected to WebSocket server 3; Ben is connected to server 17. She hits send. Server 3 has her message but no socket to Ben. With millions of open connections spread over hundreds of servers, the design must answer three questions: **where is Ben?**, **how does the message cross servers?** and **what if Ben is not there at all?** The target is a delivery time of tens of milliseconds, with no lost or doubled messages.

2 Picking the transport

Transport	Direction	How it works	Use for
WebSocket (RFC 6455)	both ways	HTTP/1.1 `Upgrade` → `101 Switching Protocols`, then framed messages on one TCP (TLS) connection; ping/pong control frames	chat, games, collaboration
Server-Sent Events	server → client	one long HTTP response of `text/event-stream`; browser auto-reconnects with `Last-Event-ID`	live feeds, scores, notifications
Long polling	server → client	request is held open until data arrives, then the client asks again	fallback when sockets are blocked
Mobile push (APNs/FCM)	server → device	OS-level channel that wakes a sleeping app; best effort	offline recipients only (lesson 28)

Chat needs both directions and low overhead per message, so WebSocket is the default. A message frame adds only 2–14 bytes of header, compared with hundreds of bytes of HTTP headers per poll.

3 The building blocks

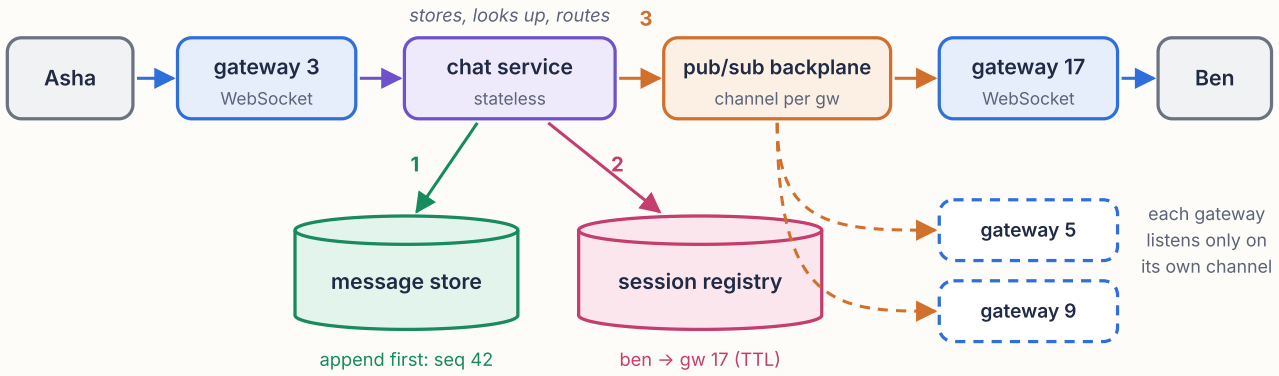


Figure 1. Asha's message: the chat service (1) appends it to the store, (2) asks the registry where Ben is, (3) publishes on gateway 17's channel. Gateways never call each other.

Connection gateways. A gateway terminates the WebSocket, authenticates it once (token at connect time), and moves frames in and out. It holds no business logic, so it is cheap to scale out. Each open connection costs a file descriptor and some kilobytes of buffers, so a tuned server holds hundreds of thousands of connections (WhatsApp reported over 2 million per Erlang server in 2012). Gateways sit behind a layer-4 load balancer; on deploy they are **drained** slowly, and clients reconnect with **jittered exponential backoff** so a restart does not cause a reconnect storm.

Session registry. When Ben connects, the gateway writes `session:ben → {gw-17, conn-881}` into a fast store such as Redis. One user usually has several sessions (phone, laptop, tablet), so store a set or hash per user, not one value. Each entry has a TTL that heartbeats refresh (lesson 27): if gateway 17 crashes without cleaning up, its entries expire on their own.

Backplane. Every gateway subscribes to one channel named after itself (`gw-17`). The chat service publishes to the channels of the recipient's gateways. Options:

- **Redis Pub/Sub:** very fast, but fire-and-forget: a message published while nobody is subscribed is simply gone. That is fine *only* because the store already holds it and the client will sync.
- **Kafka** (or another durable log): replayable and ordered per partition, at the cost of a few more milliseconds.
- **Direct RPC** from chat service to gateway: simplest at small scale, but every service must know every gateway's address.

Large systems often route by *conversation* instead of by user: Slack's published design maps each channel to a **channel server** by consistent hashing, and gateways subscribe to the channels their users have open. Either way, the rule is the same: gateways only listen; they never talk to each other.

4 Message IDs, order and acknowledgements

- **Client message ID** (e.g. a UUID, `m-7f3`) is created on Asha's phone before sending. If the network drops and she retries, the server sees the same ID and does not store a second copy: the send is **idempotent**.
- **Sequence number** is assigned by the server, per conversation (42, 43, 44 ...). It defines one order everyone agrees on, and it lets a client detect gaps. Give each conversation a single writer: e.g. partition by conversation ID so one partition (or one row with an atomic counter) owns the counter. Time-sortable IDs such as Twitter/Discord Snowflakes (timestamp + worker + counter) are another option.

- **Three ticks:** *sent* = stored by the server; *delivered* = Ben's device received it; *read* = displayed on screen. Receipts travel back over the same gateways and backplane. Send a **read watermark** ("read up to 57") instead of one receipt per message: one write covers many messages.

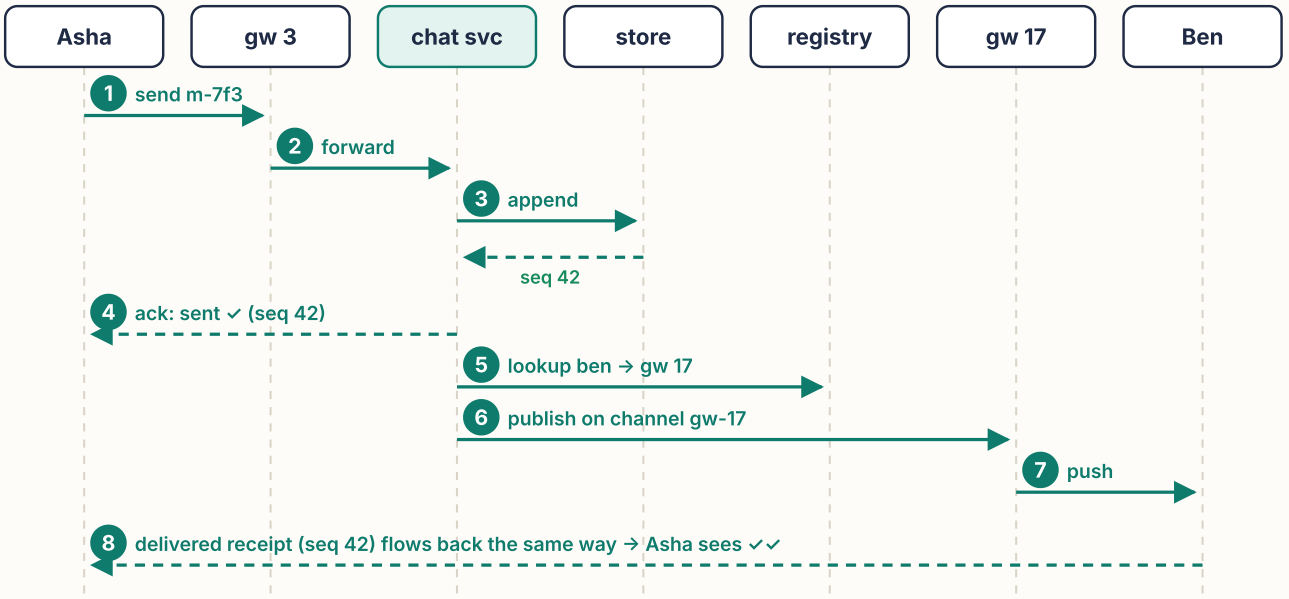


Figure 2. End to end. The message is stored (3) before anything is delivered, so a crash after step 3 loses nothing. The **sent** tick comes from the server; **delivered** and **read** come from Ben's device.

Delivery is **at-least-once** (retries can repeat a push) plus **deduplication** on the client by message ID, which is what users experience as exactly-once. Ordering is guaranteed per conversation only; across conversations nobody needs it.

5 Offline users, push and sync



Figure 3. Sync by sequence number. Because the log is the source of truth, a missed push or dropped socket only delays messages; it never loses them.

If the registry has no session for Ben, the message is already safe in the store. The chat service asks the notification system (lesson 28) to send a push through APNs or FCM, which wakes his phone. Push is best effort and may be collapsed by the OS, so it carries at most a preview and is never the delivery mechanism itself. On reconnect, the app sends its last seen sequence number per conversation (or a single per-user sync cursor across all conversations) and downloads the gap.

6 Groups and storage

Groups: the chat service stores the message once, then looks up every member's sessions and publishes once per *gateway*, not once per member: 300 members spread over 40 gateways means 40 publishes, and each gateway pushes to its local sockets. Very large channels switch to subscription:

gateways subscribe to the channel topic, and members fetch history when they open it. Receipts in big groups are usually aggregated or turned off.

Message store: chat is append-heavy and read by "latest N in this conversation", which fits wide-column stores. Partition by `(conversation_id, time_bucket)` and cluster by message ID, so one conversation's recent messages sit together and a huge, old conversation does not grow one partition forever. Discord stored messages this way in Cassandra and later moved to ScyllaDB.

WORKED EXAMPLE: GATEWAY 3 → GATEWAY 17

Asha sends "hello" with client ID `m-7f3` on gateway 3. The chat service appends it to conversation asha–ben as **seq 42** and acks Asha (one tick). It reads `session:ben` → gateway 17 and publishes the message on channel `gw-17`. Gateway 17 pushes it down Ben's socket; his app stores it, sends *delivered (42)*, and Asha sees two ticks. Each hop is about a millisecond inside one data centre, so the round trip is tens of milliseconds. If Ben's phone had been asleep, the lookup would find no session, a push would go out, and his app would fetch `after=41` on reconnect.

TRADE-OFFS

- Redis Pub/Sub is fastest but loses messages with no subscriber; Kafka is durable but adds latency and operational weight.
- Per-user routing is simple for 1:1 chat; per-channel routing scales better for big groups.
- Short heartbeats detect dead sockets fast but cost battery and traffic on millions of devices.
- Storing before delivering adds a write to the latency path, but it is what makes offline sync and retries safe.

COMMON MISTAKES

- Gateways calling each other directly: an N×N mesh that breaks every time a gateway is added or removed.
- Treating the backplane or a push notification as the source of truth instead of the message store.
- Ordering by client timestamps: phone clocks are wrong; use server-assigned sequence numbers.
- One registry value per user, so logging in on a laptop silently stops delivery to the phone.

7 Key terms

Gateway: server that holds client WebSocket connections and nothing else

Backplane: pub/sub layer that carries messages between servers

Client message ID: ID made on the sender's device, used to dedupe retries

Session registry: user → gateway(s) map with TTL, often in Redis

Sequence number: server-assigned order within one conversation

Sync cursor: last sequence a device has; it fetches everything after it

8 Interview questions

Q1. How do two users on different WebSocket servers exchange messages? BASIC

The sender's gateway hands the message to a chat service, which stores it, finds the recipient's gateway in the session registry, and publishes it over pub/sub to that gateway, which pushes it down the socket.

Q2. What if the recipient is offline? BASIC

It is already stored, so nothing is lost: send a push notification, and on reconnect the client syncs everything after its last sequence number.

Q3. Why WebSocket rather than HTTP polling for chat? BASIC

It is a persistent, bidirectional connection with tiny per-message framing, so the server can push instantly without the latency and header overhead of repeated requests.

Q4. A user is logged in on a phone and a laptop. How does routing change? SCENARIO

The registry stores a set of sessions per user; the service publishes to every gateway that holds one, and each device acks and syncs on its own cursor.

Q5. Gateway 17 crashes with 300,000 connections. What happens? SCENARIO

Clients reconnect with jittered backoff to other gateways, which write new registry entries; old entries expire by TTL; each client syncs from its cursor, so no messages are lost.

Q6. How do you send a message to a 5,000-member group efficiently? SCENARIO

Store it once, then publish once per gateway that has members (or to a channel topic the gateways subscribe to); turn off or aggregate per-member receipts.

Q7. How do you guarantee the same message is never shown twice? TRICKY

You can't avoid duplicate delivery under retries, so make it harmless: a client-generated message ID makes the server write idempotent, and the client dedupes by ID.

Q8. Redis Pub/Sub drops messages when no one is subscribed. Why is it still acceptable here? TRICKY

Because the message store, not the backplane, is the source of truth: a dropped publish only delays the message until the client's next sync.

Q9. How do you keep message order correct? TRICKY

Assign a sequence number per conversation from a single writer (partition by conversation ID); clients sort by it and fetch any gaps, never trusting device clocks.

Next: SD-Q-002 · Presence backendroadmap.com

Presence

heartbeats, expiring keys, careful fan-out

Watch the 4-min video

> **In one line:** clients send periodic **heartbeats** that refresh a **key with a TTL**, so a user goes offline automatically when the beats stop; the expensive part is **fan-out**, which you limit to friends who are online and looking, debounce, and replace with **pull** for large groups.

1 The busiest green dot

A chat app with 100 million users shows a green dot next to friends who are online. People connect, lose signal, close laptops and switch apps every second, and each change might be interesting to hundreds of friends. A rough sum shows the danger: if 50 million users are connected and each changes state about once every 10 minutes, that is ~83,000 changes a second; with 200 friends each, a naive design sends **~17 million presence events a second**, more than all the chat messages combined. Presence is a small feature with a big write and fan-out load.

2 Detecting online and offline

Online is easy: the WebSocket opened (lesson 26). **Offline** is hard, because the most common way to leave is not a clean logout: the phone enters a tunnel, the OS suspends the app, the laptop lid closes. TCP may not notice a dead peer for minutes. So the client sends a tiny **heartbeat** every ~30 s (a WebSocket ping frame or a small app message) and the server treats silence as absence.

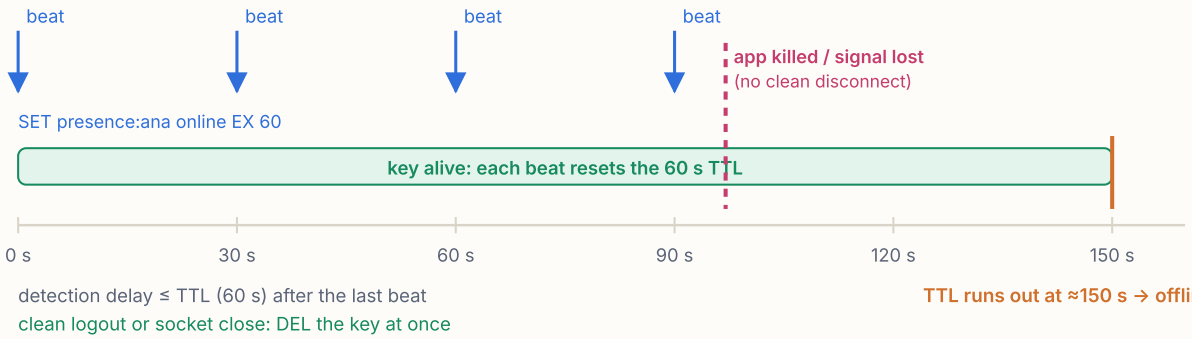


Figure 1. Each heartbeat resets the key's TTL. After the last beat, the key survives at most one TTL, then the user is offline with no cleanup job.

The interval is a trade-off: a 10 s beat detects absence faster but triples the traffic and drains phone batteries (each beat can wake the radio). Set the TTL to about **2× the interval** so one lost beat does not flip the user offline. A clean disconnect (socket close event on the gateway, or logout) deletes the key immediately, so the TTL only matters for crashes and lost signal.

3 Storing presence with expiring keys

In Redis each beat runs `SET presence:42 online EX 60` (or `EXPIRE` on an existing key). While beats arrive, the key lives; when they stop, it disappears. Crashed clients and crashed gateways cannot leave a stale green dot forever. Some details that matter at scale:

- **Shard by user ID.** Presence writes are tiny but constant: 50M users / 30 s ≈ 1.7M writes a second, spread across a cluster.
- **Let gateways batch.** The gateway already sees every beat; it can refresh keys in pipelined batches, or own presence for its sockets and write only on change, using the TTL as a safety net if the gateway itself dies.
- **Expiry events are not instant.** Redis deletes expired keys lazily (on access) and by background sampling, and keyspace notifications fire only when the key is actually removed. Use the TTL for truth, but don't promise friends an offline event to the second.
- **Keep it small.** Store the online flag (and maybe status and device) separately from profile data, so the hottest key is a few bytes.

4 Last seen, statuses and privacy

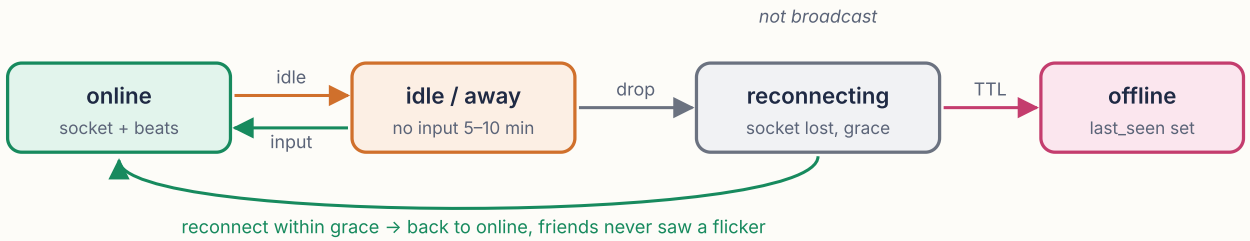


Figure 2. A practical status model. The grace state absorbs flaps (bad signal, app switch) so friends do not see on-off-on.

Last seen is a timestamp of the last activity, kept with the presence data or in the user store. It powers "last seen 10 minutes ago". Update it at most once a minute, not on every beat, and write it durably when the user goes offline. A user with several devices is online if **any** device is; status is the most active one (online beats idle beats offline). Privacy settings ("hide last seen", "show to contacts only") must be applied in the presence service on read, before anything is sent to another user.

5 The fan-out problem and how to limit it

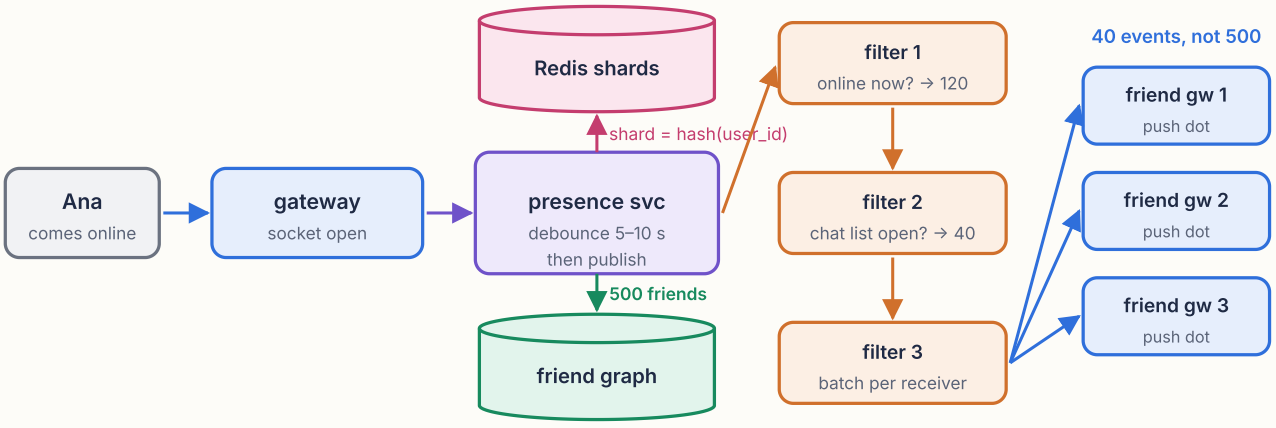


Figure 3. Ana comes online. Instead of 500 events, the presence service filters to friends who are online and have a view that shows her, after a short debounce.

Technique	What it does	Effect
Only online friends	intersect the friend list with the presence store	offline friends will fetch on open
Only interested viewers	clients subscribe to the presence of users visible on screen (open chat, contact list)	often cuts events by 10× or more
Debounce / flap suppression	announce offline only after a grace period (e.g. 10–30 s); ignore on-off-on	removes the noisiest events
Batching	group changes per receiver and send every few seconds	fewer, larger messages
Pull for large groups	no push; client fetches online members when it opens the list, refreshes periodically	cost grows with viewers, not members

For a channel with 50,000 members, pushing every status change to everyone is hopeless (each change × 50,000). Show a count ("1,204 online") from a cheap approximate counter, and load the visible part of the member list lazily as the user scrolls.

WORKED EXAMPLE: ONLINE TO FRIENDS

Ana's app beats every 30 s; each beat runs `SET presence:ana online EX 60` on her shard. When her socket opens, the presence service waits a few seconds (in case she drops straight away), then reads her 500 friends, keeps the ~120 that have a live presence key, and of those the ~40 that have subscribed to her (her chat is on their screen). Each of those gets a small `{user: ana, status: online}` event through their own gateway. When she closes the app without logging out, the beats stop, the key expires within a minute, `last_seen` is written, and those friends see "last seen just now". Total cost: about two tiny writes per user per minute plus a few dozen events per change, which scales horizontally by adding shards.

TRADE-OFFS

- Short heartbeat and TTL: accurate dots, more traffic and battery use. Long: cheaper, but ghosts stay green for longer.
- Push presence: live dots, cost grows with friends × changes. Pull: cheap, but slightly stale.
- Debounce hides flaps but delays real offline transitions by the grace period.
- Strong accuracy is not needed: presence is a hint, so prefer availability and low cost over consistency.

COMMON MISTAKES

- Relying only on logout or socket-close events: crashes and lost signal leave users online forever.
- Writing last_seen to the main database on every heartbeat.
- Broadcasting every change to every friend, including offline ones and huge groups.
- Ignoring privacy settings on the fan-out path, leaking a status the user hid.

6 Key terms

Heartbeat: small periodic message proving the client is still there

TTL key: key that expires unless refreshed; absence = offline

Grace period: delay before announcing offline, to absorb flaps

Last seen: timestamp of last activity, updated coarsely

Presence fan-out: sending a status change to interested users

Subscription: client asks for presence only of users on screen

7 Interview questions

Q1. How do you track online status for 100M users? BASIC

Clients send heartbeats that refresh expiring keys in a sharded store like Redis, so users go offline automatically when beats stop; keep a coarse last-seen timestamp alongside.

Q2. Why use a TTL rather than marking users offline on disconnect? BASIC

Most exits are not clean (lost signal, killed app, crashed gateway); a TTL makes silence mean offline without any cleanup job.

Q3. How do you stop presence updates from exploding? SCENARIO

Fan out only to friends who are online and currently looking, debounce flaps, batch updates per receiver, and let large groups pull presence.

Q4. How would you choose the heartbeat interval? SCENARIO

Balance detection time against traffic and battery: around 30 s with a TTL of about two intervals is common; tighten only where faster detection is worth the cost.

Q5. A user has the app open on a phone and a laptop and closes the laptop. What should friends see? SCENARIO

Still online: presence is per device and the user's status is the most active device; only when all devices expire does last_seen get set.

Q6. Can you rely on Redis keyspace notifications to announce 'offline' exactly when the TTL ends? TRICKY

Not precisely: expired keys are removed lazily or by background sampling, and the event fires on removal, so it can lag. It is fine for a hint, not for strict timing.

Q7. Why is presence usually eventually consistent? TRICKY

It is a hint that changes constantly; a dot a few seconds stale harms nobody, while strong consistency would multiply the write and fan-out cost.

Next: SD-Q-003 · Notification Systems backendroadmap.com

Notification Systems

the right message, once, on the right channel

Watch the 4-min video

In one line: services publish **events**; one notification service resolves recipients, checks **preferences**, renders **templates**, de-duplicates with an **idempotency key** per event-user-channel, and hands the result to **priority queues** whose workers call APNs, FCM, SMS and email providers with retries, failover and per-user rate limits.

1 Twice, at 3 a.m.

Your order ships and your phone buzzes twice with the same message. Then an email arrives at 3 a.m., although you turned email off. Each bug has a different cause: a retried event (duplicate), a missing preference check (email), no quiet hours (3 a.m.). Sending a notification is one API call; sending the *right* one, *once*, on the right channel at the right time is a system.

2 Channels and their rules

Channel	Provider / protocol	Limits and quirks
iOS push	APNs over HTTP/2, token (JWT) or certificate auth	payload up to 4 KB; `apns-priority` 10 (now) or 5 (power-saving); `apns-collapse-id` replaces an earlier alert; HTTP 410 = token no longer valid
Android / web push	FCM HTTP v1 API, OAuth 2 service account	4 KB payload; high vs normal priority; `collapse_key`; `UNREGISTERED` error = delete the token
SMS	aggregators (Twilio, Vonage ...)	160 GSM-7 characters per segment (70 with Unicode); real cost per segment; country rules and sender registration
Email	SMTP or provider API (SES, SendGrid ...)	SPF, DKIM and DMARC for deliverability; handle bounces and complaints; one-click unsubscribe header (RFC 8058) for bulk mail
In-app	your own inbox table + WebSocket (lesson 26)	fully under your control; needs read/unread state

Because each channel differs, producers must never call providers directly. The order service publishes `order_shipped`; the notification service decides who gets what, where. Adding WhatsApp later is one new channel adapter, not a change in fifty services.

3 The pipeline

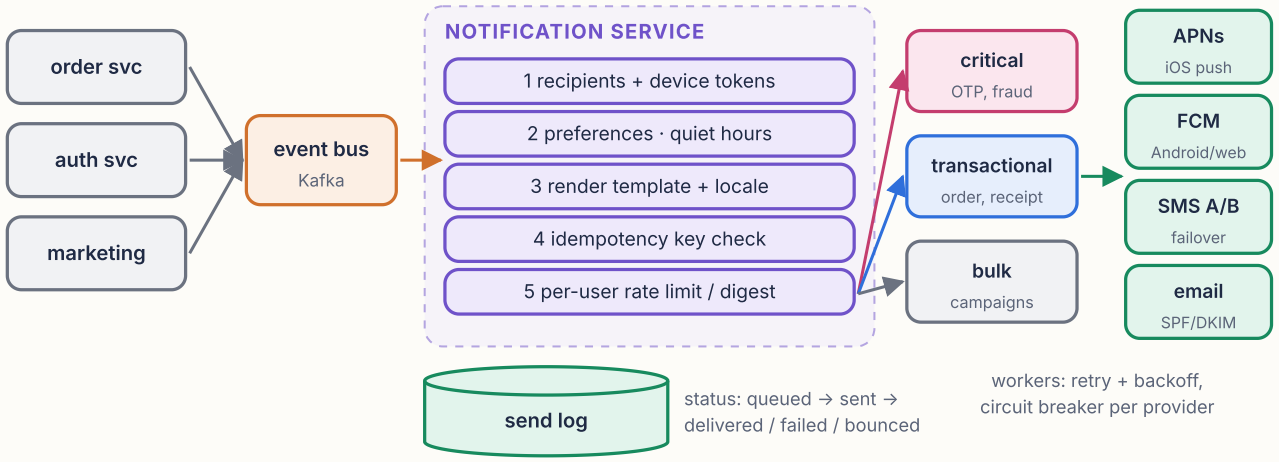


Figure 1. Events come in once; the notification service applies the same five checks to every message, then routes by priority to channel workers that talk to the providers.

Recipients and devices. A device-token registry maps users to their push tokens (one per app install). Tokens go stale when apps are uninstalled; delete them when APNs answers 410 or FCM answers `UNREGISTERED`, or you keep paying for pushes that go nowhere.

Preferences first. Before rendering anything, check: is this category on (security, orders, marketing)? Is this channel on and verified (confirmed email, phone)? Is it quiet hours in *the user's* time zone? Marketing needs explicit consent; security messages such as password resets and fraud alerts usually bypass quiet hours and cannot be turned off. Preferences live in their own store, cached, with safe defaults when the lookup fails (for marketing: don't send).

Templates. The event carries data (`order_id`, `delivery_date`); a template per channel and locale turns it into a 40-character push title, a full HTML email or a 160-character SMS. Templates are versioned outside the code, so copy changes need no deploy, and fall back by locale (`pt-BR → pt → en`). Log the template version with every send.

4 Delivery: priorities, retries, failover, limits

- **Priority queues.** Separate queues and worker pools per priority, so a 10-million-user campaign cannot delay a one-time password. Bulk sends are also throttled to what providers and your own backend (deep links!) can absorb.
- **Retries.** Retry transient errors (5xx, timeouts, 429) with exponential backoff and jitter, honouring `Retry-After`. Never retry permanent errors (invalid token, hard bounce, unsubscribed number); after N attempts, move to a dead-letter queue.
- **Failover.** For SMS and email, keep a second provider behind a circuit breaker: when error rates spike, switch. Push has no alternative provider, so the fallback is another channel (email, SMS) for critical messages.
- **Per-user rate limits and digests.** Cap each user per channel (e.g. a few pushes per hour). Past the cap, collapse into a digest ("5 people liked your photo") instead of fifty separate buzzes.
- **Status tracking.** Record every attempt: queued, sent, delivered (from provider callbacks/webhooks), opened, failed, bounced. Support can then answer "did the user get it?" in seconds.

5 No duplicates: idempotency keys

Event buses deliver **at least once**, and a worker can crash after calling the provider but before recording it. So derive a deterministic key from `(event_id, user_id, channel)` and claim it atomically before sending, e.g. Redis `SET key 'sending' NX EX 604800` or a unique index in the send log. If the key exists, skip.

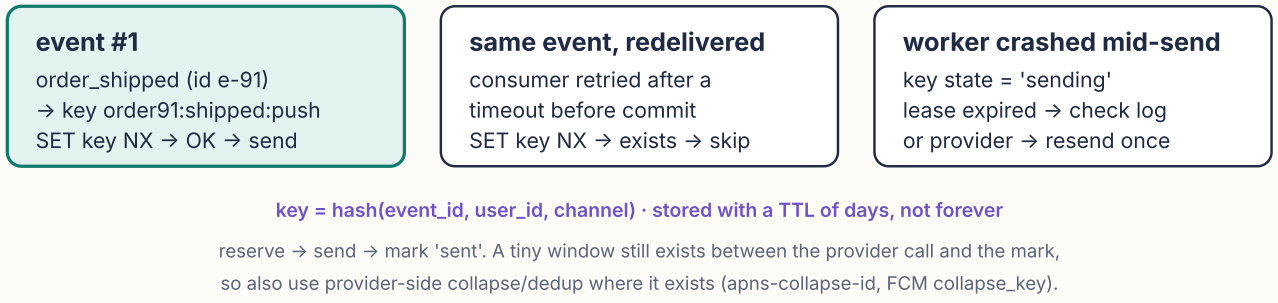


Figure 2. Three cases the key must handle. A plain "record then send" turns a crash into a lost message; "send then record" turns it into a duplicate. A short-lived 'sending' state with a lease narrows both windows.

The key covers *one logical notification*, so a genuinely new event (second shipment) gets a new key, while a redelivered one does not. Where the provider offers it, pass a collapse or dedup ID too: a second push with the same `apns-collapse-id` replaces the first on screen instead of showing twice.

WORKED EXAMPLE: ORDER SHIPPED VIA PUSH + EMAIL

The order service publishes `order_shipped {event_id: e-91, order: 91, user: 7}`. The notification service loads user 7's preferences: orders → push on, email on; it is 14:00 in their time zone, so no quiet-hours hold. It renders the push template ("Your order is on its way") and the email template (with tracking link) in their locale, and claims keys `order91:shipped:push` and `order91:shipped:email`. Both go to the transactional queue; the push leaves through FCM within a second or two, the email through the email provider. Ten minutes later Kafka redelivers e-91 after a consumer rebalance: both keys already exist, both sends are skipped, and the log shows *skipped: duplicate*.

TRADE-OFFS

- Central service: consistent rules and one place to add channels, but a critical shared dependency; give it priority isolation.
- Strict dedup window (days) costs storage; a short window risks duplicates from late redeliveries.
- Aggressive rate limits and digests reduce annoyance but can delay something the user wanted now.
- Multiple SMS/email providers improve availability but double the integration and template testing.

COMMON MISTAKES

- Checking preferences after rendering or after queueing, so a change made minutes ago is ignored.
- Quiet hours in server time instead of the user's time zone.
- One shared queue: the marketing blast delays every password-reset code.
- Never pruning dead device tokens or hard-bounced addresses, which hurts cost and email sender reputation.

6 Key terms

APNs / FCM: Apple and Google push gateways for device notifications

Preference store: per-user, per-category, per-channel opt-ins and quiet hours

Priority queue: separate lanes so critical messages never wait

Device token: per-install address for push; expires on uninstall

Idempotency key: event + user + channel; claimed once, duplicates skipped

Digest: many low-value notifications collapsed into one

7 Interview questions

Q1. What are the main parts of a notification system? BASIC

Events in, recipient and preference checks, per-channel templates, idempotency, priority queues, workers with retries and provider failover, per-user rate limits, and a send log.

Q2. How do you prevent duplicate notifications? BASIC

Give each notification an idempotency key from event, user and channel; claim it atomically before sending and skip keys already seen, while producers and consumers tolerate retried events.

Q3. A 5-million-user marketing campaign is delaying password-reset SMS. Fix it. SCENARIO

Split queues and worker pools by priority, throttle bulk sends, and reserve provider capacity for critical traffic.

Q4. Your main SMS provider starts timing out. What happens? SCENARIO

Retries with backoff absorb blips; a circuit breaker on rising error rates fails over to the second provider, and permanent errors are not retried.

Q5. How do you implement quiet hours? SCENARIO

Store the user's time zone and window; for non-critical messages inside it, schedule delivery for the window's end (or drop if stale), while security alerts bypass it.

Q6. Record the key before or after calling the provider? TRICKY

Before risks losing a message if you crash before sending; after risks a duplicate. Use a 'sending' state with a lease and reconcile on recovery, plus provider collapse IDs to hide any duplicate that slips through.

Q7. Why not let each service call FCM directly? TRICKY

Every service would re-implement preferences, quiet hours, templates, dedup and rate limits, and they would drift; users would get exactly the 3 a.m. duplicates you are trying to avoid.

Next: SD-Q-004 · Feed Generation backendroadmap.com

Feed Generation

candidates, rank, blend, paginate, cache

Watch the 4-min video

In one line: a feed is a funnel: gather a few thousand **candidates** cheaply, **rank** them with a model that predicts engagement, **blend** with rules for diversity, ads and already-seen items, then serve a page with a **cursor**, using caches that hold IDs rather than full posts.

1 Twenty items out of thousands

You open a photo app and within about 200 ms see a feed: friends' posts, a suggested reel, an ad, in a chosen order. The system picked those 20 items out of thousands of eligible ones, just for you. It cannot run an expensive model over millions of posts per request, so it works in stages, each doing more work per item on fewer items.

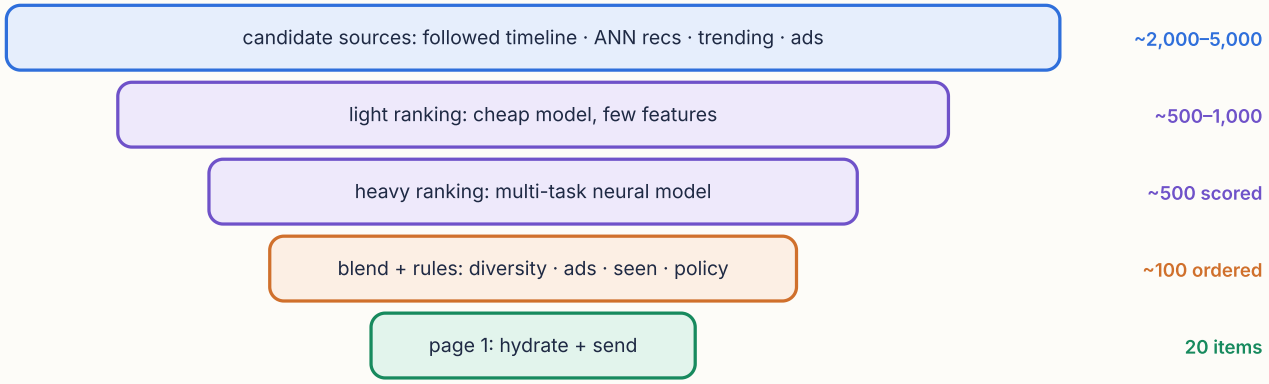


Figure 1. The ranking funnel. Numbers are typical orders of magnitude, not fixed values.

2 Candidate generation: recall first

Candidates come from several **sources** in parallel, each with a quota:

- **Followed accounts:** the user's precomputed timeline of post IDs (built by fan-out, lesson 30).
- **Recommendations:** embedding retrieval: a two-tower model maps users and posts into one vector space, and an ANN index returns the nearest posts (lessons 22–23). Also "friends engaged with this" and topic sources.
- **Trending and fresh** content (lesson 40), and **ads**, which come from a separate auction and are inserted later.

This stage optimises **recall**: a great post that never becomes a candidate cannot be ranked up. It must be cheap: cache lookups and ANN queries, a few tens of milliseconds, with eligibility filters (blocked users, deleted posts, age limits) applied early.

3 Ranking: predicting engagement

A **light ranker** (small model, few features) trims the set; then the **heavy ranker**, usually a multi-task neural network, predicts several probabilities per candidate: like, comment, share, watch to the end, and negative signals such as hide or report. A **value function** combines them, for example `score =`

$w_1 \cdot P(\text{like}) + w_2 \cdot P(\text{comment}) + w_3 \cdot P(\text{share}) - w_4 \cdot P(\text{hide})$; the weights encode what the product wants to encourage.

Features: who posted (your interaction history with them, the **affinity**), content type, the post's age and early engagement, and your context (device, time, session). They are read from a low-latency **feature store**. Models are trained on logged impressions ("shown at position 3, liked") and retrained regularly as behaviour changes, so every served feed must log what was shown.

4 Blending and rules

Pure score order produces bad feeds: five posts from one prolific friend in a row, three ads together, a post you saw yesterday. A re-ranking pass applies rules:

- **Diversity:** at most one item per author in any window of N positions; mix content types (photo, video, reel).
- **Ads and inserts:** fixed or min-gap slots (e.g. an ad no closer than every 5th position).
- **Seen filter:** drop items already shown; a per-user seen set or Bloom filter (lesson 39) makes this cheap.
- **Integrity:** remove items that break policy or were reported, and demote low-quality content.

5 Paginating a feed that keeps changing

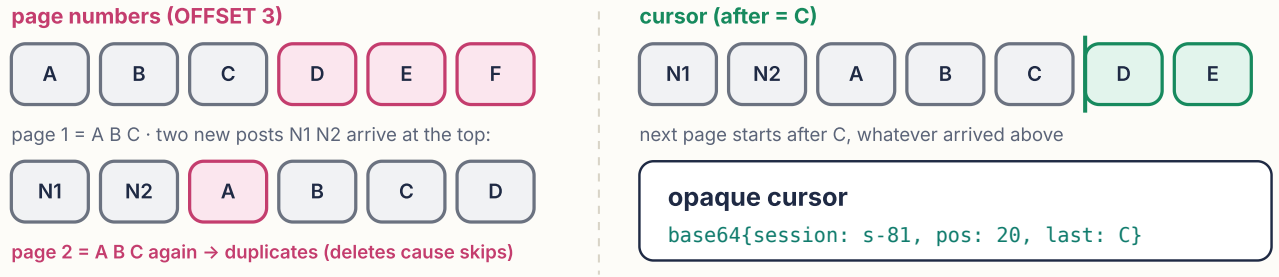


Figure 2. Offsets count from the top, which moves. A cursor remembers **where** you stopped, so new posts never cause duplicates or skips.

For a **chronological** feed, the cursor is a keyset: `(created_at, post_id)` of the last item, and the next query is "older than this". For a **ranked** feed there is no natural sort key, so the server keeps the ranked list for this session (e.g. 100 IDs in a cache with a short TTL) and the cursor points into it, plus the seen set to exclude repeats. Make the cursor **opaque** (encoded, maybe signed) so clients cannot depend on its contents.

6 Caching for a fast open

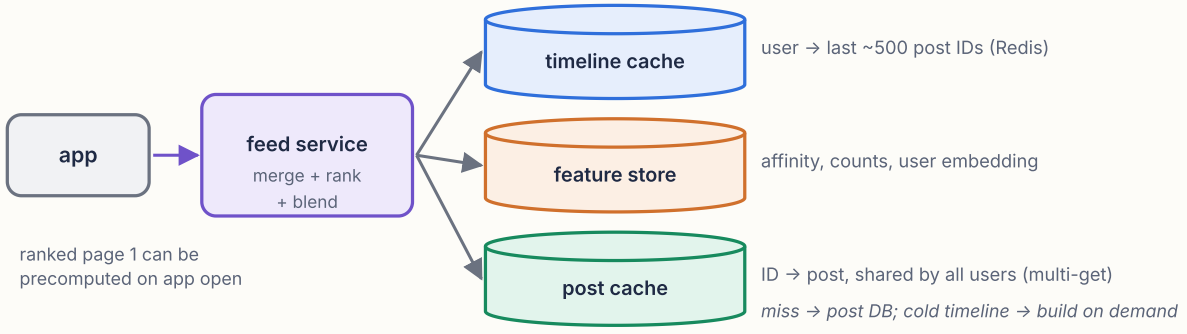


Figure 3. The read path. Timelines hold IDs only; post bodies come from one shared cache, so an edit or delete updates everyone at once.

- **Timeline cache** per user: a list of the latest few hundred post IDs (a few KB per user), not post objects. Storing full posts per follower would multiply storage and make edits impossible.
- **Post cache** shared by all users: hydrate a page with one multi-get of 20 IDs. Counters (likes) are often fetched separately because they change constantly (lesson 38).
- **Precompute** the first ranked page when signals suggest the user will open the app soon, so it appears instantly.
- **Cold cache** (inactive user): build the feed on demand from the followed accounts' recent posts. Slower, but correct.

Stage	Items in → out	Typical budget	Optimises
Candidate generation	millions → thousands	20–50 ms	recall, cheapness
Light + heavy ranking	thousands → hundreds scored	50–100 ms	precision of order
Blending / rules	hundreds → ordered list	few ms	feed quality, ads, policy
Hydration	20 IDs → 20 posts	~10 ms	latency (cache hits)

WORKED EXAMPLE: AN INSTAGRAM-STYLE HOME FEED

The user opens the app. Candidates: 600 IDs from their followed-accounts timeline, 300 recommended reels from embedding retrieval, 100 trending posts, and eligible ads from the auction. After filters (blocked, deleted, seen) about 1,000 remain. The heavy model scores them for P(like), P(comment), P(share) and P(hide); the value function orders them. Blending puts a reel every few posts, an ad every few more, and never two posts from the same account in a row. The top 20 IDs are hydrated from the post cache and returned with cursor `{session: s-81, pos: 20}`; the remaining ranked list stays cached for a few minutes, so scrolling serves positions 21–40 from it without re-ranking, and impressions are logged for the next training run.

TRADE-OFFS

- More candidates improve recall but cost latency and ranking compute.
- Heavier models rank better but need stricter funnels or precomputation.
- Session-cached rankings make pagination stable but slightly stale; re-ranking on scroll is fresher but can reorder what the user saw.
- Optimising pure engagement can favour clickbait; negative signals and diversity rules counterbalance it.

COMMON MISTAKES

- Running the expensive model on everything instead of on a small candidate set.
- Offset pagination on a feed that changes, causing duplicates and skips.
- Caching full post objects inside every user's timeline.
- Not logging impressions, so the model has no labels to learn from.

7 Key terms

Candidate generation: cheap, high-recall gathering of items to rank

Ranking model: predicts engagement probabilities per candidate

Value function: weighted mix of predictions into one score

Blending: rule-based re-ranking for diversity, ads and policy

Cursor: opaque token marking where the next page starts

Hydration: turning post IDs into full posts via a shared cache

8 Interview questions

Q1. How do you build a personalised feed? BASIC

Generate candidates (follows, recommendations, ads), rank them with an engagement model, blend with rules for diversity, ads and seen items, and cache timelines and post details.

Q2. How do you paginate a changing feed? BASIC

Use cursors, not page numbers: a keyset for chronological feeds, or a pointer into a cached ranked list for ranked feeds.

Q3. The feed takes 900 ms. Where do you look? SCENARIO

Profile per stage: candidate fan-in (too many sources or cache misses), ranking (too many candidates for the heavy model), and hydration (N+1 lookups instead of multi-get); shrink the funnel and precompute.

Q4. Users complain they see the same post every time they open the app. SCENARIO

Add a seen filter (per-user seen set or Bloom filter) in blending, and down-rank items with prior impressions.

Q5. Why can't ranking fix a post that's missing from the feed? TRICKY

Ranking only orders candidates; if retrieval never selected the post, no model sees it. Recall must be fixed at candidate generation.

Q6. Why store IDs rather than posts in each timeline? TRICKY

IDs are tiny and edits or deletes then apply everywhere through one shared post cache; copies per follower would explode storage and go stale.

Next: SD-Q-005 · Fan-Out on Write vs Read backendroadmap.com

Fan-Out on Write vs Read

push for most, pull for celebrities

Watch the 4-min video

In one line: fan-out on write pushes each new post into every follower's precomputed timeline (cheap reads, expensive writes); **fan-out on read** assembles the feed when it is opened (cheap writes, expensive reads). Real systems use a **hybrid**: push for ordinary accounts, pull for celebrities, merged at read time.

1 200 followers or 100 million

A regular user with 200 followers posts a photo. A pop star with 100 million followers posts one too. Treat both the same and one is trivial while the other means 100 million timeline writes within seconds. The question is **when** you pay for assembling feeds: when someone writes, or when someone reads.

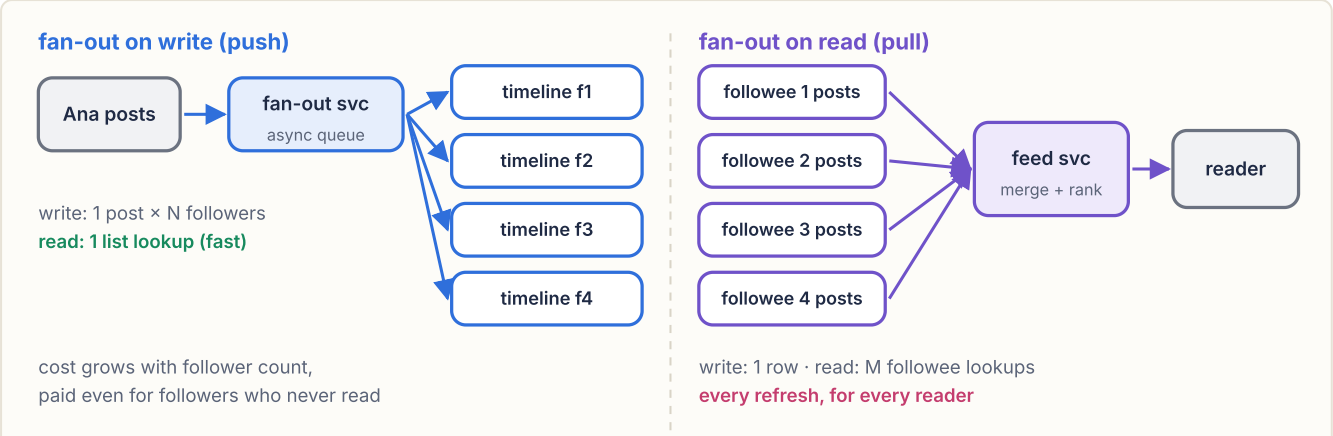


Figure 1. The two extremes. Push moves the work to the writer; pull moves it to every reader.

2 Fan-out on write (push)

When Ana posts, the post is stored once, and a background **fan-out service** reads her follower list and appends the post ID to each follower's timeline (for example a Redis list per user, trimmed to a few hundred entries). Reading a home feed is then a single list lookup plus hydration (lesson 29). Because reads vastly outnumber writes in social apps, this is usually the right default.

The numbers that made this famous come from Twitter (around 2012, as described in *Designing Data-Intensive Applications*, ch. 1): about **4.6k tweets/s** on average (12k+ at peak) against **300k home-timeline reads/s**. With an average of ~75 followers, push meant ~345k timeline inserts/s, which is far cheaper than doing 300k multi-way merges a second.

3 Fan-out on read (pull)

Posting is one write to the author's own post list. Opening the feed fetches recent posts from every account you follow, merges them by time and ranks them. Following 1,000 accounts means up to 1,000 lookups (batched or scatter-gathered) per refresh. Pull wastes nothing on inactive followers and new posts are visible instantly, but read latency and load grow with the number of followees and the refresh rate.

4 Where each one breaks

	Push (on write)	Pull (on read)
Read cost	O(1) list read	O(followees) lookups + merge
Write cost	O(followers) inserts	O(1)
Freshness	delayed by **fan-out lag**	immediate
Storage	timeline per user (IDs only)	none extra
Breaks for	celebrities: millions of writes per post, most to inactive users	heavy readers and users who follow thousands
Deletes / unfollows	must remove IDs or filter at read time	naturally correct

Push for a 100M-follower account means a burst of 100M inserts. Even at 1M inserts a second that is minutes of **fan-out lag**, during which some followers see the post and others don't, and the queue delays every other user's posts too. Pull for everybody means the busiest accounts' post lists become hot keys read millions of times a second.

5 The hybrid

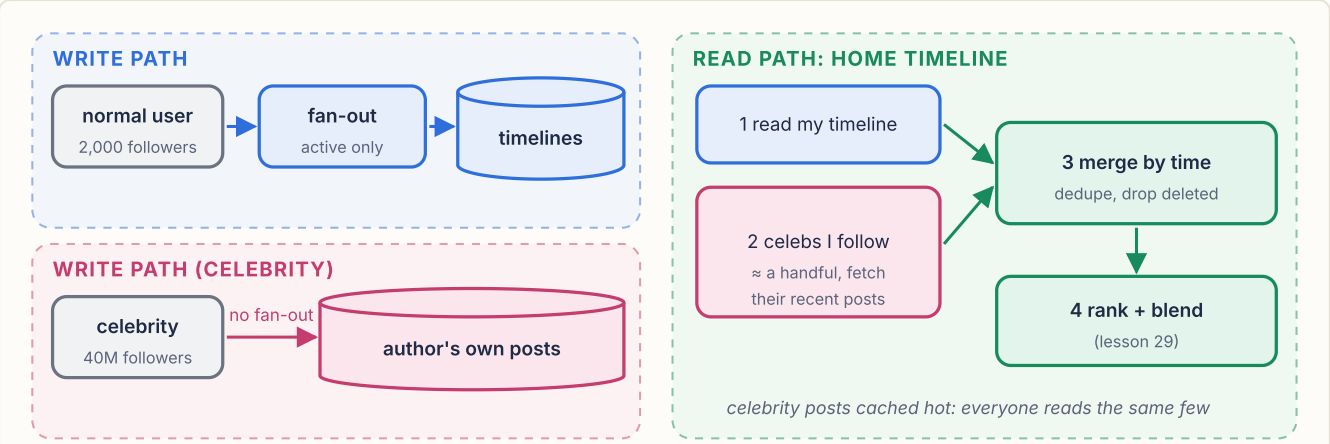


Figure 2. Ordinary authors are pushed; celebrity posts stay with the author and are merged in when a follower reads.

Pick a threshold on follower count (tunable; something like hundreds of thousands). Below it, posts fan out on write. Above it, posts are **not** fanned out. On read, the feed service takes your precomputed timeline, fetches recent posts from the handful of celebrities you follow (from a hot cache, since millions of readers ask for the same few lists), merges by time, and ranks. Most users follow few celebrities, so the read-time merge is small and bounded.

6 Making it efficient

- **Asynchronous:** the post API writes the post and enqueues one fan-out job, then returns. Workers split big follower lists into chunks (e.g. 5,000 followers per task) and process them in parallel.
- **Skip inactive users:** don't push to accounts that haven't opened the app in, say, 30 days; rebuild their timeline by pull when they return.
- **Short timelines:** keep only the latest few hundred IDs (Twitter kept about 800); older pages fall back to pull or a search index.
- **Cap per-post work** and prioritise: fan out to the most active followers first.

- **Handle the edges:** a new follow back-fills recent posts from that account; an unfollow or delete is filtered at read time, with lazy cleanup, instead of a second fan-out storm.
- **Measure fan-out lag:** the time from post to the last follower's timeline insert, as an SLO.

WORKED EXAMPLE: TWITTER'S APPROACH

A regular user with 2,000 followers tweets. The tweet is stored, and the fan-out service inserts its ID into the cached home timelines of the ~1,200 followers active this month: 1,200 small list pushes, done within a second or two. A celebrity with 40 million followers tweets: it is stored in their own tweet list and nothing is fanned out. When Ben loads his home timeline, the service reads his cached timeline (up to ~800 IDs), fetches the latest tweets of the 6 celebrities he follows from a hot cache, merges both lists by time, removes deleted or muted items, ranks, and hydrates the first page.

TRADE-OFFS

- A lower celebrity threshold reduces write storms but makes more reads merge more lists.
- Skipping inactive users saves most writes but makes a returning user's first load slow.
- Long timelines allow deep scrolling from cache but multiply memory across all users.
- Push gives predictable read latency; pull gives perfect freshness and simpler deletes.

COMMON MISTAKES

- Fanning out synchronously inside the post request.
- Storing full posts in timelines instead of IDs (edits and deletes become impossible).
- Treating the celebrity threshold as fixed: accounts cross it, so the read path must handle both kinds.
- Forgetting the hot-key problem on the pull side: celebrity post lists need replicated caching.

7 Key terms

Fan-out on write: push post IDs into followers' timelines when posted

Hybrid fan-out: push for normal users, pull for celebrities

Home timeline: precomputed list of post IDs for one user

Fan-out on read: build the feed from followees' posts when opened

Fan-out lag: time until a post reaches every follower's timeline

Celebrity threshold: follower count above which posts are pulled

8 Interview questions

Q1. Compare fan-out on write and fan-out on read. BASIC

Write pushes each post into every follower's timeline: fast reads, expensive writes. Read assembles the feed when it is opened: cheap writes, expensive reads.

Q2. How do you handle celebrities? BASIC

Hybrid: push posts from normal users, and pull celebrity posts at read time, merging them into the precomputed timeline.

Q3. Reads are 100× writes and the average user has 200 followers. Which model? SCENARIO

Push: 200 cheap inserts per post buys O(1) reads for a workload dominated by reads; add pull only for very large accounts.

Q4. A user returns after six months. What happens to their feed? SCENARIO

They were skipped by fan-out, so the timeline is rebuilt on demand by pulling recent posts from followees, then normal push resumes.

Q5. A post is deleted after being fanned out to 2 million timelines. How do you remove it? TRICKY

Don't fan out a delete storm: mark the post deleted, filter it at hydration or merge time, and let timelines trim it lazily.

Q6. Why does push give worse freshness than pull for big accounts? TRICKY

Fan-out takes time proportional to followers, so some followers get the post minutes after others; pull reads the author's list directly and is immediate.

Next: SD-Q-006 · Batch Processing

Batch Processing

bounded data, rerunnable results

Watch the 4-min video

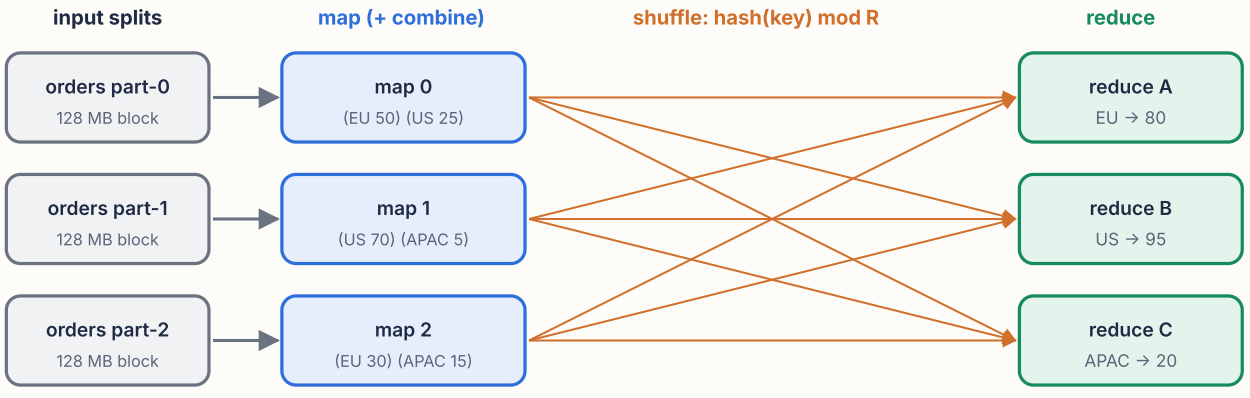
In one line: a batch job runs over a **bounded**, fixed input (such as yesterday's partition) and produces a complete result; because the input doesn't change, the job can be **rerun** to get the same answer, which makes batch the cheapest and easiest-to-correct way to compute results that may be hours old.

1 Yesterday's revenue, complete and correct

At 8 a.m. the sales team opens a dashboard of yesterday's revenue by region, computed from 300 million order rows. Nobody needs it second by second, but it must be complete (every order), correct (refunds, currency) and cheap. That is batch processing: wait until a period of data is closed, then process all of it at once on a cluster that can be shut down afterwards.

	Batch	Stream (lesson 32)
Input	bounded: a file set, a day's partition	unbounded: events forever
Latency	minutes to hours	milliseconds to seconds
Correction	fix code, rerun the partition	replay the log with a new job version
Hard parts	shuffle cost, skew, scheduling	state, event time, late data
Typical uses	reports, billing, ML training, backfills	fraud, alerts, live metrics

2 MapReduce: the model behind it



the shuffle moves every map output across the network, sorted and grouped by key: usually the slowest step

Figure 1. Revenue by region. Each map emits (region, amount) and pre-sums locally (combiner); the shuffle sends each key to one reducer, which sums it.

From Google's 2004 paper (Dean & Ghemawat): **map** reads records from input splits and emits key-value pairs; the framework **shuffles** them so all values for one key reach the same reducer (partitioned by `hash(key) mod R`, sorted by key); **reduce** combines each key's values. Failed tasks are simply re-executed on another machine, which works because tasks are deterministic and write output only on success.

- **Combiner:** a mini-reduce on the map side (sum per region before sending), which can cut shuffle volume by orders of magnitude.
- **Data skew:** if one key (a huge region, a null ID) holds most of the data, one reducer runs for hours while others idle. Fixes: salt the hot key into N sub-keys and merge, or handle it separately.
- **Data locality:** schedule map tasks where the data blocks live (HDFS) to avoid reading them over the network.

3 Spark: the usual engine today

Spark keeps intermediate data in memory where possible and plans the whole job as a **DAG of stages**. Transformations are lazy; when an action runs, the optimiser (Catalyst, for DataFrames and SQL) builds a plan. **Narrow** transformations (filter, map) stay in one stage; **wide** ones (groupBy, join) need a shuffle and start a new stage. Classic MapReduce wrote to disk between every step; a Spark pipeline of five steps may shuffle only once or twice.

- **Columnar files:** Parquet or ORC store each column separately with min/max statistics, so a query reads only the columns it needs and skips row groups that can't match (predicate pushdown).
- **Separate storage and compute:** read from S3/GCS/ADLS, run on a cluster sized for this job (often spot instances, since failed tasks just retry), then release it.
- **Tuning knobs:** number of shuffle partitions (`spark.sql.shuffle.partitions`, default 200), broadcast joins for small tables, and Adaptive Query Execution, which can coalesce partitions and split skewed ones at run time.

4 Partitioned and idempotent

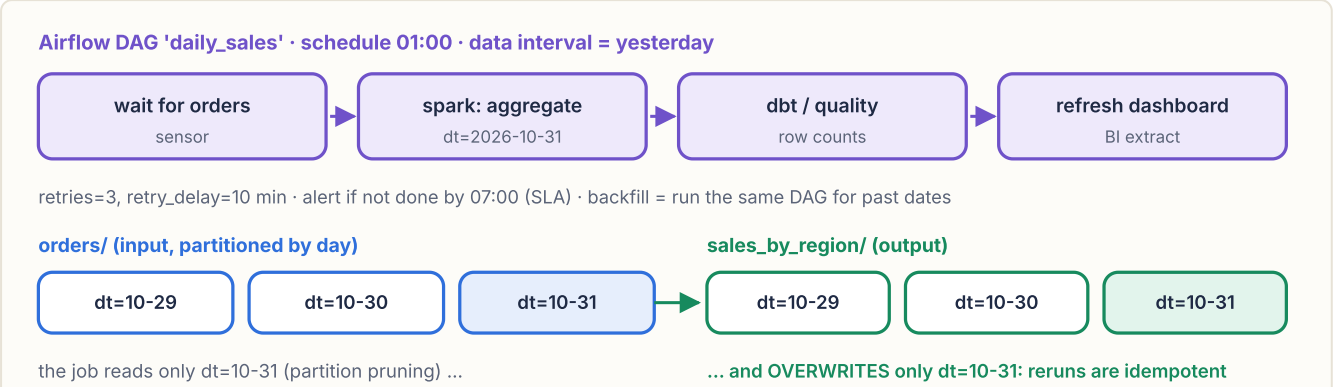


Figure 2. The job's input and output are both partitioned by day. Each run reads one input partition and replaces one output partition.

Store data in partitions such as `orders/dt=2026-10-31/`. A job then reads only what it needs (**partition pruning**) and writes to exactly one output partition by **overwriting** it, never appending. Rerunning yesterday after a bug fix replaces yesterday's results; appending would double them. In Spark, dynamic partition overwrite (`partitionOverwriteMode=dynamic`) or a table format with atomic commits (Delta Lake, Iceberg; lesson 34) makes the swap all-or-nothing, so readers never see a half-written partition. Also keep jobs deterministic: no `now()` inside the logic, use the run's logical date instead.

5 Orchestration

Jobs depend on each other: extract, then aggregate, then check, then refresh the dashboard. An orchestrator such as **Airflow** defines these as a DAG of tasks with a schedule and gives you: dependency ordering, **retries** with delays, **sensors** that wait for input to land, **backfills** (run the DAG for a range of

past dates), SLAs and alerts when something is late, and a history of every run. In Airflow, a daily run covers a **data interval** and starts after that interval ends, so the 01:00 run on Nov 1 processes Oct 31.

WORKED EXAMPLE: NIGHTLY SALES BY REGION

At 01:00 Airflow starts DAG `daily_sales` for interval Oct 31. A sensor confirms `orders/dt=2026-10-31` is complete. A Spark job reads that partition (300M rows, only the columns region, amount, currency, status), converts currency, drops cancelled orders, groups by region and sums: one shuffle of a few hundred KB after the combiner. It overwrites `sales_by_region/dt=2026-10-31`; a quality task checks row counts and totals against yesterday; the dashboard refreshes by 07:00. On Nov 6 a currency-conversion bug is found: the team fixes the code and runs a backfill for Oct 30 – Nov 5. Seven partitions are overwritten and every number is corrected, with no duplicates.

TRADE-OFFS

- Latency for simplicity: results are hours old, but cheap, complete and easy to fix.
- Bigger clusters finish faster but cost more; the shuffle, not CPU, usually limits speed.
- Small, frequent partitions give fresher results but create many small files and more scheduling overhead.
- Waiting longer before running catches more late data but delays the report.

COMMON MISTAKES

- Appending output, so every rerun doubles the numbers.
- Using wall-clock time (`now()`) inside the job, so a backfill computes the wrong day.
- Ignoring skew: one hot key makes a 10-minute job take hours.
- Starting the job on a timer without checking that the input is complete.

6 Key terms

Bounded data: a fixed input with a beginning and end

Combiner: map-side pre-aggregation that shrinks the shuffle

Idempotent rerun: running again replaces, never duplicates, output

Shuffle: moving map output across machines, grouped by key

Partition pruning: reading only the partitions a query needs

Backfill: running a pipeline for past dates

7 Interview questions

Q1. When is batch processing preferable to streaming? BASIC

When hours-old results are fine, such as daily reports, billing and model training, and you value simplicity, low cost, complete data and easy reprocessing.

Q2. What makes a batch job safe to rerun? BASIC

Partitioned inputs and outputs, with each run overwriting its own partition deterministically, so reruns are idempotent.

Q3. What are map, shuffle and reduce? BASIC

Map emits key-value pairs per record; shuffle groups all values for each key onto one machine; reduce combines each group.

Q4. A Spark job's last task runs for two hours while the others finish in minutes. Why? SCENARIO

Data skew: one key holds most rows. Salt the key, broadcast the small side of a join, or enable adaptive skew handling.

Q5. A bug corrupted a month of daily summaries. How do you fix it? SCENARIO

Fix the code, then backfill those 30 dates; each run overwrites its partition, so results are replaced without duplicates.

Q6. Why is Spark faster than classic MapReduce for multi-step jobs? TRICKY

It plans the whole DAG, pipelines narrow steps in memory and only materialises at shuffles, while MapReduce wrote to disk between every job.

Q7. Your nightly job sometimes misses orders that arrive late. What do you do? TRICKY

Wait for a completeness signal before running, and periodically re-run the last few days so late records land in the right partition.

Next: SD-Q-007 · Stream Processing backendroadmap.com

Stream Processing

answers while the events are still flowing

Watch the 4-min video

In one line: a stream processor reads an **unbounded** flow of events (usually from Kafka) and updates results continuously; the hard parts are **time** (event time vs processing time), **windows** for aggregation, **watermarks** that decide when a window is complete, and **state** that survives crashes through checkpoints.

1 Answers in seconds

A stolen card is used five times in two minutes in three countries. A nightly batch job (lesson 31) would notice tomorrow, long after the money has gone. Fraud scoring, alerting, live dashboards and real-time features need results within seconds while events keep arriving, never "complete". Frameworks such as **Apache Flink**, **Kafka Streams** and **Spark Structured Streaming** (micro-batches by default) handle parallelism, state, failure recovery and time. Kafka keeps the input durably, so a job can always replay from an earlier offset.

2 Event time vs processing time

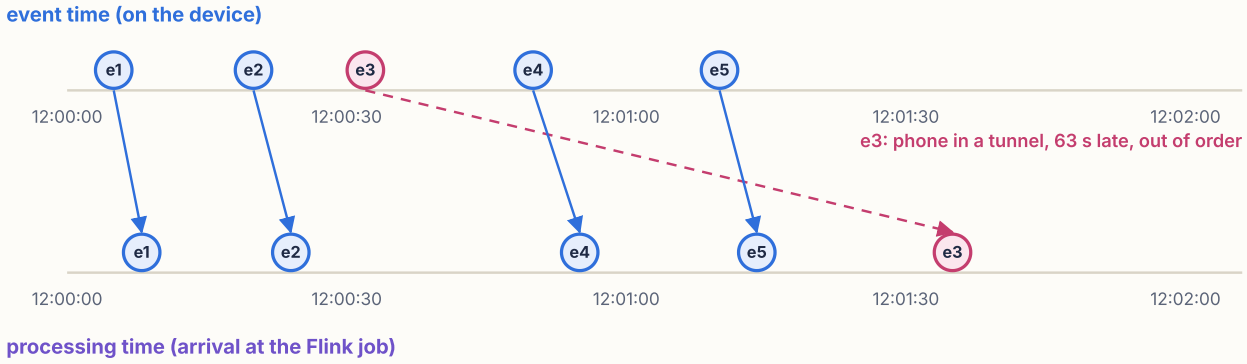


Figure 1. Events usually arrive a little after they happen, but sometimes much later and out of order.

Event time is when something happened, stamped by the source. **Processing time** is when your job sees it. Networks, retries, batching and offline phones make the gap variable. Counting by processing time is easy but wrong: a network hiccup moves events into the wrong minute, and replaying yesterday's data would put it all into today. Counting by event time gives the same answer however late or fast the data arrives, which is what makes replays correct (lesson 33).

3 Windows

An endless stream has no "total"; aggregations need boundaries. A **window** groups events by event time so each group can be computed and emitted. Choosing the window is a product decision: what question is the number supposed to answer?

A sliding window of size 5 min and slide 1 min puts each event in 5 windows, so cost grows with size ÷ slide. For "the last N minutes, right now" on every event (fraud), many systems skip windows and keep a per-key buffer of recent events in state, evicting old ones.

Window	Definition	Example question
Tumbling	fixed size, no overlap; each event in exactly one window	orders per minute
Sliding (hopping)	fixed size, advances by a smaller slide; windows overlap	error rate over the last 5 min, every minute
Session	activity separated by a gap of inactivity; size varies per key	length of one user's visit
Global + trigger	no time bounds; you decide when to emit	every 100 events per key

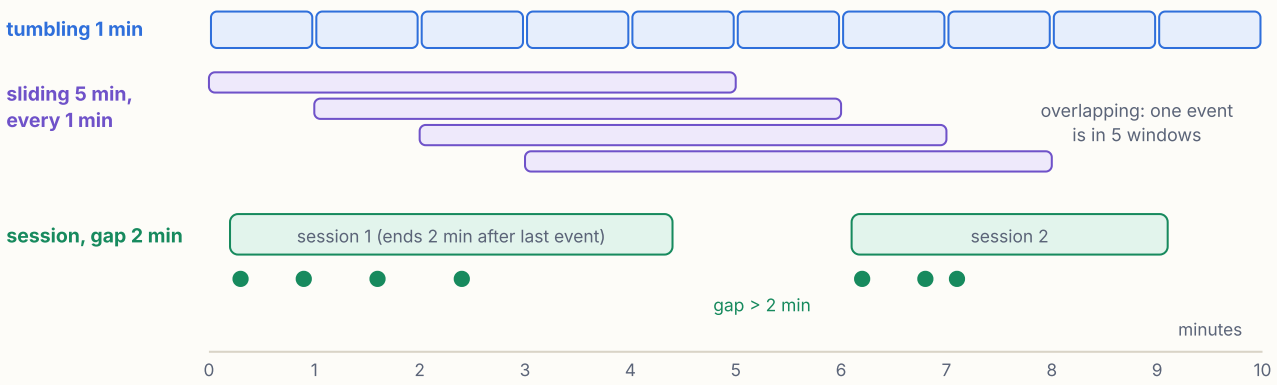
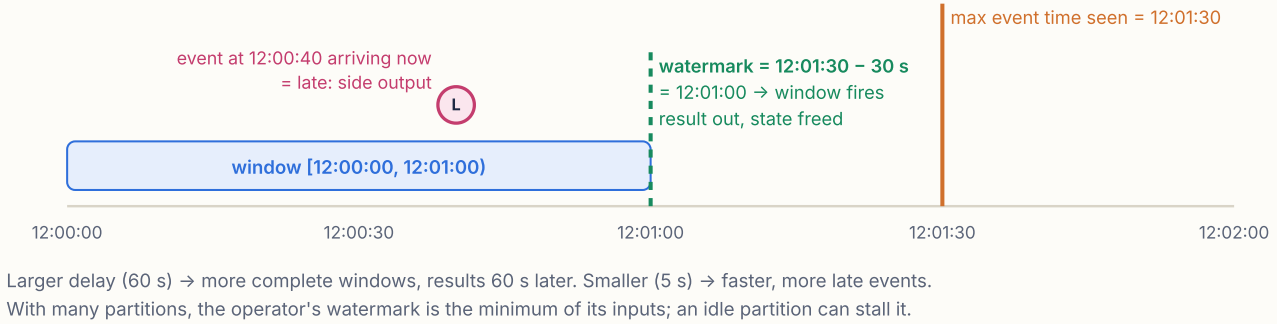


Figure 2. Three window types over the same timeline. The question being asked decides which one you need.

4 Watermarks: when is a window complete?

If events can be late, when can the job emit the result for 12:00–12:01? A **watermark** is the processor's assertion that no more events with an event time earlier than *t* are expected. The common strategy is **bounded out-of-orderness**: watermark = highest event time seen − allowed delay (in Flink, `WatermarkStrategy.forBoundedOutOfOrderness(Duration.ofSeconds(30))`). When the watermark passes a window's end, the window fires and its state is freed.



Larger delay (60 s) → more complete windows, results 60 s later. Smaller (5 s) → faster, more late events.
With many partitions, the operator's watermark is the minimum of its inputs; an idle partition can stall it.

Figure 3. A 30-second watermark delay. The window fires when the watermark reaches its end; anything older that arrives afterwards is late.

Late events can be **dropped**, sent to a **side output** for separate handling, or accepted for an extra **allowed lateness** period during which the window re-fires with an updated result (so the sink must accept updates). The delay is a dial between completeness and latency; there is no setting that gives both.

5 State, checkpoints and exactly-once

Windows and counters are **state**, kept per key next to the computation (in Flink, on the heap or in an embedded RocksDB store, so it can exceed memory). The stream is partitioned by key (`keyBy(card)`), so all events for one card reach the same parallel instance and its local state.

- **Checkpoints:** Flink periodically injects **barriers** into the stream; when every operator has seen the barrier, it snapshots its state to durable storage (S3, HDFS) together with the Kafka offsets of that point. After a crash, the job restores the last checkpoint and re-reads from those offsets: state is **exactly-once** even though some events are processed twice.
- **End-to-end exactly-once** also needs the sink to cooperate: a transactional sink (Kafka transactions, two-phase commit tied to the checkpoint) or idempotent writes (upsert by key). Otherwise outputs between the checkpoint and the crash are emitted again.
- **Backpressure:** if a sink slows down, upstream operators slow too and Kafka buffers the backlog; watch consumer lag.

WORKED EXAMPLE: REAL-TIME FRAUD SCORING

Card payments flow into Kafka topic `payments`, partitioned by card number. A Flink job does `keyBy(card)` and, per card, keeps the last 5 minutes of payments in state (count, total, set of countries), with a 30 s bounded-out-of-orderness watermark to expire old entries by event time. Each new payment is evaluated *immediately* against that state rather than waiting for a window to close. Card 4321 makes its 5th payment within 2 minutes, in a 3rd country: the rule and a model score fire, and a `fraud_score` event is emitted in well under a second, so the authorisation service can block the next attempt. Checkpoints every 10 s let the job restart without losing the per-card history.

TRADE-OFFS

- Watermark delay: completeness vs latency.
- Checkpoint interval: frequent checkpoints mean less replay after a crash, but more I/O and, with transactional sinks, more latency.
- Event time is correct and replayable; processing time is simpler and lower latency but changes with network conditions.
- Large state (long windows, many keys) costs memory, disk and recovery time; set state TTLs.

COMMON MISTAKES

- Aggregating by processing time and then wondering why replays give different numbers.
- No plan for late events: silently dropping them makes totals drift from the batch numbers.
- An idle Kafka partition holding back the watermark, so no window ever fires.
- Assuming exactly-once state means exactly-once output without a transactional or idempotent sink.

6 Key terms

Event time: when the event happened at the source

Processing time: when the stream job handles it

Tumbling / sliding window: non-overlapping vs overlapping fixed windows

Session window: activity grouped until a gap of inactivity

Watermark: assertion that no older events are expected

Checkpoint: consistent snapshot of state + input offsets

7 Interview questions

Q1. What is a watermark? BASIC
The processor's estimate of how far event time has progressed (no older events expected), so it can close windows and emit results despite late, out-of-order data.

Q2. Tumbling versus sliding windows? BASIC
Tumbling windows are fixed and don't overlap; sliding windows have a fixed size but advance in smaller steps, so they overlap and each event counts in several.

Q3. Event time or processing time for a revenue-per-minute metric? BASIC
Event time: it assigns each sale to the minute it happened, so late arrivals and replays give the same answer.

Q4. Your per-minute counts are always 3% lower than the nightly batch. Why? SCENARIO
Late events arrive after the watermark and are dropped; increase the delay, use allowed lateness with updates, or reconcile with batch.

Q5. A Flink job crashes. How does it avoid double-counting? SCENARIO
It restores state from the last checkpoint and rewinds Kafka to the offsets stored with it; with a transactional or idempotent sink, outputs are also not duplicated.

Q6. No windows are firing although data flows. What could it be? TRICKY
The watermark is stuck: one input partition is idle or has a very old timestamp, and the operator takes the minimum; configure idleness detection or fix the source.

Q7. Why might fraud detection not use a windowed aggregate at all? TRICKY
A window emits only when it closes, adding latency; evaluating each event against per-key recent state answers immediately.

Next: SD-Q-008 · Lambda vs Kappa backendroadmap.com

Lambda vs Kappa

two pipelines, or one log you can replay

Watch the 4-min video

In one line: Lambda runs a batch layer that recomputes accurate results from all raw data plus a speed layer for the latest hours, and merges them at query time, at the cost of two codebases; **Kappa** uses one streaming pipeline over a long-retention log and fixes mistakes by **replaying** the log into a new version of the output.

1 Real-time numbers that change overnight

The dashboard shows today's sales updating every second. The next morning finance reruns the numbers and they are slightly different, because late refunds and duplicate events were cleaned up overnight. You want numbers that are **fresh** (stream processing, lesson 32) and **correct and correctable** (batch processing, lesson 31). Lambda and Kappa are two ways of getting both.

2 Lambda architecture

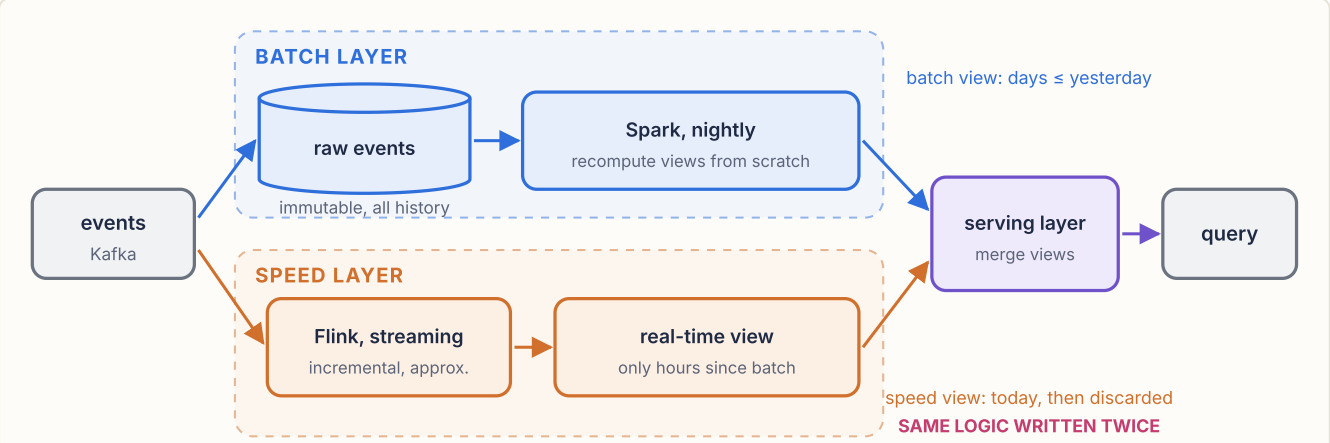


Figure 1. Lambda (Nathan Marz, around 2011). Every event goes to both layers; the serving layer hides the seam between them.

- **Batch layer:** keeps the **master dataset**, every raw event, immutable and append-only (e.g. Parquet on S3). It periodically recomputes complete views *from scratch*, so any earlier mistake disappears on the next run.
- **Speed layer:** processes the same events as a stream and maintains views for only the data the last batch run hasn't covered. These are allowed to be approximate and are thrown away once the batch catches up.
- **Serving layer:** answers queries by merging the batch view (up to the batch cut-off) with the real-time view (after it).

The strength is **robustness**: a bug, a bad deploy or corrupted stream state is fixed by correcting the code and letting the next batch recompute from raw data. Human error is survivable because raw data is never mutated.

The weakness is **duplication**. The same business logic (dedup rules, currency conversion, refund handling) exists twice, in a batch engine and a stream engine with different APIs and semantics. The two

drift, so the numbers jump at the seam between today and yesterday, and every change is built, tested and deployed twice. Jay Kreps' 2014 essay "Questioning the Lambda Architecture" proposed Kappa as the answer.

3 Kappa architecture

Remove the batch layer: **everything is a stream**. The event log (usually Kafka) is the master dataset, kept for as long as you may need to reprocess. One streaming job, one codebase, computes all results. Batch becomes a special case: a stream that starts at an old offset and runs fast.

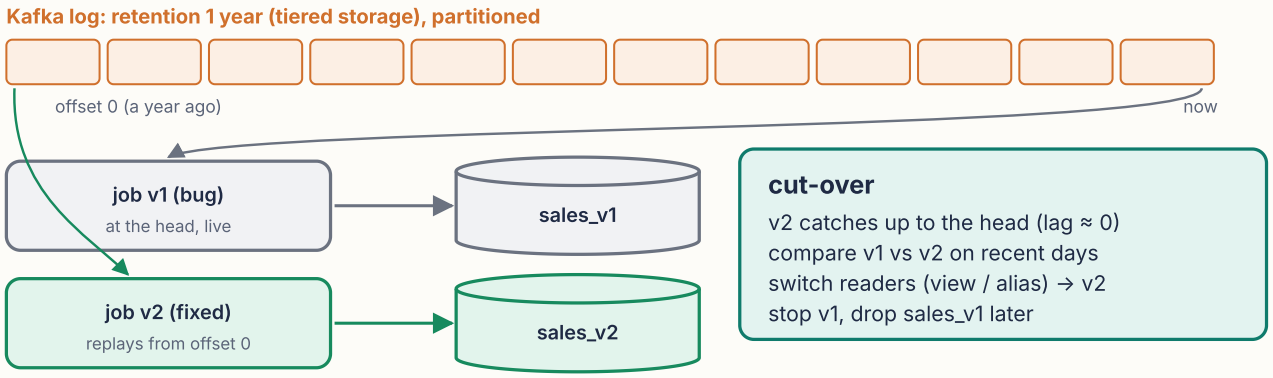


Figure 2. Correcting a mistake in Kappa: run the fixed job as a new version from the start of the log into a new table, then switch readers.

Reprocessing by replay. Deploy job v2 with a new consumer group, starting from offset 0 (or from the first day affected), writing to `sales_v2`. It processes history as fast as the cluster allows. When it has caught up with the head of the log, compare the two outputs, switch readers (a view, alias or config flag), stop v1 and later drop `sales_v1`. Because the job uses **event time**, replayed data lands in the same windows it did originally.

4 What Kappa requires

- **Enough retention:** months or years of raw events in the log. Kafka tiered storage (KIP-405) and similar features in other logs move old segments to object storage, making this affordable; or archive the log to a lakehouse table and replay from there.
- **Replay speed:** if live traffic averages 50,000 events/s and a replay runs at 1 million events/s (20× live), 90 days of history takes about 4.5 days. Plan parallelism (partitions) and sink capacity for this.
- **A capable stream engine:** event time, watermarks and late data, large state, and exactly-once output (lesson 32).
- **Versioned outputs:** writing into a new table per version, so a bad replay never corrupts what readers see.

	Lambda	Kappa
Pipelines / codebases	two (batch + stream)	one (stream)
Source of truth	raw files in the data lake	the event log
Fixing a bug	fix both; next batch run recomputes	fix once; replay into a new version
Consistency	batch and speed results may disagree	one logic, one answer
Good at	heavy batch work: model training, huge joins, ad-hoc recomputation	event-driven metrics, features, alerts
Main cost	maintaining two systems	long retention and fast replay

Which today? Usually Kappa-style, helped by engines that run the same code in batch and streaming modes (Flink, Spark Structured Streaming, Apache Beam) and by lakehouse tables (lesson 34) that can be written by streams and recomputed by batch. Lambda still makes sense when some computations simply are batch-shaped, such as training models on a year of data or giant joins that don't fit a stream.

WORKED EXAMPLE: REAL-TIME AND CORRECTED DAILY NUMBERS

Lambda: a Flink job updates `today_sales` every few seconds. At 02:00 a Spark job recomputes yesterday from raw events in S3, removing duplicate events and applying refunds that arrived late, and writes `daily_sales`. The dashboard reads `daily_sales` for past days and `today_sales` for today, and the Flink numbers for yesterday are discarded. **Kappa:** one Flink job, keyed by order, dedupes by event ID and applies refunds by event time with allowed lateness, writing upserts to `sales_v1`. A currency bug is found: `job v2` replays the Kafka log from the bug's start date into `sales_v2`, catches up in a few hours, and the dashboard switches to v2.

TRADE-OFFS

- Lambda is robust to bugs and well-suited to batch-heavy work, but doubles code, tests and operations.
- Kappa has one truth and one codebase, but depends on long log retention and replay that is fast enough.
- Replays cost compute and sink capacity; running v1 and v2 side by side doubles the load for a while.
- Unified engines reduce Lambda's duplication but do not remove the two operational paths.

COMMON MISTAKES

- Choosing Kappa with 7-day log retention, then finding a bug that started a month ago.
- Replaying into the live output table, so readers see half-old, half-new numbers.
- Using processing time in the stream job, so a replay puts a year of data into today.
- In Lambda, letting batch and stream logic drift without comparing their outputs.

5 Key terms

Batch layer: recomputes complete views from all raw data

Speed layer: streams recent data for fresh, approximate views

Serving layer: merges batch and real-time views at query time

Replay: re-reading the log from an old offset with a new job

Master dataset: immutable raw events, the source of truth

Tiered storage: old log segments kept cheaply in object storage

6 Interview questions

Q1. Lambda versus Kappa: which and why? BASIC

Lambda runs a batch layer for accurate recomputation and a speed layer for fresh results, at the cost of two codebases; Kappa uses one streaming pipeline and fixes mistakes by replaying the log.

Q2. Which would you choose today? BASIC

Usually Kappa, if the log keeps enough history and the engine handles late data; Lambda when heavy batch jobs don't fit streaming.

Q3. In Kappa, how do you deploy a fix to a metric's logic without downtime? SCENARIO

Run the fixed job as v2 from the start offset into a new table, wait until it catches up, validate against v1, then switch readers and retire v1.

Q4. Live traffic is 20k events/s; you must reprocess 60 days within 2 days. What replay rate do you need? SCENARIO

60 days of data in 2 days is 30× live speed: about 600k events/s, which sets partition count, parallelism and sink capacity.

Q5. Why do Lambda dashboards often 'jump' at midnight? TRICKY

Yesterday switches from the speed layer's approximate numbers to the batch layer's recomputed ones, and the two implementations differ.

Q6. Is Kappa possible without Kafka keeping years of data? TRICKY

Yes, if raw events are archived to a lakehouse table and the same streaming code can read that table as a bounded stream for replays.

Next: SD-Q-009 · Data Pipelines backendroadmap.com

Data Pipelines

ingest, transform, orchestrate, test

Watch the 4-min video

In one line: a data pipeline **ingests** data from apps, databases and SaaS tools, lands it **raw**, **transforms** it with SQL through bronze → silver → gold layers in a lakehouse or warehouse, is **orchestrated** on a schedule, and is guarded by **data-quality tests** so dashboards only ever read trustworthy tables. ELT is the modern default.

1 One simple question

The CEO asks: *how many users who signed up last week made a purchase?* Sign-ups are app events, purchases are in the payments database, and campaign data sits in a CRM. Answering once is a few hours of exports; answering correctly every morning is a pipeline. It has four parts: ingestion, transformation, orchestration and quality.

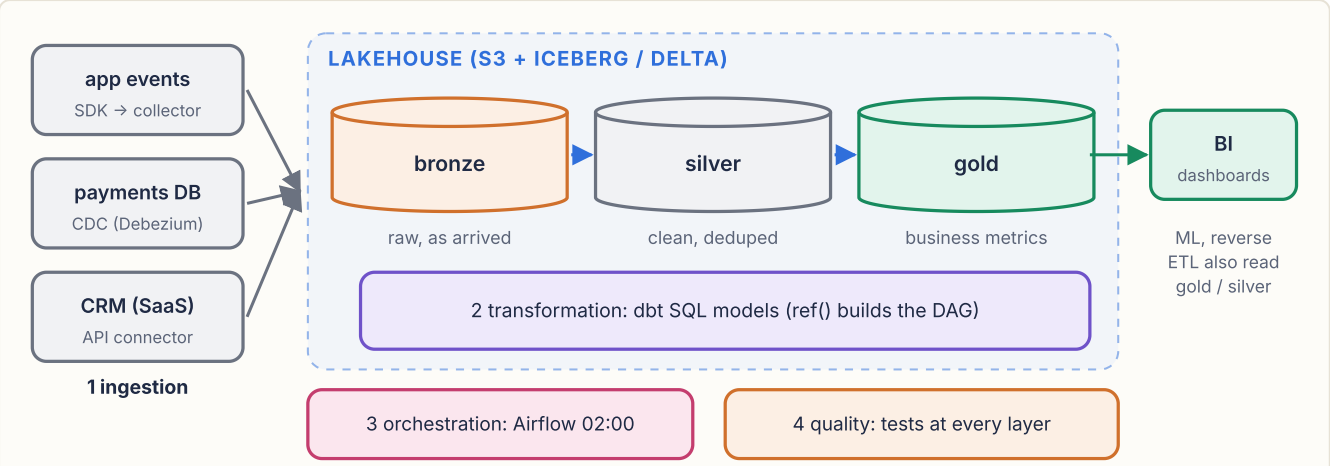


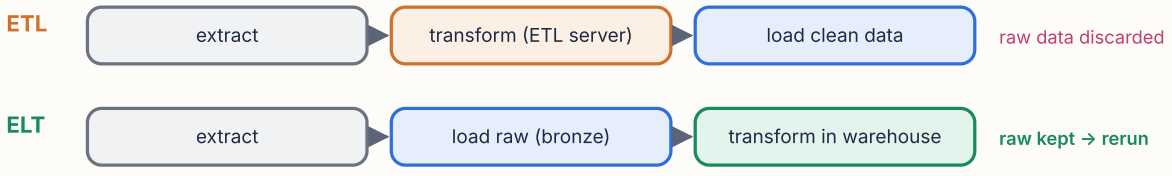
Figure 1. A typical modern pipeline. Raw data is kept, transformations are SQL, and every layer is tested before the next is built.

2 Ingestion: getting data in

Source	How	Watch out for
App and web events	SDK → collector endpoint → Kafka or straight to object storage as files	duplicates from retries (keep an event ID), clock skew, schema changes
Operational databases	**Change data capture**: Debezium reads the MySQL binlog or Postgres WAL (logical replication) and emits every insert/update/delete	full snapshot first, then the stream; ordering per key
SaaS tools (CRM, ads)	API connectors (Fivetran, Airbyte, custom), incremental by cursor	rate limits, API changes, late updates
Files from partners	SFTP / bucket drops, picked up by a sensor	partial files: wait for a success marker

Prefer CDC to nightly `SELECT * WHERE updated_at > …` extracts: the query approach misses hard deletes and rows updated without touching `updated_at`, and loads the production database.

3 ETL or ELT



ETL still fits when data must be masked or filtered before it may land (e.g. PII rules), or the target is small.

Figure 2. The difference is where the transform runs, and whether the raw data survives.

In **ETL** data is cleaned on an ETL server before loading, so only the transformed result exists; a mistake in the transform means re-extracting from sources that may have changed. In **ELT** raw data is loaded first and transformed inside a scalable warehouse or lakehouse with SQL. Cheap object storage and powerful engines (Snowflake, BigQuery, Databricks, Trino) made ELT the default: raw data is kept, so any transformation can be changed and rerun, exactly like batch reruns in lesson 31.

4 The lakehouse

A **data lake** is files (Parquet) in object storage: cheap and open, but plain files have no transactions, so a failed job leaves half-written data and concurrent writers clash. **Table formats** such as Delta Lake, Apache Iceberg and Apache Hudi add a metadata layer (Delta's transaction log, Iceberg's snapshots and manifests) on top of the same files, giving:

- **ACID commits:** a write becomes visible atomically, or not at all; `MERGE INTO` for upserts from CDC.
- **Schema evolution:** add or rename columns without rewriting data, and reject writes that don't match.
- **Time travel:** query the table as of an earlier snapshot, to debug or to roll back a bad load.
- **Many engines, one copy:** Spark, Trino, Flink and warehouses can read the same tables.

5 Medallion layers and dbt

- **Bronze:** raw, exactly as it arrived, append-only, partitioned by load date. Never edited; it is your replay source.
- **Silver:** cleaned and conformed: types cast, duplicates removed by event ID, CDC applied to give current rows, test accounts dropped, entities joined (users, orders, sessions).
- **Gold:** business-ready tables and metrics (daily active users, revenue by country, weekly conversion) that dashboards read directly, with agreed definitions.

dbt turns each transformation into a SQL `SELECT` model; `ref()` declares dependencies so dbt builds them in order, as tables, views or incremental merges, and runs tests after each build.

```
-- models/gold/signup_purchase_weekly.sql
select date_trunc('week', u.signed_up_at) as week,
       count(distinct u.user_id)           as signups,
       count(distinct o.user_id)           as buyers
from {{ ref('silver_users') }} u
left join {{ ref('silver_orders') }} o
  on o.user_id = u.user_id and o.created_at < u.signed_up_at + interval '7' day
group by 1

# models/silver/schema.yml  (tests run after every build)
- name: silver_orders
  columns:
    - name: order_id    tests: [unique, not_null]
    - name: user_id     tests: [not_null, relationships: {to: ref('silver_users'), field: user_id}]
```

Figure 3. A gold model answering the CEO's question, and the silver tests that must pass before it is built.

6 Data quality: stop bad data before gold

Pipelines rarely fail loudly; they produce plausible wrong numbers. Encode expectations as tests at every layer:

- **Schema and constraints:** `not_null`, `unique`, `accepted_values`, `relationships` (dbt's built-in generic tests), or Great Expectations suites.
- **Volume and distribution:** today's row count within the usual range; revenue not 10× yesterday's.
- **Freshness:** the source's newest record is less than N hours old (`dbt source freshness`).

A failing test **stops the downstream models** and alerts the owner: an empty dashboard tile is better than a wrong number in a board meeting. Lineage (which tables feed which) shows what a failure affects.

WORKED EXAMPLE: EVENTS → S3 → DBT → WAREHOUSE

The app sends `signup` and `purchase` events to a collector, which writes them as hourly Parquet files to `s3://lake/bronze/events/dt=…/`; Debezium streams the payments database into `bronze.payments`. At 02:00 Airflow loads new bronze partitions, then runs `dbt build`: silver models dedupe events by `event_id`, merge CDC rows into current orders, and drop internal test users; tests check uniqueness, null IDs and that every order has a user. Only if they pass are gold tables like `signup_purchase_weekly` rebuilt. The BI tool reads gold; the CEO's question is one row: 41,200 sign-ups, 6,950 buyers within 7 days.

TRADE-OFFS

- ELT keeps raw data and flexibility but stores more and can expose raw PII unless access is controlled.
- Daily batch is simple and cheap; streaming ingestion (lesson 32) gives fresher gold tables at more operational cost.
- Strict tests that block the pipeline protect trust but can delay all dashboards for one bad source.
- Managed connectors save engineering time but cost money and limit control.

COMMON MISTAKES

- Dashboards reading bronze or silver tables directly, each with its own metric definition.
- Incremental extracts by `updated_at` that silently miss deletes.
- No dedup by event ID, so retried events inflate counts.
- Tests that only alert while the bad data still flows into gold.

7 Key terms

ELT: load raw first, transform inside the warehouse

Lakehouse: object-storage files + table format with ACID

dbt model: a SELECT statement materialised as a table/view

CDC: streaming row changes from a database's log

Medallion: bronze (raw), silver (clean), gold (business)

Data test: assertion on data that blocks bad loads

8 Interview questions

Q1. Design a pipeline from app events to a BI dashboard. BASIC

Land raw events in object storage, load into a lakehouse or warehouse, transform with SQL through bronze, silver and gold, orchestrate with a scheduler, test quality at every step, and point the dashboard at gold tables.

Q2. ETL or ELT? BASIC

ELT is the modern default: load raw data first, so transformations can be changed and rerun; ETL when data must be filtered or masked before it lands.

Q3. Revenue on the dashboard doubled overnight. How would your pipeline have caught it? SCENARIO

Volume and distribution tests on silver and gold (e.g. revenue within a band of the trailing average) would fail and stop the gold build, alerting the owner.

Q4. How do you ingest a production Postgres database without loading it? SCENARIO

Use CDC from the write-ahead log (logical replication, e.g. Debezium): one initial snapshot, then a stream of changes, merged into silver tables.

Q5. What does a table format like Iceberg add over Parquet files? TRICKY

A metadata layer with atomic snapshots: ACID commits, schema evolution, time travel and safe concurrent writers, readable by many engines.

Q6. Why keep bronze if silver is clean? TRICKY

Bronze is the replay source: when a silver rule is wrong, you fix it and rebuild from bronze; without it you'd have to re-extract from sources that have changed.

Next: SD-Q-010 · Time-Series Data backendroadmap.com

Time-Series Data

append, compress, roll up, expire

Watch the 4-min video

In one line: time-series data is append-only points of (timestamp, value, tags) queried by time range and aggregated; store it in a time-series database that writes **time-chunked, heavily compressed** data, batch the writes, **downsample** into rollups with **retention per resolution**, and keep tag **cardinality** low.

1 A million points a second, forever

A factory has 200,000 sensors reporting temperature, vibration and pressure every second: around 600,000 values a second, with headroom planned for 1 million. At 1 million a second that is 86.4 billion points a day. Engineers want live charts, alerts within seconds, and trends going back years. A normal relational table with a B-tree index per row would spend its life updating indexes and still hold terabytes a day. The data's shape allows a much better design.

2 The shape of time-series data

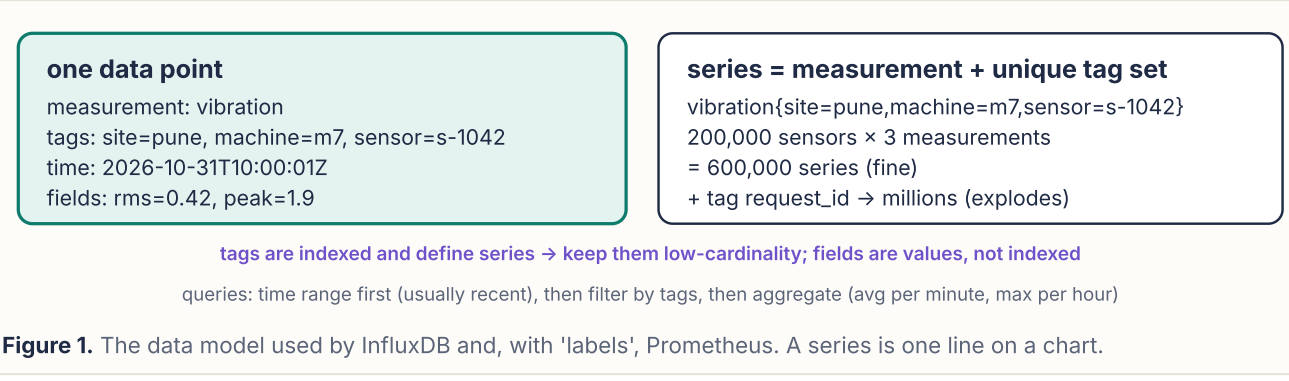


Figure 1. The data model used by InfluxDB and, with 'labels', Prometheus. A series is one line on a chart.

- **Writes** are constant, append-only and almost always for "now"; updates and deletes of single points are rare.
- **Reads** ask for a time range, usually recent, filtered by tags, and aggregated: average per minute, 99th percentile per hour.
- **Value decays** with age: per-second detail matters for days, hourly trends for years.

3 How time-series databases store it

Recent points sit in an in-memory **head** (with a write-ahead log for durability), then are flushed into immutable **time chunks** stored column by column: TimescaleDB calls them chunks of a hypertable, Prometheus uses 2-hour blocks compacted into larger ones, InfluxDB uses shards by time. A query for the last hour opens only the chunks that overlap it.

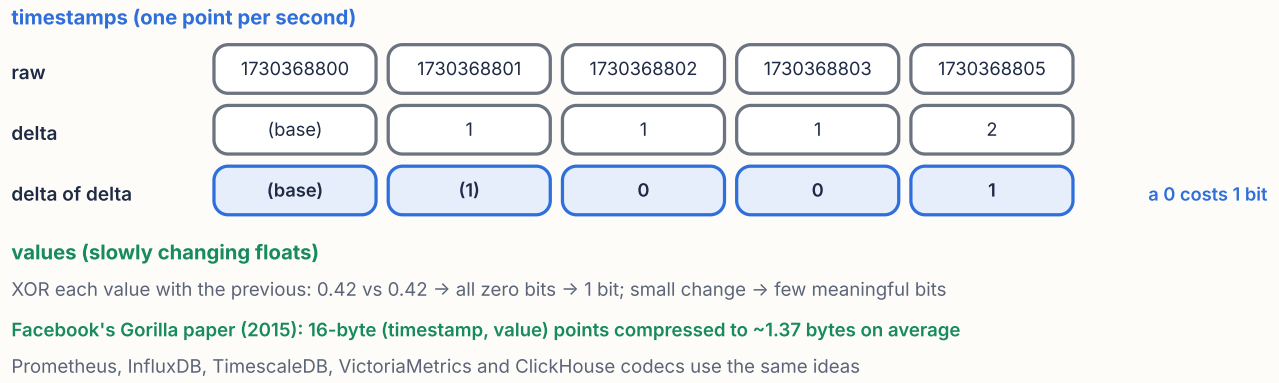


Figure 2. Why compression is so good: regular timestamps reduce to runs of zeros, and slowly changing values XOR to mostly zero bits.

With ~1.4 bytes a point, a day of 86.4 billion points is about **120 GB** instead of ~1.4 TB of raw 16-byte points. Columnar chunks also make aggregates fast, because a query reads one tightly packed column.

System	Model	Good for
Prometheus	pull-based scraping, local TSDB, PromQL	infrastructure metrics and alerting; pair with Thanos, Mimir or VictoriaMetrics for long-term storage
InfluxDB	push writes (line protocol), tags + fields	IoT and app metrics
TimescaleDB	PostgreSQL extension: hypertables, continuous aggregates	time series that need SQL and joins with business tables
ClickHouse	columnar OLAP with MergeTree tables	very high volume events and logs, ad-hoc analytics

4 Rollups and retention

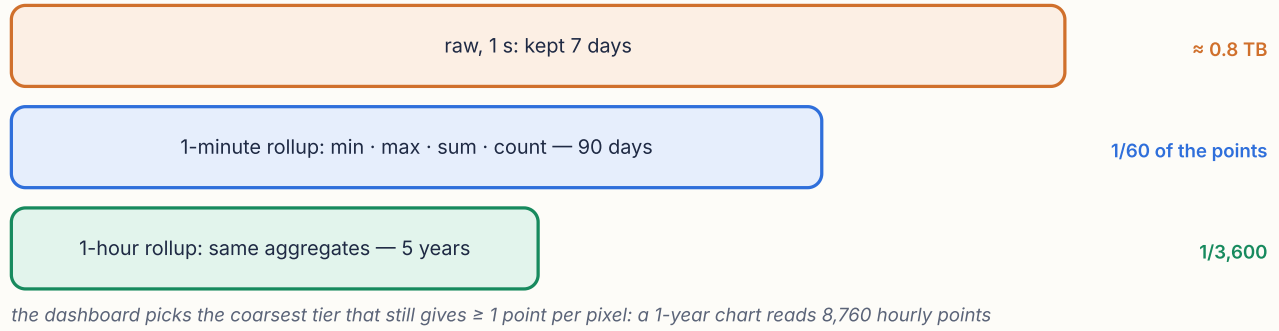


Figure 3. Three resolutions, each with its own retention. Storage is dominated by the short-lived raw tier.

A **rollup** (downsample) aggregates raw points into coarser buckets, continuously: TimescaleDB continuous aggregates, Prometheus recording rules, InfluxDB tasks. Store **min, max, sum and count**, not just the average: averages of averages are wrong when buckets have different counts, and max is what alerts and capacity planning need. (Percentiles need sketches, such as t-digest, because they can't be merged from per-minute percentiles.)

Retention runs per tier: raw after 7 days, minutes after 90, hours after 5 years. Because data lives in time chunks, expiry **drops a whole chunk** (a file delete), which is instant, unlike `DELETE FROM … WHERE ts < …` over billions of rows.

5 Cardinality: the hidden danger

Each unique tag combination is a series, and a TSDB keeps an index entry and per-series buffers for every active one. 200,000 sensors × 3 measurements is 600,000 series, fine. Add a tag with unbounded values (request ID, user ID, full URL, error message) and you create millions of series that each hold a handful of points: the index explodes, memory runs out, compression collapses. Rules: tags only for values you group or filter by with a bounded set; unique values go in fields or in logs and traces; monitor the active-series count and limit it per tenant.

WORKED EXAMPLE: IOT SENSORS AT A MILLION POINTS PER SECOND

Sensors publish over **MQTT** (a lightweight pub/sub protocol for devices) to regional gateways, which forward into **Kafka** partitioned by sensor ID, so a database outage never loses data. Ingest workers consume Kafka and write **batches of 5,000–10,000 points** per request into the TSDB, which is partitioned by time and by sensor. Raw data lives 7 days (~0.8 TB compressed); 1-minute rollups 90 days; hourly rollups 5 years. Alerts ("vibration rms > 1.5 for 30 s") run on the Kafka stream in a stream processor (lesson 32), not by polling the database. A dashboard zoomed to the last hour reads raw points; one showing the last year reads 8,760 hourly points per series instead of 31 million raw ones.

TRADE-OFFS

- Longer raw retention allows detailed forensics but multiplies the largest tier of storage.
- Bigger write batches raise throughput but add latency and data lost in a crash (unless buffered in Kafka).
- Downsampling loses detail: a 2-second spike disappears from an hourly average; keep max to preserve it.
- A purpose-built TSDB is fast and compact; Postgres + Timescale gives SQL and joins with somewhat less efficiency at extreme scale.

COMMON MISTAKES

- Putting user IDs, request IDs or URLs into tags.
- Writing one point per request instead of batching thousands.
- Storing only averages in rollups, so peaks and correct re-aggregation are lost.
- Deleting old data row by row instead of dropping time chunks.

6 Key terms

Series: one measurement with one unique tag set

Time chunk: time-bounded partition of the data, dropped whole

Rollup: pre-aggregated coarser copy (min, max, sum, count)

Cardinality: number of distinct series

Delta-of-delta / XOR: compression for timestamps / floats

Retention policy: how long each resolution is kept

7 Interview questions

Q1. How do you store and query high-volume time-series data? BASIC

A TSDB with append-only, time-chunked, compressed storage; batched writes; rollups plus retention per resolution; queries read the coarsest table that answers them.

Q2. What's the main pitfall? BASIC

High cardinality: tags with too many unique values create millions of series and blow up the index and memory. Keep tags low-cardinality.

Q3. Estimate storage for 1M points/s kept raw for 7 days. SCENARIO

86.4 billion points a day × ~1.4 bytes ≈ 120 GB/day, so ~0.85 TB for 7 days, versus ~10 TB at 16 bytes a point uncompressed.

Q4. A year-long chart takes 40 seconds to load. Fix it. SCENARIO

Serve it from an hourly (or daily) rollup chosen by zoom level, so it reads thousands of points, not billions.

Q5. Why store min, max, sum and count in rollups instead of the average? TRICKY

Sum and count re-aggregate correctly into coarser buckets and give the average; max and min keep spikes an average hides.

Q6. Why is deleting old time-series data cheap in a TSDB but expensive in a regular table? TRICKY

Data is partitioned into time chunks, so retention drops whole files; a regular table must find and delete billions of rows and update indexes.

Next: SD-Q-011 · Geospatial Indexing

Geospatial Indexing

turn two dimensions into cells you can search

Watch the 4-min video

In one line: ordinary indexes are one-dimensional, so spatial indexes map locations into **cells** (geohash prefixes, quadtree leaves, S2 or H3 cell IDs); a nearby search scans the target cell and its **neighbours**, then computes exact distances only for those few candidates.

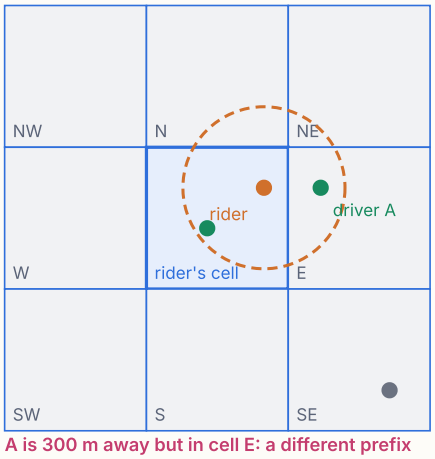
1 Nearest drivers, in milliseconds

A rider opens a ride-hailing app. In the city, 5,000 drivers report their location every 4 seconds (1,250 updates a second), and the app must find available drivers within 2 km in a few milliseconds. For one city you could almost scan them all; across hundreds of cities with millions of drivers and thousands of ride requests a second, comparing every rider with every driver does not scale. You need an index that answers "what is near this point?" directly.

2 Why ordinary indexes fail

A B-tree on `lat` finds a horizontal band around the whole planet; one on `lng` finds a vertical band. A query `WHERE lat BETWEEN … AND lng BETWEEN …` can use only one of them efficiently and must filter the band's millions of rows for the other. Spatial indexes solve this by turning 2D proximity into something a 1D structure can range-scan (a space-filling curve), or by building a tree over 2D regions.

3 Geohash



geohash: interleave longitude and latitude bits

lon bit, lat bit, lon bit … → base-32 string (Z-order curve)
length 5 ≈ 4.9 × 4.9 km · length 6 ≈ 1.2 × 0.6 km · 7 ≈ 153 m
shared prefix ⇒ same cell ⇒ nearby (usually)

```
WHERE geohash LIKE 'tdr1v%' → ordinary B-tree range scan
```

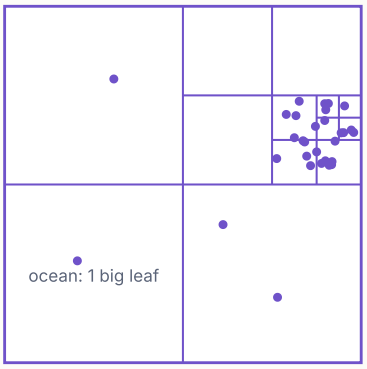
radius search done right

1 choose a length whose cell is ≥ the radius
2 search the rider's cell + its 8 neighbours
3 compute exact (haversine) distance, drop > 2 km
4 sort by distance, then by ETA on the road graph

Figure 1. Search the 3×3 block of cells around the rider, not just the rider's cell: points across a boundary can be close but share no prefix.

Geohash alternately halves longitude and latitude, writing one bit each time, and encodes the bits in base 32. Each extra character shrinks the cell about 32×. Because the bits follow a **Z-order curve**, nearby points *usually* share a prefix, so a prefix query is a range scan in any database or key-value store. The catch: two points either side of a cell boundary (or of the equator or the prime meridian) can have completely different prefixes, so always include the 8 neighbouring cells. Cells are also not equal in size: they shrink in width towards the poles.

4 Quadtrees and R-trees



quadtree: split a square into 4 when it holds > N points

N = e.g. 100; dense downtown → deep, tiny leaves; empty areas stay big
query: descend to the leaves that intersect the search circle
great for uneven, mostly static data (shops, restaurants)
moving points force splits and merges; usually rebuilt periodically

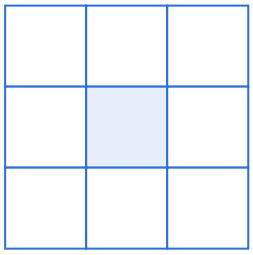
R-tree: nest bounding rectangles of objects

indexes shapes (roads, delivery zones), not just points

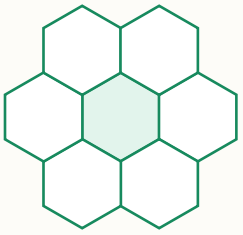
```
PostGIS GiST index: ST_DWithin(geom, point, 2000)
```

Figure 2. A quadtree adapts to density: the dense area in the top-right is split three times, the rest of the map not at all.

5 S2 and H3: modern cell systems



square: 4 neighbours at 1, 4 at 1.41



hexagon: all 6 neighbours at the same distance

H3 (Uber)

16 resolutions, 0–15
res 8 ≈ 0.74 km² per cell
gridDisk(cell, k) = k rings

S2 (Google): cube faces, Hilbert curve, 64-bit cell IDs, 31 levels (0–30)

Figure 3. Why hexagons: every neighbour is equally far away, so k rings of cells approximate a circle well.

S2 projects the sphere onto the six faces of a cube and orders cells along a Hilbert curve, which keeps neighbours closer in ID order than a Z-curve; any region can be covered by a small set of cell ID ranges. **H3** tiles the world with hexagons (plus 12 pentagons per resolution) at 16 resolutions; Uber uses it for dispatch, surge pricing and heat maps, because counting demand per equal-sized hexagon and taking "k rings" around a cell are simple and uniform.

Index	Cells	Strength	Weakness
Geohash	fixed grid, string prefix	works in any DB or KV store; simple	edge effects; cells distort near poles
Quadtree	adaptive squares	handles uneven density	in-memory structure; costly for fast-moving points
R-tree	bounding rectangles	shapes and polygons; PostGIS	updates rebalance the tree
S2	quad cells on a cube, Hilbert order	precise region coverings	more complex library
H3	hexagons, 16 resolutions	uniform neighbours; analytics	hexagons don't nest exactly into parents

6 Moving objects

Drivers move constantly, so the index lives in memory and is updated in place. **Redis GEO** commands store members in a sorted set whose score is a 52-bit interleaved geohash: `GEOADD drivers:pune`

`73.8567 18.5204 d-17` moves a driver in O(log n), and `GEOSEARCH drivers:pune FROMLONLAT 73.85 18.52 BYRADIUS 2 km ASC COUNT 20 WITHDIST` (Redis 6.2+) scans the covering cells and returns exact distances. Sorted-set members have no individual TTL, so stale drivers must be removed explicitly: keep the last update time in a second sorted set and `ZREM` drivers silent for 30 s, or rotate time-bucketed keys. Shard by city or region, since searches never cross a city boundary.

WORKED EXAMPLE: DRIVERS WITHIN 2 KM OF A RIDER

Each driver app sends its position every 4 s; the location service runs `GEOADD drivers:pune lon lat d-17` and records `ZADD drivers:pune:seen <now> d-17`; a sweeper removes drivers unseen for 30 s. A rider requests a ride: `GEOSEARCH … BYRADIUS 2 km ASC COUNT 50 WITHDIST` returns 50 candidates in about a millisecond. The service filters for *available* drivers with the right vehicle type, asks a routing service for ETAs on the road network (2 km straight-line can be 8 minutes across a river), and offers the ride to the best two or three.

TRADE-OFFS

- Smaller cells mean fewer false candidates but more cells to scan for a given radius.
- Geohash is portable and simple; S2/H3 give better geometry at the cost of a library dependency.
- Higher location update rates improve accuracy but multiply writes and battery use.
- Straight-line distance is cheap; road ETA is what matters, so compute it only for the final few candidates.

COMMON MISTAKES

- Searching only the rider's own geohash cell and missing drivers just across the boundary.
- Ranking by straight-line distance and ignoring rivers, one-way streets and traffic.
- Never removing stale positions, so offline drivers keep being offered rides.
- Treating latitude and longitude degrees as equal distances (a degree of longitude shrinks towards the poles).

7 Key terms

Geohash: base-32 string of interleaved lat/lng bits; prefix = cell

Quadtree: recursive 4-way split of space by density

S2 / H3: global cell systems: cube-based squares / hexagons

Neighbour cells: the 8 cells around a geohash cell, searched to fix edge effects

R-tree: tree of nested bounding rectangles, for shapes

Haversine: great-circle distance between two lat/lng points

8 Interview questions

Q1. How do you find nearby points efficiently? BASIC

Map locations into cells with a spatial index, search the target cell and its neighbours, then compute exact distances only for those candidates.

Q2. Geohash vs quadtree? BASIC

Geohash is a fixed grid encoded as string prefixes, works in any database but has edge effects; a quadtree splits adaptively by density, better for uneven data.

Q3. Design 'restaurants near me' for 1 million static restaurants. SCENARIO

Store restaurants with a geohash or use PostGIS (GiST) and ST_DWithin; cache popular cells; data changes rarely, so a quadtree or R-tree works well.

Q4. How do you keep driver locations fresh? SCENARIO

Update every few seconds with GEOADD, record last-seen times, sweep out drivers silent for ~30 s, and shard the index by city.

Q5. Why are hexagons popular for ride-hailing analytics? TRICKY

All six neighbours are equidistant and cells are similar in size, so rings approximate circles and per-cell counts (demand, supply) compare fairly.

Q6. Two points 50 m apart have geohashes with no common prefix. How? TRICKY

They sit either side of a major cell boundary (e.g. the equator or prime meridian); the Z-order curve jumps there, which is why neighbour cells are searched.

Next: SD-Q-012 · Leaderboards backendroadmap.com

Leaderboards

sorted sets, score bands, approximate ranks

Watch the 4-min video

In one line: keep scores in a **sorted set** (Redis ZSET: skip list + hash) so updates, rank lookups and top-N reads are all O(log n); beyond one node, **shard by score band** and add the sizes of higher bands, or estimate **approximate ranks** from a score histogram, keeping exact ranks for the top; time-windowed boards are separate, expiring keys.

1 Your rank among 100 million

A mobile game has 100 million players. After every match a score changes and the player wants to see "you are #400,012" immediately. In SQL that is `SELECT COUNT(*) FROM scores WHERE score > 9820`: even with an index, the database walks every entry above you, so a mid-table player's rank costs tens of millions of index entries, for every request, while scores keep changing. `RANK() OVER (ORDER BY score DESC)` sorts the whole table. Rank needs a structure that knows *how many* items are before a position without visiting them.

2 Redis sorted sets

```
ZADD lb:all 9820 ana              # add or update score       O(log n)
ZINCRBY lb:all 150 ana            # +150 after a match        O(log n)
ZREVRANK lb:all ana               # 0-based rank, high first  O(log n)
ZREVRANGE lb:all 0 9 WITHSCORES   # top 10                    O(log n + 10)
ZREVRANGE lb:all 1402112 1402122  # 5 above + 5 below me      O(log n + 11)
ZCOUNT lb:all 9820 +inf           # players at or above 9820  O(log n)
```

Figure 1. The core commands. Ranks are 0-based; ZREVRANK / ZREVRANGE order from highest score (in Redis 6.2+ also `ZRANGE … REV`).

A sorted set maps members to scores and keeps them ordered. Small sets use a compact list; larger ones use a **skip list** (ordered by score, then member) plus a **hash table** (member → score), so both "what is ana's score?" (O(1)) and "what is at rank k?" (O(log n)) are fast.

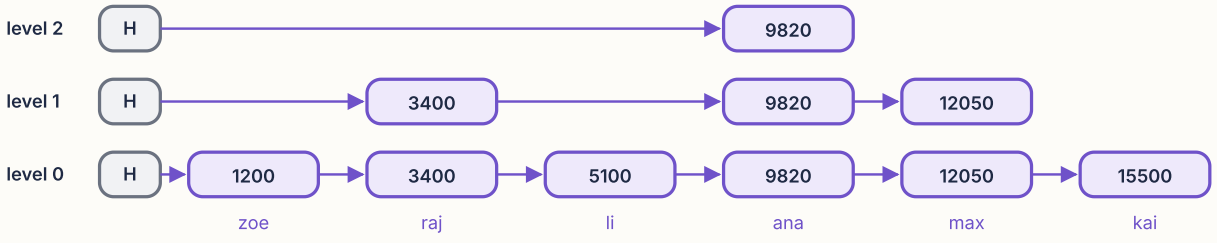


Figure 2. A skip list: sorted linked lists in levels; higher levels skip further. Spans on each link give the rank for free.

Ties. Equal scores are ordered by member name, which is rarely fair. Encode a tie-breaker into the score: a ZSET score is a double, which represents integers exactly up to 2^{53}. With points up to 1 million (20 bits)

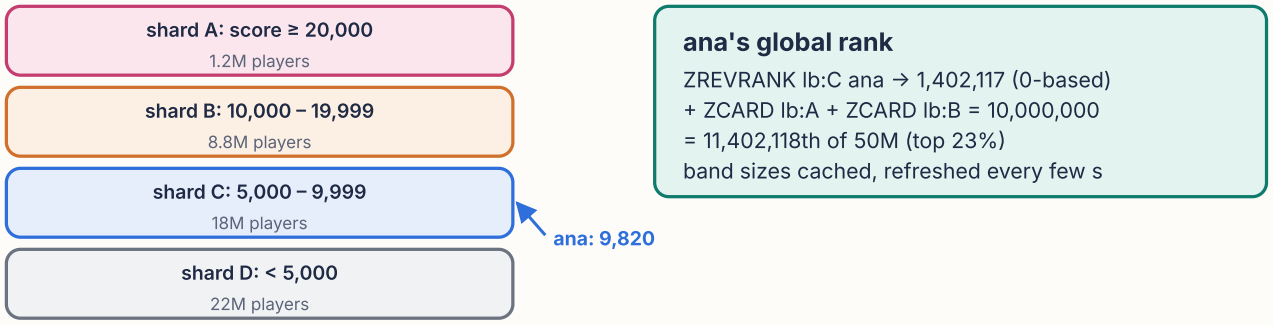
and a 32-bit "seconds until 2100" value: `score = points × 2³² + (T_end − t)`, so among equal points the earlier achiever ranks higher, within 52 bits.

Around me and friends. `ZREVRANGE` with rank ± 5 gives neighbours. For a friends board, fetch friends' scores with `ZMSCORE` (or keep a small per-user set) and sort a few hundred values in the app.

3 Beyond one node: shard by score band

One sorted set comfortably holds tens of millions of members on one node (memory is very roughly on the order of 100 bytes per member, so 50M members is several GB), but every write for every player hits one single-threaded node. Two ways to split:

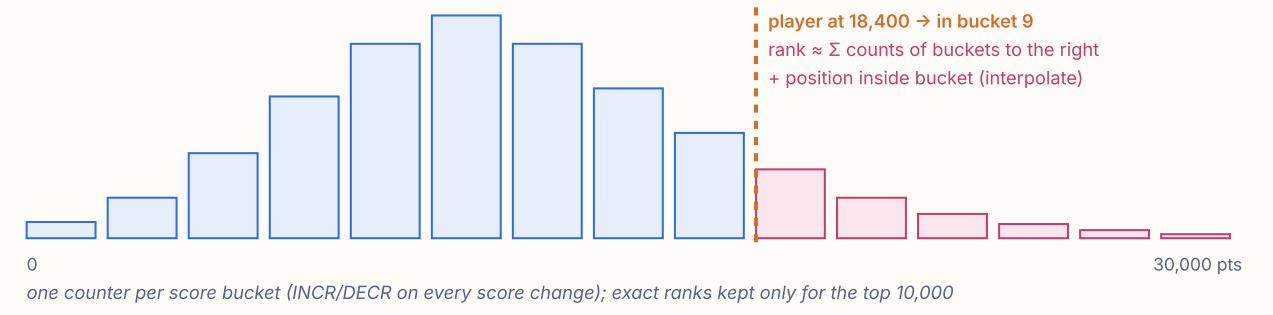
- **By player ID:** writes spread evenly, but a global rank needs every shard to count players above the score: scatter-gather on every lookup. Fine for per-region or per-shard boards.
- **By score range:** each shard holds a band. A player's global rank is their rank within their band plus the sizes of all higher bands (cached `ZCARD`s). A score that crosses a band boundary is a `ZREM` on one shard and a `ZADD` on another.



bands are chosen from the score distribution so shards hold similar load; low scores are dense, top bands sparse

Figure 3. Score-band sharding. One local rank lookup plus a few cached counts gives an exact global rank.

4 Approximate ranks



one counter per score bucket (INCR/DECR on every score change); exact ranks kept only for the top 10,000

Figure 4. A score histogram. A few thousand counters estimate any player's rank in O(buckets) with no per-player index.

At #11,402,118 nobody notices whether the number is exact; "top 23%" is often more meaningful. Keep a **histogram**: a counter per score bucket, incremented and decremented as scores move. A rank is the sum of counts above the player's bucket, plus an interpolated position inside it. Keep an exact sorted set only for the **top 10,000**, where players do compare positions, and show everyone else a rounded rank or percentile.

Approach	Rank lookup	Exact?	Use when
SQL COUNT / RANK()	O(rank) or full sort	yes	small boards, offline reports
One Redis ZSET	O(log n)	yes	up to tens of millions
ZSETs by score band	O(log n) + cached counts	yes	very large boards, exact ranks needed
Histogram + top-N ZSET	O(buckets)	top only	huge boards where "top x%" is enough

5 Daily, weekly and all-time boards

Each window is its own key: `lb:daily:2026-10-31`, `lb:weekly:2026-W44`, `lb:all`. Every score update writes to all of them (pipelined); window keys get an `EXPIRE` a few days after the window closes, so old boards clean themselves up, and a reset is just a new key name. A weekly board can also be built from daily keys with `ZUNIONSTORE lb:weekly 7 lb:daily:… AGGREGATE SUM` (or MAX, depending on the game's rules).

Redis is a fast **view**, not the source of truth: write scores durably to a database (after server-side validation against cheating), and be able to rebuild every sorted set from it.

WORKED EXAMPLE: A GLOBAL BOARD FOR 50 MILLION PLAYERS

A match ends; the game server validates the result and writes ana's new total, 9,820, to the scores table. A consumer then pipelines `ZADD` to her band shard (C: 5,000–9,999) and to `lb:daily:2026-10-31` and `lb:weekly:2026-W44`. The top 100 come from shard A with `ZREVRANGE 0 99`, cached for one second because millions of clients request it. Ana's rank is her rank in shard C, 1,402,117, plus the cached sizes of A and B (10,000,000): **#11,402,118**, shown as "top 23%" with her five neighbours from shard C.

TRADE-OFFS

- Exact ranks for everyone cost memory and hot-node writes; approximate ranks are nearly free.
- Score-band shards give exact global ranks but need rebalancing as the score distribution drifts.
- ID-based shards balance writes perfectly but make global rank a scatter-gather.
- Caching the top 100 for a second removes load but lets it lag by up to that second.

COMMON MISTAKES

- Computing rank with COUNT(*) on every request.
- Treating Redis as the only copy of scores.
- Ignoring ties, so equal scores rank alphabetically by player ID.
- Packing score and timestamp into more than 53 bits, silently losing precision in the double.

6 Key terms

Sorted set (ZSET): members ordered by score; skip list + hash

ZREVRANK: 0-based rank from the highest score

Score histogram: counts per score bucket for approximate ranks

Skip list: layered linked lists giving O(log n) search and rank

Score band: shard holding a range of scores

Composite score: points and tie-breaker packed into one number

7 Interview questions

Q1. How do you get a user's rank among 100M players? BASIC
Redis sorted sets give ZREVRANK in O(log n); shard by score range and add the sizes of higher bands, or estimate approximate ranks from a score histogram, keeping exact ranks for the top.

Q2. How do you support daily and weekly boards? BASIC
Separate sorted sets per time window, written on every update, with keys that expire after the window ends.

Q3. Two players have the same score. Who ranks higher, and how do you implement it? SCENARIO
Usually whoever reached it first: pack points and an inverted timestamp into one score that fits in 53 bits.

Q4. The leaderboard Redis node is lost. What happens? SCENARIO
Scores are durable in the database, so you fail over to a replica or rebuild the sorted sets from the database; ranks are briefly stale, not lost.

Q5. Why is SQL's COUNT(*) WHERE score > x slow even with an index? TRICKY
A B-tree doesn't store subtree counts, so it must walk every entry above x; a skip list with spans (or an order-statistic tree) gets the count in O(log n).

Q6. Why not shard the leaderboard by player ID? TRICKY
Writes balance well, but a global rank then needs every shard to count players with higher scores on every lookup.

Next: SD-Q-013 · Counting at Scale backendroadmap.com

Counting at Scale

shard it, batch it, or estimate it

Watch the 4-min video

In one line: a single counter row becomes a **hot row**, so high-volume counts use **sharded counters** or **in-memory batching** (write-behind); unique counts use **HyperLogLog**, a 12 KB sketch with ~0.8% error that merges across hours and pages. Decide per metric whether it must be exact (money) or can be approximate (views).

1 A simple number on every page

A news site shows "1.2M reads" and "34,000 unique visitors today" on every article. A popular article gets 50,000 views a minute. `UPDATE articles SET views = views + 1` makes every request queue on one row lock; and counting unique visitors exactly means remembering every visitor ID for every page, every day. Two different problems: **throughput** for plain counts, **memory** for distinct counts.

2 Plain counts without a hot row

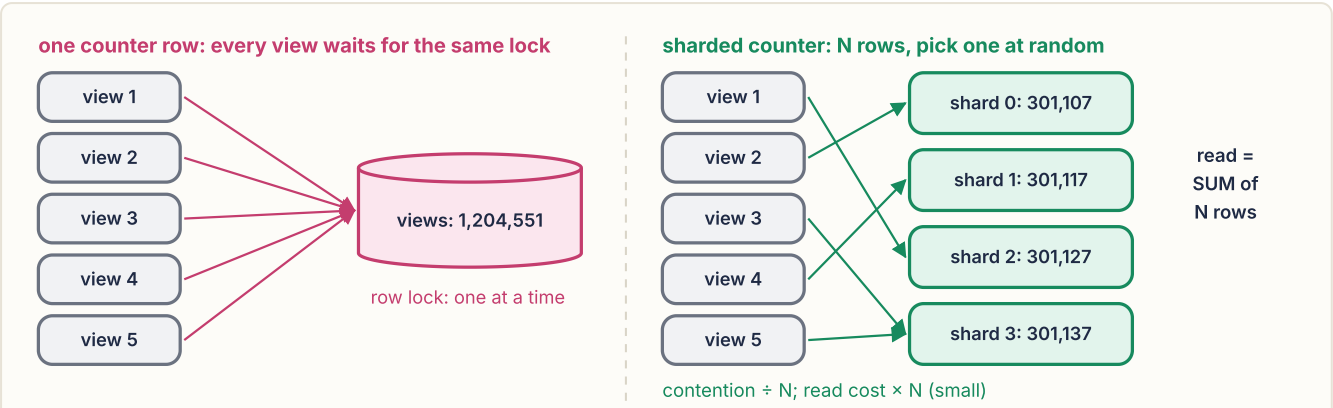


Figure 1. Spreading one logical counter across N rows divides lock contention by N; a read sums N small rows.

Every database has a per-row (or per-document) write ceiling: Firestore documents its limit as roughly one sustained write per second per document, which is why Google's docs teach **sharded counters**; a relational row serialises increments on its lock. Shard the counter into N rows (`counter_id, shard_no, n`), increment a random shard, and read with `SUM(n)`. Choose N from the peak write rate ÷ safe rate per row, and cache the sum for a few seconds.

An in-memory store helps too: Redis `INCR` is atomic and a single node handles on the order of 100,000 simple operations a second, but one key still lives on one node, so a truly viral counter may need sharded keys as well.

❸ Write-behind: count in memory, flush in batches

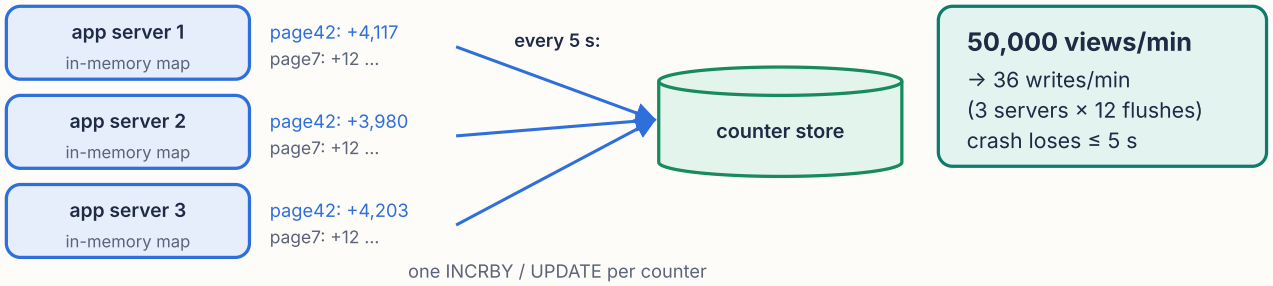


Figure 2. Each app server aggregates locally and sends one increment per counter per interval.

Each app server keeps a map of pending increments and flushes it every few seconds with one `INCRBY` or `UPDATE … SET n = n + 4117` per counter. 50,000 writes a minute become a few dozen. The cost: a crash loses up to one interval of counts, and displayed numbers lag by the interval. That is fine for views and likes counts, never for money. If counts must not be lost, publish view events to Kafka instead and let a stream job (lesson 32) aggregate per minute with checkpoints: the same batching, made durable.

❹ Unique counts: HyperLogLog

Exact unique counts need the set of IDs. One million pages × 10,000 unique visitors × 16-byte IDs is about **160 GB per day**, and weekly uniques can't be computed by adding daily numbers (the same person visits on several days). **HyperLogLog** (Flajolet et al., 2007) estimates the number of distinct items in fixed memory.

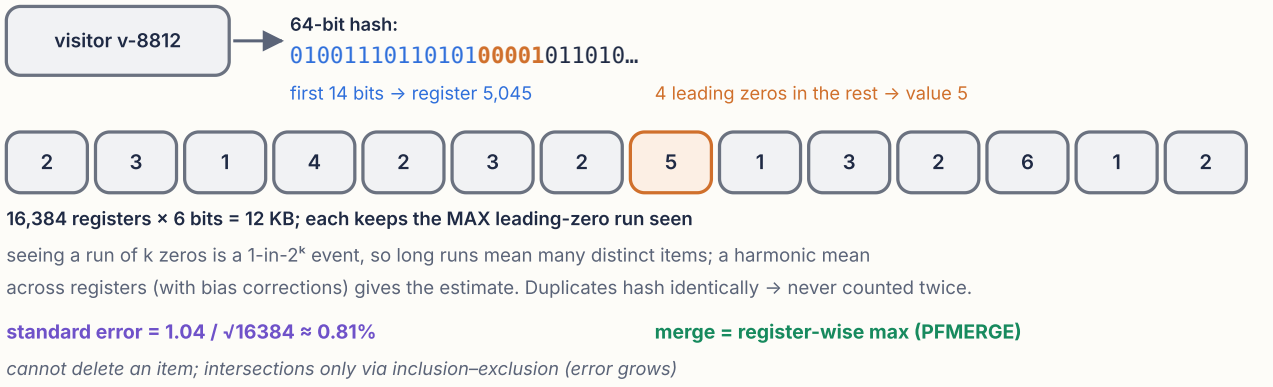


Figure 3. HyperLogLog as Redis implements it (p = 14). Rare long zero-runs in hashes reveal how many different items have been seen.

In Redis: `PFADD key item`, `PFCOUNT key` (also accepts several keys and returns the size of their union), and `PFMERGE dest k1 k2 …`. Small HLLs use a sparse encoding far below 12 KB; the dense form is capped at 12 KB however many items you add. Because merging takes the per-register maximum, hourly sketches combine into daily ones, daily into weekly, and page sketches into a site-wide one, with no double counting.

For **how often each item** appears (rather than how many distinct items), use a **Count-Min Sketch**: a small grid of counters with one hash per row; an item's estimate is the minimum of its counters, so it can overestimate (collisions) but never underestimate (lessons 39 and 40).

Metric	Exact?	Technique
Account balance, inventory, invoices	exact, durable	transactions, idempotency keys, ledgers
Likes (one per user)	exact set, approximate display	a row per (user, post) with a unique key; cached count
Page views, video plays	approximate, eventually consistent	sharded counters, write-behind, stream aggregation
Unique visitors	approximate	HyperLogLog per page per day, merged
Trending counts per item	approximate	Count-Min Sketch + heap (lesson 40)

WORKED EXAMPLE: UNIQUE VISITORS PER PAGE

Each page view runs `PFADD uv:page42:2026-10-31 v-8812`. Today's number is `PFCOUNT uv:page42:2026-10-31`; this week's is `PFCOUNT` over the seven daily keys (or `PFMERGE uv:page42:2026-W44` them once and cache). Each key is at most ~12 KB, so a million pages cost at most ~12 GB a day (less, thanks to sparse encoding for quiet pages), instead of storing billions of IDs, and every number is within about 1% (standard error 0.81%). Daily keys get a 35-day `EXPIRE`. The view count on the same page uses write-behind: each server flushes `INCRBY views:page42 n` every 5 seconds.

TRADE-OFFS

- More counter shards: more write throughput, slightly slower and less consistent reads.
- Longer flush intervals: fewer writes, but staler numbers and more loss on a crash.
- HyperLogLog: constant memory and mergeable, but ~1% error, no deletes, no list of who visited.
- Exact counting is always possible; the question is whether the metric is worth its cost.

COMMON MISTAKES

- One counter row for a viral item.
- Adding daily unique counts to get weekly uniques (counts people several times).
- Using write-behind or approximate counts for money or stock levels.
- Counting likes by incrementing a number, so double-taps and retries count twice.

5 Key terms

Hot row / key: one record receiving most of the writes

Write-behind: aggregate in memory, flush periodically

PFADD / PFCOUNT / PFMERGE: Redis HLL add, estimate, merge

Sharded counter: one count split over N rows, summed on read

HyperLogLog: fixed-size sketch estimating distinct counts

Count-Min Sketch: frequency estimates that only overestimate

6 Interview questions

Q1. How do you count unique users cheaply? BASIC

HyperLogLog: a fixed ~12 KB sketch that estimates distinct counts within about 1% and merges across hours or pages.

Q2. How do you count high-volume views without a hot row? BASIC
Shard the counter across rows and sum on read, or count in memory and flush periodically (write-behind).

Q3. Daily uniques are stored as numbers. Product wants weekly uniques. What now? SCENARIO
You can't add them; store a HyperLogLog per day instead, then PFCOUNT or PFMERGE the seven sketches.

Q4. How many counter shards do you need for 2,000 increments/s if one row handles ~200/s safely? SCENARIO
At least 10, more for headroom (say 20); reads sum 20 rows and are cached briefly.

Q5. Why can HyperLogLog merge without double-counting? TRICKY
Each register stores the maximum zero-run for its hash bucket; the same visitor hashes identically everywhere, so max-merging equals adding all items to one sketch.

Q6. When is write-behind unacceptable? TRICKY
When losing a few seconds of increments matters (payments, stock, quotas); use transactional updates or durable event logs instead.

Next: SD-Q-014 · Probabilistic Data Structures backendroadmap.com

Probabilistic Data Structures

a little error, a lot less memory

Watch the 4-min video

In one line: probabilistic structures trade a small, **bounded** error for fixed and tiny memory: a **Bloom filter** answers "definitely not" or "maybe" for set membership, a **cuckoo filter** does the same with deletes, **HyperLogLog** counts distinct items and a **Count-Min Sketch** estimates frequencies.

1 A billion keys, mostly missing

A database stores a billion keys on disk. Most lookups are for keys that don't exist (cache misses, existence checks, joins), and each one costs disk reads just to learn "not here". Keeping all keys in memory would take tens of gigabytes. A structure of ~1.2 GB that says "definitely not here" for 99% of absent keys removes almost all of those reads.

2 Bloom filters

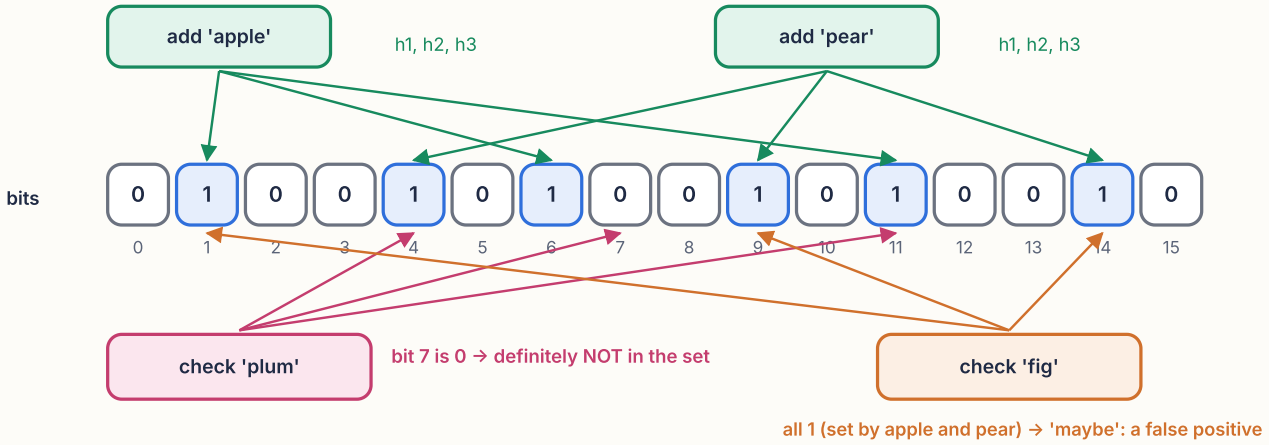


Figure 1. A 16-bit Bloom filter with k = 3 hash functions. A zero bit proves absence; all ones only suggests presence.

A Bloom filter (Burton Bloom, 1970) is an array of *m* bits and *k* hash functions. **Add:** set the *k* bits the item hashes to. **Check:** if any of those bits is 0, the item was never added; if all are 1, it *probably* was. So there are **no false negatives**, only false positives, and the rate is chosen by sizing:

$$\text{FPR} \approx (1 - e^{-kn/m})^k \quad \cdot \quad \text{best } k = (m/n)\cdot \ln 2 \quad \cdot \quad m/n \approx -1.44\cdot \log_2(\text{FPR})$$

Bits per item (m/n)	Best k	False-positive rate	1 billion items
4.8	3	≈ 10%	0.6 GB
9.6	7	≈ 1%	1.2 GB
14.4	10	≈ 0.1%	1.8 GB
19.2	13	≈ 0.01%	2.4 GB

Each extra ~4.8 bits per item cuts false positives 10×, and the item's own size doesn't matter: a 1 KB URL costs the same 10 bits as a user ID. In practice the k hashes come from two base hashes ($h_1 + i \cdot h_2$). Limits: you **can't delete** (clearing a bit might erase other items; counting Bloom filters use small counters instead of bits, at ~4× the memory), you **can't list** the items, and the false-positive rate rises if you add more items than planned, so size for growth or chain new filters (scalable Bloom filters).

3 Where Bloom filters earn their keep

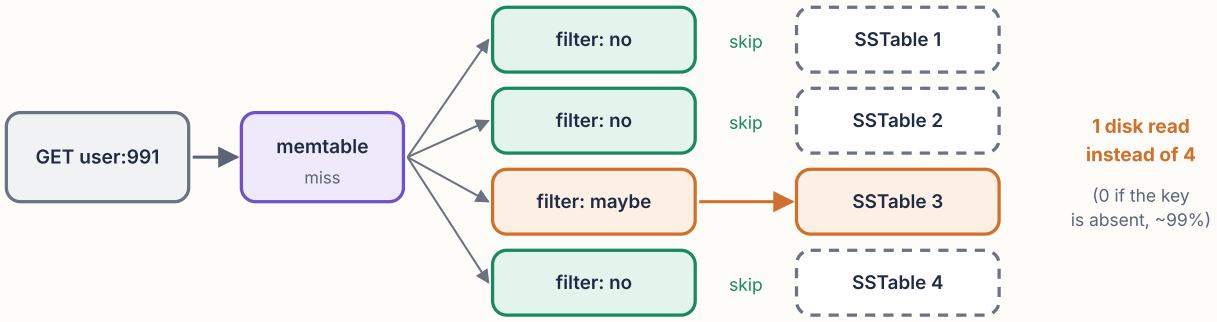


Figure 2. An LSM-tree read: each on-disk file has its own Bloom filter, kept in memory, so files that can't contain the key are never read.

LSM-tree databases (RocksDB, Cassandra, HBase, LevelDB) write data into many immutable files. A point lookup could have to check every file; a per-file Bloom filter turns most of those into memory checks. RocksDB users commonly configure about 10 bits per key (~1% FPR); Cassandra exposes the target directly as the table option `bloom_filter_fp_chance`. Other uses: CDNs and caches avoiding lookups for objects never seen, "is this username possibly taken?" before a database query, a feed's seen-items filter (lesson 29), and distributed joins that ship a filter instead of the key set.

4 Cuckoo filters: Bloom with deletes

A cuckoo filter (Fan et al., 2014) stores a short **fingerprint** (e.g. 8–16 bits) of each item in a table of buckets, typically 4 slots each. Every item has two candidate buckets: $i_1 = hash(x)$ and $i_2 = i_1$ *XOR* $hash(fingerprint)$, so either can be computed from the other plus the fingerprint. If both are full, an existing fingerprint is kicked to its alternate bucket, like a cuckoo chick. **Delete** removes a matching fingerprint. Lookups check just two buckets, and at low false-positive rates (below a few percent) it uses less space than a Bloom filter. Downsides: inserts can fail once the table is about 95% full, and deleting an item that was never added can remove another item's fingerprint.

5 The family

Structure	Question	Error	Memory	Deletes / merge
Bloom filter	is x in the set?	false positives only	~10 bits/item for 1%	no deletes; union = OR
Cuckoo filter	is x in the set?	false positives only	fingerprint + overhead per item	deletes yes
HyperLogLog	how many distinct?	±0.81% (12 KB, Redis)	fixed 12 KB	merge yes (lesson 38)
Count-Min Sketch	how often is x?	overestimates only, ≤ εN with prob. 1 − δ	w = ⌈e/ε⌉ × d = ⌈ln 1/δ⌉ counters	merge by adding grids
t-digest / KLL	what is the p99?	small rank error	a few KB	merge yes

For a Count-Min Sketch with ε = 0.1% and δ = 1%: width ⌈2.718/0.001⌉ = 2,719 and depth ⌈ln 100⌉ = 5, so 13,595 counters (~54 KB with 4-byte counters) estimate the frequency of any item to within 0.1% of the total count, with 99% probability, however many distinct items there are. It powers heavy-hitter detection in lesson 40. Redis Stack (RedisBloom) offers all of these: `BF.*`, `CF.*`, `CMS.*`, `TOPK.*`, plus built-in `PF*` for HyperLogLog.

WORKED EXAMPLE: SKIPPING DISK READS FOR MISSING KEYS

A Cassandra node holds 200 million rows spread over 40 SSTables. A `GET user:991` misses the memtable; the node checks the in-memory Bloom filter of each SSTable whose key range could contain it. 39 say "no" and are skipped without touching disk; 1 says "maybe" and is read. With a 1% false-positive rate, a lookup for an absent key reads a needless file only about once per 100 filters consulted, so 99 out of 100 useless reads disappear. The filters cost about 10 bits × 200M = 250 MB of RAM for the whole node.

TRADE-OFFS

- More bits per item: fewer false positives, more memory (≈4.8 bits per 10× improvement).
- Bloom: simplest and fastest, no deletes. Cuckoo: deletes and better space at low FPR, but inserts can fail when full.
- Sketches answer one question each; you can't recover the items or ask a different question later.
- Errors are bounded only if you size for the real item count; overfilled filters degrade quickly.

COMMON MISTAKES

- Using a "maybe" answer for permissions, billing or fraud blocks without an exact follow-up check.
- Trying to delete from a standard Bloom filter.
- Sizing for today's item count, then inserting ten times as many.
- Thinking a Bloom filter can give false negatives: a "no" is always correct.

6 Key terms

Bloom filter: bit array + k hashes; no false negatives

False positive: 'maybe present' for an item never added

Cuckoo filter: fingerprints in 2 candidate buckets; supports delete

Fingerprint: few-bit hash stored instead of the item

Count-Min Sketch: d × w counters; frequency, overestimates only

Sketch: small summary answering one question with bounded error

7 Interview questions

Q1. When would you use a Bloom filter? BASIC

For a fast, memory-cheap membership test where an occasional false positive is acceptable, such as skipping disk reads, cache checks or seen-item filters; a 'no' is always correct.

Q2. What's the general trade-off? BASIC

Bounded error in fixed, small memory: HyperLogLog for distinct counts, Count-Min Sketch for frequencies, cuckoo filters when you need deletes.

Q3. Size a Bloom filter for 100 million items at 1% false positives. SCENARIO

About 9.6 bits per item with 7 hashes: ~960 million bits ≈ 120 MB.

Q4. Users must be able to 'unsee' items in a seen filter. Which structure? SCENARIO

A cuckoo filter (or a counting Bloom filter), since standard Bloom filters can't delete.

Q5. Can a Bloom filter ever say 'not present' for an item that was added? TRICKY

No: adding sets all k bits and bits are never cleared, so a member always finds all ones. Only false positives are possible.

Q6. Why does a Count-Min Sketch only overestimate? TRICKY

Each counter for an item also absorbs collisions from other items, never subtracts; taking the minimum across rows picks the least polluted counter.

Next: SD-Q-015 · Top-K Heavy Hitters backendroadmap.com

Top-K Heavy Hitters

count in fixed memory, merge, compare with normal

Watch the 4-min video

In one line: to find the K most frequent items in a huge stream, count approximately in **fixed memory** (Count-Min Sketch + min-heap, or the Space-Saving algorithm), count in **time buckets** so results reflect the recent window, score against a **baseline** so "trending" means unusual, and **merge local top lists** from partitioned workers into a global top K.

1 Trending hashtags, right now

Thousands of posts arrive every second containing hundreds of thousands of different hashtags, most seen once or twice. The product wants the top 10 of the last hour, updated every few seconds, and it must survive the spike when something big happens. Keeping an exact counter for every hashtag ever seen, for every minute, is wasteful; most of that memory goes to the long tail that will never trend. This is the **top-K** or **heavy hitters** problem.

2 The exact way: batch

For yesterday's top hashtags, a MapReduce or Spark job is exact and simple: map each post to its hashtags, reduce by hashtag to sum counts, then each reducer keeps its **local top K** and a final step merges them. Because the shuffle sends every occurrence of a hashtag to one reducer, each count is complete and the merged top K is exact. This is the right answer for reports. It is not the answer for "last hour, every 10 seconds".

3 Streaming in fixed memory

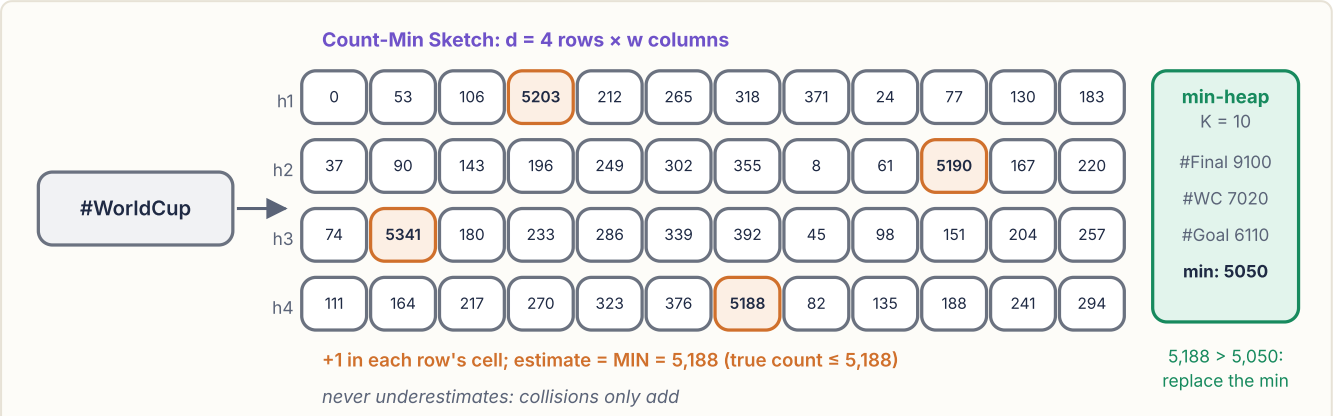


Figure 1. Each occurrence increments one counter per row; the estimate is the minimum. A size-K min-heap keeps the current leaders.

Count-Min Sketch + heap. The sketch holds approximate counts for *all* items in d × w counters (lesson 39). For each arrival: increment the sketch, read the estimate, and if it beats the heap's minimum, replace the minimum (or update the item's entry if it is already in the heap, using a side map). Memory is fixed, O(d·w + K), however many distinct hashtags appear. The error is one-sided: a rare tag that collides with popular ones may look bigger than it is, never smaller.

Space-Saving (Metwally et al., 2005) is often better in practice: keep exactly m counters. A tracked item increments its own counter; an untracked item, when all counters are in use, **evicts the item with the smallest count** and inherits that count + 1. Each count overestimates by at most the evicted minimum, and any item occurring more than N/m times is guaranteed to be tracked.

Arrival	Counters (m = 3)	What happened
a a b c	a:2 · b:1 · c:1	three items fit
d	a:2 · **d:2** · c:1	d evicts b (min 1) → 1 + 1 = 2 (over by ≤ 1)
a	a:3 · d:2 · c:1	tracked item: plain increment
e	a:3 · d:2 · **e:2**	e evicts c (min 1); a stays on top

4 Recent, and unusual

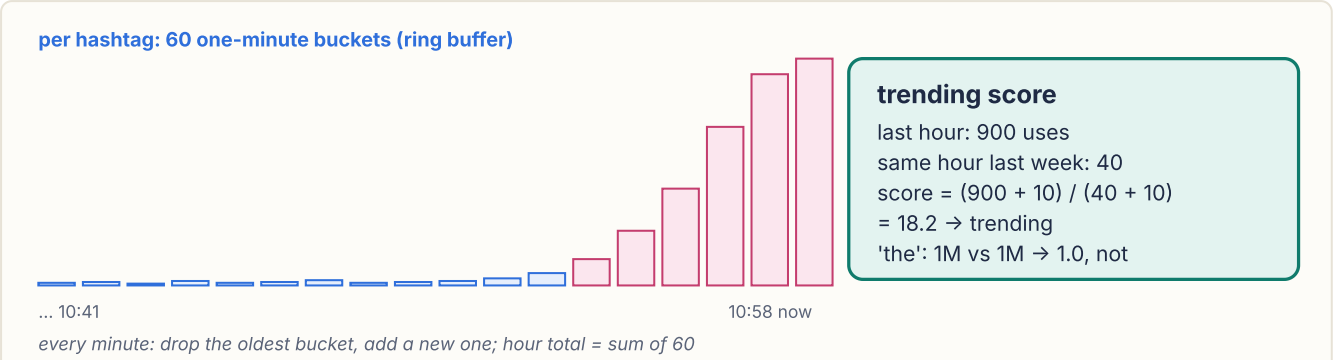


Figure 2. Bucketed counts give a sliding hour; comparing with a baseline separates a spike from a word that is always common.

Windows. Keep 60 one-minute buckets per worker (each a count map or its own sketch). The hour's count is the sum of 60; every minute the oldest bucket is dropped, so items fall out cleanly. An alternative is **exponential decay**: `score = score · e^{−λΔt} + 1` on each arrival, with λ set from a half-life (say 30 minutes); old activity fades gradually and only one number per item is stored.

Popular is not trending. "the" or #love is always near the top. Trending compares the current rate with what is normal for that item: the same hour last week, or a long-term moving average. A smoothed ratio `(current + α) / (baseline + α)` stops tiny baselines from producing huge scores; statistical versions use a z-score. Add guards: a minimum absolute count, and a minimum number of **distinct users** per hashtag (an HLL per tag, lesson 38), so one bot posting 10,000 times can't trend.

5 Distributing the work

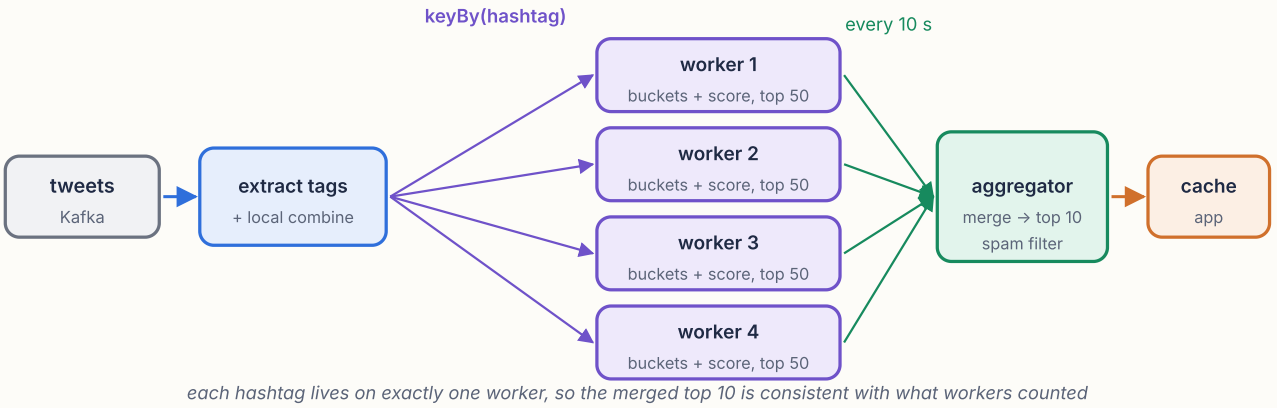


Figure 3. Partition by hashtag, pre-combine upstream, and merge small local top lists centrally every few seconds.

Partition the stream by hashtag (`keyBy`), so every occurrence of a tag reaches the same worker and its count there is complete. Each worker keeps buckets, scores, and a local top list; every 10 seconds it sends its top 50 to an aggregator that merges them into the global top 10. Two subtleties:

- **If partitioning is not by item** (e.g. posts spread round-robin), each worker sees only part of each item's count. An item moderately popular everywhere may miss every local top 10 yet be first globally. Send more candidates than K (top 50 for a top 10) and, for exactness, re-query the candidates' counts from all workers.
- **Hot keys:** keying by hashtag puts a viral tag's entire load on one worker. Pre-aggregate upstream (count per tag per second before the shuffle), which turns a million events into one message per producer per second.

WORKED EXAMPLE: TRENDING HASHTAGS IN THE LAST HOUR

Posts flow through Kafka to a Flink job that extracts hashtags, pre-sums them per second and keys by hashtag across 32 workers. Each worker keeps 60 one-minute count buckets per tag it owns, plus last week's hourly baseline loaded from a table, and an HLL of distinct authors. #WorldCupFinal has 900 uses this hour from 640 authors against a baseline of 40: score (900 + 10) / (40 + 10) ≈ 18. Every 10 seconds each worker sends its top 50 by score to an aggregator, which merges 1,600 candidates, drops blocked terms and spam, and writes the top 10 to a cache key that the app reads; the whole list is at most about 10 seconds old.

TRADE-OFFS

- Exact counting is simple but memory grows with distinct items; sketches and Space-Saving fix memory at the cost of bounded error.
- More candidates per worker (50 vs 10): more accurate global top K, more merge traffic.
- Shorter windows or faster decay react quickly but are noisier and easier to game.
- Baseline scoring finds novelty but needs history; brand-new tags need a sensible default baseline.

COMMON MISTAKES

- Ranking by raw count, so the same evergreen words always "trend".
- Merging local top-K lists from workers that each saw only part of every item's count, and calling it exact.
- No defence against one account or a bot ring pushing a tag.
- Keying by hashtag without pre-aggregation, so a viral tag overloads one worker.

6 Key terms

Heavy hitters: items whose frequency exceeds a threshold of the stream

Top-K: the K most frequent (or highest-scoring) items

Count-Min Sketch + heap: approximate counts for all, exact list of leaders

Space-Saving: m counters; new items evict the minimum and inherit it

Time buckets: per-minute counts summed into a sliding window

Baseline: normal rate for an item, used to score trending

7 Interview questions

Q1. Design a top-K trending system. BASIC

Partition the stream; workers count in time buckets (Count-Min Sketch + heap if the item space is huge), score against a baseline, and merge local top lists into a global top K every few seconds.

Q2. Exact or approximate? BASIC

Exact for batch reports; approximate for real-time streams, where fixed memory matters more than perfect counts.

Q3. The word 'the' is always trending. Fix it. SCENARIO

Score by the ratio of the current rate to the item's normal rate (same hour last week or moving average), with smoothing and minimum counts.

Q4. A bot ring makes a hashtag trend. What do you add? SCENARIO

Count distinct authors per tag (HLL) with a minimum, weight by account quality, and filter spam and blocked terms in the aggregator.

Q5. Why might merging each worker's top 10 miss the true global #1? TRICKY

If workers don't own whole items, an item can be 11th on every worker but first in total; send more candidates or partition by item.

Q6. What does Space-Saving guarantee? TRICKY

With m counters, every item with frequency above N/m is tracked, and each reported count overestimates by at most the minimum counter value.

Q7. Sliding buckets or exponential decay? TRICKY

Buckets give an exact, explainable 'last hour' at 60× the counters; decay needs one number per item and fades smoothly, but has no hard window.

Next: SD-R-001 · Tenant Isolation backendroadmap.com

Tenant Isolation

silo, pool or bridge: chosen layer by layer

Watch the 4-min video

In one line: tenant isolation is a dial, not a switch: **silo** gives each tenant dedicated resources, **pool** shares everything and isolates in code, and real SaaS products set the dial per layer (compute, data, network) and per tier, with a routing layer that hides the difference from the product.

1 The bank's three questions

Your SaaS serves 3,000 small businesses on shared infrastructure. A bank wants to sign up, and its security questionnaire asks three things: **where exactly is our data** (which database, region and backups), **who shares our servers**, and **what happens if another customer's workload goes wild** or a bug crosses tenant boundaries. Each answer is set by your isolation model, and that model also sets your cost per tenant, how you deploy, and how fast you can onboard a new customer.

Two different risks hide in those questions. **Data isolation**: can tenant A ever read or change tenant B's data? **Performance isolation**: can tenant A's load slow down tenant B (the noisy neighbour, lesson 44)? A design can be strong on one and weak on the other. Note also that isolation is not the same as authentication: a correctly logged-in user is still the main threat if the code forgets to check whose data they are asking for (lesson 43).

2 Silo, pool and bridge

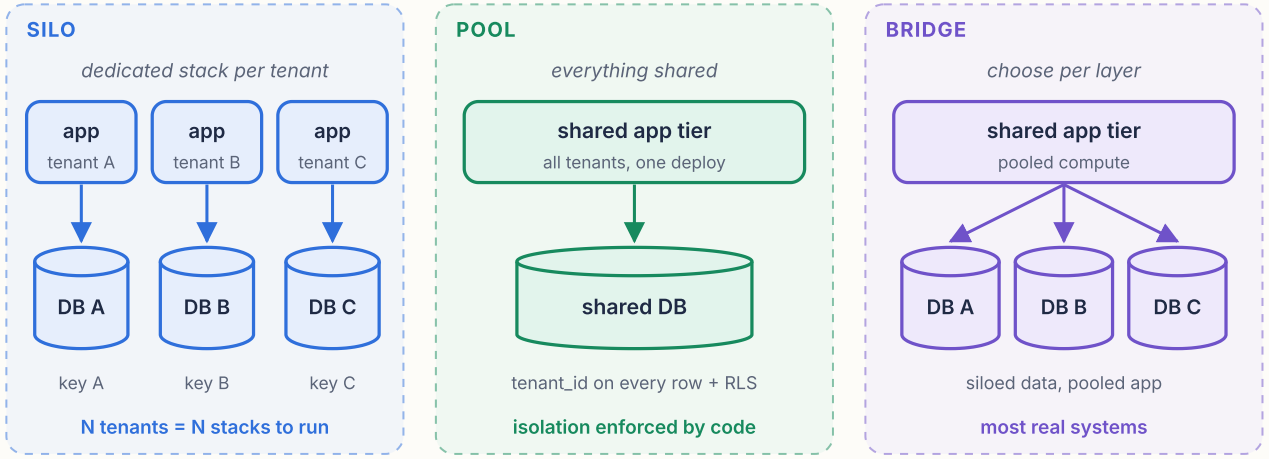


Figure 1. The three models from the AWS SaaS Lens. The bridge mixes them layer by layer; here the app is pooled and the data siloed.

Silo: each tenant gets its own stack: compute, database, often its own cloud account or subscription and network. A bug in one tenant's stack cannot touch another's data, blast radius is one tenant, and cost per tenant is easy to measure. But you pay for idle capacity in every stack, onboarding means provisioning infrastructure (minutes to hours, so it must be automated with infrastructure as code), and every release must roll out to N stacks. Silo only works if every stack runs the **same code version**; per-tenant forks turn a SaaS into a consulting business.

Pool: all tenants share the same services and databases, and every row and request carries a `tenant_id`. One deployment, high utilisation (the peaks of different tenants rarely line up), onboarding is an insert into a table. But isolation is **logical**: one missing filter leaks data, one heavy tenant slows everyone, and per-tenant cost must be estimated from metering (lesson 45) rather than read off a bill.

Bridge: choose silo or pool separately for each layer. Common combinations: pooled web and API tier with a database (or schema) per tenant; pooled compute with siloed storage for regulated tenants; pooled everything except a dedicated worker pool for heavy batch tenants.

	Silo	Pool	Bridge (app pooled, data siloed)
Data isolation	physical, easy to prove	logical: code + RLS	physical for data
Noisy neighbour	none (own resources)	main risk	in the shared app tier
Cost per tenant	high, includes idle capacity	lowest	medium
Onboarding	provision a stack	insert a row	create a database
Deploy / migrate	N times	once	app once, schema N times
Cost attribution	read off the cloud bill	needs metering	partly direct
Fits	regulated, very large tenants	many small tenants	most growing SaaS

3 Isolation happens at three levels

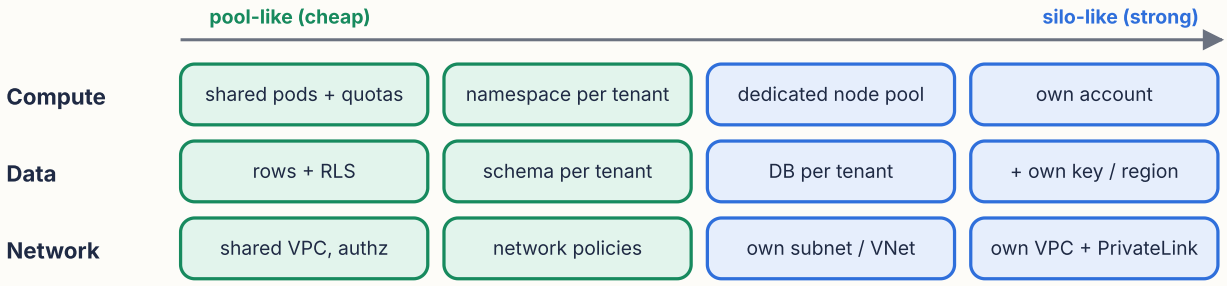


Figure 2. Each layer has its own ladder of options. Pick a rung per layer, and per tier, instead of one model for the whole system.

- **Compute.** Shared pods with per-tenant quotas and concurrency limits; a Kubernetes namespace per tenant with a `ResourceQuota` and `NetworkPolicy`; a dedicated node pool (taints and tolerations keep other tenants' pods off it, lesson 57); or a separate cloud account or subscription, the strongest boundary because even the cloud IAM is separate. Remember that containers share the host kernel (lesson 55): for untrusted tenant code, use VM-level isolation such as gVisor, Kata or Firecracker micro-VMs.
- **Data.** Rows with `tenant_id` plus row-level security, a schema per tenant, or a database per tenant (lesson 42). Add **per-tenant encryption keys** (envelope encryption with a key in a KMS): the tenant's data keys are wrapped by its own master key, the key can be customer-managed, and deleting it on offboarding makes every copy, including backups, unreadable ("crypto-shredding").
- **Network.** In a pool, tenants never get network access to your internals, so network isolation is about your own components: network policies so a compromised service can't reach every database. In a silo, a VPC or VNet per tenant, and private connectivity (AWS PrivateLink, Azure Private Link) when the tenant connects from its own network.

Do not forget the layers in between: caches (key prefix per tenant), object storage (prefix or bucket per tenant, with scoped credentials), search indexes, queues, logs and analytics. A tenant's data leaks

through whichever layer you forgot.

4 Tier it, and route by tenant

The practical answer is **tiering**: free and SMB tenants go into the pool, where cost matters most; enterprise and regulated tenants get silos, and their higher price pays for the dedicated resources and the residency (lesson 46) and compliance story. A **tenant router** resolves the tenant from the signed token or the subdomain, looks up its tier and location in a **tenant catalog**, and forwards the request. The catalog is read on every request, so it is cached at the router; it is also the place you change when a tenant moves between tiers.

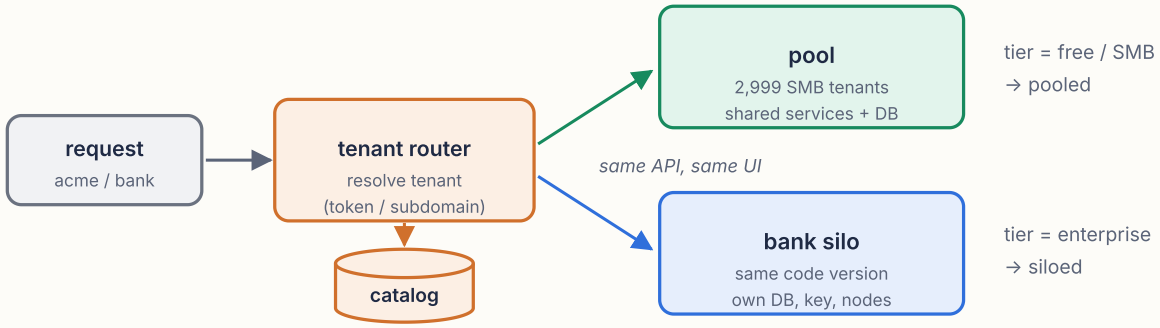


Figure 3. A router plus a catalog make tiers invisible to the product: the same code, API and UI everywhere, different isolation underneath.

Because tenants upgrade, design the **move** from pool to silo from day one: keep `tenant_id` on every row even in siloed databases, so the same code and the same migration tooling (lesson 42) work in both places. At the extreme, the silo becomes a full deployment stamp (lesson 50), and the router becomes a global routing layer.

WORKED EXAMPLE: ENTERPRISE DEDICATED, SMB POOLED

3,000 SMB tenants share the pooled services and one Postgres cluster: every table has `tenant_id`, every query is filtered by the repository layer, and row-level security is the backstop. The bank runs the **same container images and release**, but on its own dedicated node pool (tainted so no other pods land there), with its own database and its own customer-managed KMS key. The tenant router reads `tenant=bank → tier=enterprise, db=pg-bank-01, pool=np-bank` from the catalog; every other tenant maps to the pool. The bank's answers: its data lives in one named database, in one region, encrypted with a key it controls; nobody else's workload runs on its nodes; and a runaway SMB tenant can at worst slow the shared API tier, which has per-tenant rate limits.

TRADE-OFFS

- Silo buys isolation and simple compliance with cost and operational effort that grow with every tenant.
- Pool buys efficiency and one-step deploys with isolation that depends on code being right every time.
- Bridge and tiering get both, but you now run two shapes of system and need routing, a catalog and migration tooling.
- The stronger the isolation, the slower onboarding is, unless provisioning is fully automated.

COMMON MISTAKES

- Choosing one model for the whole system instead of per layer and per tier.
- Letting siloed tenants drift onto custom code versions or hand-made infrastructure.
- Isolating the database but sharing an unprefixed cache, bucket or search index.
- Selling "dedicated" to an enterprise without a tested way to move it there from the pool.

5 Key terms

Silo model: dedicated resources per tenant

Bridge model: silo or pool chosen per layer

Crypto-shredding: deleting a tenant's key to make its data unreadable

Pool model: shared resources; tenant_id isolates

Tenant catalog: maps each tenant to its tier and location

Blast radius: how many tenants one failure affects

6 Interview questions

Q1. Compare silo and pool tenancy models. BASIC

Silo gives each tenant dedicated resources: strong isolation and easy compliance, but higher cost and heavier operations. Pool shares everything: cheap and simple, but isolation relies on code and noisy neighbours are a risk.

Q2. What do most systems do? BASIC

A bridge, or tiering: pool smaller tenants, silo enterprise ones, and route each tenant to its tier through a catalog, with the same code everywhere.

Q3. At which levels can you isolate tenants? BASIC

Compute (quotas, namespaces, node pools, accounts), data (rows + RLS, schema, database, per-tenant keys) and network (policies, separate VPCs, private links).

Q4. A bank asks for its data to be isolated, but you run a pooled system. What do you offer? SCENARIO

A bridge: same application code, but a dedicated database with its own customer-managed key and optionally a dedicated node pool, routed by the tenant catalog and priced as an enterprise tier.

Q5. Customers on the free tier keep growing. When does a tenant leave the pool? SCENARIO

When its load or data share starts to hurt others or it needs residency or compliance guarantees; move it with a bulk copy plus change data capture and a catalog flip (lesson 42).

Q6. Is a separate database per tenant enough isolation? TRICKY

Not alone: caches, object storage, search indexes, queues, logs and shared compute can still mix tenants, and app code still decides which database to open.

Q7. Why is per-tenant cost harder in the pool model? TRICKY

Shared resources produce one bill, so you must attribute cost from per-tenant metering (requests, CPU time, storage), not read it from the cloud invoice.

Q8. Do containers give enough compute isolation for tenants running their own code? TRICKY

Usually not: containers share the host kernel, so untrusted code calls for sandboxed runtimes or micro-VMs (gVisor, Kata, Firecracker) or dedicated nodes.

Next: SD-R-002 · Shared vs Dedicated Databases backendroadmap.com

Shared vs Dedicated Databases

shared schema, schema per tenant, or a database each

Watch the 4-min video

In one line: multi-tenant data lives in a shared schema keyed by `tenant_id`, a schema per tenant, or a database per tenant; mature products mix all three behind a **tenant catalog**, and design for the day a big tenant must move.

1 The tenant that outgrew you

Your largest customer now owns **40%** of the rows in the shared database. Its nightly reports saturate I/O and slow every other tenant, and its new contract requires a dedicated database in Germany. You must move it with no downtime and no lost rows. If `tenant_id` is on every row and the application already looks up where each tenant lives, this is a routine operation. If tenant data is mixed with global data, or tenant IDs are implied by joins, it is a months-long project.

2 Three data models

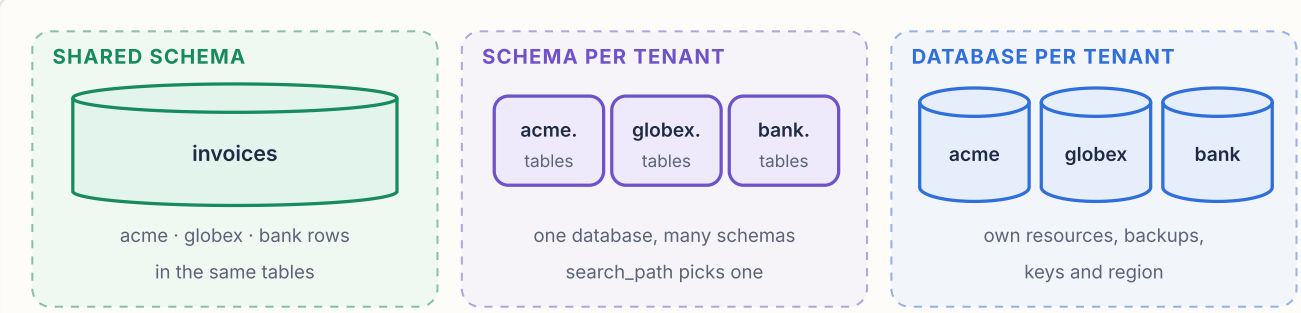


Figure 1. Same data, three layouts. Isolation, cost and operational load all rise from left to right.

Shared schema. One set of tables; every table has `tenant_id`, and every primary key and index starts with it, e.g. `PRIMARY KEY (tenant_id, id)`, so a tenant's rows are clustered and queries never scan other tenants. One migration updates everyone. This is what Citus does at scale: tables distributed by `tenant_id`, so one tenant's rows sit on one node and joins stay local. The risk is a missing `WHERE tenant_id = …`, so use Postgres **row-level security** as a second wall:

```
ALTER TABLE invoices ENABLE ROW LEVEL SECURITY;
ALTER TABLE invoices FORCE ROW LEVEL SECURITY;   -- applies to the owner too
CREATE POLICY tenant_isolation ON invoices
  USING (tenant_id = current_setting('app.tenant_id')::uuid);
-- per transaction, from the verified token (lesson 43):
SET LOCAL app.tenant_id = '6f1c...';
```

Figure 2. RLS policy. FORCE matters: table owners bypass RLS by default, as do superusers and roles with BYPASSRLS. Use a non-owner app role.

Schema per tenant. One database, a schema per tenant (`acme.invoices`, `globex.invoices`), selected with `search_path`. Per-tenant backup and restore (`pg_dump -n acme`) and per-tenant custom columns become easy. The cost is the catalog: 5,000 tenants × 60 tables is 300,000 tables plus their indexes, which slows migrations, `pg_dump` and backups, and every connection caches catalog entries it touches. Watch connection pooling too: with PgBouncer in transaction mode, a plain `SET search_path` can leak to

the next client of that server connection; use `SET LOCAL` inside each transaction or schema-qualified names.

Database per tenant. Strongest isolation: separate CPU and I/O, backups, encryption keys, maintenance windows and even regions; noisy neighbours disappear and restoring one tenant never touches another. But everything scales with the tenant count. Connections are the classic trap: 20 app pods × 1,000 databases × a pool of 5 is 100,000 connections. Use lazily opened, small pools, a proxy (PgBouncer, RDS Proxy), or pooled capacity such as Azure SQL **elastic pools**, where many databases share one vCore budget. Migrations become a fleet operation: record the schema version per database, run migrations in waves, and keep code compatible with both the old and new schema while the fleet catches up (expand, then contract).

	Shared schema	Schema per tenant	Database per tenant
Isolation	logical (code + RLS)	namespace	physical
Cost per tenant	lowest	low	highest
Schema migration	one run	N schemas	N databases, in waves
Restore one tenant	hard (filter a full backup)	easy-ish	easy
Noisy neighbour	yes	yes (same server)	no
Scale limit	table size, hot tenants	catalog size	connections, ops effort
Tenant count it suits	thousands to millions	hundreds to low thousands	tens to hundreds (or automated fleets)

3 How to choose: count, size, compliance

Tenant sizes almost always follow a power law: a few tenants hold most of the data. So the honest answer is a mix. Many small tenants go into a shared schema; a few large or regulated tenants get dedicated databases; schema per tenant is a middle step when customers need per-tenant restore or customisation. A **tenant catalog** records where each tenant lives, and the data access layer picks the connection for every request.

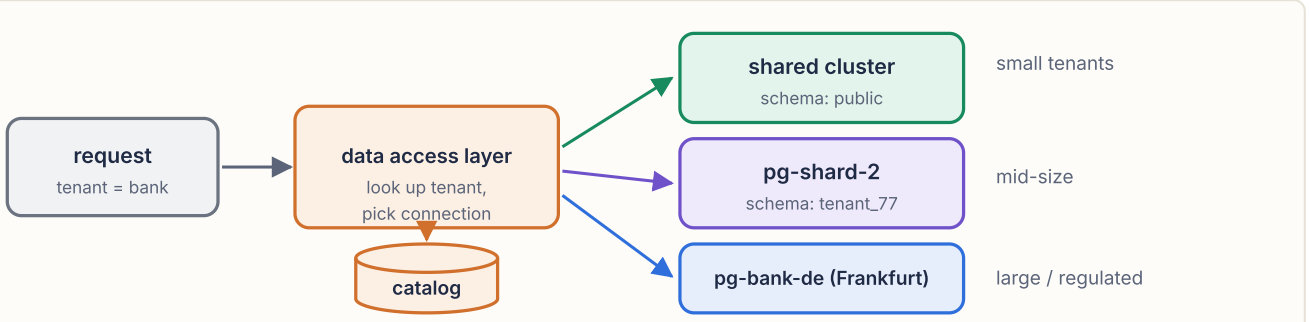


Figure 3. The catalog (tenant → cluster, database, schema, region, status) is tiny, read on every request, and therefore cached.

Keep three rules everywhere, even in dedicated databases: **tenant_id on every row** (so moving is a filter, not a rewrite), **globally unique IDs** such as UUIDs or ULIDs (two databases' sequences would collide when you merge or move), and **no cross-tenant foreign keys or joins**, so one tenant's data can be lifted out cleanly.

4 Design for moving tenants

Copying one tenant is a filtered export, but the tenant keeps writing while the copy runs. The standard answer is **snapshot + change data capture**: record the log position (Postgres LSN, MySQL binlog position), bulk-copy a consistent snapshot, then replay every change after that position, filtered by tenant. Debezium on the WAL, or Postgres logical replication, which since version 15 supports **row filters** on a publication (`CREATE PUBLICATION bank_pub FOR TABLE invoices WHERE (tenant_id = '…')`), both work. Make the apply step idempotent (upsert by primary key), so replaying a change twice is harmless.

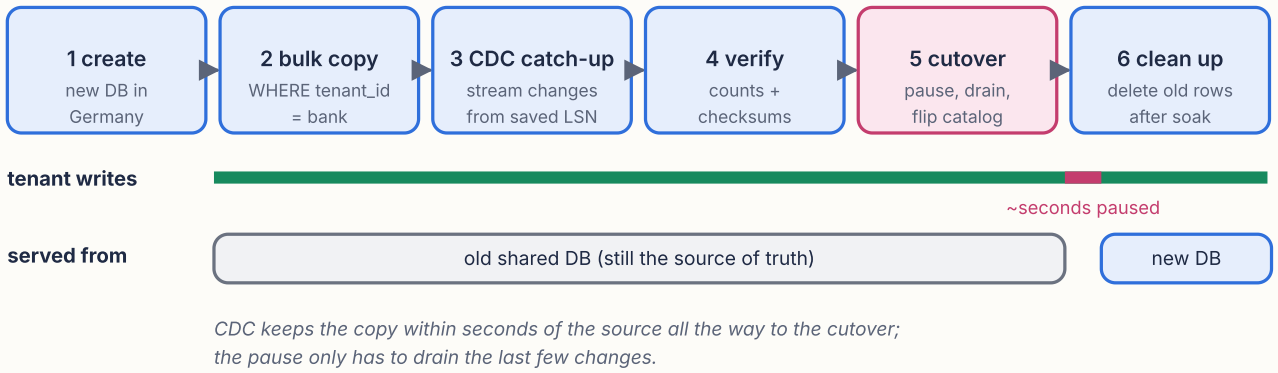


Figure 4. Zero-downtime tenant move. Only step 5 pauses the tenant, for seconds, and only that tenant.

WORKED EXAMPLE: MOVING THE BIG TENANT TO GERMANY

(1) Create `pg-bank-de` in Frankfurt with the same schema version. (2) Note the current LSN, then copy every table `WHERE tenant_id = 'bank'` from a consistent snapshot. (3) Stream changes after that LSN, filtered to the bank, until replication lag is under a second. (4) Verify: row counts per table and checksums over ordered rows (e.g. a hash of each table's primary keys and updated_at). (5) Set the bank's catalog status to `migrating`, which makes the API hold or reject its writes with a retry; wait until lag is zero; flip the catalog to `pg-bank-de`; invalidate cached catalog entries; resume. The pause is seconds and only the bank sees it. (6) Keep the old rows for a soak period as a rollback path, then delete them in small batches. Don't forget the bank's files in object storage, its search index documents and its cache keys.

COMMON MISTAKES

- Tables without `tenant_id` ("it's implied by the parent row"): you can't filter them for a move.
- RLS enabled but the app connects as the table owner, so the policy never applies.
- Database per tenant with a full connection pool per database in every pod.
- Flipping the catalog before change data capture has fully drained, losing the last writes.

5 Key terms

Shared schema: all tenants in the same tables, keyed by tenant_id

Schema per tenant: one database, a namespace each

Database per tenant: a physical database per tenant

Tenant catalog: tenant → where its data lives

Row-level security: database-enforced row filter per session

Change data capture: streaming committed changes from the log

6 Interview questions

Q1. How do you choose a multi-tenant data model? BASIC

Shared schema with tenant_id for many small tenants, schema per tenant for moderate isolation, a database per tenant for large or regulated tenants; most systems mix them with a tenant catalog for routing.

Q2. Why should every index in a shared schema start with tenant_id? BASIC

Every query is per tenant, so leading with tenant_id keeps lookups and scans inside one tenant's rows and makes the data easy to shard by tenant.

Q3. How do you move a big tenant to its own database? SCENARIO

Bulk copy by tenant_id from a snapshot, sync later changes with change data capture, verify counts and checksums, then briefly pause that tenant's writes, flip the catalog and resume.

Q4. You have 2,000 tenants, each in its own database. What breaks first? SCENARIO

Usually connections and operations: pools per database per pod explode, and migrations, monitoring and backups must run as fleet jobs. Use pooled proxies or elastic pools and wave-based migrations.

Q5. RLS is enabled, but a query still returns other tenants' rows. Why? TRICKY

The session's role bypasses RLS: it is the table owner without FORCE ROW LEVEL SECURITY, a superuser, or has BYPASSRLS; or the tenant setting leaked from a pooled connection.

Q6. Why do you need globally unique IDs in a sharded multi-tenant system? TRICKY

Moving or merging tenants copies rows between databases; per-database sequences would collide, while UUIDs or ULIDs move as they are.

Q7. What's the downside of schema per tenant at large scale? TRICKY

Catalog size: hundreds of thousands of tables slow migrations, dumps and connection start-up, and every schema change runs thousands of times.

Next: SD-R-003 · Tenant-Aware Authorization backendroadmap.com

Tenant-Aware Authorization

what can you do, and in which tenant?

Watch the 4-min video

In one line: take the tenant from a **signed token**, set it once per request, enforce it automatically in the data layer with **row-level security** as a backstop, carry it to every downstream call, queue and job, and give support staff a separate, audited break-glass path.

1 Invoice 124

A support engineer at Acme changes `/invoices/123` to `/invoices/124` and sees Globex's invoice. Login worked. The role check worked: she may read invoices. Nobody asked **whose** invoices. This is **Broken Object Level Authorization** (BOLA, also called IDOR), number one in the OWASP API Security Top 10 (2023). In a multi-tenant system every authorization decision has two parts: what may you do, and in which tenant.



the bug in the hook: steps 1 and 2 passed, step 3 was never asked

Figure 1. Authorization in SaaS is three checks. The tenant check is the one that is easy to forget, so it must not depend on remembering.

Hard-to-guess IDs (UUIDs instead of 123, 124) make enumeration harder but fix nothing: IDs leak through URLs, logs, emails and exports. The fix is a tenant check on every object access, enforced by structure rather than discipline.

2 The tenant comes from the token

At sign-in the identity provider issues a JWT with the user ID, roles and a tenant claim (Microsoft Entra ID, for example, puts the directory in `tid`; with your own IdP you add a custom claim). The server verifies the **signature** against the issuer's published keys (JWKS), pins the allowed algorithms (never accept `alg: none`), and checks `iss`, `aud` and `exp`. Because the token is signed, the client cannot change its tenant. Never trust a tenant ID from a header, query string or body.

Users who belong to several tenants (an accountant serving five clients) are the exception. Let them pick a tenant in the URL or a header, but check it on the server against their **memberships**, or better, exchange the login token for a short-lived, tenant-scoped token (OAuth 2.0 Token Exchange, RFC 8693) so everything downstream sees one verified tenant.

3 Set the context once, enforce it below

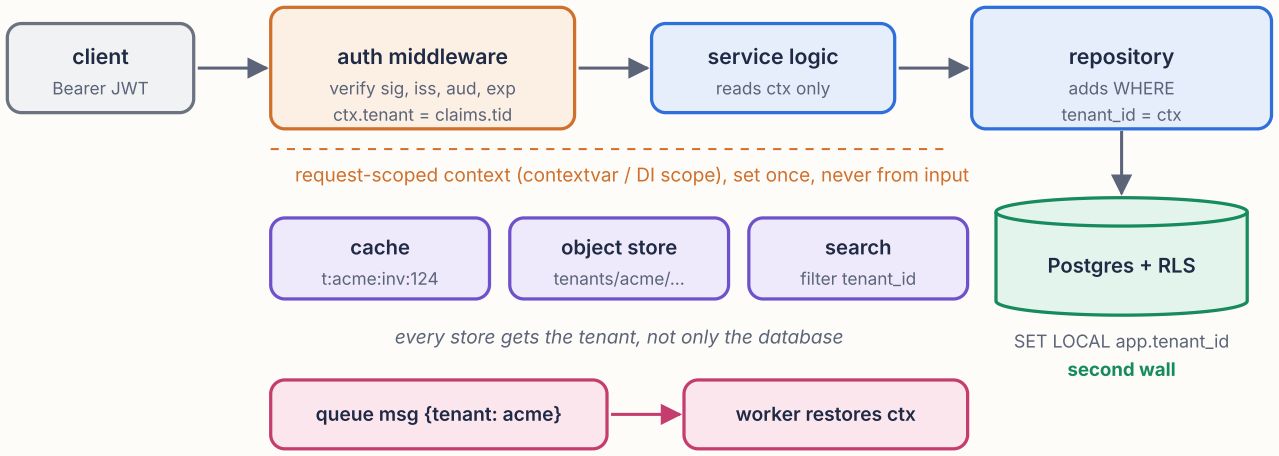


Figure 2. Tenant context is established at the edge and read by every layer. Each store also partitions by tenant.

A middleware or dependency reads the verified claims and stores the tenant in a **request-scoped context** (a Python `contextvar`, a scoped DI service in .NET, a request attribute in Spring). Services and repositories read it from there; nobody passes it by hand, so nobody can forget it.

- **Data layer, automatically.** The repository adds the tenant filter to every query, or the ORM does it globally: EF Core global query filters (`HasQueryFilter`), Hibernate 6 `@TenantId`. Writing a query without the filter should require an explicit, reviewable opt-out.
- **Database backstop.** Postgres RLS returns only rows matching `current_setting('app.tenant_id')`, set with `SET LOCAL` in each transaction so a pooled connection never carries one tenant's setting into another's request. Connect as a role that is not the table owner (lesson 42).
- **Every other store.** Cache keys prefixed `t:{tenant}:`; object storage under `tenants/{tenant}/` with credentials or pre-signed URLs scoped to that prefix; a tenant filter on every search query (or an index per tenant); tenant in every analytics row.
- **Answer 404, not 403**, for another tenant's object, so the API doesn't confirm that it exists.

4 Context must travel

A request that calls another service passes the user's token on, or exchanges it for a downstream token that keeps the tenant claim. Internal calls between your own services can use a platform-signed header, never an unsigned one that any caller could set. A message on a queue carries the tenant in its envelope; the worker **restores the context before touching data**. Fail closed: a job or query that runs with no tenant must raise an error, not run unfiltered. Scheduled jobs that truly span tenants (billing runs, cleanup) loop over tenants explicitly, setting the context for each.

Test it like a feature: an automated suite creates two tenants and calls every endpoint with tenant A's token and tenant B's object IDs; every call must return 404. Run it in CI so new endpoints can't skip it.

5 Break-glass access for your own staff

Your support engineers and operators sometimes need to see a customer's data. Treat that as a separate, exceptional path, never as an ordinary tenant role: a distinct platform role, a reason and ticket, often an approval (by a second engineer, or by the customer, as with Microsoft's Customer Lockbox), access limited in time and to one tenant, and every action written to the tenant-visible audit trail (lesson 48).



Figure 3. Break-glass access: requested, approved, time-boxed, audited, and revoked automatically.

6 Worked example: a FastAPI tenant dependency

```
async def current_tenant(token: str = Depends(oauth2_scheme)) -> str:
    key = jwks.get_signing_key_from_jwt(token).key
    claims = jwt.decode(token, key, algorithms=["RS256"],   # never 'none'
                        audience="api://invoices", issuer=ISSUER)  # checks exp
    return claims["tid"]

@app.get("/invoices/{inv_id}")
async def get_invoice(inv_id: UUID, tenant: str = Depends(current_tenant)):
    async with tenant_session(tenant) as db:      # SET LOCAL app.tenant_id
        inv = await repo.invoice(db, inv_id, tenant)  # AND tenant_id = :tenant
    if inv is None:
        raise HTTPException(status_code=404)      # not 403: don't confirm it exists
    return inv
```

Figure 4. PyJWT verifies signature, audience, issuer and expiry; the route can't run without a tenant, and the repository requires it.

WORKED EXAMPLE: ACME ASKS FOR GLOBEX'S INVOICE

The `current_tenant` dependency verifies the JWT and returns `acme`. Every tenant route declares it, and a test fails any route that doesn't. `tenant_session` opens a transaction and runs `SET LOCAL app.tenant_id = 'acme'`; the repository queries `WHERE id = :id AND tenant_id = :tenant`. Invoice 124 belongs to Globex, so the query finds nothing and the API answers 404. If a future developer forgets the filter in a new query, RLS still returns nothing, and the cross-tenant test suite fails the build.

COMMON MISTAKES

- Reading the tenant from an `X-Tenant-ID` header or request body.
- Filtering by tenant in list endpoints but not in get-by-ID, update and delete.
- Background jobs that run without a tenant and therefore see everything.
- Giving support staff a normal admin role inside customer tenants instead of audited break-glass access.

7 Key terms

BOLA / IDOR: accessing another object by changing its ID

Tenant context: request-scoped tenant set once at the edge

Token exchange: swap a token for a scoped downstream one (RFC 8693)

Tenant claim: signed tenant ID inside the token

Row-level security: database-enforced per-tenant row filter

Break-glass: exceptional, approved, audited staff access

8 Interview questions

Q1. How do you ensure tenant context is never missed? BASIC
Take it from a signed token claim, set it once per request in middleware, enforce it automatically in the data layer with row-level security as a backstop, and propagate it to downstream calls, queues and jobs.

Q2. How should support staff access tenant data? BASIC
Through a separate break-glass role that is approved, time-limited, scoped to one tenant and audited where the customer can see it, never through ordinary tenant roles.

Q3. Why not read the tenant from a request header? BASIC
Anyone can set a header; only a signed token claim, or a server-side membership check, proves which tenant the caller belongs to.

Q4. A user belongs to three tenants. How does the API know which one a request is for? SCENARIO
The client names the tenant, the server checks it against the user's memberships, and ideally issues a short-lived token scoped to that tenant for the rest of the call chain.

Q5. A worker processes queued jobs for all tenants. How do you keep it tenant-safe? SCENARIO
Each message carries its tenant; the worker sets the tenant context (and the RLS setting) before any data access and fails if the tenant is missing.

Q6. Should another tenant's invoice return 403 or 404? TRICKY
404: a 403 confirms the object exists, which leaks information and helps enumeration.

Q7. Are random UUIDs enough to stop invoice-ID guessing? TRICKY
No. They make guessing harder, but IDs leak through links, logs and exports; only a tenant check on every access prevents BOLA.

Q8. RLS uses a session setting. What goes wrong with connection pooling? TRICKY
A plain SET survives on the pooled connection and the next request may inherit another tenant's value; use SET LOCAL inside each transaction.

Next: SD-R-004 · Noisy Neighbour backendroadmap.com

Noisy Neighbour

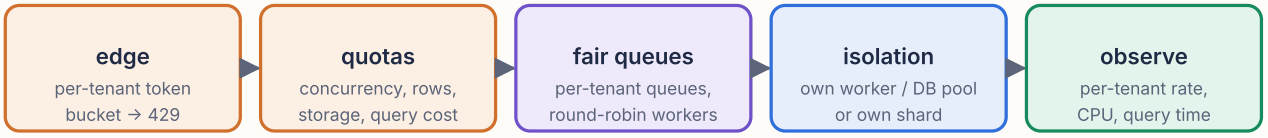
one tenant's spike must not become everyone's outage

Watch the 4-min video

In one line: stop one tenant from degrading the rest with layers: per-tenant **rate limits** at the edge, **quotas** on expensive resources, **fair scheduling** of background work, **isolation** of heavy tenants in their own pools or shards, and per-tenant monitoring.

1 The Monday import

9 a.m. Monday: one customer starts a bulk import of two million records. Within minutes every other customer's dashboard crawls, API calls time out and support tickets pile up. The importer did nothing wrong; the platform simply let one tenant take far more than its share of CPU, database connections, I/O and queue time. In a pooled system (lesson 41) fairness is not automatic. You have to build it at every shared resource.



from cheap and coarse at the front door to expensive and precise at the back

Figure 1. Defence in depth against noisy neighbours.

2 Per-tenant rate limits at the edge

Give each tenant its own **token bucket**: capacity 400 tokens, refilled at 200 per second. A burst of 400 requests passes, then the tenant is held to 200 per second on average. A request that finds the bucket empty gets **429 Too Many Requests** (RFC 6585) with a `Retry-After` header, so well-behaved clients back off. Limits come from the tenant's plan (lesson 49): enterprise gets more than free, nobody gets unlimited.

- **Key by tenant and by user or API key**, so one script inside a tenant can't use the whole tenant's budget.
- **Weight by cost.** A search or export costs more tokens than a simple read; counting only requests lets one expensive endpoint slip through.
- **Distributed counters.** Many gateway nodes share a limit through Redis (an atomic Lua script per check), or each node enforces limit ÷ nodes locally, trading accuracy for zero extra latency.

3 Quotas on expensive resources

Rate limits protect the front door; the heavy work happens behind it. Add quotas on what is actually scarce: **concurrent jobs** per tenant, rows per import, storage, exports per hour, and query cost. In the database, a per-role or per-session `statement_timeout` stops one runaway report from holding locks and I/O for an hour. **Soft limits** warn (email, banner, header); **hard limits** stop or queue the work. Quotas turn a sudden spike into a bounded, predictable load, and the same usage numbers feed billing (lesson 45).

4 Fair scheduling for background work

A single shared FIFO queue is unfair by design: a tenant that enqueues 10,000 jobs puts everyone else behind them. Keep a **queue per tenant** (or a partition per tenant group) and let workers pick across tenants **round-robin**, or **weighted fair** so paid tiers get a bigger share. Fairness doesn't waste capacity: when other tenants are idle, the big tenant still gets every worker.

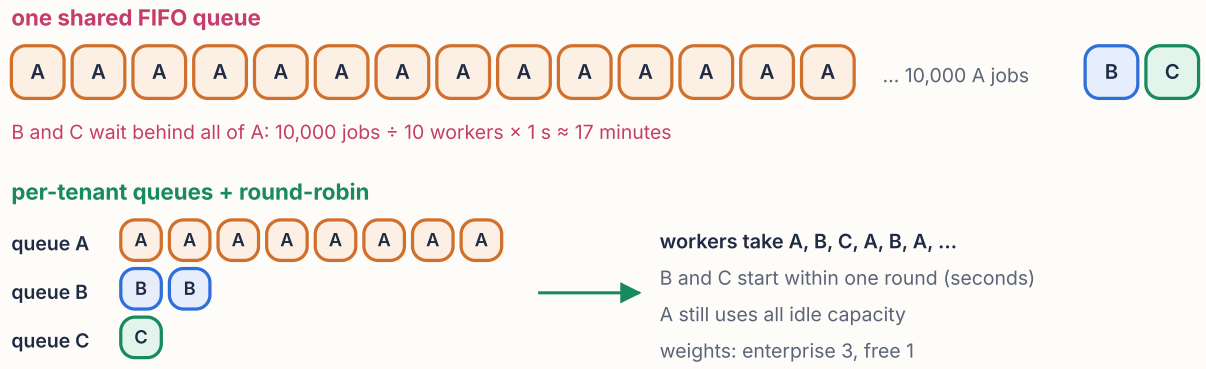


Figure 2. Same jobs, same workers. Only the pick order changed, and B's wait drops from minutes to seconds.

Implementation sketch: a Redis list per tenant plus a set of tenants with pending work; each worker rotates through that set, pops one batch, and re-adds the tenant if work remains. With Kafka, keying by tenant gives per-tenant ordering but a hot tenant still overloads its partition, so large tenants are usually split out to their own topic or consumer group.

5 Isolate heavy tenants

Some tenants are simply big. Use the **bulkhead** pattern: give the heaviest tenants their own worker pools, their own database connection pool (so they can exhaust only their own connections), or their own shard or database (lesson 42). **Shuffle sharding**, described in the AWS Builders' Library, is a clever middle ground: each tenant is assigned a small random subset of workers, so a noisy tenant can take down only its own subset, and almost no other tenant shares exactly the same subset.



Figure 3. Shuffle sharding. With plain sharding into 4 pairs, X's whole pair (a quarter of tenants) would suffer; with shuffle sharding, Y survives.

Tenant **placement** then becomes an operational lever: spread tenants across cells or shards so one noisy tenant affects only its neighbours, move a tenant that grows too large to a dedicated shard, and at the top end, give it a dedicated stamp (lesson 50).

6 Watch per-tenant metrics

You can't fix a neighbour you can't see. Track request rate, error rate, CPU time, database time and queue depth **per tenant**, and alert on any tenant whose share jumps. Careful with cardinality: a metrics label per tenant across 50,000 tenants overwhelms Prometheus-style systems, so label the top tenants

(say top 100 by usage) individually and aggregate the rest, and keep full detail in logs or the metering pipeline.

WORKED EXAMPLE: THE BULK IMPORT, REDESIGNED

The import API now accepts the file, stores it in object storage and returns `202 Accepted` with a job ID. The file is split into batches of 5,000 rows (400 batches for 2 million rows) placed on the tenant's own queue. Workers pick batches by **weighted round-robin** across tenants, the importer is capped at **4 concurrent batches**, and its writes go through a separate, small database connection pool. Edge limits still return 429 with Retry-After if its scripts poll too hard. The import takes a little longer; nobody else notices it.

TRADE-OFFS

- Strict limits protect neighbours but frustrate big customers; tie them to plans and offer higher tiers.
- Fair queues add bookkeeping (many queues, rotation) compared with one FIFO.
- Dedicated pools isolate well but waste idle capacity; shuffle sharding isolates with shared capacity.
- Local, per-node limits are fast but approximate; central counters are exact but add a network hop.

COMMON MISTAKES

- Global rate limits only: one tenant uses the whole global budget and everyone gets 429s.
- Counting requests, not cost, so a cheap-looking export endpoint melts the database.
- One shared job queue in arrival order.
- Shared connection pools, so one tenant's slow queries starve everyone of connections.

7 Key terms

Noisy neighbour: one tenant degrading others on shared resources

Quota: cap on a resource: soft warns, hard blocks

Bulkhead: separate resource pools so failures stay contained

Token bucket: rate limit with a refill rate and burst capacity

Fair scheduling: giving every tenant a turn, optionally weighted

Shuffle sharding: each tenant on a random small subset of workers

8 Interview questions

Q1. How do you stop one tenant from degrading everyone? BASIC

Per-tenant rate limits at the edge, quotas on expensive resources, fair scheduling with per-tenant queues, isolation of heavy tenants in their own pools or shards, and per-tenant monitoring.

Q2. Why isn't a single shared queue enough? BASIC

One tenant's backlog blocks everyone behind it; per-tenant queues with round-robin or weighted fair picking give every tenant a turn.

Q3. What does a 200/s rate with a burst of 400 mean? BASIC

A token bucket of capacity 400 refilled at 200 tokens per second: a 400-request burst passes, then the tenant averages at most 200 requests per second.

Q4. One tenant's reporting queries slow the shared database at night. What do you do? SCENARIO
Add per-tenant query timeouts and concurrency limits, move its reports to a read replica or warehouse, give it its own connection pool, and if it keeps growing, move it to its own database.

Q5. How would you find which tenant is causing a slowdown? SCENARIO
Per-tenant metrics on request rate, CPU and database time, with the top tenants labelled individually, plus tenant IDs in traces and slow-query logs.

Q6. Why limit by cost rather than request count? TRICKY
Requests differ by orders of magnitude in cost; counting them equally lets a tenant stay under its request limit while saturating the database with expensive calls.

Q7. What's the advantage of shuffle sharding over plain sharding? TRICKY
With plain sharding a noisy tenant hurts everyone in its shard; with shuffle sharding few or no tenants share its exact worker set, so most still have a healthy worker.

Next: SD-R-005 · Usage Metering backendroadmap.com

Usage Metering

billing you can prove, quotas you can enforce

Watch the 4-min video

In one line: emit an immutable **usage event** with a unique ID for every billable action, write it to a durable log, **deduplicate and aggregate** per tenant, meter and period, check quotas against a fast counter, and send period totals to billing, so every number on an invoice can be traced back to events.

1 The disputed bill

Your AI platform charges per API call and per token. A large customer disputes the month: they count 800,000 calls, you billed 1.2 million. If your number came from a database counter incremented in the request path, you can't prove anything: counters lose updates when a write fails, double-count when a client retries, and keep no history. Billing needs a pipeline built like an accounting ledger: append-only facts, then derived totals you can recompute.

2 Usage events: facts, not counters

```
{ "event_id": "01J9Z6…",            // unique; set once, reused on retry
  "tenant_id": "acme",  "meter": "input_tokens",
  "quantity": 1834,     "unit": "token",
  "occurred_at": "2026-11-03T09:14:07Z",  // event time, not arrival time
  "dims": { "model": "large-v2", "api_key": "k_51", "request_id": "r-88…" } }
```

Figure 1. A usage event. Small, immutable, with an ID that survives retries and the dimensions you'll need to explain the bill.

Every billable action emits an event: tenant, meter (`api_calls`, `input_tokens`, `output_tokens`, `storage_gb_hours`), quantity, event time and a unique ID. Events go to a durable, replayable log such as Kafka (and are archived to object storage), not straight into a counter. The log is the source of truth: if an aggregation bug is found, you fix the job and recompute. Decide what is billable up front and write it into the contract: for example, count only successful calls, and count tokens actually processed.

3 Never bill a retry twice

Events get duplicated: a client retries a timed-out request, a producer re-sends after a lost ack, a consumer re-reads after a crash. So the ID must be **deterministic per billable action**, typically derived from the request ID or the client's `Idempotency-Key`, not generated fresh on each attempt. The pipeline then drops any ID it has already seen.

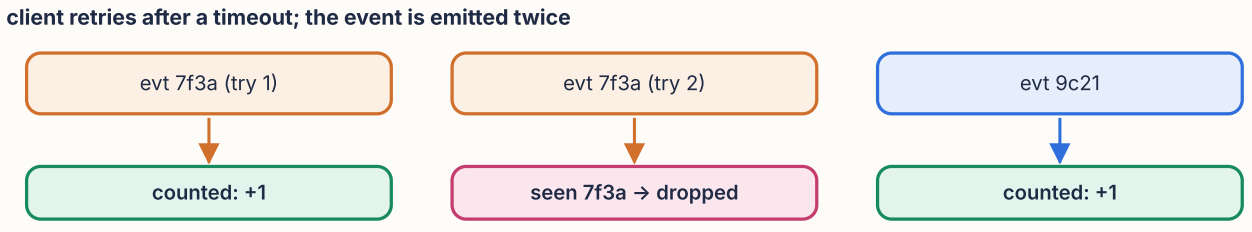


Figure 2. Deduplication by event ID: the retried request carries the same ID and is counted once.

Where to deduplicate: Kafka's idempotent producer and transactions give exactly-once processing *inside* Kafka (no duplicates from producer retries, atomic read-process-write), but they can't know two requests from a client are the same action. So keep the ID dedupe in the stream job (Flink keyed state of seen IDs with a TTL longer than any retry window, e.g. 7 days), and make the sink idempotent: upsert the total for `(tenant, meter, day)` rather than incrementing it.

4 Aggregate by event time, close periods carefully

A stream job rolls events up per tenant and meter: per minute for dashboards and quota checks, per day and per month for billing. Aggregate by **event time**, so a call at 23:59:58 on 31 October that arrives at 00:00:03 lands in October. Watermarks tell the job when a window is probably complete; allowed lateness accepts stragglers for a while. After a billing period is **closed and invoiced**, never rewrite it silently: late or corrected usage becomes an explicit adjustment line on the next invoice.

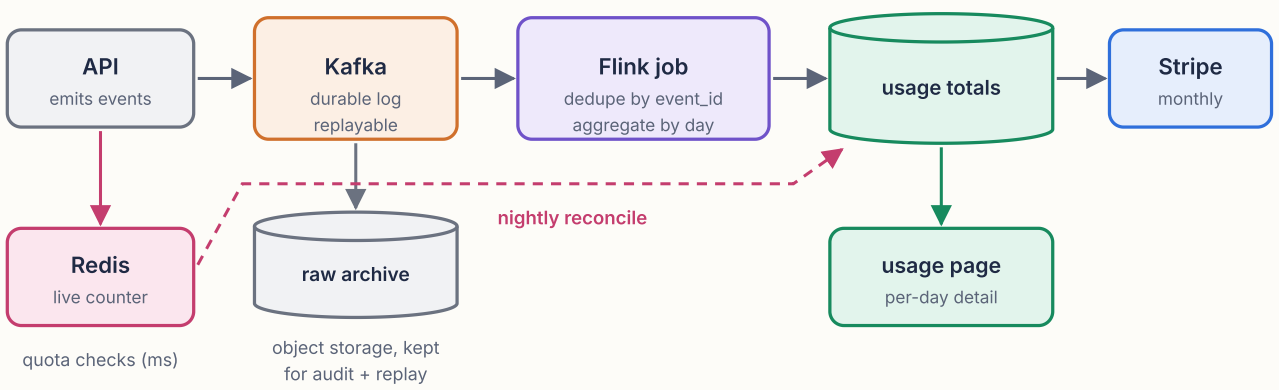


Figure 3. Metering pipeline. The slow, exact path feeds billing; the fast, approximate path feeds quota checks; reconciliation ties them together.

5 Quotas on the same numbers

A **soft limit** warns: at 80% of the plan the customer gets an email and a response header. A **hard limit** blocks (429 with a clear quota-exceeded error) or switches to **overage** pricing, depending on the plan. Quota checks run on every request and can't wait for the pipeline, so a fast counter tracks this period in near real time: `INCRBY usage:acme:input_tokens:2026-11 1834` in Redis, with the key expiring after the period. For tokens, you don't know the cost until the model answers, so **reserve** an estimate (e.g. max_tokens) before the call and settle the difference after.

	Fast counter (Redis)	Durable totals (pipeline)
Purpose	quota decisions, live usage bar	invoices, disputes, analytics
Latency	milliseconds	seconds to minutes
Accuracy	approximate: may miss or double a few	exact: deduplicated, replayable
If lost	rebuild from durable totals	recompute from the log / archive

Reconcile nightly: compare the counter with the durable totals and correct the counter. The counter is **advisory**; the durable totals are what you bill.

6 Billing integration

At period end, send each tenant's per-meter totals to the billing system (Stripe's usage-based billing, or open-source tools such as Lago or OpenMeter), which applies the plan's prices, tiers and overage rules

and issues the invoice. Send **aggregates with an idempotency key** per (tenant, meter, period), not millions of raw events, so a retried upload can't double-bill. Give customers a usage page with daily totals and the same dimensions you bill on (API key, model), so most disputes are settled before they reach support.

WORKED EXAMPLE: API CALLS AND TOKENS PER TENANT PER MONTH

Each request emits two events, `api_calls = 1` and `input/output_tokens` with the model name, both with IDs derived from the request ID. Kafka → Flink deduplicates by ID and writes daily totals per tenant, meter and model; Redis holds the running month for quota checks. On the 1st, monthly totals go to Stripe with idempotency keys. The dispute: the daily breakdown shows the customer's own count matches on most days, and the gap sits on three days when its client retried aggressively. Because the retries reused the request ID they were **not** billed; the remaining gap traces to a second API key the customer had forgotten, with per-key totals and sample request IDs to prove it.

COMMON MISTAKES

- Billing from a counter in the request path, with no events behind it.
- Generating a new event ID on each retry, so deduplication never matches.
- Aggregating by arrival time, so usage shifts between periods.
- Silently rewriting a closed month instead of issuing an adjustment.

7 Key terms

Usage event: immutable record of one billable action

Idempotency key: ID that makes a retry the same action

Soft / hard limit: warn vs block or overage

Meter: what is counted: calls, tokens, GB-hours

Event time: when usage happened, not when it arrived

Reconciliation: correcting fast counters from durable totals

8 Interview questions

Q1. Design usage metering for billing. BASIC

Emit immutable usage events with unique IDs, write them to a durable log, deduplicate and aggregate per tenant, meter and period, use a fast counter for live quota checks, and send period totals to the billing system.

Q2. Soft vs hard limits? BASIC

Soft limits warn the customer; hard limits block usage or switch it to overage pricing.

Q3. A customer disputes their bill. What do you need to answer? SCENARIO

Per-day, per-meter totals with the billed dimensions (API key, model), traceable to raw events and request IDs kept in the archive.

Q4. How do you enforce a monthly token quota without slowing every request? SCENARIO

Check and increment an in-memory counter in Redis per tenant and month, reserve an estimate before the model call, settle afterwards, and reconcile nightly with durable totals.

Q5. Kafka has exactly-once semantics. Why do you still need event IDs? TRICKY

Kafka's guarantees cover producer retries and processing inside Kafka; a client retrying the same request creates a new message that only an application-level ID can recognise.

Q6. Usage arrives after the month is invoiced. What do you do? TRICKY

Never rewrite the closed period; record it as an adjustment on the next invoice, with the original dates.

Q7. Why not bill straight from the Redis counter? TRICKY

It's approximate and has no history: it can lose or double increments and can't be recomputed or audited.

Next: SD-R-006 · Data Residency

Data Residency

keep each tenant's data inside its jurisdiction

Watch the 4-min video

In one line: run a complete **regional stamp** of the data plane in each jurisdiction, give every tenant a **home region** in a small global directory, route its requests there, and hunt down every side channel (logs, backups, support tools, analytics, third parties) that would copy its data across the border.

1 The clause that stops the deal

A German bank is ready to sign, but one clause blocks it: *all of our customer data must stay in the European Union*. Your platform runs in one US region with backups in another US region. To win the deal you need data residency, and it is far easier to design in early than to retrofit.

2 Residency, sovereignty, localisation

Term	Means	Typical driver
Data residency	data is stored (and often processed) only in named regions	customer contracts, GDPR transfer rules
Data sovereignty	data is subject only to that jurisdiction's laws, including who operates it and who can be compelled to hand it over	public sector, banks; concern about foreign laws such as the US CLOUD Act
Data localisation	a law requires certain data to stay in the country	e.g. the Reserve Bank of India's 2018 rule that payment system data be stored only in India

GDPR itself does not say "store data in the EU". It restricts **transfers** of personal data outside the EEA (Chapter V) unless the destination has an adequacy decision (for example the 2023 EU-US Data Privacy Framework for certified US companies) or safeguards such as standard contractual clauses (SCCs) are in place. The 2020 *Schrems II* ruling, which struck down the earlier Privacy Shield, made many EU customers prefer simply keeping data in the EU, which is why residency shows up in contracts. Sovereignty goes further and may require EU-resident operators and EU-controlled keys; cloud providers now sell "sovereign cloud" offerings for this.

3 Regional stamps and home regions

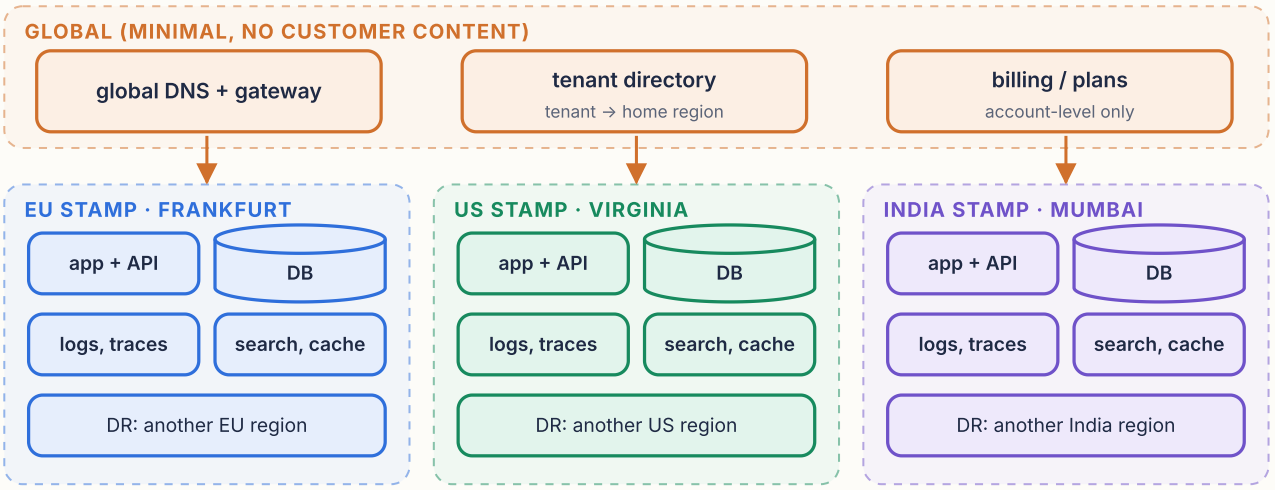


Figure 1. Each region runs a full copy of the data plane, including its own logs, search and backups. The global layer holds only routing and account data.

Instead of one global deployment, run a complete copy of the data plane per region: an **EU stamp** in Frankfurt, a US stamp in Virginia, an India stamp in Mumbai, each with its own databases, object storage, search, caches, queues, logs and backups (deployment stamps, lesson 50). Every tenant gets a **home region** at sign-up, recorded in a global tenant directory. A global routing layer (DNS plus an API gateway or front door) looks up the tenant and forwards the request; a German bank employee travelling in New York is still served from Frankfurt. Regional hostnames such as `bank.eu.app.com` make routing trivial and the location visible.

Login is the subtle part: to find a user's tenant from their email address, a global index of emails would itself be personal data stored globally. Common answers: route by tenant subdomain or a region picker before login, or keep only a keyed hash of the email domain in the global directory and resolve the user inside the home region.

4 Accidental leaks: the hard part

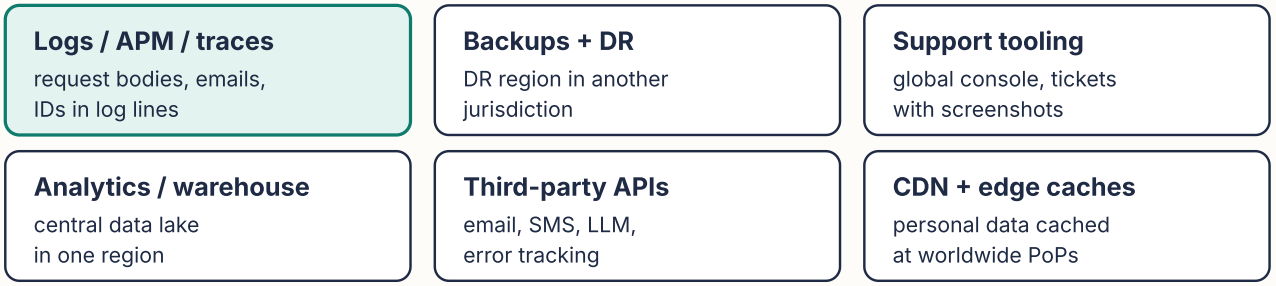


Figure 2. Where residency usually breaks. Each one needs a regional answer or scrubbing.

- **Observability.** Logs, traces and error reports contain emails, IDs and sometimes request bodies. Keep the pipeline regional (a regional Elasticsearch or the vendor's EU site), or scrub personal data before anything leaves the region.
- **Backups and DR.** Replicate to a second region **inside** the jurisdiction: Frankfurt to Paris or to Germany North, never to a US region. Check the provider's default backup geo-redundancy setting, which may pick a paired region for you.
- **Support.** An EU-hosted support console; tickets and screenshots in a regional helpdesk instance; access by approved staff (break-glass, lesson 43).

- **Analytics.** No central data lake of raw customer data: aggregate or anonymise in-region and export only non-personal results.
- **Third parties.** Email and SMS providers, error trackers and LLM APIs are sub-processors; choose EU endpoints or EU data-processing options and list them in your data processing agreement.
- **Edge.** A CDN caching personalised responses stores them at points of presence worldwide; mark them uncacheable or restrict the edge locations.

5 Keep global data minimal

Some data is genuinely global: the tenant directory, billing accounts, plan definitions, product configuration. Keep it small, use opaque IDs, keep personal data out of it, and document every cross-border flow in a data map. When a transfer is unavoidable, use the legal mechanisms (SCCs plus a transfer impact assessment) and technical ones: encrypt with keys held in the customer's region, ideally customer-managed (BYOK) in an in-region key vault or HSM, so data that crosses a border is useless without a key that never does.

WORKED EXAMPLE: EU CUSTOMERS' DATA ONLY IN EU REGIONS

The company deploys an EU stamp with a primary in Frankfurt and disaster recovery in Paris. The tenant directory records `german-bank → EU`. Global DNS and the API gateway route `german-bank.eu.app.com` to the EU stamp. Logs and traces go to an EU-hosted observability stack; support engineers use the EU support console; transactional email goes through the provider's EU region; the database is encrypted with the bank's own key in an EU key vault; backups replicate Frankfurt → Paris and never leave the EU. The contract's annex lists every sub-processor and its location.

TRADE-OFFS

- Every region you add is another full stack to run, patch, monitor and pay for.
- Cross-region features (global search, shared workspaces between an EU and a US tenant) become hard or impossible.
- DR inside one jurisdiction may mean regions that are closer together than you'd otherwise pick.
- Central analytics and ML on raw customer data must be redesigned around in-region processing.

COMMON MISTAKES

- Moving the database to the EU but shipping logs and traces to a US observability tenant.
- Default geo-redundant backups replicating outside the jurisdiction.
- A global user table of emails used for login routing.
- Forgetting sub-processors: email, error tracking, AI APIs.

6 Key terms

Data residency: data stored and processed only in named regions

Home region: the region a tenant's data lives in

SCCs: standard contractual clauses for GDPR transfers

Sovereignty: data subject only to one jurisdiction's laws

Tenant directory: global map of tenant → home region

Sub-processor: third party that processes customer data for you

7 Interview questions

Q1. How do you meet data residency requirements in a global SaaS? BASIC
Deploy regional stamps with all customer data inside each region, assign every tenant a home region in a global directory, route requests there, and keep logs, backups, support tools and third-party processing regional too.

Q2. What's the hardest part? BASIC
Accidental cross-border leaks through logs, analytics, backups, support tooling and external services.

Q3. Residency vs sovereignty? BASIC
Residency is about where data is stored and processed; sovereignty is about which jurisdiction's laws apply, including who operates the system and who can be compelled to access it.

Q4. An EU user logs in from the US. Where is the request served? SCENARIO
From the EU stamp: routing is by the tenant's home region in the directory, not by the user's location.

Q5. How do you do disaster recovery for an EU-only tenant? SCENARIO
Replicate to a second region within the EU (e.g. Frankfurt to Paris) and verify the provider's default backup redundancy doesn't pick a region outside it.

Q6. Does GDPR require data to be stored in the EU? TRICKY
No; it restricts transfers outside the EEA unless there's adequacy or safeguards like SCCs, but many customers contractually require EU storage to avoid transfer risk.

Q7. How do users log in if you can't keep a global email index? TRICKY
Route by tenant subdomain or regional URL, or a region picker, or keep only a hash of the email domain globally and resolve the user inside the home region.

Next: SD-R-007 · Enterprise SSO

Enterprise SSO

their login, their joiners, their leavers

Watch the 4-min video

In one line: for each enterprise tenant, trust **its own identity provider** via SAML or OpenID Connect, find it by email domain (**home realm discovery**), provision and deprovision users with **SCIM**, and map directory groups to app roles per tenant.

1 Three requests from every enterprise

A company with 20,000 employees buys your product. Its IT team asks: (1) our people sign in with **our corporate login**, with our password and MFA rules; (2) **new hires get access automatically**; (3) when someone leaves, their **access disappears the same day**. Every enterprise customer asks for these three; they are usually a precondition of the deal and a line in their security review.

2 Single sign-on, per tenant

Each customer has its own identity provider (IdP): Okta, Microsoft Entra ID, Google Workspace, Ping, ADFS. Your app is the **service provider** (SP, or relying party) and trusts each customer's IdP **separately**. Two protocols cover nearly everyone:

	SAML 2.0	OpenID Connect
Age / base	2005, XML	2014, on top of OAuth 2.0, JSON / JWT
What you receive	signed XML assertion, POSTed by the browser to your ACS URL	ID token (signed JWT) via the authorization code flow
Per-tenant config	IdP entity ID, SSO URL, signing certificate; your entity ID + ACS URL	issuer / discovery URL, client ID + secret, redirect URI
Where it dominates	enterprise IT, older IdPs	modern IdPs, mobile and SPA apps

Validate strictly. For SAML: the XML signature with the tenant's certificate (beware XML signature-wrapping attacks; use a maintained library), `Audience` equal to your entity ID, `Destination` / `Recipient`, the `NotOnOrAfter` time window, and `InResponseTo` matching the request you sent (IdP-initiated logins have no request to match, so they are easier to replay; many apps disable them or treat them carefully). Support **two certificates** per tenant during rotation, or SSO breaks for a whole company on rotation day.

3 Home realm discovery

With hundreds of IdPs, the login page must know which one to use. The user types an email; the domain `acme.com` maps to tenant `acme` and its SSO configuration; the user is redirected to Acme's Okta. Alternatives: a tenant subdomain (`acme.app.com`) or a tenant-specific login URL.

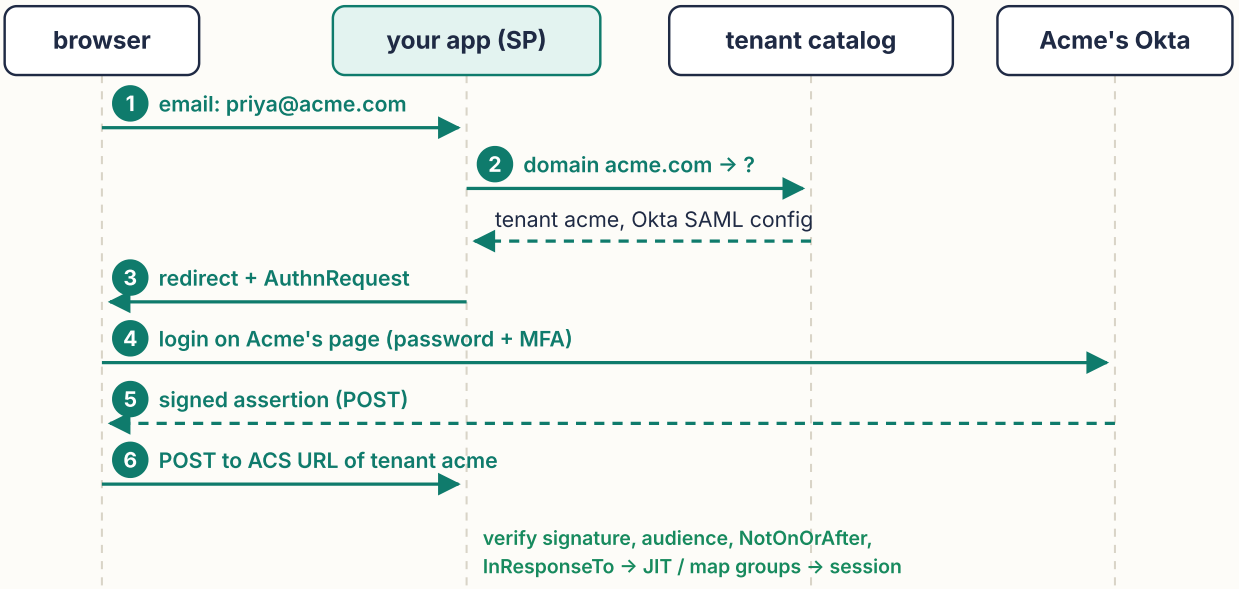


Figure 1. SP-initiated SSO with home realm discovery. The app never sees the password; it trusts the IdP's signed statement.

Two security rules make this safe. **Verify domain ownership** before a tenant can claim a domain (a DNS TXT record), otherwise anyone could register `acme.com` and capture Acme's logins. And identify SSO users by **(IdP, subject ID)**, not by email alone: an email asserted by one tenant's IdP must never log into an account owned by another tenant. Once SSO is enforced for a domain, disable password login for it, keeping one break-glass admin in case the IdP itself is misconfigured.

4 Just-in-time provisioning and its gap

JIT provisioning creates the account on first login from the assertion's attributes: name, email, groups. It's simple and needs no extra integration, but it only sees users when they log in. A leaver never logs in again, so JIT never hears about it: the account lingers, and so do any API keys, long-lived refresh tokens and sessions it created. For many security teams that alone fails the review.

5 SCIM: lifecycle from the directory

SCIM 2.0 (RFC 7643 schema, RFC 7644 protocol) is a standard REST API that **your app exposes** and the customer's IdP calls: `/Users` and `/Groups` with POST, PUT, PATCH and DELETE, authenticated with a per-tenant bearer token. When HR offboards an employee, the IdP sends a deactivation, and your app must do more than flip a flag: revoke sessions and refresh tokens, disable the user's API keys, and record it in the audit trail (lesson 48).

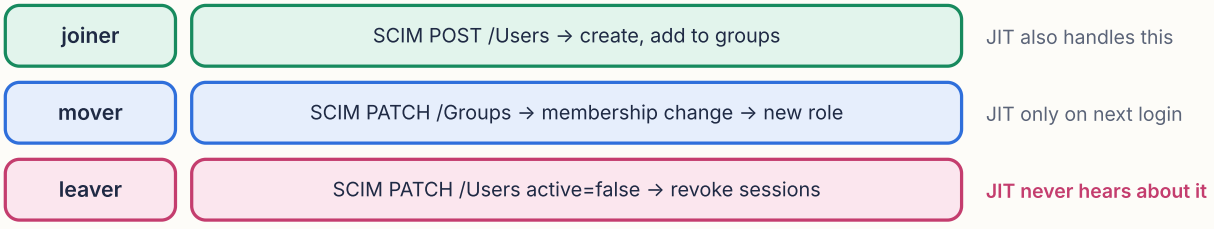


Figure 2. Joiner, mover, leaver. JIT handles only the first well; SCIM handles all three.

```
PATCH /scim/v2/Users/2819c223  HTTP/1.1
Authorization: Bearer <per-tenant SCIM token>
Content-Type: application/scim+json

{ "schemas": ["urn:ietf:params:scim:api:messages:2.0:PatchOp"],
  "Operations": [ { "op": "replace", "value": { "active": false } } ] }
```

Figure 3. A SCIM deactivation as Okta and Entra ID typically send it.

How fast? Okta pushes changes to SCIM apps as they happen; Microsoft Entra ID provisioning runs in cycles, roughly every 40 minutes after the initial sync. If customers need faster cut-off, pair SCIM with short session lifetimes and re-validation of sessions against the user's status.

6 Map directory groups to roles

Permissions should be managed in the customer's directory, not by hand in your app. The tenant admin maps groups to roles (`finance-analysts → viewer`, `finance-admins → admin`); you store the mapping per tenant, keyed by the group's stable ID rather than its display name, and apply it at login and on every SCIM group update. Moving someone between groups in Okta changes their role in your app.

WORKED EXAMPLE: A CUSTOMER'S OKTA TENANT MAPPED TO APP ROLES

Acme's admin adds your app from the Okta catalog, uploads Okta's SAML metadata (entity ID, SSO URL, certificate) in your tenant settings, verifies `acme.com` with a DNS TXT record, and creates a SCIM token. Okta pushes users and groups; the admin maps `sales-managers` to the **manager** role. A new hire, priya@acme.com, is created by SCIM on day one, signs in through Okta and lands with the manager role. When she leaves, Okta deactivates her; your app revokes her sessions and keys within minutes.

COMMON MISTAKES

- Relying on JIT only, so leavers keep access.
- Letting a tenant claim an email domain without verifying it.
- Linking SSO identities to existing accounts by email address.
- Deactivating the user but not revoking sessions, refresh tokens and API keys.

7 Key terms

Identity provider (IdP): the customer's login system, e.g. Okta, Entra ID

Service provider (SP): your app, trusting the IdP

Home realm discovery: picking the IdP from the email domain

JIT provisioning: create the account at first login

SCIM: standard API for user and group lifecycle

Group-role mapping: directory group → app role, per tenant

8 Interview questions

> **Q1. How do you support SSO for many enterprise customers?** BASIC
> Store a separate SAML / OpenID Connect configuration per tenant, use home realm discovery by email domain, provision users with SCIM, and map directory groups to app roles per tenant.

Q2. Why isn't just-in-time provisioning enough? BASIC

It creates users at login but never removes leavers or updates movers who don't log in; SCIM handles deprovisioning and group changes.

Q3. SAML or OIDC? BASIC

Support both if you sell to enterprises: SAML is still common in corporate IT; OIDC is simpler, JSON-based and preferred by modern IdPs.

Q4. A customer's IdP certificate is rotated and SSO breaks. How do you prevent this? SCENARIO

Allow two active certificates per tenant, read the IdP's metadata URL to pick up new certificates, and alert before the current one expires.

Q5. An employee is fired at 10:00. How do you make sure they can't use the app at 10:05? SCENARIO

SCIM deactivation from the IdP, which revokes sessions, refresh tokens and API keys; plus short session lifetimes in case provisioning runs on a cycle.

Q6. Why must tenants verify email domains? TRICKY

Otherwise any tenant could claim another company's domain and route its users' logins to an IdP it controls.

Q7. Why not link an SSO login to an existing account by matching email? TRICKY

The email is asserted by an IdP you don't control; a malicious or misconfigured IdP could assert someone else's email and take over their account.

Next: SD-R-008 · Enterprise Audit Trails backendroadmap.com

Enterprise Audit Trails

who did what, to what, when, and from where

Watch the 4-min video

In one line: an audit trail is a product feature: structured, tenant-scoped records of every security-relevant action, written through a pipeline that can't lose them, stored **append-only and tamper-evident**, and searchable, exportable and streamable by the customer.

1 The incident question

An enterprise customer's security team is investigating an incident: *who exported our customer list last Tuesday, from which IP, and who changed the admin's permissions an hour before?* If your answer is "let me grep our server logs", you've lost their trust, and probably your SOC 2 auditor's. Enterprise customers expect to answer that question themselves, in minutes, from your product.

2 What an audit entry records

```
{ "id": "aud_01JA…", "tenant_id": "acme", "seq": 48213,
  "ts": "2026-11-03T02:07:41Z",
  "actor": { "type": "user", "id": "u_731", "email": "c.lee@contractor.io",
             "via": "sso", "impersonated_by": null },
  "action": "customer_list.exported", "result": "success",
  "target": { "type": "report", "id": "rep_customers" },
  "ip": "203.0.113.45", "user_agent": "Mozilla/5.0 …", "request_id": "r-9f2…",
  "changes": null,                 // {before, after} for updates
  "prev_hash": "a41c…", "hash": "7be0…" }
```

Figure 1. One audit event: actor, action, target, time, origin, result, and for changes the before and after values.

The core is **who** (actor: user, API key, service, or your own staff, with how they authenticated and whether someone was impersonating them), **did what** (a stable action name from a catalog, like `role.changed`, `report.exported`, `sso.config.updated`), **to what** (target type and ID), **when** (UTC timestamp from a synced clock), **from where** (IP, user agent, request ID for correlation) and **with what result** (success, denied, failed: failed logins and denied attempts are often the most interesting). Record `before` / `after` for changes, but never secrets: log that an API key was created, not the key.

What to cover: sign-in and sign-out, failed logins, MFA changes, user and role changes, SSO and SCIM events (lesson 47), permission and sharing changes, data exports and bulk downloads, API key lifecycle, settings and security policy changes, and every access by your own staff.

3 Audit logs are not debug logs

	Debug / application logs	Audit logs
Audience	your engineers	customers, security teams, auditors
Content	free text, noisy, anything	structured schema, defined action catalog
Completeness	sampled, rate-limited, best effort	every security-relevant action, never sampled
Retention	days to weeks	1 to 7+ years, per tenant
Mutability	rotated and deleted freely	append-only, tamper-evident
Access	internal tooling	tenant-scoped UI, API, SIEM stream

So audit events go through a **dedicated pipeline**. The classic gap is "the change committed but the audit event was lost". Close it with a **transactional outbox**: write the audit row in the same database transaction as the change, and let a relay publish it. For the most sensitive actions (exports, permission changes), **fail closed**: if the audit write fails, the action fails.

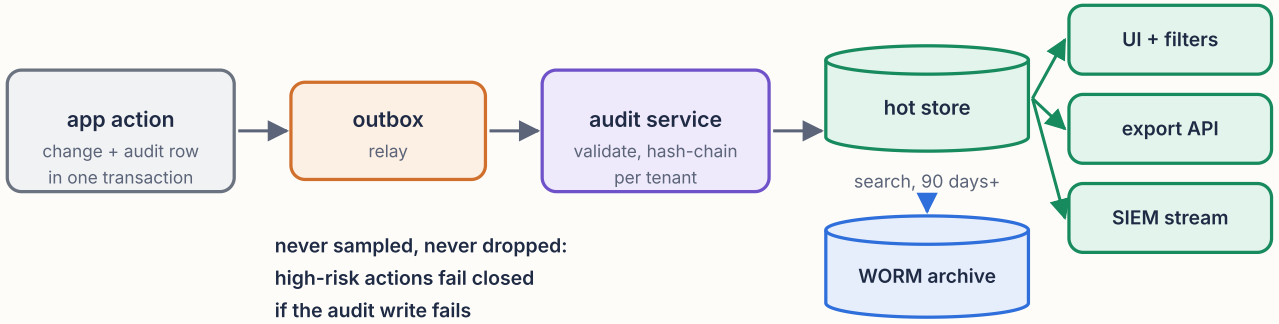


Figure 2. Audit pipeline: transactional capture, per-tenant chaining, hot searchable store, write-once archive, and three ways out for the customer.

4 Tamper evidence

An attacker's first move is often to hide their tracks, and insiders exist. Three layers:

- **Append-only access.** The application role can insert audit rows but not update or delete them; no human has write access in production.
- **Write-once storage (WORM).** Amazon S3 Object Lock in compliance mode blocks deletion or overwrite, even by the root account, until the retention date; Azure Blob immutable storage with a locked time-based retention policy does the same.
- **Hash chaining.** Each entry stores the hash of the previous one, per tenant: h_n = SHA-256($h_{n-1} \parallel \text{entry}_n$). Deleting or editing any entry breaks every link after it, which a verifier detects.

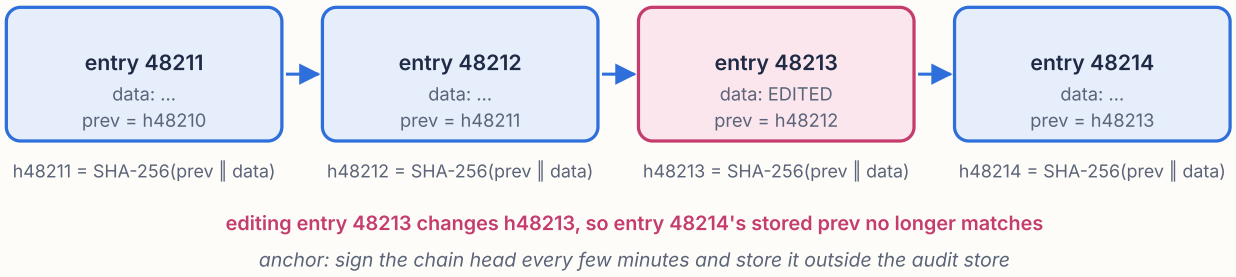


Figure 3. A hash chain makes changes detectable. Anchoring the head stops an attacker from simply recomputing the whole chain.

A chain alone has one hole: someone who can rewrite the whole store can recompute every hash. So periodically **anchor** the head: sign it with a key the audit service doesn't hold, and store the signed checkpoint somewhere else (another account, the WORM archive, or give it to the customer). Per-tenant chains also let you hand a customer a verifiable export of just their entries.

5 Customer access, retention and compliance

Customers need three ways in: an **audit page** with filters (actor, action, target, time range, IP), an **export API** (paginated JSON or CSV), and **streaming to their SIEM**, such as Splunk (via its HTTP Event Collector) or Microsoft Sentinel. Every query is scoped to the tenant (lesson 43). Retention is set per tenant: for example one year by default and seven years for regulated customers, with old entries moved from the hot store to the archive.

Frameworks expect this. SOC 2 auditors ask how you log access to customer data, how long you keep logs, who can change them and how you review them; ISO 27001:2022 covers it in Annex A control 8.15 (Logging); PCI DSS requires at least 12 months of audit history with the last three months immediately available. Your own staff's access, especially break-glass access, must appear in the same tenant-visible trail. Audit logs hold personal data (emails, IPs), so GDPR still applies: define retention, and restrict who can read them.

WORKED EXAMPLE: A CUSTOMER ADMIN EXPORTS HER AUDIT LOG

The admin filters the audit page to last week and action `customer_list.exported`. One row stands out: a contractor exported the full customer list at 02:07 from an IP never seen before. Filtering by that actor shows a role change to **admin** an hour earlier, made by an account that had a failed-MFA streak. She exports the week as JSON through the API, with each entry's hash so her team can verify the chain, and forwards it to Sentinel. Her export itself becomes a new audit event: `audit_log.exported`.

COMMON MISTAKES

- Mixing audit events into application logs, where sampling and rotation drop them.
- Writing the audit event after the commit, outside the transaction, so crashes lose it.
- Hiding your own staff's access from the customer's trail.
- Logging secrets or full payloads in before/after values.

6 Key terms

Audit trail: tenant-visible record of security-relevant actions

Actor / action / target: who did what to which resource

Hash chain: each entry stores the previous entry's hash

SIEM: security event platform, e.g. Splunk, Sentinel

WORM: write once, read many storage

Anchoring: signing and storing the chain head elsewhere

7 Interview questions

Q1. What do enterprise customers expect from audit logs? BASIC

Tenant-scoped records of who did what, to what, when and from where for all security-relevant actions; tamper-evident storage; search, export APIs and SIEM streaming; and configurable retention.

Q2. How do you make audit logs tamper-evident? BASIC

Append-only, write-once storage with hash-chained entries, so any change breaks the chain, plus periodically signed chain heads stored elsewhere.

Q3. How are audit logs different from debug logs? BASIC

They're a customer-facing, structured, complete and immutable record kept for years; debug logs are internal, noisy, sampled and short-lived.

Q4. How do you guarantee an audit event exists for every committed change? SCENARIO

Write it in the same database transaction as the change (transactional outbox) and fail closed for sensitive actions if the audit write fails.

Q5. A regulated customer needs seven years of audit logs. How do you store them affordably? SCENARIO

Keep recent months in a searchable hot store and move older entries to compressed, write-once object storage with a per-tenant retention policy.

Q6. Hash chaining is enough to stop an attacker with full database access, right? TRICKY

No: they can recompute the whole chain. Anchor the chain head by signing it with a separate key and storing checkpoints outside their reach.

Q7. Should your support engineers' access appear in the customer's audit log? TRICKY

Yes: staff access, especially break-glass, is exactly what customers and auditors want to see.

Next: SD-R-009 · Tenant Configuration backendroadmap.com

Tenant Configuration

defaults, overrides, settings, and entitlements by name

Watch the 4-min video

In one line: model per-tenant behaviour as **layered configuration** (plan defaults, tenant overrides, tenant settings), have code check **named entitlements** rather than plan names, enforce them on the server, cache the effective config with invalidation, and audit every change.

1 The deal that becomes an if statement

Sales closes a deal: this enterprise customer gets advanced analytics, SSO, seven-year data retention and a higher API limit. A starter customer gets none. If each difference becomes `if customer == 'acme'` or `if plan == 'enterprise'` in code, every deal is a deployment, every pricing change is a code search, and nobody can say what a given tenant actually has. Tenant configuration turns those differences into data.

2 Configuration in layers

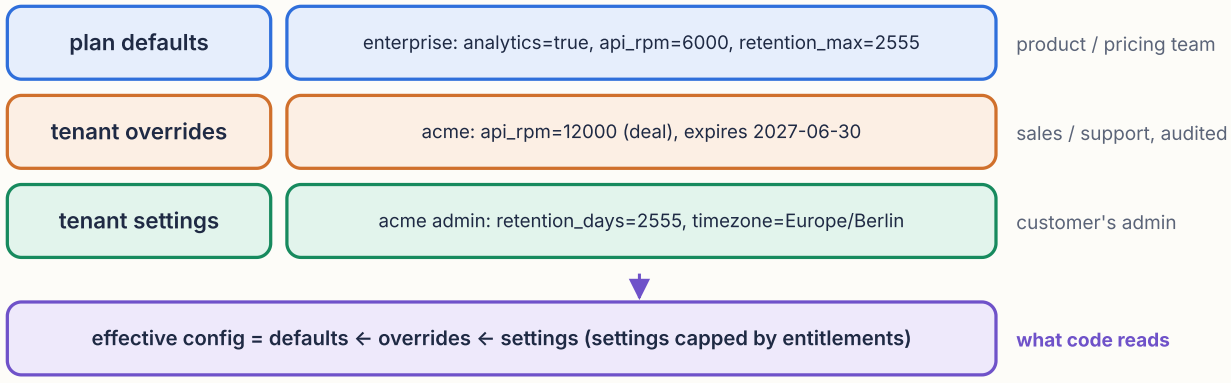


Figure 1. Three layers owned by three different people, merged into one effective configuration per tenant.

- **Plan defaults** define what each tier includes (starter, business, enterprise). Owned by product and pricing; changing a plan changes every tenant on it.
- **Tenant overrides** capture deals and exceptions: a custom limit, a trial of a higher-tier feature, an early-access grant. Each has an owner, a reason and, ideally, an **expiry date**; overrides without expiry quietly become permanent.
- **Tenant settings** are choices the customer's admin makes: logo, time zone, password policy, retention period. Settings live inside the bounds of entitlements: an admin can choose retention up to the entitled maximum, not beyond it.

The **effective configuration** is computed by merging the layers, and that's the only thing application code reads.

3 Entitlements: check capabilities, not plans

Entitlements are what a tenant has paid for. They come in three shapes: **booleans** (`advanced_analytics`, `sso`), **limits** (`api_rpm`, `seats`, `retention_days_max`), and **enumerations** (support tier). Code asks for the

capability by name. Then moving SSO from enterprise to business is a data change, a custom deal needs no deployment, and a new plan is a new row.

```
# never:   if tenant.plan == 'enterprise': ...
# instead: ask for the capability by name
cfg = tenant_config(ctx.tenant)              # cached, effective config
if not cfg.entitled('advanced_analytics'):
    raise Forbidden(code='upgrade_required', feature='advanced_analytics')
limit = cfg.limit('api_rpm')                 # numeric entitlement
```

Figure 2. Check the named entitlement; return a machine-readable error the UI can turn into an upgrade prompt.

Limits connect to other lessons: `api_rpm` feeds the per-tenant rate limiter (lesson 44), quota limits feed metering and billing (lesson 45), and billing systems such as Stripe can be the source of which plan a tenant is on, with your config service deriving entitlements from it.

4 Entitlements vs feature flags

	Feature flag	Entitlement
Question	is this feature ready, and for whom?	has this tenant paid for it?
Owner	engineering / product	product, sales, billing
Lifetime	temporary: removed after rollout	permanent part of the pricing model
Typical targeting	% of tenants, internal users, a region	plan, overrides, contract
If wrong	bug exposure	revenue leak or angry customer



flag off → hidden (not ready yet) · flag on, not entitled → upgrade prompt · both → available

e.g. an AI-summaries beta: flag on for 5% of tenants, entitlement only in business and enterprise

Figure 3. A feature reaches a tenant only when the flag is on and the tenant is entitled.

5 Caching, invalidation and storage

Configuration is read on almost every request, so each service caches the effective config in memory. Two mechanisms together: **change events** (the config service publishes `tenant acme changed, version 42` and services evict the entry, so an upgrade takes effect in seconds) and a short **TTL** (say 60 seconds) that bounds staleness if an event is lost. If the config service is down, keep serving the **last-known-good** config rather than failing every request.

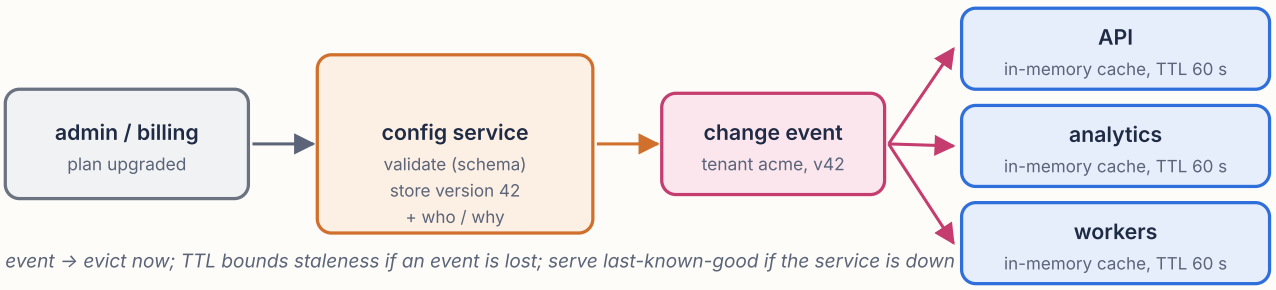


Figure 4. A central, validated, versioned config service, with services caching the effective config and evicting it on change.

Store configuration centrally with a **schema** (reject `api_rpm = "lots"`), **versions** and **history**: every change records who made it, when and why, and lands in the audit trail (lesson 48). Manual overrides by your own staff deserve an approval step. And **enforce on the server**: the UI can hide a button, but the API must refuse the call, because customers call your API directly.

WORKED EXAMPLE: AN ENTERPRISE-ONLY FEATURE WITH A TRIAL

`advanced_analytics` is `true` in the enterprise plan defaults and `false` elsewhere. A business customer gets a 90-day trial through a tenant override `advanced_analytics = true, expires_at = 2027-02-01`. The analytics service checks the entitlement from its cached effective config. Expiry needs no cron job: the merge ignores overrides whose `expires_at` has passed, so the next evaluation after the date returns false (within the cache TTL). The API then answers `403 upgrade_required` and the UI shows an upgrade prompt, while the tenant's data is kept, so upgrading later restores the dashboards.

COMMON MISTAKES

- Checking plan names or customer names in code.
- Overrides without expiry or owner, so nobody knows why acme has 10× the limit.
- Enforcing entitlements only in the UI.
- Using one flag system for both rollout and paid access, then deleting a "flag" that was really a product tier.

6 Key terms

Plan defaults: what each tier includes

Tenant override: per-tenant exception, ideally with expiry

Tenant setting: a choice the customer's admin makes

Entitlement: a feature or limit the tenant has paid for

Effective config: the merged result code reads

Feature flag: temporary switch controlling rollout

7 Interview questions

Q1. How do you manage per-tenant features? BASIC

Layer plan defaults, tenant overrides and settings; check named entitlements, not plan names; enforce them on the server; cache the effective config with invalidation; audit every change.

Q2. How are entitlements different from feature flags? BASIC

Flags control rollout and are temporary; entitlements control what a tenant has paid for and are part of the pricing model.

Q3. Product wants to move SSO from enterprise to business. What changes? SCENARIO

Only data: set the sso entitlement to true in the business plan defaults; caches refresh via change events and no deployment is needed.

Q4. A customer upgrades but still sees the old limits for ten minutes. Why, and how do you fix it? SCENARIO

Services are caching the old effective config with a long TTL; publish change events that evict the tenant's entry and keep the TTL short as a backstop.

Q5. The config service goes down. What should services do? TRICKY

Keep using the last-known-good cached config, rather than failing requests or falling back to the most restrictive defaults.

Q6. How do you expire a trial without a scheduled job? TRICKY

Store expires_at on the override and ignore expired overrides when merging, so the effective config changes on the next evaluation.

Q7. Why must entitlements be enforced in the API and not just the UI? TRICKY

Customers and scripts call the API directly; a hidden button doesn't stop an unentitled request.

Next: SD-R-010 · Deployment Stamps backendroadmap.com

Deployment Stamps

grow by adding copies, not by growing the copy

Watch the 4-min video

In one line: run many identical, independent **stamps** (scale units, cells), each a full copy of the application and its data serving a subset of tenants, with a small **control plane** that places tenants and a **global router** that sends each tenant's traffic to its stamp.

1 One deployment, one blast radius

Your SaaS runs as one huge deployment serving 10,000 tenants. One bad configuration change takes it down, and every tenant is offline together. Growth means making the biggest database bigger, until there is no bigger instance to buy. Large SaaS providers avoid both problems the same way: they stop scaling the deployment **up** and start scaling it **out** as whole copies. Azure calls this the Deployment Stamps pattern; AWS calls it cell-based architecture.

2 What a stamp is

A stamp is a complete, independent copy of the application and its data: web tier, services, databases, caches, queues, storage. It serves a subset of tenants, perhaps a few hundred, and has a **known, tested maximum capacity**, found by load testing one stamp. To grow, you don't make a stamp bigger; you deploy another one from the same template (infrastructure as code, lesson 61). Every stamp runs the same code and configuration; a "special" hand-modified stamp defeats the pattern.

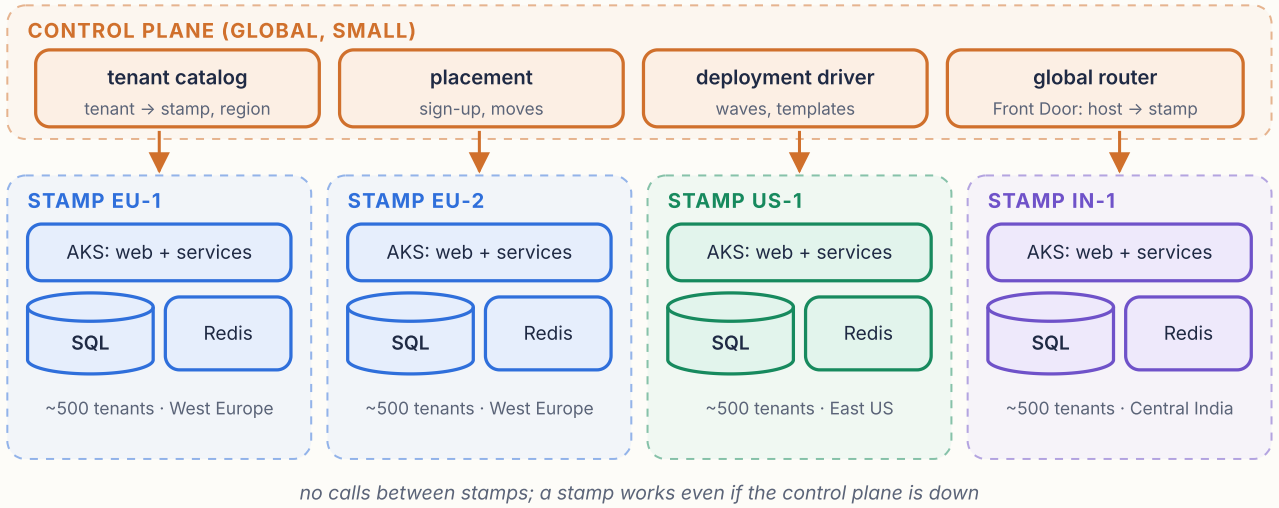


Figure 1. Control plane above, identical stamps below. The router sends each tenant to the stamp the catalog names.

The rule that makes stamps work: **no synchronous calls between stamps**, and nothing in the request path that is shared by all stamps. A shared global database or cache would put the whole fleet back into one blast radius.

3 Limiting the blast radius

With 10,000 tenants in 20 stamps of 500, a stamp failure, a corrupted database or a noisy tenant (lesson 44) affects **5%** of tenants, not 100%. Releases roll out stamp by stamp in **waves**, starting with an internal

or canary stamp, with bake time and automated health gates in between. This mirrors how the cloud providers deploy their own services region by region.

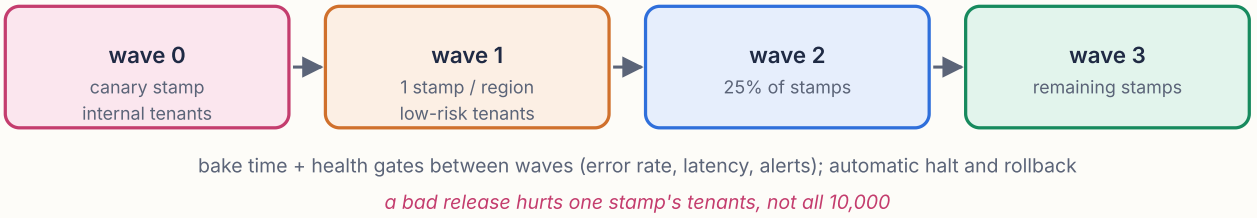


Figure 2. Safe deployment across stamps. Each wave widens exposure only after the previous one proves healthy.

Schema changes must be backward compatible (expand, then contract), because during a rollout some stamps run version N and others N+1, and a rollback must work on a stamp that already migrated.

4 The control plane and global routing

Above the stamps sits a small **control plane**: the tenant catalog (tenant → stamp, region, status), sign-up and placement, stamp inventory and capacity, and the deployment driver. A **global routing layer** such as Azure Front Door or a DNS-plus-gateway setup maps each request to its stamp, typically by host name (`acme.app.com` → stamp EU-1) or by a tenant claim in the token, with the catalog cached at the edge.

Design the control plane for **static stability**: if it goes down, existing tenants keep working, because routing uses its cached catalog and stamps don't depend on the control plane per request. Only sign-ups, moves and deployments pause. The control plane is the one global component, so keep it small, replicated and off the hot path.

5 Tenant placement

Input	Effect on placement
Residency (lesson 46)	only stamps in the tenant's allowed jurisdiction
Size and growth	big tenants spread across stamps; very large ones get a dedicated stamp
Compliance / tier	regulated or enterprise tenants on isolated stamps (lesson 41)
Stamp capacity	fill to a target (e.g. 70% of tested max), then open a new stamp
Release risk	internal and friendly tenants on the canary stamp

Placement is not one-time. Tenants grow, so rebalance by moving them between stamps with the copy + change-data-capture + catalog-flip process from lesson 42. Keep tenant data self-contained (tenant_id everywhere, globally unique IDs) so a move is routine.

6 Regional failover

Each stamp can have a **paired stamp** in another region of the same jurisdiction, with data replicated asynchronously (so a few seconds of RPO). If a region fails, the control plane promotes the paired stamps and updates routing tenant by tenant, never moving a tenant outside its allowed jurisdiction. Rehearse it: a failover runbook that has never run is a hope, not a plan.

WORKED EXAMPLE: THE AZURE DEPLOYMENT STAMPS PATTERN FOR SAAS

A global control plane holds the tenant catalog. Azure Front Door routes `{tenant}.app.com` to stamps in West Europe, East US and Central India. Each stamp is one Bicep or Terraform template: an AKS cluster, Azure SQL databases (elastic pools for small tenants), Azure Cache for Redis, Service Bus and storage. Load tests showed one stamp handles about 600 average tenants, so placement fills stamps to 500. New releases go to a canary stamp of internal tenants, then one stamp per region, then the rest in waves. When West Europe's stamps reach 70% of target, the platform team deploys EU-3 from the same template and sends new EU sign-ups there.

TRADE-OFFS

- Fixed overhead per stamp (clusters, databases, monitoring) makes small stamps inefficient: pick a size that amortises it.
- Cross-tenant features (global search, sharing between tenants on different stamps) become hard.
- Fleet operations (deploys, migrations, dashboards) must be automated; you run N systems.
- In return: linear scaling, bounded blast radius, safe rollouts, and residency almost for free.

COMMON MISTAKES

- A shared global database or cache in the request path of every stamp.
- Stamps drifting apart through manual changes.
- Deploying to all stamps at once, which throws away the blast-radius benefit.
- A control plane that every request depends on.

7 Key terms

Deployment stamp / cell: independent full copy serving some tenants

Control plane: catalog, placement and deployment above the stamps

Static stability: keeps working when the control plane fails

Scale unit: a stamp with a known, tested capacity

Global router: sends each tenant to its stamp

Deployment wave: a step of a staged rollout across stamps

8 Interview questions

Q1. What is the deployment stamps pattern? BASIC

Many identical, independent copies of the full application, each serving a subset of tenants, with a control plane that places tenants and a router that sends traffic to their stamp.

Q2. Why use it? BASIC

Scale by adding stamps, limit the blast radius of failures and deployments, and support data residency and regional failover.

Q3. A stamp is at capacity and one tenant keeps growing. What do you do? SCENARIO

Open a new stamp from the template for new sign-ups and move the growing tenant (or some neighbours) with copy + CDC + catalog flip; a very large tenant may get a dedicated stamp.

Q4. How do you roll out a risky release across 40 stamps? SCENARIO

Canary stamp first, then one stamp per region, then larger waves, with bake time and automatic health gates and rollback between waves.

Q5. What happens to tenants if the control plane is down? TRICKY

Nothing visible if it's statically stable: routing uses its cached catalog and stamps serve traffic on their own; only sign-ups, moves and deploys wait.

Q6. Why must stamps not call each other synchronously? TRICKY

Cross-stamp calls couple their availability, so one stamp's failure spreads and the blast radius is no longer one stamp.

Q7. How is a stamp different from a region? TRICKY

A region is the provider's location; a stamp is your unit of deployment. A region can host several stamps, and each stamp usually lives in one region.

Next: SD-V-001 · Cloud Service Models backendroadmap.com

Cloud Service Models

how much of the stack do you want to own?

Watch the 4-min video

In one line: IaaS, PaaS, FaaS and SaaS differ in **how much of the stack the provider runs**: the further you move toward SaaS, the less you patch, scale and operate, and the less you control; under the **shared responsibility model** your data, identities and configuration are always yours.

1 Three engineers, one database

The team needs a relational database. One engineer wants SQL Server on virtual machines for full control. Another wants Azure SQL Database so Microsoft handles patching and failover. A third asks whether a SaaS product could replace the custom app altogether. All three are valid; they differ in which layers of the stack you own, and therefore what you pay for in money versus engineering time.

2 The models, layer by layer

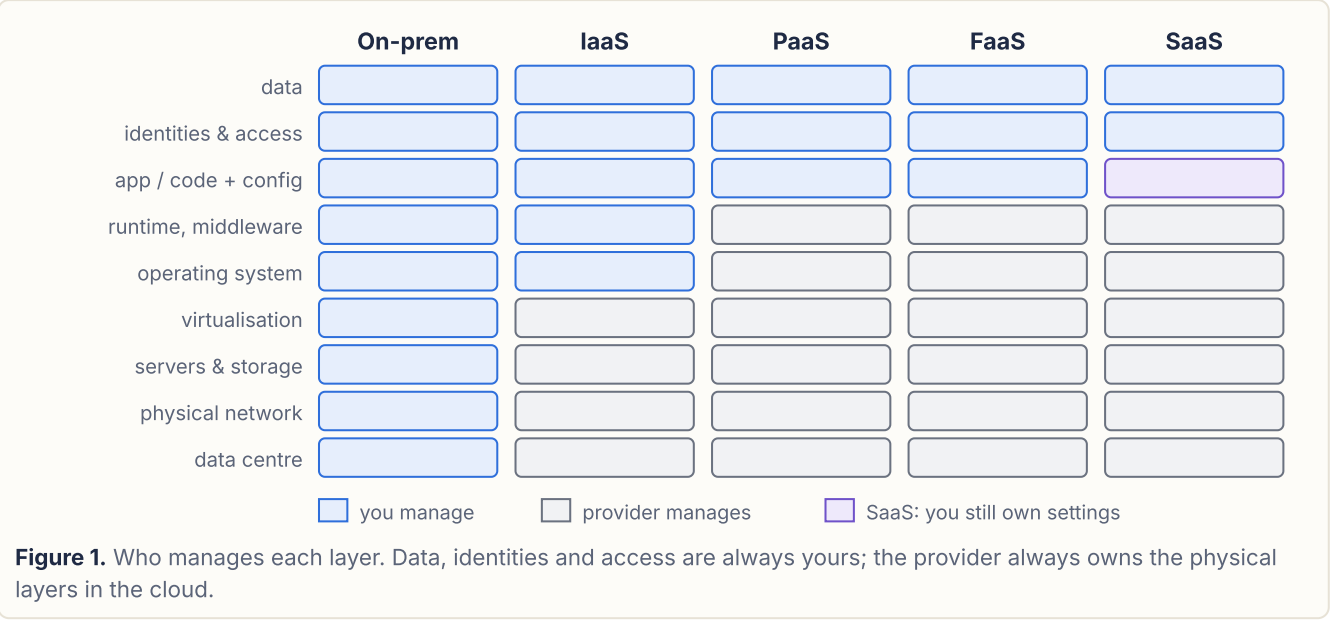


Figure 1. Who manages each layer. Data, identities and access are always yours; the provider always owns the physical layers in the cloud.

- **IaaS** is the closest to your own data centre: you install and patch the OS and software, configure backups and high availability, and size the machines. Maximum control and portability, maximum operational work.
- **PaaS** hands over the OS, runtime and much of operations: patching, backups, HA, often scaling. You bring code, data and configuration and accept the service's limits (versions, extensions, settings).
- **FaaS / serverless** goes further: deploy functions, pay per execution, never see a server. Limits shape the design: AWS Lambda runs at most 15 minutes per invocation; Azure Functions on the Consumption plan time out after 5 minutes by default (10 max). Expect **cold starts** when scaling from zero. 'Serverless' also describes PaaS that bills per use, like serverless database tiers.
- **SaaS**: you configure and use; your job is users, access, data and settings.

Model	You get	Examples	You pay for
IaaS	VMs, disks, networks	Azure VMs, AWS EC2, Google Compute Engine	provisioned capacity per second/hour, whether busy or not
CaaS (between)	a managed container platform	AKS, EKS, GKE; Azure Container Apps, Cloud Run, Fargate	nodes, or vCPU/memory per second
PaaS	a managed platform: DB, app host, queue	Azure SQL, App Service, RDS, Cloud SQL, Service Bus	service tier (vCores, DTUs, instance size)
FaaS	functions run on events, scale to zero	Azure Functions, AWS Lambda, Cloud Run functions	invocations + execution time × memory
SaaS	a finished application	Microsoft 365, Salesforce, GitHub	per user or per usage, monthly

3 The shared responsibility model

The provider is responsible for **security of the cloud**: physical data centres, hardware, the host network and the virtualisation layer. You are responsible for **security in the cloud**. Which layers that covers depends on the model (Figure 1), but some are **always yours**: your data and its classification, your identities and access (who has admin rights, MFA, keys), your endpoints, and how you configure the service (public network access, firewall rules, encryption settings, backups retention). Most cloud breaches come from customer-side misconfiguration, such as a storage container left public or an over-privileged key in a repo, not from the provider's infrastructure.

4 When to self-manage

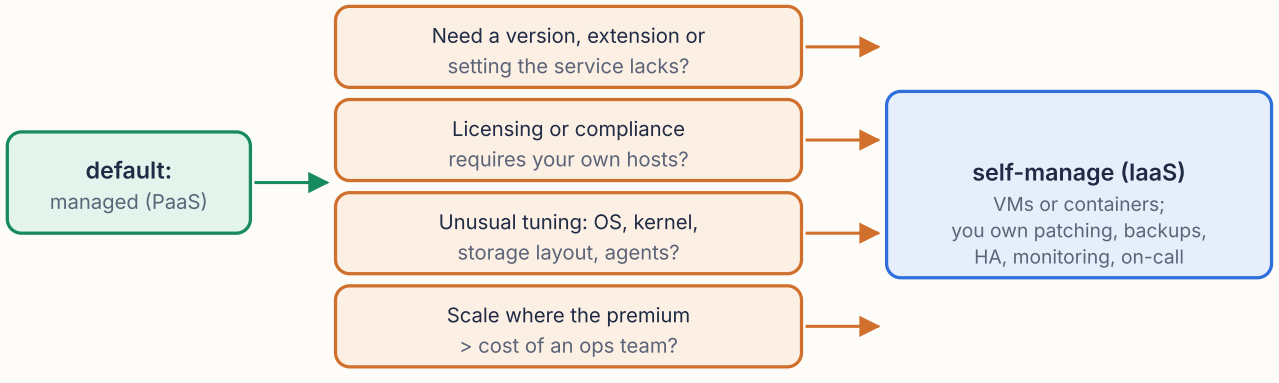


Figure 2. Start from managed services and require a concrete reason to take on the operations yourself.

Managed services usually win because engineering time is expensive and patching, failover and backups are undifferentiated work. Reasons to self-manage are specific: a database extension or version the service doesn't offer; software that must run with OS-level agents or custom kernels; licensing that ties you to your own hosts; compliance demanding dedicated hardware (though dedicated-host options exist); or a very large, steady footprint where the managed premium exceeds the cost of a team that runs it well. Also count **lock-in**: PaaS APIs and features differ across providers, while VMs and containers move more easily. A middle path is often best, e.g. Azure SQL Managed Instance (near-full SQL Server compatibility, still managed) or SQL Server on a VM with the provider's automation extension for patching and backups.

WORKED EXAMPLE: AZURE SQL (PAAS) VS SQL SERVER ON VMS (IAAS)

Task	SQL Server on VMs	Azure SQL Database
OS + SQL patching	you schedule and test	automatic, in maintenance windows
Backups	you configure, store and test restores	automatic; point-in-time restore, 1–35 days (default 7)
High availability	you build Always On availability groups / failover cluster	built in; zone-redundant option
Scaling	resize VMs, re-plan storage	change tier or vCores; serverless tier can auto-pause
Control	full: any version, agent, setting	limited: no OS access, some features differ

The team's app uses standard SQL features. Azure SQL costs somewhat more than the raw VMs, but replaces most of a DBA's routine work (patching, backup jobs, failover drills) and comes with a financially backed SLA, so they choose PaaS and keep the VM option for a legacy system that needs SQL Server Agent jobs and cross-database queries, which they move to Managed Instance instead.

TRADE-OFFS

- **Control vs operations**: every layer you own is a layer you patch, monitor and get paged for.
- **Unit price vs total cost**: VMs look cheaper per hour; PaaS often wins once people's time is counted.
- **Portability vs features**: managed services are convenient but provider-specific.
- **FaaS**: no idle cost and automatic scale, but time limits, cold starts and per-request cost at high steady load.

COMMON MISTAKES

- Assuming 'managed' means the provider secures your data and access settings too.
- Comparing VM price to PaaS price without counting engineering and on-call time.
- Putting long-running or steady high-throughput work on FaaS and paying more than a container would cost.
- Choosing IaaS for 'control' without a concrete feature or requirement that needs it.

5 Key terms

IaaS: rent VMs, disks, networks; you run the OS up

PaaS: provider runs OS and platform; you bring code and data

FaaS: event-driven functions, pay per execution

SaaS: finished application; you manage users and data

Shared responsibility: provider secures the cloud, you secure what's in it

Managed premium: extra price for operations you don't do

6 Interview questions

Q1. When would you self-manage instead of using a managed service? BASIC

When you need control, features or versions it lacks, when licensing or compliance demands it, or at a scale where the managed premium exceeds your operational cost.

Q2. What is the shared responsibility model? BASIC

The provider secures the underlying cloud; you always own your data, identities and configuration, with more layers shifting to the provider from IaaS to SaaS.

Q3. What's the difference between PaaS and FaaS? BASIC

PaaS runs your app on a managed platform that is usually always on; FaaS runs individual functions per event, scales to zero and bills per execution, with time limits and cold starts.

Q4. A startup's API has spiky traffic and idle nights. Which model? SCENARIO

FaaS or a scale-to-zero container platform for the API, with managed databases; it avoids paying for idle capacity, as long as cold starts and time limits are acceptable.

Q5. Your PostgreSQL needs an extension the managed service doesn't support. Options? SCENARIO

Check whether the provider's other tiers or flexible-server options support it, redesign around it, or self-manage on VMs/Kubernetes and accept the operational load.

Q6. A storage bucket on a managed service was leaked. Whose fault under the shared model? TRICKY

Usually the customer's: access configuration is always the customer's responsibility, even on fully managed services.

Q7. Is serverless always cheaper? TRICKY

No: at steady high utilisation, per-request pricing can exceed reserved VMs or containers; serverless wins on spiky or low traffic and on saved ops time.

Next: SD-V-002 · Regions and Availability Zones backendroadmap.com

Regions and Availability Zones

spread across zones, plan for regions

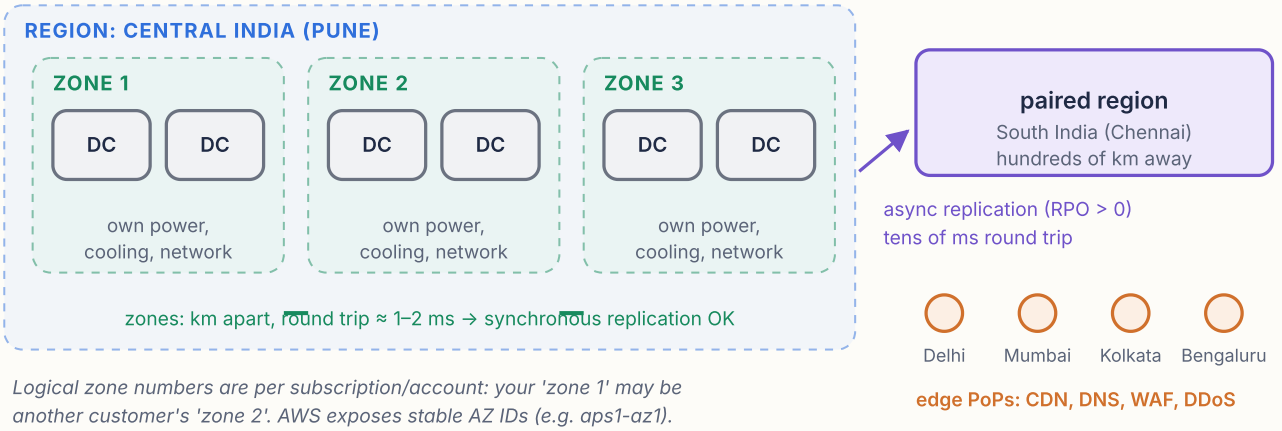
Watch the 4-min video

In one line: a **region** is a geographic area of data centres; inside it, **availability zones** are independent sites a few kilometres apart, close enough for synchronous replication; deploying across zones survives a data-centre failure, but not a region outage, a bad global deploy or corrupted data, which need multi-region designs, safe rollouts and backups.

1 Same outage, different outcome

A power failure takes out one data centre in a cloud region. Company A, running on VMs in a single zone, is offline for six hours. Company B, in the same region, doesn't notice: its app and database were spread across three availability zones and traffic simply moved. The difference was a deployment decision, not luck.

2 Regions, zones and edge



Logical zone numbers are per subscription/account: your 'zone 1' may be another customer's 'zone 2'. AWS exposes stable AZ IDs (e.g. aps1-az1).

Figure 1. A region contains at least three zones (where zones are offered), each with independent facilities; edge locations sit much closer to users.

A **region** is a set of data centres in one metro or geography: Central India (Pune), South India (Chennai) and West India (Mumbai) on Azure; `ap-south-1` (Mumbai) on AWS. Regions are isolated from each other, so a disaster in one rarely affects another. Choose a region for:

- **Latency** to your users (roughly 1 ms of round trip per 100 km of fibre, plus routing).
- **Data residency and regulation**: e.g. the RBI's 2018 directive requires payment-system data in India to be stored only in India; GDPR constrains transfers out of the EU (lesson 46).
- **Service and capacity availability**: new services, GPU types and zone support arrive region by region.
- **Price**, which differs between regions, and a **paired / secondary region** for disaster recovery.

An **availability zone** is one or more data centres with independent power, cooling and networking. Zones in a region are far enough apart that a fire, flood or power failure hits only one, but connected by high-bandwidth, low-latency links: Microsoft describes round-trip latency between zones of under about 2 ms; AWS places its AZs within about 100 km of each other. That latency is low enough for **synchronous**

replication (RPO = 0) on every commit. Note that cross-zone traffic is not always free: AWS, for example, bills data transfer between AZs.

3 Zonal, zone-redundant, regional, global

Flavour	Behaviour	Examples	Your job
Zonal	pinned to one zone; fails with it	a VM, a managed disk, a zonal IP	deploy several, one per zone, and balance
Zone-redundant	provider spreads it across zones automatically	ZRS storage, zone-redundant Azure SQL, Standard LB, multi-AZ RDS	turn it on (often at creation only)
Regional	runs in the region; provider decides placement	many PaaS control planes, queues	check its documented zone resilience
Global	spans regions	DNS, Front Door / CloudFront, Entra ID / IAM	remember it is a shared dependency

SLAs reflect this. For Azure VMs, two or more instances across zones carry a 99.99% connectivity SLA, versus 99.95% in an availability set within one data centre and 99.9% for a single VM with premium storage. Stateful zonal pieces are the usual trap: a pod whose persistent disk is zonal can't restart in another zone unless the disk is zone-redundant or the data is replicated at the application level (lesson 59).

4 What multi-AZ does and doesn't buy you

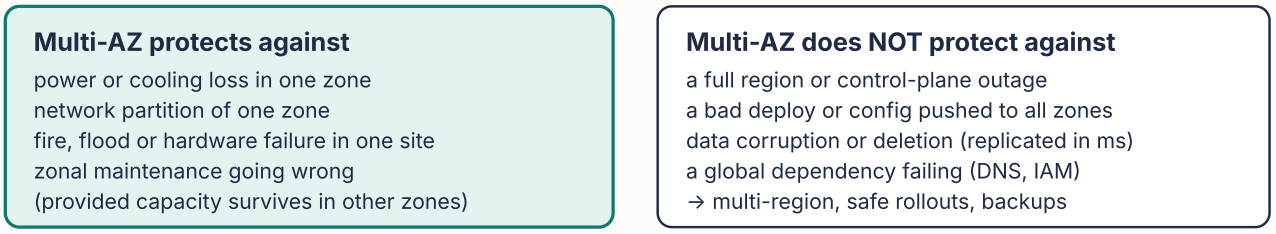


Figure 2. Zones remove the data centre as a single point of failure. Other failure modes need other defences.

Static stability matters as much as spreading: if losing a zone forces you to launch new capacity during the incident, you depend on the control plane exactly when it is busiest. Pre-provision so the survivors can absorb the load: with three zones, run each at no more than about two-thirds of its capacity (in total, ~150% of peak), so two zones can carry 100%. Region-level failures do happen, including control-plane outages such as AWS us-east-1 in December 2021, which is why critical systems add a second region (active-passive with asynchronous replication, or active-active) plus backups that a bad write can't reach.

5 Edge locations

Edge points of presence are small sites in hundreds of cities, much closer to users than regions. They host CDN caches, DNS resolvers, TLS termination, WAF and DDoS protection, and lightweight edge functions (lesson 69). Pushing static content and simple logic to the edge cuts latency for distant users and absorbs attacks, while the data and core logic stay in the region. Providers also offer in-between options such as AWS Local Zones and Azure Extended Zones for latency-sensitive workloads in specific metros.

WORKED EXAMPLE: THREE ZONES IN CENTRAL INDIA

The app runs on an AKS cluster whose node pool is created with zones 1, 2 and 3; pods use `topologySpreadConstraints` on `topology.kubernetes.io/zone` so 2 pods land in each zone. A zone-redundant Standard Load Balancer fronts them. Azure SQL runs in a zone-redundant configuration with synchronous replicas in other zones. Each zone has spare capacity for half the peak load.

Zone 2 loses power. Its nodes, 2 pods and a database replica disappear. Load-balancer health probes fail within seconds and traffic goes to zones 1 and 3; committed transactions are safe because replication was synchronous, and if the primary had been in zone 2, a replica would be promoted automatically. The remaining 4 pods absorb the load without waiting for autoscaling; Kubernetes reschedules the lost pods onto surviving zones. Users see a few failed requests that clients retry.

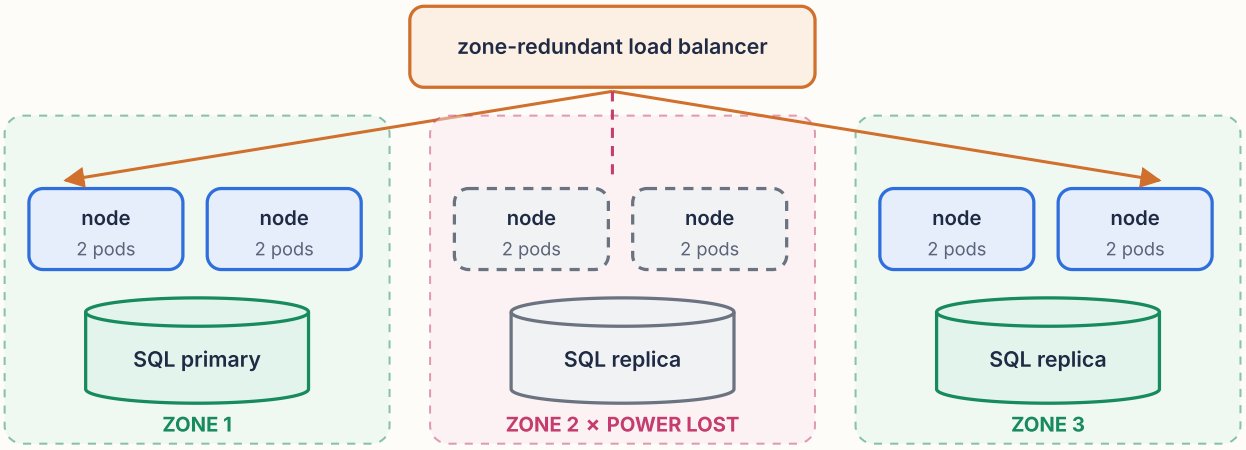


Figure 3. The worked example during the zone 2 failure: the load balancer stops sending traffic there and the other zones carry the load.

TRADE-OFFS

- **Multi-AZ**: survives a data-centre failure for modest cost, but adds cross-zone latency and (on some clouds) transfer fees.
- **Multi-region**: survives a region failure, but brings async replication (RPO > 0), failover complexity and roughly double cost.
- **Spare capacity per zone**: static stability costs money every day, but makes zone loss a non-event.
- **Region choice**: closest-to-users vs where the services, capacity and compliance rules are.

COMMON MISTAKES

- Running 'highly available' replicas that all landed in one zone.
- Making the app zone-redundant but leaving a zonal database, cache or disk as a single point of failure.
- Assuming multi-AZ protects against bad deploys or accidental deletes.
- Relying on autoscaling during a zone outage instead of pre-provisioned headroom.

6 Key terms

Region: geographic area with several data centres

Availability zone: independent site(s) in a region, ~1–2 ms apart

Zone-redundant: service spread across zones by the provider

Paired / secondary region: distant region for disaster recovery

Static stability: survive a failure without new provisioning

Edge PoP: small site near users: CDN, DNS, WAF

7 Interview questions

Q1. What failures does a multi-AZ deployment protect against? BASIC

Failures of a single data centre or zone, like power, cooling, networking or hardware, because zones are independent but close enough for synchronous replication.

Q2. What doesn't it protect against? BASIC

A full region outage, bad global deployments or replicated data corruption, which need multi-region designs, safe rollouts and backups.

Q3. How do you choose a region? BASIC

Latency to users, data-residency and compliance rules, availability of the services and capacity you need, price, and a suitable second region for DR.

Q4. You run 3 zones at 80% utilisation each. What happens when one fails? SCENARIO

The other two would need 120% each, so they overload until autoscaling adds capacity, if it can; keep each zone at ≤ ~66% for static stability.

Q5. Why might synchronous replication across regions be a bad idea? SCENARIO

Every commit waits for a round trip of tens of milliseconds or more, hurting latency and availability; regions usually replicate asynchronously and accept RPO > 0.

Q6. Your Kubernetes pods are spread across zones but the app still went down with zone 1. Why? TRICKY

A zonal dependency: a persistent disk, a single-zone database or cache, or a NAT/egress component pinned to zone 1.

Q7. Is 'zone 1' the same physical zone for every customer? TRICKY

Not necessarily: providers map logical zone names per subscription or account, so coordinate across accounts with AZ IDs or the provider's zone mapping.

Next: SD-V-003 · Cloud Networking backendroadmap.com

Cloud Networking

private by default, public by exception

Watch the 4-min video

In one line: put workloads in a private **virtual network** split into tiered subnets, expose only the load balancer, allow each tier only from the tier above with **security groups**, reach managed services through **private endpoints**, send outbound traffic through a **NAT gateway**, and watch the data-transfer bill.

1 The database with a public IP

A team launches its first cloud app. The database gets a public IP so developers can connect from home, every VM can talk to every other VM, and outbound traffic leaves however the cloud defaults allow. It works until a security scan finds Postgres listening on the internet, protected only by a password. Internet-wide scanners find open database ports within hours. Good cloud networking makes that state impossible by construction.

2 Virtual networks, address space and subnets

A **VPC** (AWS, Google Cloud) or **VNet** (Azure) is your private address space, normally from the RFC 1918 ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. A `/16` such as 10.0.0.0/16 holds 65,536 addresses; a `/24` subnet holds 256, of which AWS and Azure each reserve 5 (the first four and the last: network address, gateway, DNS and other provider uses, and broadcast), so 251 are usable. Split by tier:

- **Public subnet:** only load balancers, application gateways and NAT. On AWS a subnet is "public" because its route table sends 0.0.0.0/0 to an internet gateway.
- **Private subnet:** app servers, containers, Kubernetes nodes. No public IPs.
- **Data (isolated) subnet:** databases and private endpoints, reachable only from the app subnet.

A zonal detail: an AWS subnet lives in one availability zone, so you create one subnet per tier **per AZ**; an Azure subnet spans the region's zones. Size generously: Kubernetes pods that take VNet IPs (Azure CNI, AWS VPC CNI) can consume thousands of addresses (lesson 60).

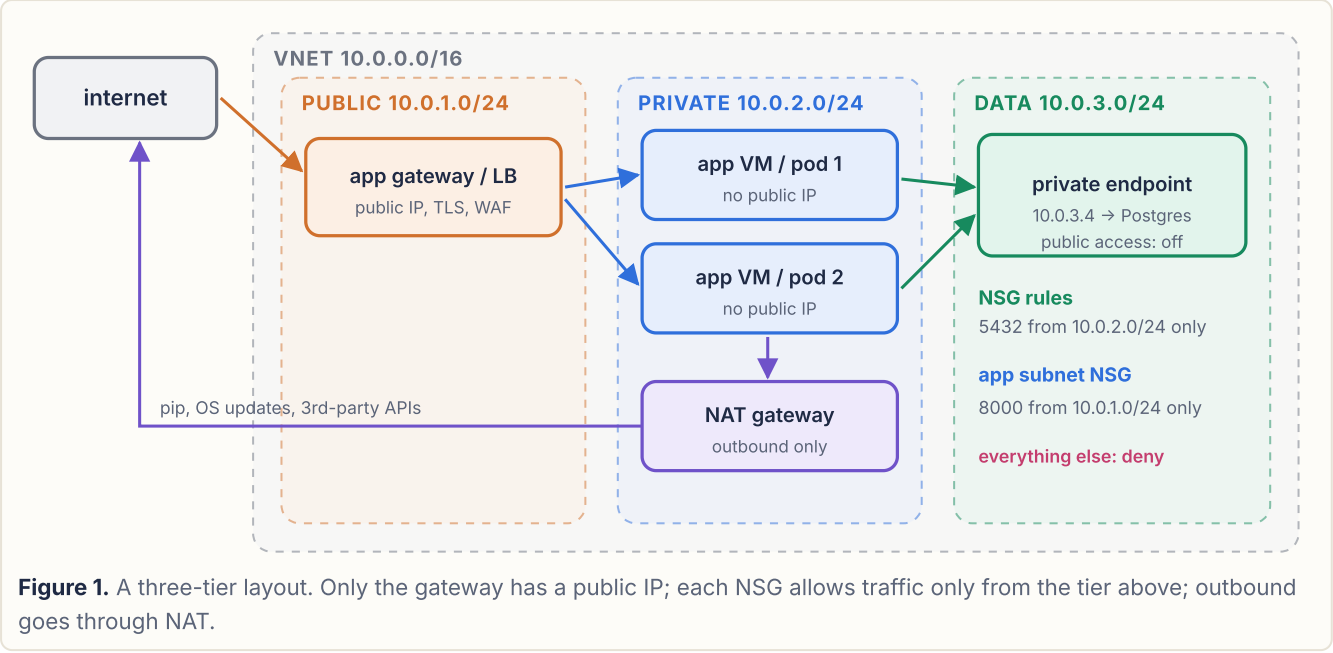


Figure 1. A three-tier layout. Only the gateway has a public IP; each NSG allows traffic only from the tier above; outbound goes through NAT.

3 Security groups: allow lists

Security groups (AWS) and **network security groups** (Azure) are stateful firewalls attached to resources or subnets: allow the inbound connection and the reply is allowed automatically. Write them as allow lists by tier: the app tier accepts port 8000 only from the load balancer; the database accepts 5432 only from the app tier. AWS lets a rule reference another security group ("from sg-app") instead of IP ranges, and Azure has application security groups for the same purpose, so rules survive scaling and IP changes.

Know the defaults. An Azure NSG has default rules that **allow all traffic inside the VNet** (AllowVNetInBound, priority 65000) before the final deny, so tier isolation needs explicit deny or narrow allow rules at lower priority numbers. AWS also has stateless **network ACLs** at subnet level, a coarse second layer where you must allow return traffic yourself.

4 NAT gateways and private endpoints

Private resources still need outbound access: package updates, container image pulls, third-party APIs. A **NAT gateway** lets them open connections out while remaining unreachable from outside, and gives all outbound traffic a fixed public IP that partners can allow-list. On AWS a NAT gateway is zonal, so production uses one per AZ.

Managed services (databases, storage, key vaults, queues) have public endpoints by default. A **private endpoint** (Azure Private Link, AWS PrivateLink interface endpoints) gives the service a private IP inside your subnet, so traffic never crosses the internet, and you can then **disable public access** on the service entirely. The catch is DNS: the service's normal hostname must resolve to the private IP inside your network, via a private DNS zone such as `privatelink.postgres.database.azure.com`, or clients silently keep using the public endpoint. On AWS, **gateway endpoints** for S3 and DynamoDB are free and keep that traffic off the NAT gateway.

5 Connecting networks: peering and hub-and-spoke

Peering links two virtual networks directly with private routing. It is **not transitive**: if A peers with B and B with C, A cannot reach C through B. With many networks, use a **hub-and-spoke** design: a central hub holds shared services (firewall, VPN or ExpressRoute / Direct Connect gateway to on-premises, DNS), and each workload spoke peers only with the hub. On AWS a **Transit Gateway** plays the hub role as a managed router; on Azure, a hub VNet or Virtual WAN.

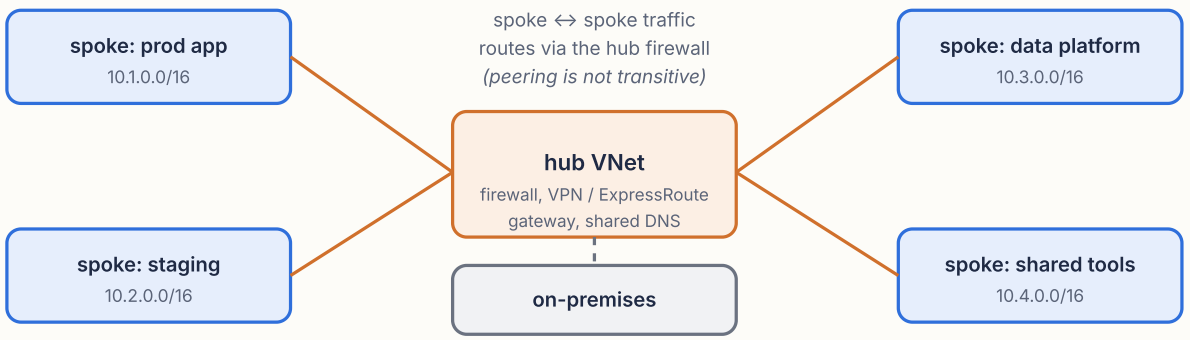


Figure 2. Hub-and-spoke: one place for shared firewalls, inspection and on-premises connectivity.

Plan address ranges early. Overlapping CIDR ranges cannot be peered or routed, and renumbering a live network is painful. Keep an IP address plan (e.g. a /16 per spoke from a reserved 10.0.0.0/8), and never reuse the default range in every environment.

6 What network traffic costs

Traffic	Typical AWS list price	Notes
Within one AZ (private IPs)	free	keep chatty services together
Between AZs in a region	$0.01/GB each direction	replication and cross-zone calls add up
Between regions	about $0.02/GB (varies by pair)	cross-region replication, DR
Out to the internet	about $0.09/GB for the first 10 TB/month	first 100 GB/month free; a CDN is usually cheaper
NAT gateway	$0.045/hour + $0.045/GB processed (us-east-1)	on top of transfer charges

Prices vary by provider and region (Azure and Google Cloud have their own tables), but the shape is the same: inside a zone is cheap, crossing zones, regions and the internet costs more each step. Classic surprises: a service pulling large container images or S3 data through the NAT gateway, chatty microservices spread across zones, and large downloads served from origin instead of a CDN. Cost optimisation is lesson 64.

WORKED EXAMPLE: AN APP REACHING A MANAGED DATABASE PRIVATELY

VNet `10.0.0.0/16` has three subnets: `10.0.1.0/24` for Azure Application Gateway (the only public IP), `10.0.2.0/24` for the app servers, and `10.0.3.0/24` holding a private endpoint for Azure Database for PostgreSQL at `10.0.3.4`. Public network access on the database is **disabled**. A private DNS zone makes the server's hostname resolve to 10.0.3.4 inside the VNet. The data subnet's NSG allows 5432 only from 10.0.2.0/24 and denies other VNet traffic; the app subnet allows 8000 only from the gateway subnet. A NAT gateway on the app subnet handles outbound calls. The security scan now finds one open port on the internet: 443 on the gateway.

COMMON MISTAKES

- Public IPs on databases or VMs "just for debugging" (use a bastion or VPN).
- Relying on Azure's default NSG rules, which allow all traffic inside the VNet.
- Private endpoint created but DNS still resolving to the public address.
- Overlapping address ranges that block peering later.

7 Key terms

VPC / VNet: private address space in the cloud

Security group / NSG: stateful allow-list firewall

Private endpoint: private IP for a managed service

Subnet: a slice of the range, usually one per tier

NAT gateway: outbound-only internet access for private resources

Hub-and-spoke: shared hub network that spokes peer with

8 Interview questions

Q1. Design a secure network layout for a three-tier app in the cloud. BASIC
One virtual network with public, private and data subnets; only the load balancer is public; security groups allow each tier only from the tier above; managed services via private endpoints; a NAT gateway for outbound access.

Q2. What drives network cost? BASIC
Cross-zone, cross-region and internet egress traffic, plus NAT gateway processing; traffic within a zone is usually free.

Q3. How many usable addresses does a /24 subnet have on AWS or Azure? BASIC
251: 256 minus the 5 addresses each provider reserves.

Q4. A private app server needs to call a third-party API that allow-lists IPs. How? SCENARIO
Route outbound traffic through a NAT gateway with a static public IP and give that IP to the partner.

Q5. You added a private endpoint for your database, but traffic still goes over the public endpoint. Why? SCENARIO
DNS: the hostname still resolves to the public IP; link a private DNS zone (privatelink...) to the VNet so it resolves to the private endpoint.

Q6. VNet A peers with B, and B with C. Can A reach C? TRICKY
No, peering isn't transitive; use a hub with a firewall or router (or Transit Gateway / Virtual WAN), or peer A and C directly.

Q7. Your NAT gateway bill is larger than your compute bill. What's likely happening? TRICKY
Heavy traffic such as S3 reads or image pulls is going through NAT; use gateway or private endpoints for cloud services and cache or mirror images.

Next: SD-V-004 · Well-Architected Frameworks backendroadmap.com

Well-Architected Frameworks

six lenses on the same design

Watch the 4-min video

In one line: AWS, Azure and Google Cloud each publish a well-architected framework: pillars (reliability, security, cost, operations, performance, and on AWS sustainability), design principles and review questions, used as **lenses** on one design to surface risks and trade-offs before customers find them.

1 Two weeks before launch

Your architect asks three questions. *What happens if this region goes down?* Nobody knows. *Who can delete the production database?* Eleven people. *What will this cost at ten times the traffic?* Silence. None of these are exotic; they are simply questions nobody was responsible for. A well-architected review is a structured way to ask them all, every time, and to record the answers.

2 The frameworks and their pillars

Provider	Pillars	Tools
AWS Well-Architected	Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability (added 2021)	Well-Architected Tool (marks high and medium risk issues); lenses such as SaaS and Serverless
Azure Well-Architected	Reliability, Security, Cost Optimization, Operational Excellence, Performance Efficiency	Well-Architected Review assessment; Azure Advisor recommendations by pillar
Google Cloud (Architecture / Well-Architected Framework)	the same core areas, with security combined with privacy and compliance	framework guides and design checklists

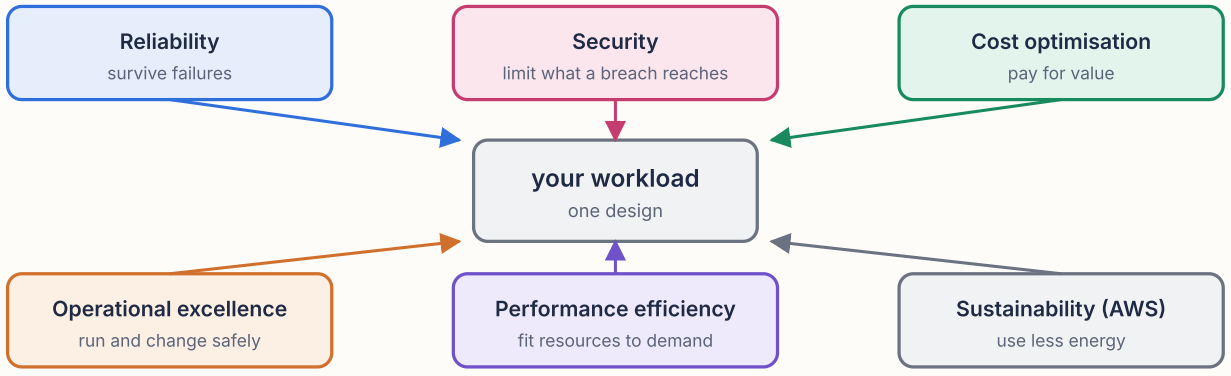


Figure 1. The pillars are not separate checklists to tick once; each is a different way of looking at the same system.

3 Reliability and security

Reliability is about recovering from failure and meeting demand. Set recovery targets per workload: **RTO** (how long you may be down) and **RPO** (how much data you may lose). Then design to them: redundancy

across availability zones (lesson 52), health checks and automatic failover, timeouts and retries with backoff, and backups you have **actually restored**. The key review question: what is our RTO, and have we proven we can meet it with a drill?

Security covers identity, data and network. Least privilege (no standing owner rights, short-lived credentials, managed identities instead of keys), encryption in transit (TLS) and at rest, network isolation (lesson 53), secrets in a vault, and detection: audit logs and alerts on suspicious activity. **Assume breach**: for each component, ask what an attacker who controls it could reach next, and shrink that.

4 Cost optimisation and operational excellence

Cost optimisation means paying only for what delivers value: right-sizing (many instances run at low average CPU), autoscaling, reservations or savings plans for steady load, spot capacity for interruptible work, storage tiering, and deleting what nobody uses. Measure **unit cost**, such as cost per tenant or per 1,000 requests, so growth and waste can be told apart. Tag every resource with an owner, environment and cost centre, and set budget alerts. More in lessons 64 and 65.

Operational excellence is how you run and change the system: infrastructure as code (lesson 61), CI/CD with safe deployment (canaries, feature flags, fast rollback), monitoring and alerting on user-facing SLOs, runbooks, and blameless post-incident reviews. Prefer small, frequent, reversible changes; the DORA metrics (deployment frequency, lead time for changes, change failure rate, time to restore) are a common way to measure it.

5 Performance efficiency and sustainability

Performance efficiency is using resources well as demand changes: choosing the right compute (CPU, memory or GPU optimised; ARM-based instances often give better price-performance), storage and database types, caching, CDNs, and **load testing** before launch against the expected peak. Measure before scaling: slow requests usually come from the data layer (missing indexes, N+1 queries), not from too few servers.

Sustainability, AWS's sixth pillar, asks you to minimise the energy and hardware a workload consumes: raise utilisation, remove idle and over-provisioned capacity, scale to zero where possible, keep only the data you need, and prefer efficient hardware and regions with lower-carbon electricity. It mostly aligns with cost: the greenest server is often the one you can switch off (lesson 73).

6 The pillars pull against each other

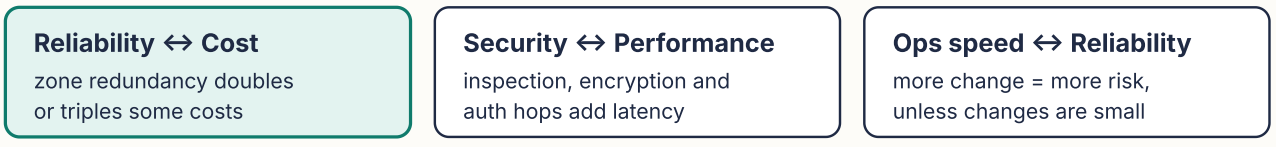


Figure 2. Typical tensions. A review doesn't remove them; it makes the choice explicit and documented.

More redundancy raises reliability and cost. Deeper inspection raises security and latency. Faster change raises delivery speed and, without guard-rails, risk. The value of a review is turning accidental trade-offs into deliberate ones: "single-zone dev environment, accepted, because it saves 40% and has no SLA" is a decision; nobody noticing is not.



Figure 3. The review loop. Findings become a tracked backlog, not a slide.

WORKED EXAMPLE: A REVIEW BEFORE LAUNCH

The team answers the Azure Well-Architected Review questions for each pillar, with evidence (screenshots, configs, drill results). Findings: the Azure SQL database is not zone-redundant; eleven people hold Owner on the production subscription; no restore has ever been tested; there are no budget alerts; there's no load test above 2× current traffic. They fix the high-risk items before launch: enable zone redundancy, replace standing Owner rights with least-privilege roles and just-in-time elevation, and run a restore drill that measures a real RTO of 35 minutes. Budget alerts and a 10× load test go on the backlog with owners and dates. Six months later the re-review list is much shorter.

COMMON MISTAKES

- Treating the framework as a one-off checklist instead of a recurring review.
- Answering "yes, we have backups" without having restored one.
- Reviewing only reliability and security, never cost and operations.
- Recording findings without owners, priorities and dates.

7 Key terms

Pillar: one lens: reliability, security, cost, ops, performance, sustainability

Least privilege: only the access a role needs, only when needed

High-risk issue: finding to fix before launch

RTO / RPO: max downtime / max data loss

Unit cost: cost per tenant, request or transaction

DORA metrics: four measures of delivery performance

8 Interview questions

Q1. Name the well-architected pillars. BASIC

Reliability, security, cost optimisation, operational excellence, performance efficiency, and on AWS sustainability.

Q2. One practice for each? BASIC

Zone redundancy with tested restores; least privilege and encryption; right-sizing and autoscaling; infrastructure as code with safe deployments; caching and load testing; higher utilisation with no idle resources.

Q3. You have one week before launch. Which review findings do you fix first? SCENARIO

The high-risk ones that cause data loss, outages or breaches: untested restores, single-zone databases, excessive admin rights, public exposure; schedule the rest with owners.

Q4. How do you show a reliability design actually works? SCENARIO

Define RTO and RPO, then prove them with restore drills, failover tests and chaos experiments, and record the measured times.

Q5. Give an example where two pillars conflict and how you'd decide. TRICKY

Multi-region active-active improves reliability but roughly doubles cost and adds consistency complexity; decide from the business's RTO, RPO and budget, and document it.

Q6. Is a well-architected review a compliance audit? TRICKY

No: it's a self-assessment against best practices to find risks and trade-offs; compliance audits (SOC 2, ISO 27001) check controls against a standard.

Next: SD-V-005 · Containers and Docker

Containers and Docker

ship the app with everything it needs

Watch the 4-min video

In one line: a container image packages your code with its runtime, libraries and start command; containers run it as isolated processes that **share the host kernel** (namespaces + cgroups), so they're light and fast; build images in cache-friendly layers with multi-stage builds, and ship them by immutable tag from a scanned registry.

1 It works on my machine

The FastAPI service runs perfectly on a developer's laptop, then fails in production: the server has Python 3.9 instead of 3.12 and is missing `libpq`. Environments drift: OS packages, runtime versions, environment variables. Containers fix this by shipping the application **together with** everything it needs, as one immutable artifact that runs the same on a laptop, a CI runner and a Kubernetes cluster.

2 Images and containers

An **image** is a packaged filesystem plus metadata: OS libraries, the language runtime, your dependencies and code, environment defaults and the start command. Under the OCI image specification it is a stack of read-only **layers** (tar archives) plus a config, each addressed by its SHA-256 digest, so identical layers are stored and downloaded once. A **container** is a running instance: the runtime stacks the image layers with a union filesystem (overlayfs), adds a thin writable layer on top, and starts the process. Anything written there disappears with the container, so state belongs in volumes or external services.

3 Containers are not virtual machines

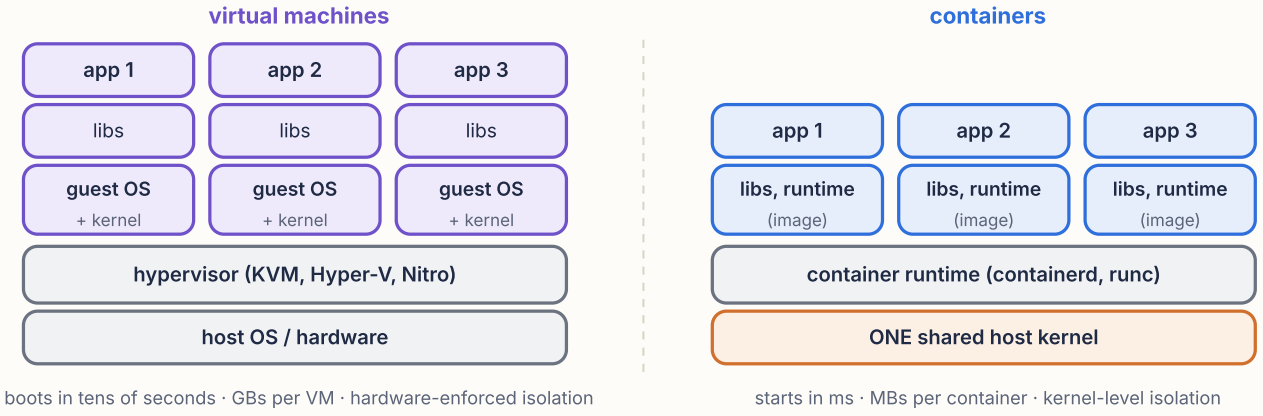


Figure 1. A VM virtualises hardware and runs a whole OS; a container is an isolated process on the host's kernel.

A container is an ordinary Linux process with two kernel features applied. **Namespaces** give it its own view of the system: PID (its own process tree, where it is PID 1), network (own interfaces and ports), mount (own filesystem), UTS (hostname), IPC and user namespaces. **Control groups (cgroups)** limit what it may consume: CPU, memory, I/O and number of processes; these are what Kubernetes requests and limits set (lesson 57).

	Virtual machine	Container
Isolation boundary	hypervisor, hardware-assisted	kernel namespaces + cgroups
Contains	full guest OS and kernel	app + user-space libraries only
Start time	tens of seconds to minutes	milliseconds to a second or two
Footprint	GBs of memory and disk	what the process needs; layers shared
Kernel	any (Windows on Linux host is fine)	the host's (Linux images need a Linux kernel)
Security	strong; a guest escape is rare	weaker: a kernel bug can break out

When tenants run untrusted code, the industry answer is to combine both: **sandboxed runtimes** such as gVisor (a user-space kernel) or micro-VMs like Firecracker and Kata Containers, which keep container ergonomics but put a VM-style boundary around each workload.

4 Layers and build caching

Each Dockerfile instruction that changes the filesystem (`RUN`, `COPY`, `ADD`) creates a layer, and the builder reuses a cached layer when the instruction and its inputs are unchanged. A changed layer invalidates every layer after it, so **order matters**: copy `requirements.txt` and install dependencies first, then copy the source. A code change then rebuilds a one-second layer instead of reinstalling every package. Add a `.dockerignore` (`.git`, `.venv`, tests, secrets) so the build context is small and unrelated file changes don't bust the cache.

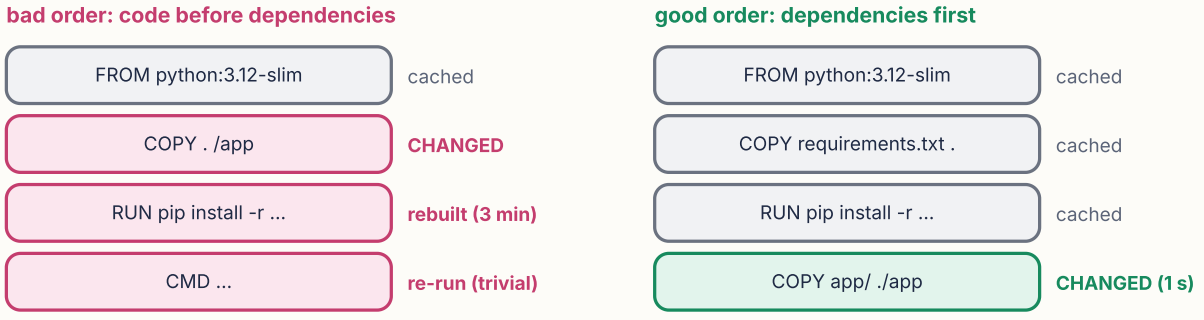


Figure 2. Same Dockerfile, two orders. The left one reinstalls every dependency on each code change.

5 Multi-stage builds and small, safe images

Build tools (compilers, headers) are needed to build but not to run. A **multi-stage build** compiles in a first stage and copies only the results into a fresh, slim final stage. Smaller images pull and start faster (which matters when autoscaling) and contain fewer packages with known vulnerabilities. Choosing a base for Python: `python:3.12-slim` (Debian) is the usual choice; Alpine uses musl libc, so many Python wheels must be compiled from source; distroless images strip even the shell and package manager.

Three details in the file below matter. A file deleted in a later layer still exists in the earlier one, so `COPY` + `RUN rm` doesn't shrink an image; the bind mount avoids the copy altogether. **Run as non-root** (`USER 10001`) so a compromised process has fewer privileges. And use the **exec form** of `CMD` (a JSON array): the shell form wraps the process in `/bin/sh -c`, which doesn't forward SIGTERM, so the app is killed hard instead of shutting down gracefully.

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim AS build
RUN apt-get update && apt-get install -y --no-install-recommends build-essential
WORKDIR /src
COPY requirements.txt .
RUN pip wheel --no-cache-dir --wheel-dir /wheels -r requirements.txt

FROM python:3.12-slim                     # fresh, slim final stage
WORKDIR /app
RUN --mount=type=bind,from=build,source=/wheels,target=/wheels \
    pip install --no-cache-dir --no-index /wheels/*   # wheels never become a layer
COPY app/ ./app
RUN useradd --uid 10001 --no-create-home appuser
USER 10001
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Figure 3. Multi-stage FastAPI image. The bind mount (BuildKit, the default builder since Docker Engine 23) installs the wheels without storing them in a layer.

6 Registries, tags, scanning and signing

Images live in a **registry**: Docker Hub, Azure Container Registry, Amazon ECR, GitHub Container Registry, Google Artifact Registry. Tag each build with an **immutable version**, typically the git commit SHA, not `latest`: `latest` moves, so two nodes can run different code under the same name and rollbacks become guesswork. For full certainty deploy by digest (`shop-api@sha256:…`). In CI, **scan** images for known CVEs (Trivy, Grype, or the registry's scanner), generate an SBOM, and **sign** them (Sigstore cosign, Notation) so the cluster's admission policy only runs images your pipeline built.

WORKED EXAMPLE: A SMALL FASTAPI IMAGE

The first version was `FROM python:3.12` (the full Debian image with compilers), `COPY . .`, then `pip install`: about **1.1 GB**, rebuilt from scratch on every commit, running as root. The multi-stage version above builds wheels in stage one, installs them in a slim stage two, copies only `app/`, runs as UID 10001 and starts uvicorn in exec form: about **150 MB**. Code-only commits rebuild in seconds because the dependency layers are cached, and the CI pipeline pushes `shopreg.azurecr.io/shop-api:3f9c2e1` after a clean scan.

COMMON MISTAKES

- Copying the whole source tree before installing dependencies.
- Deploying `:latest`, so you can't tell or pin what's running.
- Running as root and baking secrets into image layers (they stay in history).
- Treating containers as a strong security boundary for untrusted code.

7 Key terms

Image: layered, immutable filesystem + start command

Container: running instance of an image

Namespaces: kernel feature: isolated view of processes, network, mounts

cgroups: kernel feature: CPU, memory, I/O limits

Multi-stage build: build in one stage, ship only results

Registry: stores and serves images by tag and digest

8 Interview questions

Q1. How are containers different from VMs? BASIC

Containers share the host kernel and isolate processes with namespaces and cgroups, so they're lightweight and start in milliseconds; VMs run a full guest OS on a hypervisor, with stronger isolation and more overhead.

Q2. How do you keep images small and fast to build? BASIC

Order layers for caching, use multi-stage builds with slim bases, copy only runtime artifacts, and use a .dockerignore.

Q3. Image vs container? BASIC

An image is the immutable, layered package; a container is a running instance of it with its own writable layer.

Q4. Every code change triggers a five-minute pip install. Why? SCENARIO

The source is copied before dependencies are installed, so any code change invalidates the install layer; copy requirements.txt and install first, then copy the code.

Q5. A pod takes 30 seconds to stop on every deploy. What's a likely cause? SCENARIO

The app doesn't receive SIGTERM (shell-form CMD wraps it in sh), so it ignores the stop signal until the grace period ends and it's killed; use exec form and handle SIGTERM.

Q6. You deleted a secret file in a later RUN step. Is it gone from the image? TRICKY

No, it remains in the earlier layer and can be extracted; use build secrets (RUN --mount=type=secret) so it never enters a layer.

Q7. Why is :latest a bad tag in production? TRICKY

It's mutable: different nodes can pull different images under the same name, and you can't reliably roll back or know what's running.

Next: SD-V-006 · Kubernetes Core Objects backendroadmap.com

Kubernetes Core Objects

pods, Deployments, Services, Ingress

Watch the 4-min video

In one line: you declare desired state as objects (a **Deployment** for replicated pods, a **Service** for a stable address, an **Ingress** for HTTP routing, ConfigMaps and Secrets for configuration) and controllers continuously reconcile reality towards it; **labels and selectors** tie the objects together.

1 What you need after docker run

You've containerised the FastAPI app (lesson 55). In production you need three copies, restarted when they crash, spread over machines, updated without downtime, reachable at a stable internal address, and exposed on `api.shop.com` with TLS. Kubernetes provides each of these with a small set of object types.

2 Declarative state and the cluster

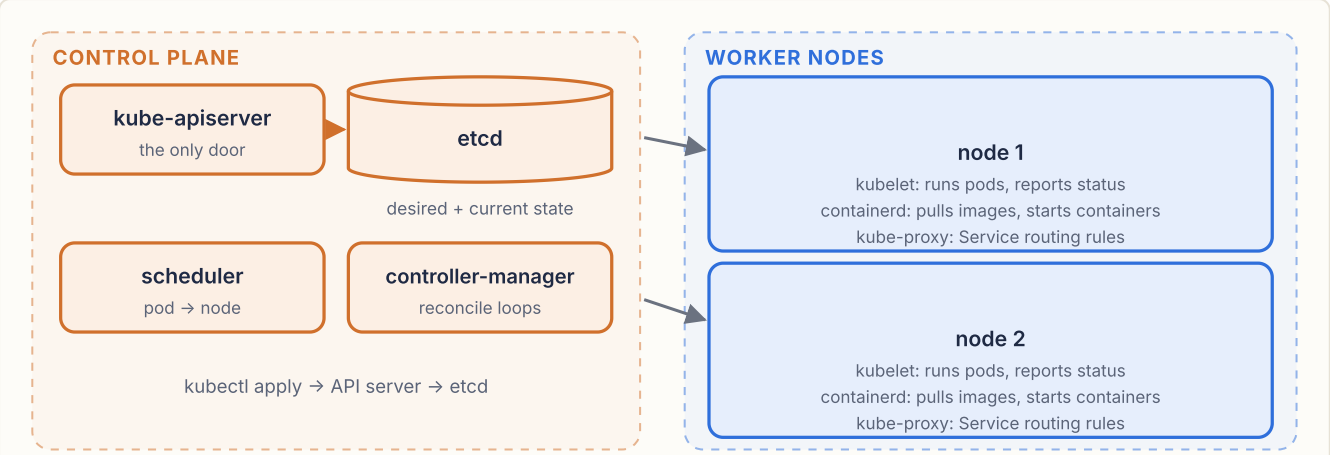


Figure 1. Everything goes through the API server and is stored in etcd; controllers and kubelets watch it and act.

You never tell Kubernetes "start a container"; you submit YAML saying what should exist. The **API server** validates and stores it in **etcd**. **Controllers** in the controller manager run reconcile loops: observe current state, compare with desired, act to close the gap. The **scheduler** assigns new pods to nodes (lesson 57). On each node, the **kubelet** starts the pod's containers through the container runtime and reports status. Because everything is a loop, the system self-heals: delete a pod and a controller simply creates a new one.

3 Pods: the disposable unit

A **pod** is one or more containers that share a network namespace (one IP, talking over localhost) and volumes, always scheduled together on one node. Typical multi-container pods have a main container plus a sidecar (log shipper, proxy). Pods are **disposable**: when a node dies, its pods are gone, and replacements get new names and new IPs elsewhere. So you almost never create bare pods; you create a controller that manages them.

4 Deployments and ReplicaSets

A **Deployment** declares "run 3 replicas of this pod template". It creates a **ReplicaSet**, whose only job is to keep exactly that many matching pods running. When you change the template (a new image tag), the Deployment creates a new ReplicaSet and performs a **rolling update**: scale the new one up and the old one down step by step, by default with `maxSurge: 25%` and `maxUnavailable: 25%`. A new pod counts only when it passes its **readiness probe**, so a broken release stalls instead of replacing healthy pods. Old ReplicaSets are kept (10 by default), so `kubectl rollout undo` is a fast rollback.

Probes matter: **readiness** decides whether a pod receives traffic; **liveness** restarts a stuck container; **startup** protects slow starters from liveness kills. For stateful workloads, a **StatefulSet** gives stable names and volumes (lesson 59); for one pod per node, a **DaemonSet**; for run-to-completion work, a **Job** or **CronJob**.

5 Services: a stable address

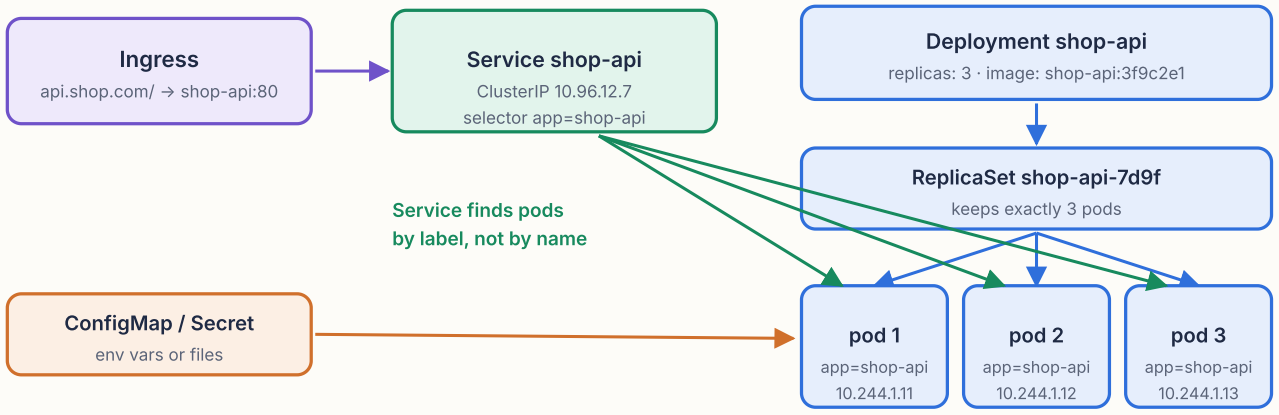


Figure 2. Labels are the glue: the Deployment owns pods labelled app=shop-api; the Service routes to whatever ready pods carry that label. All live in namespace shop.

A **Service** gives a changing set of pods one stable virtual IP and DNS name, `shop-api.shop.svc.cluster.local` (or just `shop-api` from the same namespace). Its **selector** picks pods by label; Kubernetes keeps an **EndpointSlice** listing the ready pod IPs, and kube-proxy (iptables or IPVS rules on every node, or an eBPF data plane such as Cilium) load-balances connections to them. Types: **ClusterIP** (internal only, the default), **NodePort** (a port in 30000–32767 on every node), **LoadBalancer** (asks the cloud for an external load balancer), and **headless** (no virtual IP; DNS returns the pod IPs, used by StatefulSets). Networking details are in lesson 60.

6 Ingress: HTTP from outside

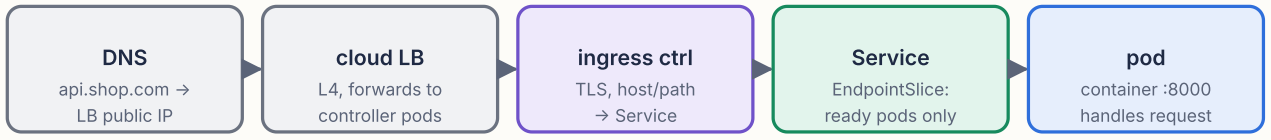


Figure 3. The request path. Many controllers proxy straight to pod IPs from the EndpointSlices.

An **Ingress** is a set of HTTP routing rules: host `api.shop.com`, path `/` → Service `shop-api` port 80, with TLS from a Secret. The Ingress object does nothing by itself; an **ingress controller** (NGINX-based, Traefik, HAProxy, or a cloud one such as Azure Application Gateway for Containers or the AWS Load Balancer Controller) watches Ingress objects and configures a real proxy, usually behind one cloud load

balancer. The newer **Gateway API** (GA since 2023) splits this into Gateway and HTTPRoute objects with richer routing, and is the direction new clusters are moving.

7 ConfigMaps, Secrets and namespaces

ConfigMaps hold non-sensitive configuration (up to 1 MiB each); **Secrets** hold passwords, keys and certificates. Both are mounted as environment variables or files; files update in place when the object changes, environment variables only on restart. Secrets are only **base64-encoded**, not encrypted, by default: enable encryption at rest for etcd, restrict them with RBAC, or sync them from a real vault (External Secrets Operator, Secrets Store CSI driver). **Namespaces** group objects per team or environment, and carry their own RBAC rules, ResourceQuotas and network policies.

```
apiVersion: apps/v1                    apiVersion: v1
kind: Deployment                       kind: Service
metadata: {name: shop-api}             metadata: {name: shop-api}
spec:                                  spec:
  replicas: 3                            selector: {app: shop-api}
  selector:                              ports: [{port: 80, targetPort: 8000}]
    matchLabels: {app: shop-api}       ---
  template:                            apiVersion: networking.k8s.io/v1
    metadata:                          kind: Ingress
      labels: {app: shop-api}          metadata: {name: shop-api}
    spec:                              spec:
      containers:                        tls: [{hosts: [api.shop.com],
      - name: api                               secretName: shop-tls}]
        image: shop-api:3f9c2e1          rules:
        ports: [{containerPort: 8000}]   - host: api.shop.com
        readinessProbe:                    http:
          httpGet: {path: /health,           paths:
                    port: 8000}              - path: /
        envFrom:                               pathType: Prefix
        - configMapRef: {name: shop-cfg}       backend: {service:
                                                 {name: shop-api, port: {number: 80}}}
```

Figure 4. The worked example as YAML: Deployment (left), Service and Ingress (right). The labels and the port numbers must line up.

WORKED EXAMPLE: DEPLOYMENT + SERVICE + INGRESS FOR FASTAPI

The Deployment runs 3 replicas of `shop-api:3f9c2e1`, each with a readiness probe on `/health` and config from the `shop-cfg` ConfigMap. The ClusterIP Service `shop-api` selects `app=shop-api` and maps port 80 to container port 8000. The Ingress routes `api.shop.com` to that Service with the `shop-tls` certificate. Changing the image to `shop-api:8b1d0aa` starts a rolling update: one new pod is added, becomes ready, enters the Service's endpoints, one old pod is removed, and so on, with zero downtime. If the new pods never become ready, the rollout stalls with the old pods still serving.

COMMON MISTAKES

- Creating bare pods, which nobody recreates when a node dies.
- No readiness probe, so traffic hits pods that are still starting.
- A Service selector that doesn't match the pod labels: the Service has no endpoints.
- Believing Secrets are encrypted because they look scrambled (base64).

8 Key terms

Pod: co-scheduled containers sharing an IP; disposable

Deployment: desired replicas + rolling updates

ReplicaSet: keeps N matching pods running

Service: stable virtual IP and DNS name for pods

Ingress: HTTP host/path routing, implemented by a controller

Labels / selectors: how objects find each other

9 Interview questions

Q1. Explain the path of a request from Ingress to a pod. BASIC

The domain resolves to the ingress controller's load balancer; the controller matches host and path rules and forwards to the Service; the Service's virtual IP load-balances to a ready pod matching its selector, whose container handles it.

Q2. Why not create pods directly? BASIC

Pods are disposable; Deployments keep the desired count running through a ReplicaSet and handle rolling updates and rollbacks.

Q3. What's the difference between readiness and liveness probes? BASIC

Readiness controls whether the pod gets traffic; liveness restarts the container when it's stuck.

Q4. A Service returns connection errors though the pods are running. What do you check? SCENARIO

Whether its selector matches the pods' labels and the pods are ready (kubectl get endpointslices), and whether targetPort matches the container port.

Q5. A new release crashes on start. What does the Deployment do? SCENARIO

New pods never become ready, so with maxUnavailable limits the rollout stalls while old pods keep serving; you roll back with kubectl rollout undo.

Q6. Does Ingress traffic always pass through the Service's ClusterIP? TRICKY

No: many ingress controllers read the EndpointSlices and proxy directly to pod IPs; the Service is used for discovery.

Q7. Are Kubernetes Secrets secure by default? TRICKY

They're only base64-encoded in etcd; you need encryption at rest, tight RBAC, or an external secret store.

Next: SD-V-007 · Kubernetes Scheduling backendroadmap.com

Kubernetes Scheduling

requests, limits, placement and disruption budgets

Watch the 4-min video

In one line: the scheduler places pods by their **requests**, the kernel enforces **limits** (CPU throttles, memory OOM-kills), QoS classes decide who is evicted first, and **affinity, taints, topology spread** and **disruption budgets** keep replicas where they should be and alive through maintenance.

1 Three symptoms, one cause

Your cluster looks half empty, yet a new pod has been **Pending** for an hour. Another service keeps restarting with **OOMKilled**. And during a routine node upgrade, all three replicas of your API went down together. All three come from how Kubernetes schedules pods and manages resources, and all three are fixed in the pod spec.

2 How the scheduler picks a node

For each unscheduled pod, kube-scheduler runs two phases. **Filtering** removes nodes that can't run it: not enough unreserved CPU or memory, a taint the pod doesn't tolerate, a required node affinity that doesn't match, a volume in another zone, a spread constraint that would be violated. **Scoring** ranks the rest (spreading, preferred affinities, image already present, balanced utilisation), and the pod is bound to the best one. If no node passes filtering, the pod stays Pending and the event message lists the reasons per node. If a higher-priority pod can't fit, the scheduler can **preempt** lower-priority pods (PriorityClass), and the cluster autoscaler can add nodes (lesson 58).

3 Requests and limits

Each container may declare, for CPU and memory, a **request** and a **limit**. CPU is in cores or millicores (`500m` = half a core), memory in bytes (`512Mi`). **Requests are reservations**: the scheduler only places a pod where the sum of requests fits within the node's **allocatable** capacity (node capacity minus what's reserved for the OS and kubelet). Actual usage is irrelevant to placement. **Limits are caps** enforced at runtime by cgroups (lesson 55).

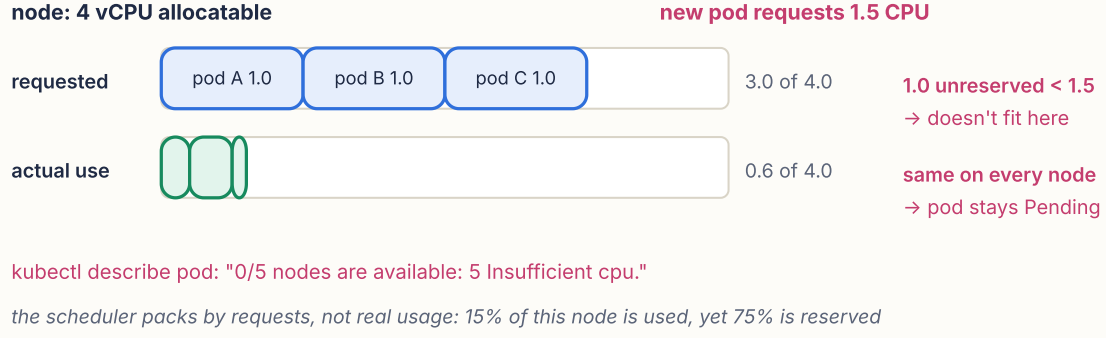


Figure 1. Why a pod is Pending on a quiet cluster: over-sized requests reserve capacity that nobody uses.

So set requests from measured usage (for example the p90–p95 of real consumption plus headroom), not guesses. Requests that are too high waste money and cause Pending pods; too low, and nodes are over-

committed, leading to throttling, evictions and noisy neighbours.

4 Over the limit: throttling vs OOMKilled

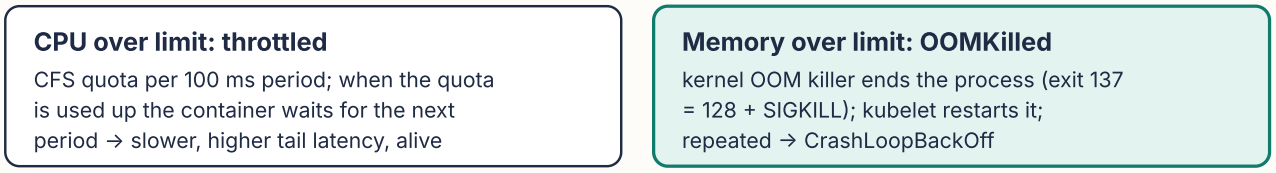


Figure 2. CPU is compressible, memory is not.

CPU is compressible. A CPU limit becomes a CFS quota: with a 1-CPU limit the container may use 100 ms of CPU time per 100 ms period, across all its threads. A multi-threaded app can burn its quota in 25 ms and then sit idle for 75 ms, which shows up as latency spikes rather than errors. Many teams therefore set CPU **requests** but no CPU limit for latency-sensitive services, letting them use idle CPU while requests still guarantee their share.

Memory is not compressible. A container that exceeds its memory limit is killed by the kernel's OOM killer, shown as `OOMKilled` with exit code 137, and restarted. Common causes: a JVM or Node heap sized larger than the container limit, unbounded caches, or a memory leak. The usual advice is **memory limit = memory request**, so what you reserve is what you can use.

5 QoS classes and eviction

QoS class	Rule	Under node memory pressure
Guaranteed	every container has CPU and memory requests = limits	evicted last; lowest OOM score
Burstable	at least one request or limit set, not Guaranteed	evicted when using more than requested
BestEffort	no requests or limits at all	evicted first

When a node runs short of memory or disk, the kubelet **evicts** pods to protect the node. It ranks candidates by whether their usage exceeds their requests, then by pod priority, then by how far over their requests they are, which in practice follows the QoS order above. Critical services should be Guaranteed, or at least Burstable with realistic memory requests; BestEffort is for work you don't mind losing.

6 Steering placement

- **Node affinity**: require or prefer nodes with labels, e.g. `requiredDuringSchedulingIgnoredDuringExecution` on `accelerator=nvidia`. **Pod anti-affinity** keeps replicas off the same node.
- **Taints and tolerations**: a taint on a node repels pods unless they tolerate it, the reverse of affinity. Effects: `NoSchedule`, `PreferNoSchedule`, and `NoExecute` (also evicts running pods). Taint GPU nodes so only GPU workloads land there; give those workloads both a toleration and an affinity, since a toleration only allows, it doesn't attract.
- **Topology spread constraints**: spread replicas evenly over a topology key such as `topology.kubernetes.io/zone` or `kubernetes.io/hostname`. `maxSkew: 1` means no zone may have

more than one pod more than the emptiest zone; `whenUnsatisfiable` chooses between refusing to schedule (`DoNotSchedule`) and best effort (`ScheduleAnyway`).

7 Pod disruption budgets

Voluntary disruptions are ones the cluster chooses: node drains for upgrades, autoscaler scale-down. A **PodDisruptionBudget** limits them: with `minAvailable: 2` for a 3-replica service, a drain evicts one pod, waits until its replacement is ready elsewhere, and only then evicts the next. The eviction API refuses requests that would break the budget, so the upgrade slows down instead of causing an outage. PDBs don't protect against **involuntary** disruptions such as a node crash or a zone outage; that's what spreading is for. Avoid budgets that can never be satisfied (`minAvailable` equal to replicas), which block node drains forever.

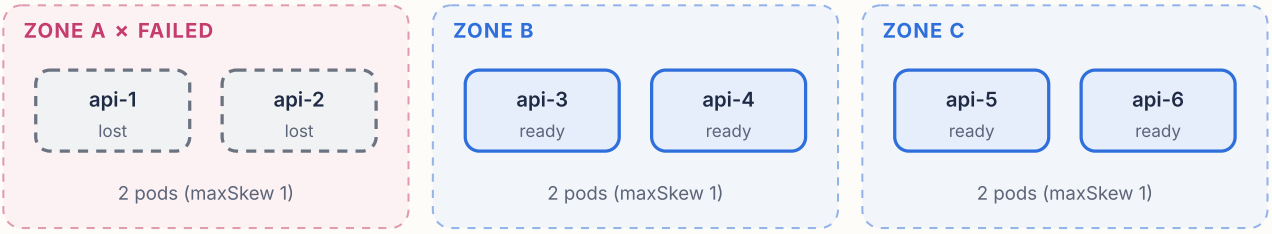


Figure 3. Spread plus budget: spreading survives involuntary failures; the PDB protects against voluntary ones.

```
spec:
  replicas: 6
  template:
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: DoNotSchedule      # or ScheduleAnyway
        labelSelector: {matchLabels: {app: api}}
      containers:
      - name: api
        resources:
          requests: {cpu: 500m, memory: 512Mi}
          limits:   {memory: 512Mi}           # memory limit = request
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: {name: api}
spec: {minAvailable: 4, selector: {matchLabels: {app: api}}}
```

Figure 4. The worked example's spec: zone spread, realistic requests, memory limit equal to request, and a budget of 4.

WORKED EXAMPLE: SIX REPLICAS ACROSS THREE ZONES

The API Deployment has 6 replicas with a zone spread of `maxSkew: 1`, so the scheduler places 2 pods in each of zones a, b and c. A PodDisruptionBudget keeps at least 4 available, so node upgrades drain at most 2 pods at a time. When zone a fails, its 2 pods are lost and 4 keep serving, sized to carry peak load alone. Watch the constraint during the outage: with `DoNotSchedule`, replacements may be blocked from piling into zones b and c while zone a still counts as a domain, so teams either accept running on 4 until the zone returns, or use `ScheduleAnyway` to favour recovery over perfect balance.

COMMON MISTAKES

- Copy-pasted requests far above real usage: Pending pods and wasted nodes.
- No memory requests (BestEffort) on important services.
- JVM or runtime heap larger than the container memory limit.
- No PDB, or a PDB with minAvailable equal to replicas that blocks every drain.

8 Key terms

Request: reserved CPU/memory used for scheduling

Limit: runtime cap: CPU throttles, memory OOM-kills

QoS class: Guaranteed, Burstable, BestEffort

Taint / toleration: node repels pods unless they tolerate it

Topology spread: even distribution across zones or nodes

PodDisruptionBudget: limit on voluntary evictions

9 Interview questions

Q1. What happens when a pod exceeds its memory limit? BASIC

The container is OOMKilled (exit 137) and restarted; exceeding a CPU limit only throttles it.

Q2. How do you keep replicas available? BASIC

Realistic requests, topology spread across zones (and nodes), and pod disruption budgets during maintenance.

Q3. Requests vs limits? BASIC

Requests are what the scheduler reserves and guarantees; limits are the maximum a container may use, enforced by cgroups.

Q4. A pod is Pending though nodes show 20% CPU usage. Why? SCENARIO

Scheduling uses requests, not usage: the nodes' allocatable capacity is already reserved by other pods' requests. Right-size requests or add nodes.

Q5. A latency-sensitive service shows p99 spikes but low average CPU. What do you suspect? SCENARIO

CFS throttling from a CPU limit: bursts use up the quota early in each 100 ms period. Check throttling metrics; raise or remove the CPU limit and keep requests.

Q6. Does a PodDisruptionBudget protect against a zone outage? TRICKY

No, PDBs only govern voluntary evictions through the eviction API; zone failures are involuntary, so you need spreading and spare capacity.

Q7. You tainted GPU nodes and added tolerations to GPU pods. Why do GPU pods still land on CPU nodes? TRICKY

A toleration only permits scheduling on tainted nodes; add node affinity or a nodeSelector to require GPU nodes.

Next: SD-V-008 · Kubernetes Autoscaling backendroadmap.com

Kubernetes Autoscaling

pods follow the metric, nodes follow the pods

Watch the 4-min video

In one line: pod autoscalers (HPA, VPA, KEDA) decide how many pods you need and how big they are; node autoscalers (Cluster Autoscaler, Karpenter) add machines when pods are **Pending** and remove them when they sit idle. Scale on the signal that really measures the backlog, and remember that nodes, especially GPU nodes, take minutes to arrive.

1 Peaks and valleys

An AI inference service gets **50 requests a minute** overnight and **5,000** at 9 a.m. Sizing GPU pods for the peak all day wastes most of the money; sizing for the night means a queue that takes an hour to clear. Kubernetes solves this at **two levels**: the pod level (how many replicas, how big) and the node level (how many machines to run them on). The two are separate controllers that only meet through one thing: pods that can't be scheduled.

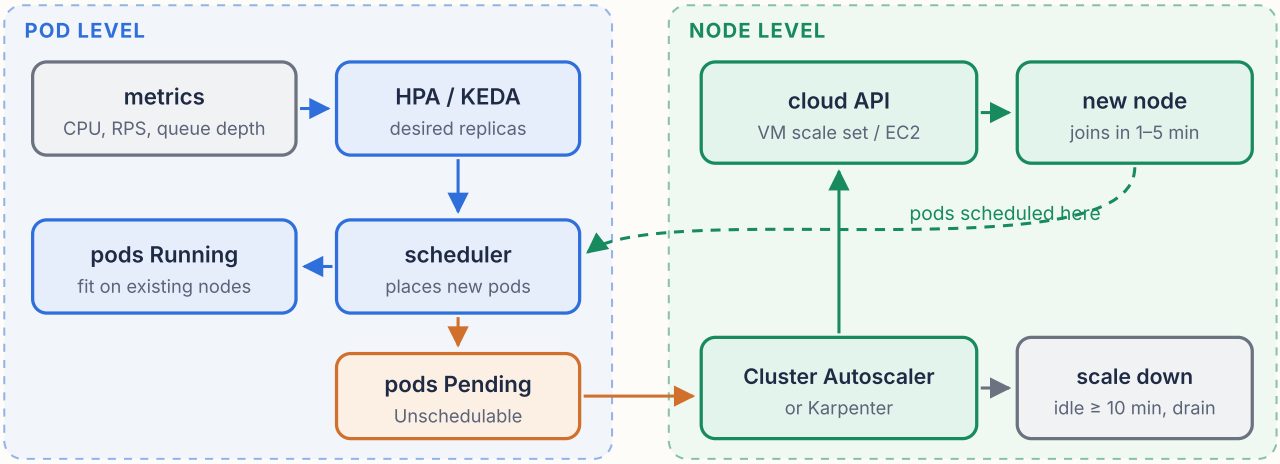


Figure 1. The two-level loop. Pod autoscalers create pods; pods that don't fit stay **Pending**, which is the only signal the node autoscaler reacts to.

2 Horizontal Pod Autoscaler: how many pods

The HPA is a control loop (every **15 s** by default) that reads a metric and sets `replicas` on a Deployment or StatefulSet. Its core formula is:

```
desired = ceil( current replicas × current metric ÷ target metric )
```

With 4 pods at 90% CPU and a 60% target: ceil(4 × 90 ÷ 60) = **6 pods**. Important details:

- **CPU utilisation is a percentage of the pod's request**, not of the node. Pods without CPU requests can't be scaled on CPU at all (lesson 57).
- A **tolerance** of 10% stops it reacting to tiny changes; `minReplicas` and `maxReplicas` bound the result.
- Scale-up is fast; scale-down waits for a **stabilisation window** (300 s by default) and uses the highest recommendation in that window, so a brief dip doesn't remove pods you need a minute later. The `behavior` field tunes both directions (for example, at most 50% fewer pods per minute).

- The `autoscaling/v2` API accepts **resource** metrics (CPU, memory, from metrics-server), **custom** per-pod metrics (requests per second via Prometheus Adapter) and **external** metrics (a queue length in a cloud service). With several metrics, it takes the largest result.

CPU is a good signal for CPU-bound request handlers. It is a poor one for queue workers (an idle worker and a blocked worker both show low CPU) and for GPU work, where the CPU may be nearly idle while the GPU is saturated.

3 Vertical Pod Autoscaler: how big is a pod

The VPA watches actual usage over days and recommends CPU and memory **requests**. It has three parts: a **recommender** (computes targets from usage history), an **updater** (evicts pods whose requests are far off) and an **admission webhook** (rewrites requests when pods are recreated). `updateMode: Off` only publishes recommendations, which is the safe way to start; `Initial` applies them at pod creation; `Recreate` / `Auto` evict running pods to apply them. Newer Kubernetes versions can resize pods in place, which removes the restart for many changes.

Use VPA for right-sizing, and for things that can't scale out (a singleton, a stateful component). **Don't** let HPA and VPA both act on CPU or memory for the same workload: VPA raises requests, so utilisation % drops, so HPA removes pods, so usage per pod rises, and they chase each other. Combining is fine when HPA scales on a different metric (RPS, queue depth).



Figure 2. Four autoscalers, four questions. Most production clusters run HPA or KEDA plus a node autoscaler; VPA mostly in recommendation mode.

4 Pods need nodes: Cluster Autoscaler and Karpenter

The **Cluster Autoscaler** (CA) checks about every 10 s for pods marked *Unschedulable*. It simulates the scheduler against each node pool's template and grows the pool whose nodes would fit them (an *expander* such as least-waste, priority or price picks between pools). For scale-down it looks for nodes whose pods' **requests** add up to under 50% of capacity for 10 minutes (both defaults), checks that those pods can move elsewhere, then drains the node respecting PodDisruptionBudgets. Pods with local storage, no controller, or the `safe-to-evict: "false"` annotation block removal.

Two consequences surprise people. First, the CA works on **requests, not real usage**: a cluster at 15% real CPU but 95% requested is "full". Second, it can only add nodes of the shapes you pre-defined in node pools.

Karpenter (created by AWS, now also behind AKS node auto-provisioning) skips node groups. It reads the pending pods' requests, selectors and tolerations, chooses the cheapest instance types that fit them from a broad allowed list (including spot), and launches those machines directly. It also **consolidates**: it replaces several half-empty nodes with fewer or cheaper ones.

	Cluster Autoscaler	Karpenter
Unit it scales	pre-defined node pools / groups	individual instances chosen per batch of pods
Instance choice	fixed per pool	any type allowed by a NodePool's requirements
Scale-down	utilisation < threshold for N minutes	empty-node removal + consolidation onto cheaper nodes
Where	every major cloud	AWS; Azure (node auto-provisioning); others via providers

5 KEDA: scale on the backlog, and to zero

For queue consumers the right question is "how much work is waiting?". **KEDA** (Kubernetes Event-Driven Autoscaling) has scalers for dozens of sources: Kafka consumer lag, RabbitMQ, SQS, Azure Service Bus, Prometheus queries, cron schedules. A `ScaledObject` names the target and a per-replica target such as **20 messages per worker**. KEDA itself handles **0 ↔ 1** (activation and scale to zero after a cooldown) and creates an HPA behind the scenes for **1 ↔ N**, fed through the external metrics API. For long jobs that must not be killed mid-way, a `ScaledJob` starts one Kubernetes Job per message instead of scaling a Deployment.

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata: { name: inference-workers }
spec:
  scaleTargetRef: { name: inference-worker }   # the Deployment
  minReplicaCount: 2          # warm minimum (0 = scale to zero)
  maxReplicaCount: 120        # cost ceiling
  pollingInterval: 15         # seconds between queue checks
  cooldownPeriod: 600         # only used when dropping to zero
  triggers:
  - type: azure-servicebus
    metadata: { queueName: inference, messageCount: "20" }
```

Figure 3. A KEDA ScaledObject for the worked example. The `messageCount` target becomes the HPA's per-pod average.

6 GPU nodes need extra care

A new GPU worker often needs a new node, and every step of that chain is slow. Plan for the whole chain, not just the HPA reaction:

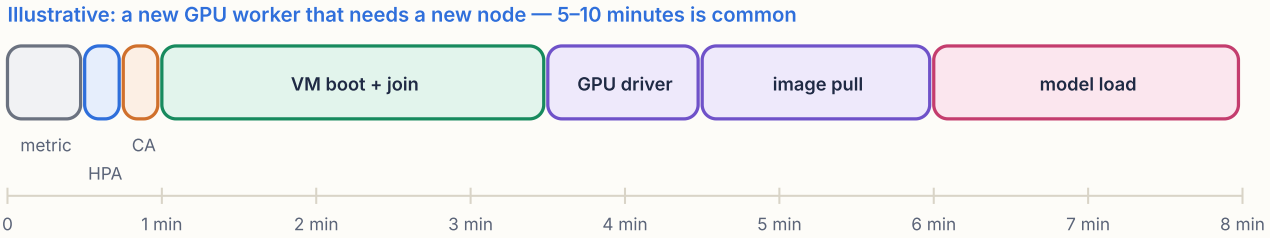


Figure 4. Why GPU scale-up takes minutes. Pod autoscaling reacts in under a minute; the node, drivers, multi-GB image and model weights dominate.

- **Warm minimum:** keep `minReplicaCount` > 0 for latency-sensitive paths. A cheaper trick is **overprovisioning**: low-priority placeholder pods reserve a spare node and are evicted the moment a real pod needs the room, so the next real pod starts in seconds.

- **Separate GPU pools with taints** (e.g. `nvidia.com/gpu:NoSchedule`) so ordinary pods never land on a $3/hour node and keep it from scaling down. GPU pods request `nvidia.com/gpu: 1` and tolerate the taint.
- **Scale on queue depth or in-flight requests**, not CPU. GPU utilisation can be used via DCGM exporter metrics, but backlog maps more directly to user wait.
- **Slow scale-down:** longer cooldowns and node-removal delays avoid paying a 6-minute start for a node you deleted 5 minutes earlier.
- **Faster starts:** pre-pull images into the node image, cache model weights on local NVMe or a shared volume, use smaller images.

WORKED EXAMPLE: INFERENCE WORKERS ON QUEUE DEPTH

Requests go to a queue. KEDA targets **20 messages per worker**, min 2, max 120. Overnight the queue holds about 30 messages, so ceil(30 ÷ 20) = 2 workers, the floor. At 9 a.m. the queue reaches **2,000**: ceil(2,000 ÷ 20) = **100** workers. The HPA's default scale-up policy lets replicas at most double (or +4) every 15 s, so 2 → 4 → 8 → … reaches 100 within a couple of minutes. Most new pods are Pending: if each GPU node runs 4 workers (one GPU each), about 24 more nodes are needed, and the CA or Karpenter requests them. The first new nodes are serving after 5–10 minutes; the 2 warm workers and the overprovisioned spare node absorb the start. In the evening the queue drains; after the scale-down stabilisation window the workers return to 2, nodes become empty, and after the scale-down delay the node autoscaler deletes them.

COMMON MISTAKES

- No resource requests: HPA can't compute CPU %, and the CA can't tell what fits.
- HPA and VPA both on CPU for the same Deployment, fighting each other.
- Scaling queue workers on CPU, so a growing backlog of slow I/O work never triggers scale-out.
- No `maxReplicas` or node pool maximum: a bug or traffic spike becomes an unbounded bill.

TRADE-OFFS TO STATE

- **Scale to zero** saves the most money but adds a cold start of minutes for GPU work.
- **Aggressive scale-down** saves cost; **slow scale-down** saves latency and churn.
- **Headroom** (warm pods, placeholder pods) costs a little all day to avoid a slow first minute at every spike.
- Predictable peaks (9 a.m. daily) can also be met with a **cron** trigger that scales up at 8:45.

7 Key terms

HPA: changes replica count to keep a metric near its target

Cluster Autoscaler: adds nodes for Pending pods; removes underused nodes

KEDA: event-driven scaling on queues, lag, cron; supports zero

VPA: changes pods' CPU/memory requests from observed usage

Karpenter: provisions right-sized instances directly for pending pods

Stabilisation window: delay that smooths scale-down decisions

8 Interview questions

Q1. How do pod and node autoscaling interact? BASIC

HPA or KEDA change the replica count; pods that can't fit stay Pending, which triggers the Cluster Autoscaler or Karpenter to add nodes; nodes that stay underused are drained and removed later.

Q2. What should queue workers scale on? BASIC

Queue depth or consumer lag, using KEDA (or an external-metric HPA), not CPU.

Q3. How does the HPA compute the replica count? BASIC

desired = ceil(current replicas × current metric ÷ target), within min/max, ignoring changes within a 10% tolerance, with a stabilisation window on scale-down.

Q4. Your cluster shows 20% real CPU usage but the Cluster Autoscaler keeps adding nodes. Why? SCENARIO

It schedules on requests, not usage: pods request far more than they use. Right-size requests (VPA recommendations) and the nodes will pack tighter.

Q5. Users wait 8 minutes when traffic jumps at 9 a.m. on your GPU service. What do you change? SCENARIO

Keep a warm minimum or placeholder pods for one spare node, add a cron pre-scale before 9, pre-pull images and cache model weights to cut start time, and scale on queue depth so scale-out starts early.

Q6. Why not run HPA and VPA together on CPU? TRICKY

VPA changes requests, which changes the utilisation percentage HPA sees, so the two controllers keep reversing each other; use VPA for sizing and HPA on a different metric.

Q7. Why doesn't the Cluster Autoscaler remove an obviously idle node? TRICKY

Something on it blocks eviction: a PodDisruptionBudget with no room, a pod with local storage or no controller, a safe-to-evict=false annotation, or a system pod that can't move.

Q8. How does KEDA scale to zero if the HPA can't go below one? TRICKY

KEDA handles activation itself: it watches the source, scales the Deployment between 0 and 1 directly, and lets its managed HPA handle 1 to N.

Next: SD-V-009 · Stateful Workloads on Kubernetes backendroadmap.com

Stateful Workloads on Kubernetes

stable names, sticky disks, and software that knows how to run a database

Watch the 4-min video

In one line: a StatefulSet gives each pod a stable name, ordered startup and its own persistent volume; an **operator** adds the database know-how (replication, failover, backups, upgrades). Prefer a managed database unless you need portability, a missing capability, or you have the expertise.

1 Stateless is easy, state is not

Kubernetes treats pods as replaceable: kill one, start another anywhere, nothing is lost. A database breaks every part of that. Each replica holds **its own data**, replicas have **roles** (one primary, several followers), peers must find **specific** members by name, and losing a disk can mean losing data. Running state on Kubernetes needs different building blocks, and an honest decision about whether to do it at all.

2 StatefulSet: identity that survives restarts

	Deployment	StatefulSet
Pod names	random suffix (`api-7d9f-x2k`)	ordinal: `kafka-0`, `kafka-1`, `kafka-2`
Storage	usually none, or one shared volume	`volumeClaimTemplates`: one PVC per pod, re-attached to the same ordinal
Network identity	one Service IP for all pods	headless Service gives each pod its own DNS name
Start / update order	any order, in parallel	0→N on create; updates N→0, one at a time, waiting for Ready
On scale-down / delete	pods and anything they wrote are gone	PVCs are **kept** by default, so data survives a mistake

A **headless Service** (`clusterIP: None`) doesn't load-balance; DNS returns the pods themselves, and each pod gets a record like `kafka-1.kafka-hl.shop.svc.cluster.local`. Clients can bootstrap from any broker, and peers can say "replicate from kafka-0". `podManagementPolicy: Parallel` drops the ordering for systems that don't need it, and an update `partition` lets you upgrade only the highest ordinals first, as a canary.

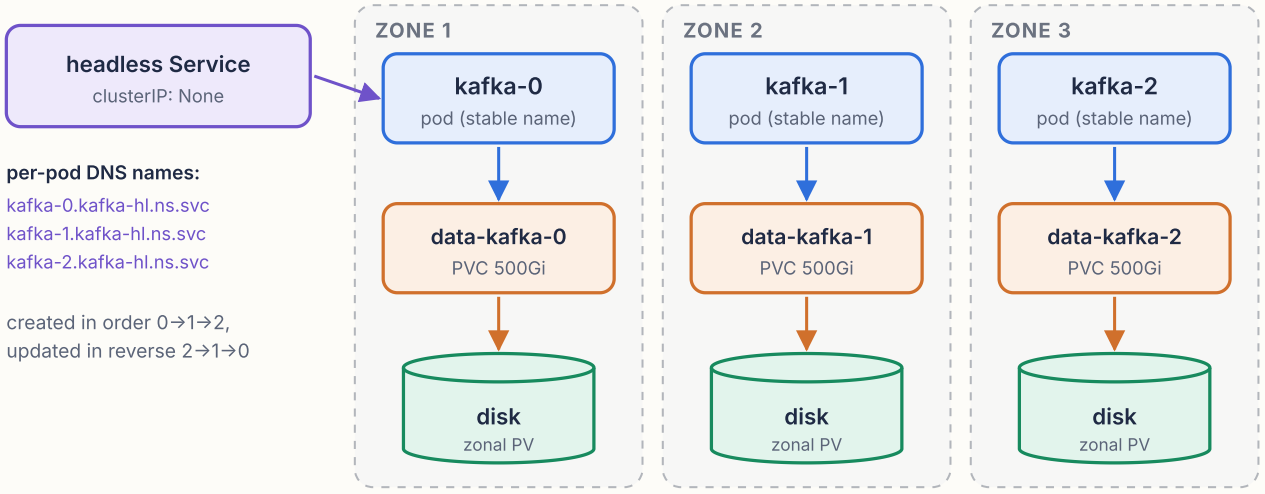


Figure 1. A three-broker StatefulSet spread over zones. Each pod keeps its name and its PVC; if `kafka-1` is rescheduled, it comes back as `kafka-1` with the same disk.

3 Persistent volumes and storage classes

A pod asks for storage with a **PersistentVolumeClaim** ("500 Gi, ReadWriteOnce, class premium-ssd"). A **StorageClass** names a CSI driver (Azure Disk, AWS EBS, GCE PD, Ceph…) and parameters; it **dynamically provisions** a cloud disk and binds it as a **PersistentVolume**. Settings that matter:

- `volumeBindingMode: WaitForFirstConsumer`: create the disk only after the pod is scheduled, in that pod's zone. Without it a disk can be created in zone 1 for a pod placed in zone 2.
- **Zonal disks:** most block disks live in one zone. If the pod moves, the disk is detached and re-attached on the new node, but only within that zone. A full zone outage leaves the pod stuck until the zone returns, so the database's own replication across zones is what protects you.
- `reclaimPolicy`: `Delete` (the default for dynamic volumes) removes the disk when the PV is released; `Retain` keeps it. Use Retain for anything precious.
- `allowVolumeExpansion: true` lets you grow a PVC in place. Access modes: RWO (one node), RWX (many nodes, file storage), RWOP (exactly one pod).

4 Operators: the know-how in software

Starting pods is the easy 10%. Someone must configure replication, **elect a new primary** on failure, take and test **backups**, perform **version upgrades** safely, rotate certificates and rebalance data. An **operator** encodes that runbook in a controller: you declare a **custom resource** (`kind: Kafka`, `kind: Cluster` for Postgres), and the controller runs the same reconcile loop as built-in controllers: observe, compare with the spec, act, repeat.

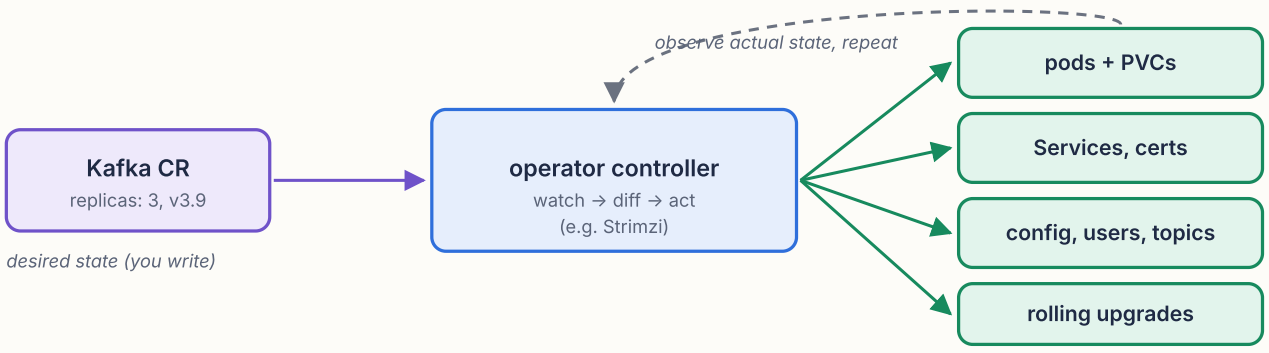


Figure 2. The operator pattern: a custom resource is the desired state; the controller creates and keeps fixing everything underneath it.

Mature examples: **Strimzi** (Kafka), **CloudNativePG**, Crunchy PGO and Zalando's operator (PostgreSQL), **Vitess** (sharded MySQL), and vendor operators for MongoDB, Redis and Elasticsearch. Quality varies widely; the Operator Framework's capability levels run from 1 (basic install) to 5 ("auto pilot": auto-scaling, auto-tuning). Check that failover, backup/restore and upgrades are actually automated before you trust one.

5 Should you run databases on Kubernetes?

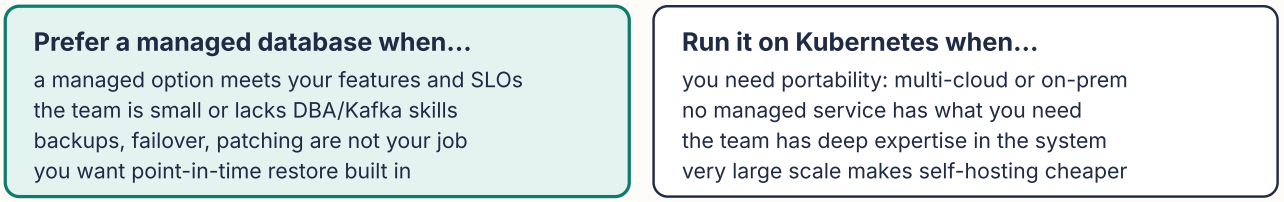


Figure 3. The decision is mostly about people and capabilities, not technology.

A managed database (RDS, Azure Database for PostgreSQL, Cloud SQL, Event Hubs, MSK...) costs more per CPU, but the provider runs backups with point-in-time restore, failover, patching and storage growth, and it does them every day for thousands of customers. Your team's time goes to the product. The costs of self-running are mostly invisible until a bad night: a failed upgrade, a full disk, a restore nobody has practised. Compare the total cost of ownership, including people (lesson 65).

6 If you do run state on Kubernetes

- **Use a mature operator** rather than hand-written StatefulSets.
- **Spread replicas across zones** with topology spread constraints or anti-affinity, and make the system itself replica-aware (Kafka `rack`, Postgres synchronous standby in another zone).
- **PodDisruptionBudgets** (e.g. `maxUnavailable: 1`) so node drains and upgrades (lesson 63) never take down two replicas at once.
- **Fast storage classes** with Retain, monitoring of disk usage and IOPS, and alerts well before full.
- **Backups outside the cluster** (object storage), and **regular restore tests**. A backup you've never restored is a hope, not a backup.
- Generous `terminationGracePeriodSeconds` for clean shutdown, a written **recovery runbook**, and dedicated node pools if noisy neighbours hurt latency.

WORKED EXAMPLE: KAFKA WITH THE STRIMZI OPERATOR

The team installs Strimzi and applies a `Kafka` resource (below): three brokers, 500 Gi each, rack-aware across zones, replication factor 3 with `min.insync.replicas: 2`. Strimzi creates the broker pods with stable names, one PVC each, the bootstrap and per-broker Services, a cluster CA and TLS certificates, and the topic and user operators. (Recent Strimzi versions manage pods through their own `StrimziPodSet` resource instead of StatefulSets, with the same guarantees: stable identity, one volume per broker.) To upgrade, they change `version`. The operator rolls brokers **one at a time**, and before restarting each one it checks that no partition would drop below its min in-sync replicas, so producers with `acks=all` keep writing throughout.

```
apiVersion: kafka.strimzi.io/v1beta2
kind: Kafka
metadata: { name: events }
spec:
  kafka:
    version: 3.9.0           # change this to upgrade
    replicas: 3
    rack: { topologyKey: topology.kubernetes.io/zone }
    storage: { type: persistent-claim, size: 500Gi, class: premium-ssd }
    config: { default.replication.factor: 3, min.insync.replicas: 2 }
```

Figure 4. The whole cluster as one custom resource. The operator turns it into dozens of Kubernetes objects.

COMMON MISTAKES

- Running a database as a Deployment: pods share or lose volumes and get random names.
- Immediate volume binding, so disks and pods end up in different zones.
- No PodDisruptionBudget: a node pool upgrade drains two of three replicas together.
- Backups stored in the same cluster or never restored in a test.

7 Key terms

StatefulSet: workload with stable pod names, ordered rollout, per-pod PVCs

PVC / PV: a claim for storage / the actual provisioned volume

Operator: controller + custom resource that automates running a system

Headless Service: DNS returns individual pod addresses, no virtual IP

StorageClass: recipe for dynamically provisioning volumes via a CSI driver

PodDisruptionBudget: limit on voluntary evictions at one time

8 Interview questions

Q1. Should you run databases on Kubernetes? BASIC

Prefer managed databases when they meet your needs, since they handle backups, failover and patching; run state on Kubernetes for portability, a missing capability or strong expertise, using StatefulSets, persistent volumes and a mature operator.

Q2. What does an operator do? BASIC

It automates operational knowledge such as failover, upgrades and backups through a custom controller that reconciles a custom resource.

Q3. What does a StatefulSet give you that a Deployment doesn't? BASIC

Stable ordinal pod names and DNS records, a persistent volume per pod that follows it across restarts, and ordered, one-at-a-time rollout.

Q4. A zone goes down. What happens to kafka-1, whose disk was in that zone? SCENARIO

The pod can't start elsewhere because its zonal disk can't attach in another zone; it stays Pending, and the cluster keeps serving from the other two brokers' replicas until the zone returns.

Q5. You're upgrading the node pool under a 3-node Postgres cluster. How do you avoid an outage? SCENARIO

A PDB with maxUnavailable 1 so drains go one node at a time, an operator that switches over the primary before its node is drained, and replicas spread across zones.

Q6. You scaled a StatefulSet from 3 to 2 and back to 3. Is kafka-2's data gone? TRICKY

No: by default its PVC is retained on scale-down, so the new kafka-2 re-attaches the same volume (unless the retention policy is set to delete).

Q7. Why can a pod and its new disk end up in different zones? TRICKY

With Immediate binding the disk is provisioned before scheduling, possibly in another zone; WaitForFirstConsumer provisions it where the pod is placed.

Next: SD-V-010 · Kubernetes Networking backendroadmap.com

Kubernetes Networking and Ingress

every pod gets an IP; Services give it a name; policies decide who may talk

Watch the 4-min video

In one line: the CNI gives every pod a routable IP, a Service gives a changing set of pods one stable virtual IP and DNS name, kube-proxy or eBPF on each node translates that IP to a ready pod, an ingress controller or Gateway brings outside traffic in, and NetworkPolicies (default deny + explicit allows) stop the traffic you don't want.

1 Three traffic questions

Inside one cluster: the orders service calls payments **by name** (east-west traffic); a browser reaches the API through **one load balancer** (north-south traffic); and a compromised pod in the dev namespace tries to connect to the production database. Knowing each layer is also how you debug the classic "it can't connect": DNS? Service selector? Pod not Ready? A NetworkPolicy dropping packets?

2 Every pod gets an IP: the CNI

The Kubernetes network model has three rules: every pod gets its own cluster-wide IP; pods can reach all other pods **without NAT**; and node agents can reach all pods on their node. A **CNI plugin** implements them: it allocates an address when a pod starts, wires its virtual interface, and makes sure packets reach the right node. Each node usually owns a slice of the pod range (for example a /24, 256 addresses). Three common designs:

- **Overlay** (VXLAN, e.g. Flannel, Calico in overlay mode, Azure CNI Overlay): pod IPs live in a private range and are tunnelled between nodes. Saves VNet addresses; small encapsulation cost.
- **VPC-native** (AWS VPC CNI, classic Azure CNI, GKE VPC-native): pods get real VNet/VPC addresses, routable without tunnels. Simple and fast, but large clusters can exhaust subnets.
- **eBPF-based** (Cilium): programs the kernel directly for routing, load balancing and policy, and can replace kube-proxy entirely.

3 Services, DNS and kube-proxy

Pods come and go, so their IPs are useless as addresses. A **Service** selects pods by label and gets a stable **ClusterIP** (a virtual IP that no interface owns) and a DNS name `<service>.<namespace>.svc.cluster.local`. The control plane keeps **EndpointSlices** listing the Service's pod IPs, including only pods that pass their **readiness probe**. On every node, **kube-proxy** (iptables, IPVS or nftables mode) or Cilium's eBPF programs rewrite connections to the virtual IP into connections to one of those pods.

Service type	What it gives you	Typical use
ClusterIP	internal virtual IP + DNS name	service-to-service calls
Headless (`clusterIP: None`)	DNS returns pod IPs directly	StatefulSets, client-side balancing (lesson 59)
NodePort	a port (30000–32767) open on every node	building block for external load balancers
LoadBalancer	a cloud load balancer with an external or internal IP	exposing one service, or the ingress controller itself
ExternalName	a DNS CNAME to an outside name	pointing at a managed database by a cluster name

4 Service-to-service, step by step

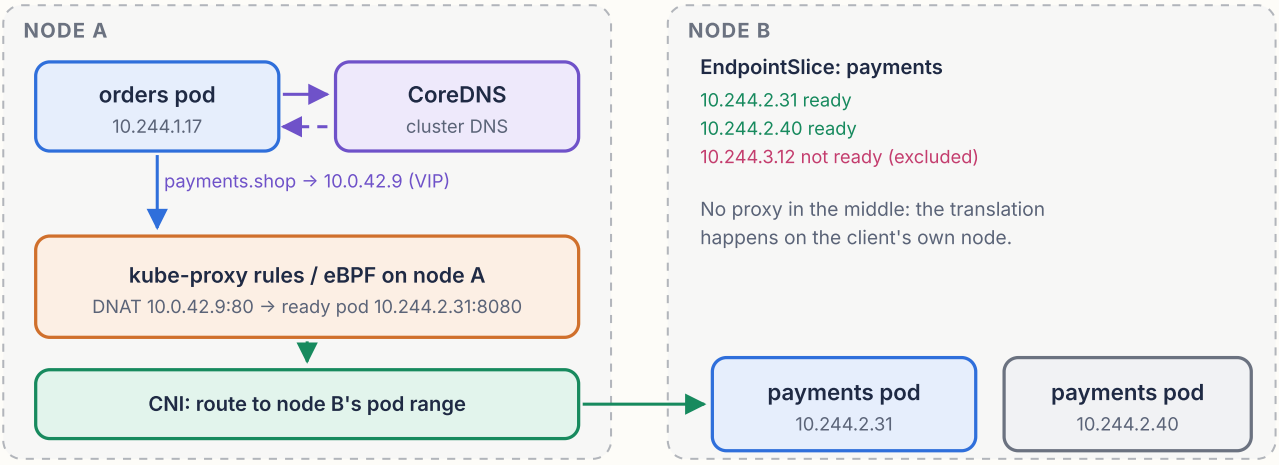


Figure 1. orders → payments. DNS gives a virtual IP; node A's rules pick a ready pod and rewrite the destination; the CNI carries the packet to node B.

1. orders resolves `payments`. The pod's `/etc/resolv.conf` search list expands it to `payments.shop.svc.cluster.local` (from another namespace you'd use `payments.shop`).
2. CoreDNS answers with the Service's ClusterIP, e.g. 10.0.42.9.
3. orders opens a TCP connection to 10.0.42.9:80. Before it leaves node A, the kernel rules DNAT it to a ready pod, 10.244.2.31:8080, chosen at random (iptables) or by a scheduler such as round robin (IPVS).
4. The CNI routes the packet to node B; the reply is un-NATed on the way back. There is no central proxy hop.

Two consequences worth knowing. Balancing is **per connection**, not per request: a gRPC or HTTP/2 client that keeps one long-lived connection sends **all** its requests to one pod, so use client-side balancing over a headless Service or a service mesh. And the default `ndots:5` setting makes a lookup of an external name like `api.stripe.com` try the cluster search domains first; add a trailing dot or tune ndots if DNS load becomes a problem.

5 Traffic from outside: ingress and Gateway API

Giving every service its own cloud load balancer is expensive and scatters TLS. Instead, one `LoadBalancer` Service fronts an **ingress controller**: a proxy (NGINX, Traefik, HAProxy, Envoy-based) running as pods. **Ingress** resources tell it to route by host and path to Services, and to terminate TLS.

Cloud controllers (AWS Load Balancer Controller, Azure Application Gateway for Containers) instead program the cloud's L7 load balancer directly.



Figure 2. North-south: one public load balancer, one ingress tier, then host and path rules to ordinary ClusterIP Services.

The Ingress API is frozen: it covers host/path routing and TLS, and everything else lives in controller-specific annotations. The **Gateway API** (GA since 2023) replaces it with role-oriented, portable objects and built-in features such as header matching, **weighted traffic splitting** for canaries, redirects, and TCP/gRPC routes. Most controllers now support it, and new clusters should start there. (The community ingress-nginx project announced its retirement in late 2025, with maintenance ending in March 2026, which pushed many teams to migrate.)

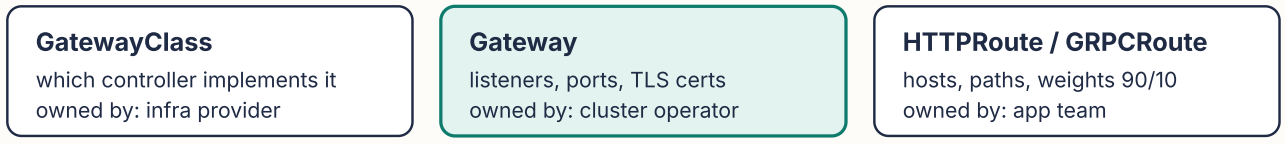


Figure 3. Gateway API splits responsibilities so app teams can change routes without touching shared load balancer settings.

6 NetworkPolicies: firewalls for pods

By default every pod can reach every pod, in every namespace: convenient and dangerous. A **NetworkPolicy** selects pods and lists allowed sources (ingress) or destinations (egress) by pod labels, namespace labels, IP blocks and ports. Rules to remember:

- Once **any** policy selects a pod for a direction, everything not explicitly allowed in that direction is dropped. Policies are **additive**; there is no "deny" rule.
- **Default deny** per namespace: a policy with `podSelector: {}` and `policyTypes: [Ingress, Egress]` and no rules. Then allow only needed flows. If you deny egress, remember to allow DNS (port 53 to kube-dns), or everything "breaks".
- Policies are **enforced by the CNI**. A plugin without policy support (basic Flannel) silently ignores them; Calico, Cilium and Azure's network policy engine enforce them.
- In a `from` list, `namespaceSelector` and `podSelector` in the **same item** mean AND; as **separate items** (each with its own dash) they mean OR. One misplaced dash opens the database to every pod in the cluster with the label.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: postgres-from-orders, namespace: data }
spec:
  podSelector: { matchLabels: { app: postgres } }
  policyTypes: [Ingress]
  ingress:
  - from:
    - namespaceSelector:                        # same list item:
        matchLabels: { kubernetes.io/metadata.name: shop }
      podSelector:                              # ns AND label
        matchLabels: { app: orders }
    ports: [ { protocol: TCP, port: 5432 } ]
```

Figure 4. The worked example's policy. Both selectors sit under one list item, so a source must be in namespace `shop` AND labelled `app=orders`.

WORKED EXAMPLE: LOCKING DOWN THE DATABASE

The `data` namespace first gets a default-deny ingress policy. Then the policy above allows TCP 5432 to `app=postgres` pods only from pods labelled `app=orders` in namespace `shop`. The namespace is matched by the label `kubernetes.io/metadata.name`, which Kubernetes sets automatically, so nobody can gain access just by adding a label to their own namespace. When the compromised pod in `dev` tries to connect, the CNI drops its SYN packets and the connection times out. orders keeps working. To check the setup, run a test pod in each namespace and try `nc -zv postgres.data 5432`: it should succeed only from orders.

COMMON MISTAKES

- Service selector labels that don't match the pods: the Service exists but has no endpoints.
- A pod failing readiness, so it silently receives no traffic.
- Default-deny egress without a DNS exception.
- Writing NetworkPolicies on a CNI that doesn't enforce them, and assuming you're protected.

7 Key terms

CNI: plugin that gives pods IPs and routes between nodes

ClusterIP: stable virtual IP for a Service

EndpointSlice: the list of ready pod IPs behind a Service

kube-proxy: per-node agent programming Service translation rules

Ingress / Gateway API: objects routing external HTTP(S) traffic to Services

NetworkPolicy: label-based allow-list firewall for pods

8 Interview questions

Q1. How does service-to-service traffic work in Kubernetes? BASIC

The pod resolves the Service's DNS name to its ClusterIP; kube-proxy or eBPF rules on the client's node rewrite the connection to one ready pod; the CNI routes pod-to-pod traffic across nodes.

Q2. How do you restrict it? BASIC

With NetworkPolicies, enforced by the CNI: start from default deny per namespace and allow only required flows by namespace, pod labels and ports.

Q3. What's the difference between Ingress and a LoadBalancer Service? BASIC
A LoadBalancer Service gives one Service an L4 cloud load balancer; an ingress controller is one shared L7 entry point that routes many hosts and paths to many Services and terminates TLS.

Q4. A Service returns connection refused, but the pods are running. What do you check? SCENARIO
Whether the Service has endpoints (selector labels match, pods Ready), whether targetPort matches the container port, and whether a NetworkPolicy blocks the source.

Q5. Your gRPC service has 10 pods but one gets almost all traffic. Why? SCENARIO
kube-proxy balances per connection and gRPC reuses one long-lived HTTP/2 connection; use client-side balancing with a headless Service or a mesh / L7 proxy that balances per request.

Q6. You added a NetworkPolicy allowing only port 5432 egress, and now the app can't reach the database by name. Why? TRICKY
Egress is now default-deny for those pods, including DNS; add an egress rule to kube-dns on port 53 UDP and TCP.

Q7. What's the difference between two selectors in one 'from' item and in two items? TRICKY
One item ANDs them (that namespace and that label); separate items OR them, which is usually far broader than intended.

Next: SD-V-011 · Infrastructure as Code backendroadmap.com

Infrastructure as Code

if it isn't in Git, it doesn't exist

Watch the 4-min video

> **In one line:** describe infrastructure declaratively in versioned files, review a **plan** before every **apply**, and guard the **state** that maps code to real resources (remote, encrypted, locked, split); reuse through versioned modules and catch **drift** by planning regularly.

1 Built by clicking

Production was built in the cloud portal two years ago. Now you need an identical copy in a new region. Nobody remembers every setting, half the decisions were never written down, and someone changed a firewall rule by hand last month. With **infrastructure as code** (IaC), networks, clusters, databases and permissions live in files that are reviewed, versioned and can recreate the same environment again and again: for a new region, for disaster recovery, or for a short-lived test environment.

2 Declarative tools

Declarative tools take the **desired end state** and work out the create/update/delete calls themselves, so running them twice changes nothing the second time (they are idempotent). Imperative scripts (a series of CLI calls) break on the second run or when something already exists.

Tool	Language	Where state lives	Notes
Terraform / OpenTofu	HCL	a state file you store	multi-cloud, huge provider ecosystem; OpenTofu is the open-source fork after the 2023 licence change
Bicep / ARM	Bicep DSL	Azure Resource Manager itself	Azure only; no state file; `what-if` previews changes
CloudFormation / CDK	YAML/JSON, or code via CDK	AWS stacks	AWS only; change sets preview; built-in drift detection
Pulumi	TypeScript, Python, Go, C#	Pulumi Cloud or your bucket	real programming languages; same plan/apply model (`preview` / `up`)

3 Plan, then apply

`terraform plan` refreshes the real resources, compares them with the code and the state, and prints exactly what will be **created (+)**, **changed in place (~)**, **destroyed (−)** or **replaced (−/+)**. `apply` executes it. Save the plan (`plan -out=tfplan`, then `apply tfplan`) so what runs is exactly what was reviewed, not a fresh plan computed minutes later against a changed world.

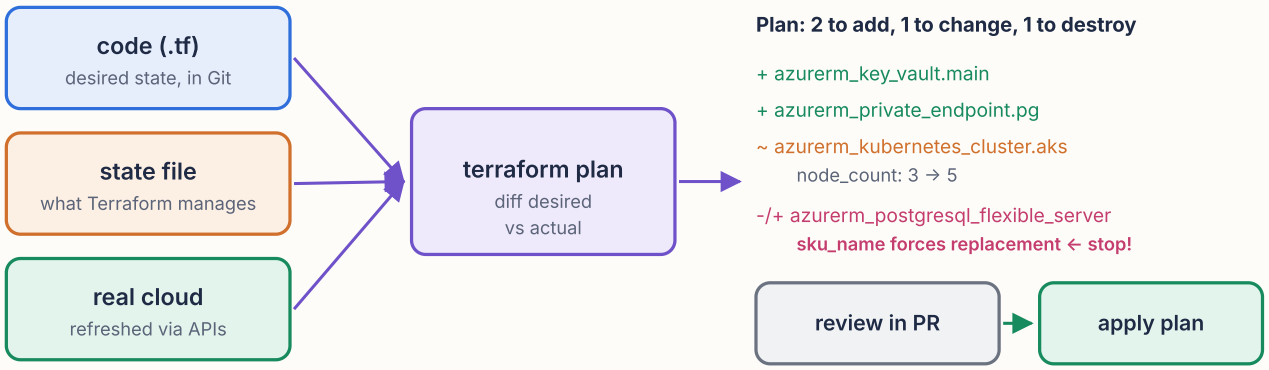
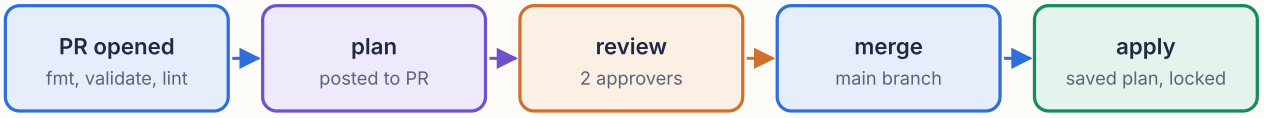


Figure 1. A plan is a three-way comparison. The line to catch in review is **-/+**: a changed attribute that forces the database to be destroyed and recreated.

Reviewing the plan in the pull request catches the expensive mistakes: a renamed resource that Terraform will delete and recreate (fix with a `moved` block), an attribute that forces replacement of a database, or a destroy you didn't expect. For critical resources add `lifecycle { prevent_destroy = true }`.



Policy checks (OPA / Sentinel / Checkov) and cost estimates (Infracost) can run at the plan step.

Figure 2. A typical IaC pipeline. CI holds the cloud credentials (ideally via workload identity, no stored secrets); humans never apply from laptops.

4 State: the map from code to reality

Terraform's **state** is a JSON document mapping every resource address in your code (`module.aks.azurerm_kubernetes_cluster.main`) to a real cloud ID and its last known attributes. It's how Terraform knows that it owns a resource, what to change and in which order to delete. Three things go wrong:

- **Lost state:** Terraform no longer knows it manages anything. The next apply tries to create everything again and collides with existing names; recovery means importing every resource by hand.
- **Concurrent applies:** two runs read the same state, both create or both modify, and the last writer's state wins, leaving duplicates or orphans.
- **Secrets inside:** attributes such as generated database passwords and access keys are stored in state in plain text. Whoever can read the state can read them.

5 Protecting state

- **Remote backend**: an Azure storage account or S3 bucket, with encryption at rest and **versioning** (soft delete) so you can roll back a corrupted state.
- **Locking**: only one apply at a time. The azurerm backend uses a blob lease; the S3 backend now supports a native lock file (older setups use a DynamoDB table).
- **Tight access**: only the pipeline's identity can write; few humans can read. Treat the bucket like a secrets store.
- **No hand edits**: use `terraform state mv`, `import` and `moved` blocks, which are reviewed and reversible.

- **Split states** to limit the blast radius: by environment and by layer, so a bad app change can't plan a VNet deletion and plans stay fast.

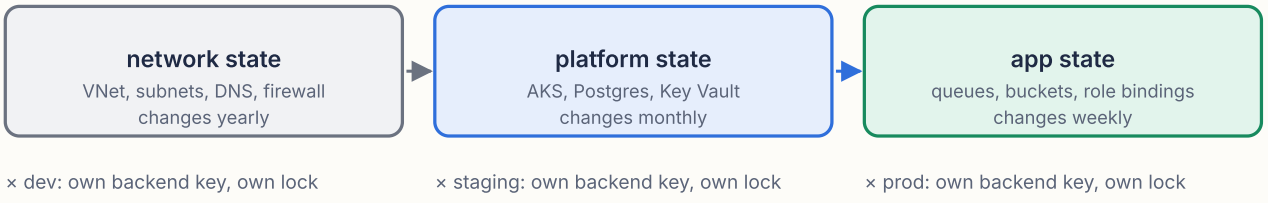


Figure 3. Split state by layer and environment. Each box is a separate backend key, lock and pipeline.

6 Modules and drift

Modules package a reusable unit (a network, an AKS cluster with standard settings) with input variables and outputs. Pin module versions with a Git tag or registry version, so upgrading a module is a deliberate, reviewed change for each environment. Keep environments thin: the same modules, different variable files.

Drift is any change made outside the code: a portal click, a hot-fix, another tool. Detect it with a scheduled `terraform plan -detailed-exitcode` (exit code 2 means changes are pending) or `plan -refresh-only`, and alert. Fix it either way round: bring the change into code if it was right, or apply to put things back if it wasn't. Restricting write access in production to the pipeline is what makes drift rare.

```
terraform {
  backend "azurerm" {                # remote, encrypted, versioned
    resource_group_name  = "tfstate-rg"
    storage_account_name = "tfstateprod01"
    container_name       = "tfstate"
    key                  = "platform/prod.tfstate"   # lock = blob lease
  }
}
module "aks" {
  source     = "git::https://git.example.com/tf-modules//aks?ref=v2.3.0"
  region     = var.region           # only variables differ per region
  node_count = var.aks_nodes
  subnet_id  = module.network.aks_subnet_id
}
```

Figure 4. Remote state with locking, and a version-pinned module whose only per-region differences are variables.

WORKED EXAMPLE: AKS, POSTGRES AND KEY VAULT WITH TERRAFORM

Modules define the **network** (VNet, subnets, private DNS zones), an **AKS** cluster, an **Azure Database for PostgreSQL** flexible server reachable only through a private endpoint, and a **Key Vault** for secrets. Each environment's state lives in an Azure storage account with versioning, blob-lease locking and access limited to the pipeline identity. A pull request runs `plan` and posts the output; reviewers check for replacements and destroys; after approval and merge the pipeline applies the saved plan. For the new region, the team adds `westeurope.tfvars` (region, CIDR ranges, node counts) and a new backend key, then runs the same code. The database password is generated and written to Key Vault, so applications never see it in Git, only in state, which is why state access is restricted.

COMMON MISTAKES

- Local state on a laptop, or state committed to Git with secrets inside.
- Applying without reading the plan, missing a forced replacement of a database.
- One giant state for everything: slow plans and a huge blast radius.
- Unpinned modules and providers, so an upstream release changes production unannounced.

7 Key terms

Declarative: describe the end state; the tool computes the steps

Plan / apply: preview the diff / execute it

State: mapping of code addresses to real resource IDs and attributes

State locking: only one run may modify state at a time

Module: reusable, versioned, parameterised group of resources

Drift: real infrastructure differs from code

8 Interview questions

Q1. Why is Terraform state important and how do you protect it? BASIC

It maps code to real resources so Terraform knows what it manages and what to change; protect it with remote, encrypted, versioned storage, locking, restricted access, no hand edits, and split states to limit blast radius.

Q2. What is drift? BASIC

Changes made to infrastructure outside the code; detect it with regular plans and fix it by reconciling, either updating the code or re-applying.

Q3. Why declarative rather than a script of CLI calls? BASIC

Declarative tools compute the difference from current state, so they are idempotent, show a reviewable plan, and can recreate or update environments safely.

Q4. The plan shows your production database will be replaced. What do you do? SCENARIO

Stop; find the attribute forcing replacement or the renamed address; use a moved block or revert the change, or plan a controlled migration; add prevent_destroy to protect it.

Q5. How do you build an identical environment in a new region? SCENARIO

Use the same version-pinned modules with a new variables file and a new state backend key, run plan in a PR, review, and apply through the pipeline.

Q6. Two engineers ran apply at the same time and you now have duplicate resources. What was missing and how do you clean up? TRICKY

State locking and a single apply path; clean up by importing or removing the orphans with state commands, then enforce locking and pipeline-only applies.

Q7. Why can Bicep work without a state file when Terraform can't? TRICKY

Bicep deploys to Azure Resource Manager, which itself knows every resource in a deployment scope; Terraform is cloud-agnostic and must keep its own record of what it created.

Next: SD-V-012 · GitOps

GitOps

Git says what should run; the cluster pulls it and keeps it that way

Watch the 4-min video

In one line: in GitOps the desired state of every environment lives in Git, and an agent inside the cluster (Argo CD, Flux) **pulls** it and **continuously reconciles** the cluster to match, so deployments are pull requests, drift is corrected automatically, and rollback is a `git revert`.

1 The deploy that undid a fix

The CI pipeline has admin credentials to every cluster and deploys with `kubectl apply`. During an incident someone fixes production by editing a Deployment by hand. Git no longer describes what's running, nobody notices, and the next routine deploy quietly removes the fix. Two problems: **powerful credentials outside the cluster**, and **no one checks** that the live state still matches the declared state.

2 Four principles

The OpenGitOps project (CNCF) defines GitOps with four principles:

1. **Declarative**: the system's desired state is expressed declaratively (Kubernetes manifests, Helm values, Kustomize overlays).
2. **Versioned and immutable**: that state is stored with full history, in practice Git, so every change has an author, a review and a way back.
3. **Pulled automatically**: software agents fetch the desired state; nobody pushes it into the cluster.
4. **Continuously reconciled**: agents keep comparing actual with desired state and correct the difference, not just once at deploy time.

3 Pull, not push

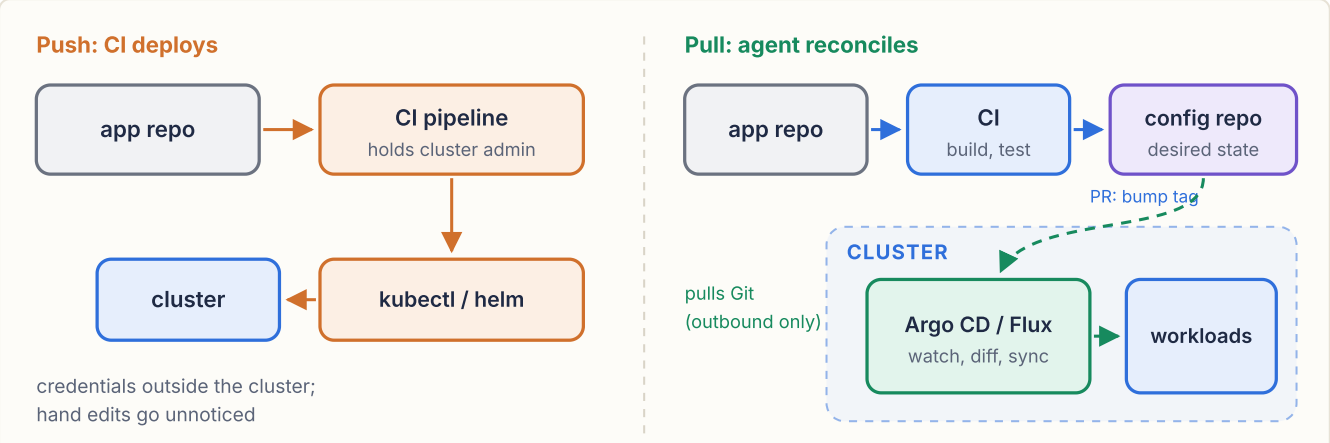


Figure 1. Push vs pull. In GitOps, CI never touches the cluster: it builds images and proposes changes to the config repo.

Argo CD or Flux runs **inside** each cluster (or in a management cluster) and needs only **read** access to Git and the image registry, through outbound connections. CI's job shrinks to testing and building images, then updating a tag in Git. Cluster credentials never leave the cluster, which removes the most valuable

secret from your CI system. A single central Argo CD managing many clusters is also common; it is easier to operate but holds credentials for all of them, so protect it like the old CI.

4 Reconciliation and drift

The agent re-reads Git on a timer (Argo CD's default is about every 3 minutes; a Git webhook makes it near-instant) and compares the rendered manifests with the live objects. Each application is **Synced** or **OutOfSync**, and separately **Healthy**, **Progressing** or **Degraded** (pods ready? rollout finished?).

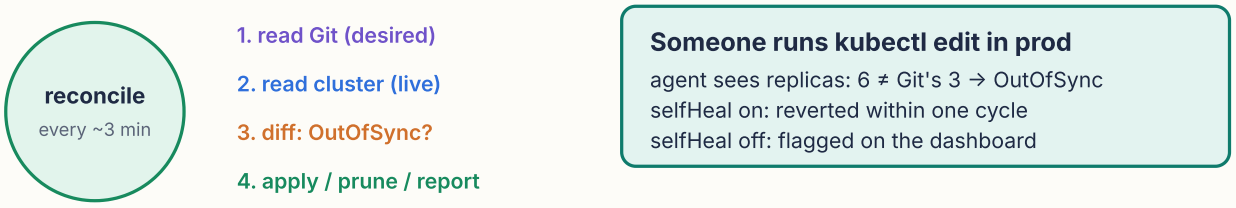


Figure 2. The reconcile loop. With self-heal, the hand edit from the opening story is undone within minutes instead of silently surviving until the next deploy.

The fix for the opening story is not "never edit by hand" but "hand edits don't last": during an incident you commit the fix to Git (fast, reviewed by one person), and the agent applies it. With `prune` enabled, objects deleted from Git are deleted from the cluster too, so nothing lingers.

5 Promotion and rollback through pull requests

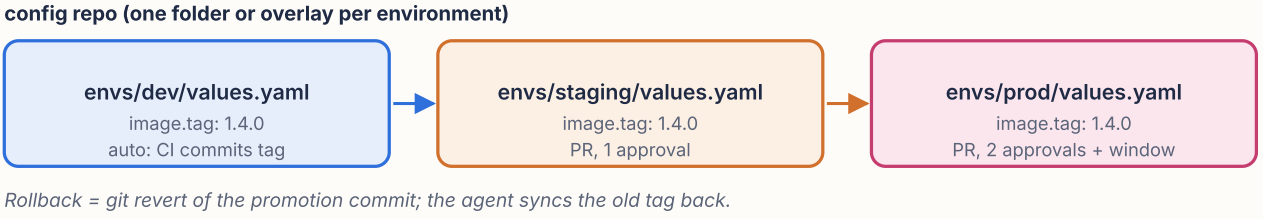


Figure 3. Promotion is a change of one line per environment, and the repository's branch protection becomes your deployment approval process.

CI writes the new image tag into the dev environment automatically (directly, or via Argo CD Image Updater / Flux image automation). Promoting to staging and production is a pull request that changes the tag in those folders; required reviewers, status checks and CODEOWNERS on `envs/prod/` are the approval gate, and the Git log is the audit trail of who deployed what, when. **Rollback** is reverting the commit. Use immutable tags or image digests, never `latest`, or Git can't say exactly what's running.

6 Caveats

- **Secrets** can't sit in Git in plain text. Options: **Sealed Secrets** (encrypted with the cluster's public key; only the in-cluster controller can decrypt), **SOPS** (encrypted values in files, decrypted by Flux or a plugin), or the **External Secrets Operator** (Git stores only a reference; the operator fetches the value from Key Vault, Secrets Manager or Vault). The last keeps secrets out of Git entirely and is the most common choice.
- **CI is still needed**: GitOps replaces only the deployment step, not tests, builds and scans.
- **Two repos** are common: application code, and environment configuration. This keeps config history clean and avoids CI loops where a tag bump triggers another build.
- **Ordering and migrations**: database migrations and dependent resources need sync waves or hooks (Argo CD) or `dependsOn` (Flux).

- **Progressive delivery** (canary, blue-green with automatic analysis) is a layer on top: Argo Rollouts or Flagger.
- GitOps fits declarative, Kubernetes-style resources best; cloud resources can join through Crossplane or Terraform controllers, but many teams keep those in an IaC pipeline (lesson 61).

WORKED EXAMPLE: ARGO CD SYNCING HELM CHARTS

A config repository holds a Helm chart per service and a values file per environment. Argo CD in each cluster has an `Application` pointing at its environment's values (below), with automated sync, prune and self-heal. CI builds **orders 1.4.0**, pushes the image, and opens a PR changing `image.tag` in `envs/prod/orders.yaml`. Two reviewers approve; the PR merges. The Git webhook notifies Argo CD, which renders the chart, sees one changed Deployment, and syncs it within a minute. The dashboard shows Progressing while new pods roll out, then Healthy. If error rates rise, the on-call engineer reverts the merge commit and Argo CD rolls back to 1.3.2.

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: orders-prod, namespace: argocd }
spec:
  source:
    repoURL: https://git.example.com/platform/config.git
    path: charts/orders
    targetRevision: main
    helm: { valueFiles: [ ../../envs/prod/orders.yaml ] }
  destination: { server: https://kubernetes.default.svc, namespace: orders }
  syncPolicy:
    automated: { prune: true, selfHeal: true }   # delete removed objects, undo drift
```

Figure 4. An Argo CD Application: which Git path to render, where to deploy it, and whether to prune and self-heal.

COMMON MISTAKES

- Keeping CI's cluster credentials "just in case", so push deploys continue beside GitOps.
- Mutable tags like `latest`: Git doesn't change, so nothing deploys, or everything does.
- Plain Kubernetes Secrets committed to the config repo (base64 is not encryption).
- Self-heal off and nobody watching OutOfSync alerts: drift returns.

7 Key terms

Desired state: what Git declares should be running

Drift: live state differs from Git

Promotion: moving a version to the next environment via PR

Reconciliation: loop that makes live state match desired state

Sync / prune / self-heal: apply changes / delete removed objects / undo manual edits

Sealed / External Secrets: ways to keep secret values out of plain Git

8 Interview questions

Q1. How does GitOps differ from push-based CI/CD? BASIC

In push-based CI/CD the pipeline holds cluster credentials and applies changes; in GitOps Git declares the desired state and an in-cluster agent pulls and continuously reconciles it, correcting drift.

Q2. How do you promote and roll back? BASIC

With pull requests that change the environment's configuration (e.g. the image tag), and by reverting the commit to roll back.

Q3. Name the four OpenGitOps principles. BASIC

Declarative, versioned and immutable, pulled automatically, continuously reconciled.

Q4. Production is down and the fix is one config value. Do you kubectl edit? SCENARIO

Commit the change to Git and let the agent sync (or sync manually from Argo CD); a direct edit would be reverted by self-heal or lost at the next deploy.

Q5. How do you handle database passwords with GitOps? SCENARIO

Keep only references in Git and use the External Secrets Operator to pull values from a secret manager, or commit encrypted Sealed Secrets/SOPS files.

Q6. Why might CI loop forever in a single-repo GitOps setup? TRICKY

CI's commit of the new image tag triggers CI again; separate config repos or path filters and skip-CI rules prevent it.

Q7. Does GitOps remove the need for CI? TRICKY

No: CI still tests, scans and builds images and proposes the config change; GitOps replaces only how changes reach the cluster.

Next: SD-V-013 · Immutable Infrastructure backendroadmap.com

Immutable Infrastructure

never fix a running server; replace it

Watch the 4-min video

In one line: instead of patching or reconfiguring running servers, build a new versioned, tested **image** with the change, start new servers from it, move traffic, and destroy the old ones; drift disappears, environments become identical, and rollback means redeploying the previous image.

1 The snowflake server

Server **web-07** has run for three years. It has been patched by hand two hundred times, someone installed a debugging tool in a hurry, and its config no longer matches the other servers. Nobody dares restart it, because nobody knows whether it would come back the same. That's a **snowflake**: unique, fragile, impossible to reproduce. Configuration management tools (Ansible, Chef, Puppet) reduce the problem by re-applying a desired config, but any change they don't manage still accumulates.

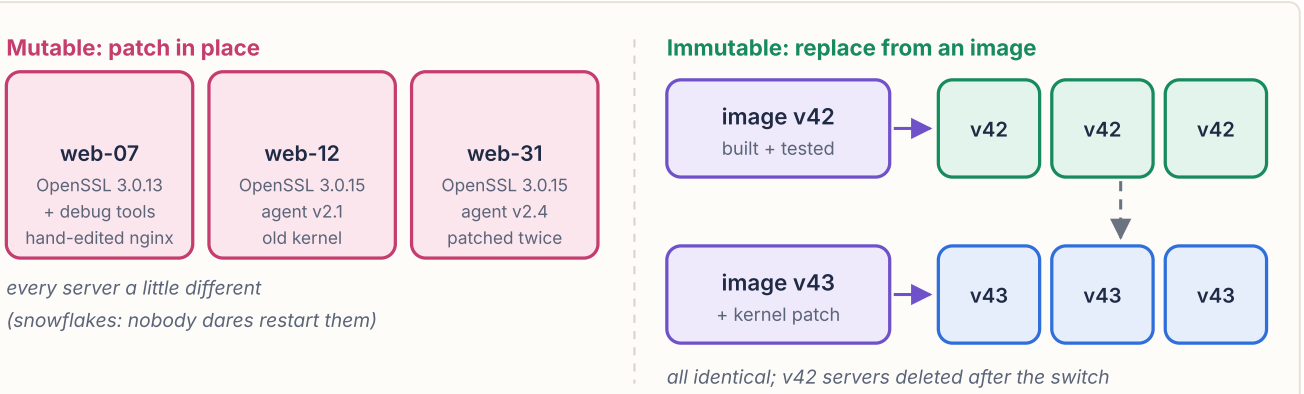


Figure 1. Mutable servers drift apart over time. Immutable servers are all built from one image version, and change only by being replaced.

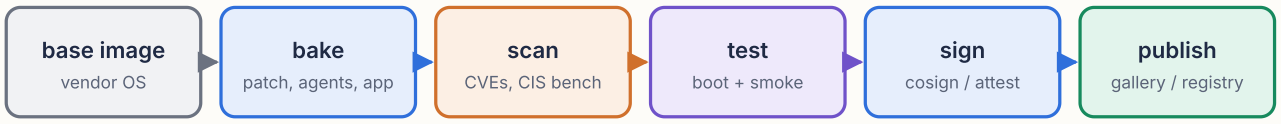
2 Never change a running server

The rule is simple: no logging in to patch, upgrade or reconfigure. Every change, from a kernel patch to a log-level setting baked into the image, goes through the same path: change code, build a new image, roll it out by replacement. Servers become **cattle, not pets**: numbered, identical, and replaceable without ceremony. Containers made this the default for applications (you never patch inside a running container); the same idea now applies to VMs and Kubernetes nodes.

	Mutable (patch in place)	Immutable (replace)
How changes happen	SSH, scripts, config management runs	new image version, rolling replacement
State of the fleet	drifts; each server has a history	identical per image version
Rollback	undo steps (often impossible)	redeploy the previous image
Security scanning	scan each server	scan the image once; it covers every server running it
Cost of a change	fast for one fix, risky at scale	build + rollout time (minutes to an hour)

3 Golden images

A **golden image** is a machine or container image built automatically by a pipeline, containing the OS, patches, security agents, configuration and often the application itself. **Packer** builds VM images (AMIs, Azure managed images) from a template; Azure VM Image Builder and EC2 Image Builder are managed equivalents; Dockerfiles or Buildpacks build container images. Each image gets a version, is published to a registry or image gallery, and is promoted through environments like any release.



Triggered by code changes AND on a schedule (new base image or CVE), so patching is a pipeline run, not a ticket.

Figure 2. An image pipeline. The same image version runs in dev, staging and production; only injected configuration and secrets differ.

Bake vs fry. "Baking" puts everything in the image, so boot is fast and nothing can fail at startup; "frying" configures at boot with cloud-init or a config tool, so images are fewer but boots are slower and less predictable. The usual balance: bake the OS, patches, agents and app; inject only **environment-specific** values (endpoints, feature flags, secrets from Key Vault) at start. Never bake secrets into images. Taking it further, immutable OSes like Bottlerocket, Flatcar and Talos have read-only root file systems and no package manager, and containers can set `readOnlyRootFilesystem: true`.

4 Deployment means replacement

A **rolling update** (VM scale set or auto-scaling group instance refresh, Kubernetes node pool upgrade), a **blue-green** switch, or a new node pool brings up servers from the new image; **health checks** confirm they work; traffic moves; the old servers are terminated. Rollback is deploying the previous image version, which you know worked, rather than trying to reverse a series of in-place steps.

5 Why it's worth it

- **No configuration drift**: server 1 and server 40 are identical; staging and production run the same bits.
- **Reproducible problems**: start the same image locally or in a test environment and you have the same server.
- **Security**: one image scan covers the fleet; attackers who change a server lose their foothold at the next replacement; you can say exactly which image every server runs, which simplifies audits.
- **Disaster recovery**: rebuilding an environment is starting images with IaC (lesson 61), not reconstructing years of hand changes.

6 New habits it requires

- **Nobody logs in to fix production.** SSH is off or break-glass only, audited; fixes go through the image pipeline. Debugging uses logs, metrics and traces, or an ephemeral debug container.
- **State and logs live outside the server**: managed databases, object storage, a log pipeline. A server can disappear at any moment and nothing may be lost with it.
- **Fast image builds**: if a critical CVE patch takes three days to reach production, people will start patching by hand again. Aim for hours, with automatic rebuilds when a base image updates.
- **Safe replacement**: PodDisruptionBudgets, connection draining and surge capacity so replacing every server is a non-event, done routinely (weekly), not once a year.

WORKED EXAMPLE: REBUILDING NODES TO APPLY A KERNEL PATCH

A critical kernel CVE is published. The image pipeline is triggered: it starts from the patched base image, bakes the node agents, runs a vulnerability scan (which now shows the CVE fixed), boots a test cluster from it and runs smoke tests, then publishes **node-image v43**. The Kubernetes node pool is upgraded with a **surge** of one extra node: a new v43 node joins, one old node is **cordoned** (no new pods) and **drained** (pods evicted, respecting each PodDisruptionBudget, so for example only one of three database replicas moves at a time), then deleted. Repeat until every node runs v43. Nobody logs in to a server, and the rollout record shows exactly when each node changed.

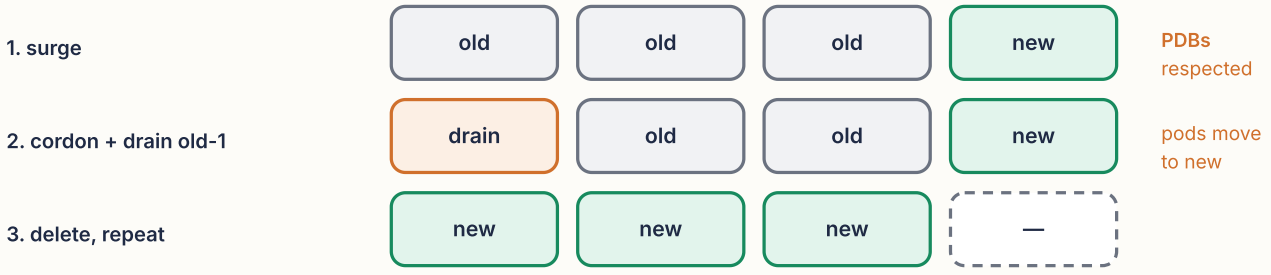


Figure 3. A surge upgrade of a node pool: add a new node, drain an old one into it, delete, repeat. Capacity never drops below normal.

TRADE-OFFS

- Every change, even one setting, costs an image build and a rollout.
- Stateful servers need state moved off them first (lesson 59).
- Large images slow boots and autoscaling; keep images lean and cache layers.
- Needs mature automation: build pipeline, IaC, health checks, observability.

COMMON MISTAKES

- "Immutable" images, then a config tool or SSH session changes servers after boot.
- Secrets or environment-specific config baked into the image.
- Logs written to local disk, lost when the server is replaced.
- Image builds so slow or manual that urgent patches bypass them.

7 Key terms

Snowflake server: unique, hand-modified, unreproducible server

Golden image: versioned, tested image with OS, patches, agents, app

Pets vs cattle: hand-cared-for servers vs identical, replaceable ones

Bake vs fry: configure at build time vs at boot time

Configuration drift: servers diverging from their intended config

Cordon / drain: stop new pods on a node / evict existing ones

8 Interview questions

Q1. Why prefer immutable infrastructure? BASIC

Replacing servers from versioned, tested images instead of patching in place removes drift, makes environments identical and reproducible, simplifies rollback to a known-good image, and lets one image scan cover every server.

Q2. What does it require? BASIC

Automated image builds, state and logs stored outside the servers, and deployments that replace instances safely with health checks.

Q3. What is a golden image? BASIC

A pipeline-built, versioned and tested image containing the OS, patches, agents and usually the app, used to start every server.

Q4. A critical CVE drops on Friday. How does an immutable shop patch 400 servers? SCENARIO

Rebuild the image from the patched base, scan and test it, then roll the fleet by replacement with surge and health checks; nobody SSHes in.

Q5. Developers want SSH to debug production. What do you offer instead? SCENARIO

Good logs, metrics and traces, ephemeral debug containers or a break-glass, audited access path whose changes are never kept; the fix still goes through the pipeline.

Q6. Can a database server be immutable? TRICKY

The server can, if data lives on a separate persistent volume or managed storage: replace the VM or pod, re-attach the volume or replica, and the data outlives the instance.

Q7. Isn't Ansible enough to prevent drift? TRICKY

It reduces drift for what it manages, but anything outside its playbooks still accumulates and servers keep a history; replacement guarantees a known starting state every time.

Next: SD-V-014 · Cloud Cost Optimization backendroadmap.com

Cloud Cost Optimization

make cost visible, owned, then efficient

Watch the 4-min video

In one line: first make cost **visible and owned** (tags, allocation, unit economics), then remove waste (right-size, delete idle, turn off non-production), buy the steady baseline cheaply (reservations, savings plans), run interruptible work on **spot**, tier storage, and cut network egress; keep it with budgets and monthly reviews.

1 A $200,000 surprise

The bill arrives: **$200,000** this month, up 40%, and finance wants a third cut. Nobody can say which team, service or feature spends what. That's the normal state of a cloud account that grew fast. **FinOps**, the practice formalised by the FinOps Foundation, treats cost as an engineering metric with owners, in a loop of three phases.

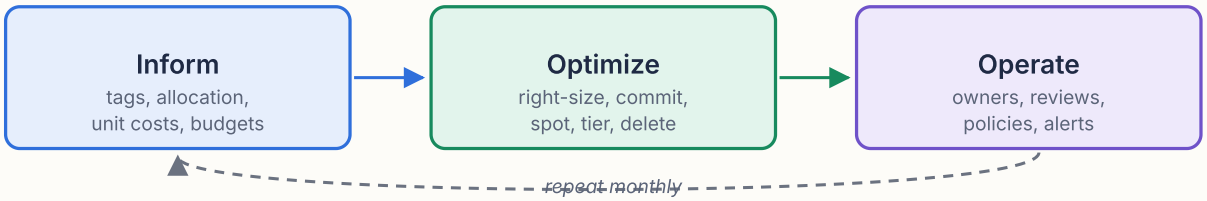


Figure 1. The FinOps lifecycle. You can't optimise what you can't attribute, and savings fade unless someone owns them.

2 Visibility first

- **Tag everything** with `team`, `service`, `environment` (and `cost-center`). Enforce tags with policy (Azure Policy, AWS tag policies / SCPs) so untagged resources can't be created. In Kubernetes, shared nodes need allocation by namespace or label: OpenCost or Kubecost split node cost by requests.
- **Showback** (each team sees its costs) usually comes before **chargeback** (costs hit team budgets).
- **Unit economics**: cost per request, per tenant, per active user, per order. A bill that grows with revenue is fine; a rising **cost per customer** is the real problem. If 10% traffic growth brings 40% cost growth, something is inefficient.

3 Right-size and clean up

Many VMs and pods run at 10% CPU because someone guessed big. Use 2–4 weeks of utilisation (p95 CPU and memory, not averages) to pick smaller instance sizes and lower Kubernetes requests (VPA recommendations, lesson 58). Move to newer generations, which are usually cheaper per unit of work (and ARM instances such as Graviton or Cobalt where the software runs on them). Look hard at databases: an oversized managed database is often the single biggest line item.

Then delete **zombies**: idle dev environments, unattached disks, unused public IPs, old snapshots and images, load balancers with no targets, forgotten test clusters. Each is small; together they are often 5–10% of a bill. Automate the hunt (Azure Advisor, AWS Trusted Advisor / Compute Optimizer) and give every resource an owner tag so someone can say "delete it".

4 Commit and discount

Pricing model	Typical discount vs on-demand	You give up	Use for
On-demand	—	nothing	spiky or unknown load
Savings plans (AWS, Azure)	roughly 15–65%	commit $/hour for 1 or 3 years; flexible on instance type	steady compute baseline
Reserved instances / reservations	up to ~70%+	commit to a type/region for 1 or 3 years	very stable fleets, databases
Spot / preemptible	up to ~90%	can be reclaimed: 2 min notice on AWS, ~30 s on Azure and GCP	batch, CI, stateless workers, some training

Commit only to the **baseline** you will certainly use: the load at 3 a.m. on a Sunday, after right-sizing (right-size first, or you lock in waste). Many teams aim to cover most of that baseline with commitments and keep the variable part on-demand.

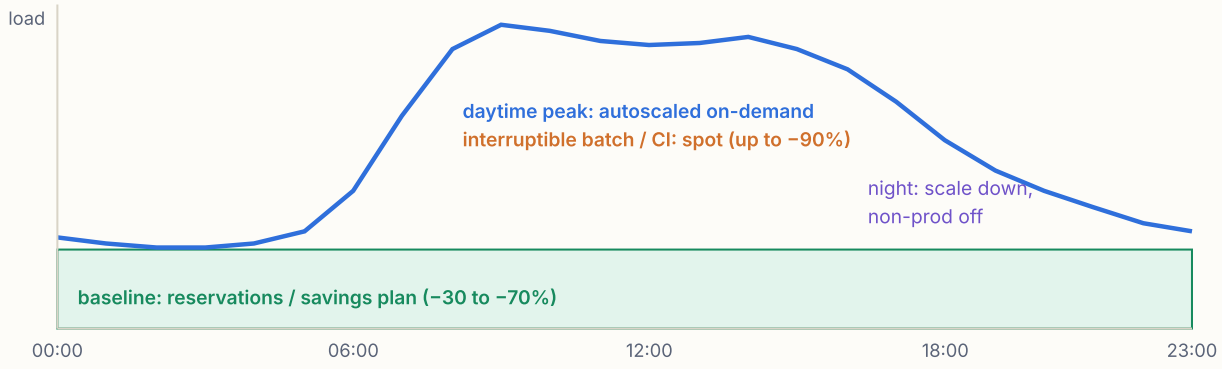


Figure 2. Layer the pricing models on the demand curve: commitments under the always-on baseline, on-demand autoscaling for the daily peak, spot for work that can be interrupted.

5 Follow demand, tier storage

Autoscaling (lesson 58) turns fixed capacity into capacity that follows the curve. **Non-production off-hours** is the easiest win: running dev/test 12 hours × 5 weekdays is 60 of 168 hours a week, about **64% less** than running all the time. Schedules, scale-to-zero, and ephemeral per-PR environments that delete themselves make it stick.

Storage tiering: object storage tiers (S3 Standard → Infrequent Access → Glacier tiers; Azure Hot → Cool → Cold → Archive) cost progressively less per GB stored but more per access, with minimum storage durations. Lifecycle rules move data by age and delete what you don't need (old logs, temporary exports, incomplete multipart uploads). Also check disk types (for example gp3 is cheaper than gp2 for the same size on AWS) and log retention, since observability data is a surprisingly large bill.

6 Network, budgets and making it stick



Figure 3. Costs that don't show up in architecture diagrams.

Data transfer out to the internet costs on the order of $0.05–0.09 per GB at list price on the big clouds; between regions and even between availability zones is also charged. Serve static content through a CDN, keep chatty services in the same zone where resilience allows, compress responses, and use private endpoints to avoid NAT gateway processing fees for traffic to cloud services. Lesson 65 puts numbers on these.

Set **budgets and anomaly alerts** per team and subscription/account, so a runaway job is found in a day, not at month end. Review the top cost drivers with each team monthly, track savings, and add guardrails (allowed instance types, maximum node counts, required tags) to the platform (lesson 70).

WORKED EXAMPLE: MOVING NIGHTLY BATCH TO SPOT

Nightly processing runs on **20 on-demand VMs for 6 hours**: 20 × 6 × 30 = 3,600 VM-hours a month. At an illustrative $0.40/hour that's about **$1,440**. The team moves it to a **spot node pool** spread over several instance types (so one type running out doesn't stop the job), makes each job **checkpoint progress every few minutes** to object storage, and keeps a small **on-demand fallback** pool that the scheduler uses if spot capacity is unavailable. When a node is reclaimed, the job gets the eviction notice, writes a final checkpoint if it can, and restarts elsewhere from the last checkpoint, losing at most a few minutes. With spot prices around 70–80% below on-demand and a little rework, the batch bill falls by about **70%**, to roughly $430 a month, and the job still finishes before morning.

COMMON MISTAKES

- Buying 3-year reservations before right-sizing, locking in waste.
- Running stateful or non-checkpointed jobs on spot and losing hours of work.
- Optimising the total bill without unit costs, so growth hides inefficiency.
- One-off clean-ups with no owners or budgets: costs creep back within months.

7 Key terms

FinOps: practice of making cloud cost visible, owned and optimised

Right-sizing: matching instance size or requests to real usage

Spot: spare capacity, deeply discounted, can be reclaimed

Unit economics: cost per request, tenant or user

Savings plan / reservation: discount for 1–3 year commitment

Showback / chargeback: report costs to teams / bill them

8 Interview questions

Q1. How would you cut a cloud bill by 30%? BASIC

Make cost visible with tags and unit economics; right-size and delete idle resources; use reservations or savings plans for steady load and spot for interruptible work; autoscale and shut down non-production at night; tier storage; and reduce egress.

Q2. What is unit economics? BASIC

Cost per request, tenant or user, so you can see whether efficiency improves as you grow, independent of total growth.

Q3. When is spot a bad fit? BASIC

For work that can't tolerate interruption without losing progress: stateful databases, long jobs without checkpoints, latency-critical services without fallback capacity.

Q4. Costs grew 40% while traffic grew 10%. Where do you look first? SCENARIO

Cost per unit by service and team: new or oversized resources, unattached storage, logs and data transfer growth, idle environments, and expired commitments now billed on-demand.

Q5. How much should you commit with reservations or savings plans? SCENARIO

Only the steady baseline after right-sizing, typically the minimum load over the period, leaving peaks to autoscaled on-demand and spot.

Q6. Why can moving to a cheaper storage tier increase the bill? TRICKY

Cooler tiers charge more per read and retrieval and have minimum storage durations, so frequently accessed or short-lived data costs more there.

Q7. Your Kubernetes cluster runs on shared nodes. How do you charge teams? TRICKY

Allocate node cost by each namespace's resource requests (or max of requests and usage) with a tool like OpenCost, plus shared costs split by a stated rule.

Next: SD-V-015 · Cost-Aware Architecture backendroadmap.com

Cost-Aware Architecture Decisions

every box on the diagram has a monthly price

Watch the 4-min video

In one line: treat cost as a design dimension like latency and availability: estimate the main drivers (compute, storage, data transfer, GPUs, managed premiums), compare options by **total cost of ownership** including people's time, and state the trade-off out loud: "twice the cost for half the latency in Europe".

1 "Roughly what would that cost?"

In a design interview you propose replicating every database across three regions, serving video from your own servers, and calling a large GPU model on every request. The interviewer asks one question: **roughly what would that cost each month?** Strong engineers can answer to the nearest order of magnitude, and know which choice dominates the bill. You don't need exact prices; you need the drivers, a few anchor numbers, and the arithmetic.

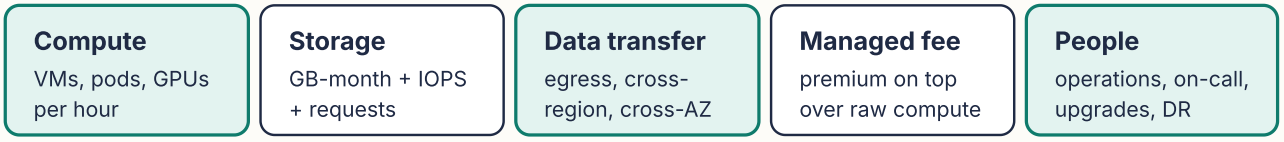


Figure 1. The five cost drivers. The three marked are the ones most often missed in designs.

2 Every decision has a price

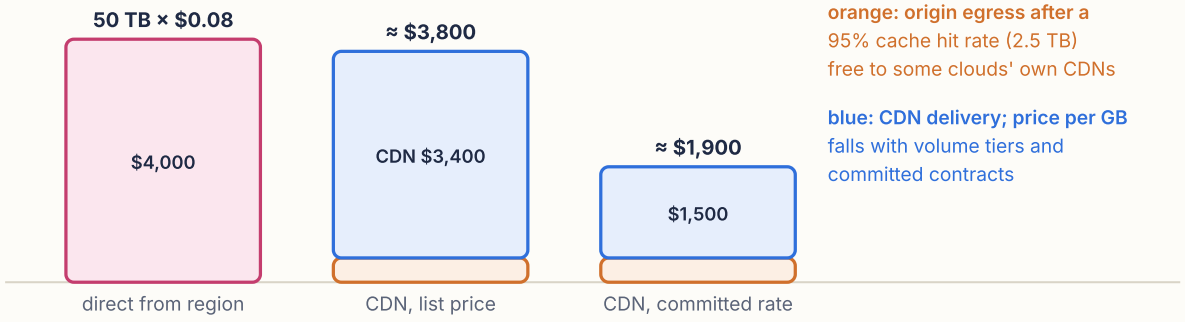
- **Another region** roughly doubles infrastructure for an active-active design, plus cross-region replication traffic. Active-passive with a smaller warm standby costs less, at the price of a slower failover (lesson 52).
- **Synchronous cross-region writes** add latency and inter-region transfer charges on every write.
- **Managed services** charge a premium over raw compute (often 1.5–3× per unit), in exchange for operations you don't do.
- **Higher availability** (lesson 54) usually means more replicas and zones, so more compute and cross-zone traffic.
- **Retention**: keeping every event, log and trace forever is a storage and query cost that grows every month.

3 Data transfer: the classic surprise

Inbound data is usually free; **outbound** is not. Internet egress from the big clouds lists at roughly **$0.05–0.09 per GB**, falling with volume; inter-region transfer is around $0.02 per GB; even traffic between availability zones in one region is charged on some clouds (about $0.01 per GB each way on AWS). A NAT gateway adds a per-GB processing charge (about $0.045 on AWS) on top of egress, so a fleet downloading container images and calling public APIs through NAT pays twice.

WORKED NUMBERS: 50 TB OF VIDEO A MONTH

Served directly from a region at about $0.08/GB: 50,000 GB × $0.08 ≈ **$4,000 a month**. Through a CDN with a 95% cache hit rate, only 2.5 TB leaves the origin (often free or cheap when the CDN belongs to the same cloud), and the CDN's own per-GB price drops with volume tiers and committed contracts. At list prices the saving is modest; with a committed rate or a cheaper CDN, cutting the bill in half or more is realistic, and users get lower latency as a bonus.



Illustrative monthly costs ($ thousands, bar height). Real prices vary by provider, region and contract.

Figure 2. The CDN changes two numbers: how much leaves your origin, and the price per GB for the rest.

4 GPUs dominate AI systems

A single data-centre GPU costs several dollars per hour on demand. At $3 per GPU-hour, one GPU running all year is 3 × 8,760 ≈ **$26,000**; a node with eight of them is over $200,000 a year. So size GPU fleets from first principles:

```
GPUs at peak ≈ (peak requests/s × tokens per request) ÷ tokens/s per GPU
                                                      × headroom
```

Example with illustrative numbers: 40 requests/s at peak × 500 output tokens = 20,000 tokens/s. If one GPU serving this model with batching sustains about 2,500 tokens/s, you need 8 GPUs, about 10 with headroom: around **$22,000 a month** at $3/hour if they run all the time. The levers that change this most are usually not the hardware: a **smaller model** where it is good enough (often several times cheaper per token), **caching** repeated prompts or answers, **batching** for higher throughput per GPU, shorter prompts and outputs, and autoscaling to the daily curve (lesson 58).

5 Total cost of ownership

The price list is only part of the cost. Self-hosting Kafka on VMs looks cheaper than a managed event service until you add the people: upgrades, security patches, monitoring, capacity planning, on-call, disaster-recovery tests. A fully loaded engineer costs on the order of $10,000–20,000 a month depending on location, so even a third of one is a large line item. Add the **opportunity cost**: those hours aren't spent on the product.

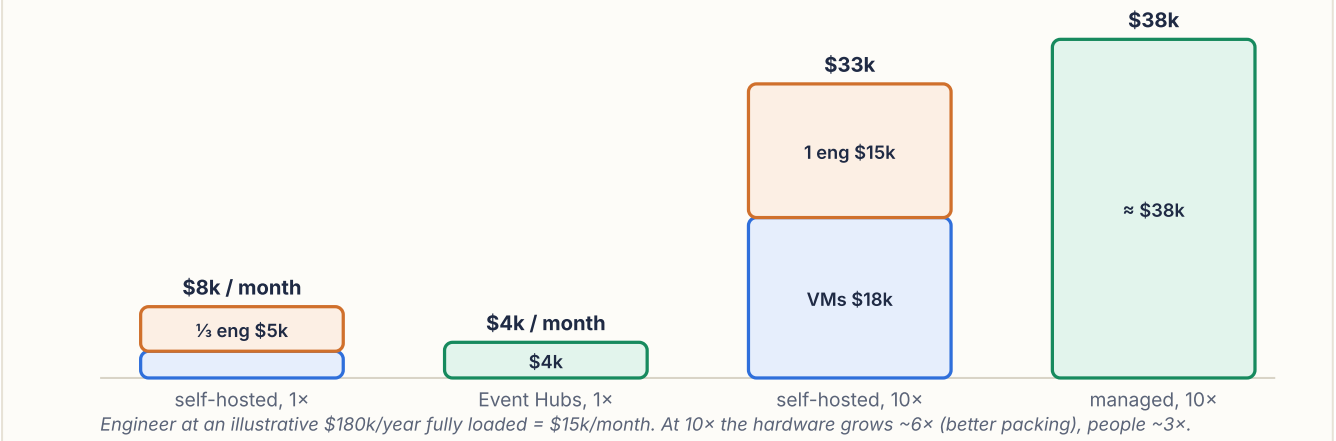


Figure 3. Illustrative TCO for the worked example. Managed wins clearly at 1×; at 10× the gap narrows or reverses, because people scale more slowly than hardware.

At small and medium scale, managed services usually win. At very large scale, self-hosting can pay for itself because the platform team's cost is spread over a big fleet and you can tune hardware precisely. Lock-in, features, compliance and skills also count, but the question to ask is always "**total** cost, at **our** scale, including **people**".

6 Hidden costs to name

Hidden cost	Why it bites	Mitigation
Egress and cross-region transfer	billed per GB, grows with users and replication	CDN, compression, keep replication async and selective
Cross-zone traffic	chatty microservices spread over zones	zone-aware routing; co-locate chatty pairs
NAT gateway and load balancer processing	per-GB and per-hour fees on top of egress	private endpoints to cloud services, fewer LBs (shared ingress)
Observability	logs and metrics ingestion priced per GB	sampling, retention limits, drop debug logs in prod
Storage requests	millions of tiny object PUT/GETs	batch small objects, cache reads
Operational time	self-hosted systems need people	count it in TCO; prefer managed below large scale

7 In the interview

1. Name the **two or three dominant drivers** for this design (GPUs for an AI assistant; egress for video; storage for logging).
2. Estimate them with round numbers you can defend ("egress about 8 cents a GB", "a GPU a few dollars an hour").
3. **Compare two options** and state the trade-off: "this costs about twice as much but halves latency for European users", or "we accept eventual consistency to avoid cross-region write charges".
4. Mention the cheap wins: caching, a CDN, a smaller model, autoscaling, tiering.

WORKED EXAMPLE: SELF-HOSTED KAFKA VS AZURE EVENT HUBS

Self-hosted: six VMs with premium disks, about **$3,000 a month**, plus about a third of an engineer to operate it: upgrades, broker failures, rebalancing, monitoring, on-call. At an illustrative $15,000 a month per engineer, that's $5,000, so **about $8,000** in total. **Event Hubs** at the required throughput costs about **$4,000 a month**, with the operations included and a Kafka-compatible endpoint so clients don't change. For a team of five, the managed option has the lower total cost and frees a third of a person. At ten times the scale, hardware may grow to about $18,000 (better packing) and people to one engineer, about $33,000, while the managed bill might be around $38,000: the answer may flip, and it's worth re-checking with real quotes and a capacity test.

COMMON MISTAKES

- Comparing a managed service's price with VM prices only, ignoring people.
- Forgetting data transfer, which can exceed compute for media and analytics.
- Sizing GPUs for peak and running them 24/7 at 20% utilisation.
- Giving precise-looking numbers without stating assumptions.

8 Key terms

TCO: total cost of ownership, including people and operations

Managed premium: extra price of a managed service over raw resources

Tokens per second per GPU: throughput used to size LLM serving

Egress: data leaving the cloud or region, billed per GB

Cache hit rate: share of requests served without reaching the origin

Opportunity cost: what engineers could build instead of operating infrastructure

9 Interview questions

Q1. How do you include cost in a design discussion? BASIC

Treat cost as a design dimension: estimate the main drivers (compute, storage, data transfer, GPUs, managed premiums), compare options by total cost of ownership including people's time, and state the trade-offs explicitly.

Q2. What are common hidden costs? BASIC

Egress and cross-region transfer, NAT and load balancer processing, observability ingestion, and the operational time of self-hosted systems.

Q3. Estimate egress for 50 TB of video a month and how to cut it. SCENARIO

About 50,000 GB × $0.08 ≈ $4,000; a CDN with a high hit rate cuts origin egress sharply and, with volume or committed pricing, lowers the per-GB cost.

Q4. How many GPUs for 40 requests/s at 500 tokens each? SCENARIO

20,000 tokens/s divided by measured throughput per GPU (say 2,500 tokens/s with batching) gives 8, about 10 with headroom; then look at smaller models, caching and batching to reduce it.

Q5. Managed or self-hosted Kafka for a five-person team? SCENARIO

Managed: once you count a third of an engineer for operations, its total cost is lower, and the team stays on the product; revisit at much larger scale.

Q6. Why does an active-active second region more than double some costs? TRICKY

Besides duplicated infrastructure, every replicated write crosses regions and is billed per GB, and you need capacity in each region to absorb the other's traffic on failover.

Q7. Why can a cheaper architecture be the more expensive choice? TRICKY

Lower cloud spend can mean more engineering time, slower delivery, or higher incident risk; TCO and opportunity cost, not the invoice alone, decide.

Next: SD-V-016 · Data Consistency Across Systems

Data Consistency Across Systems

write once, propagate reliably, and reconcile anyway

Watch the 4-min video

In one line: never write the same change separately to several systems (dual writes); write it once to the source of truth, propagate it with a **transactional outbox** or **change data capture** into **idempotent, version-aware** sinks, and run **reconciliation** jobs that find and repair the drift that still happens.

1 Every system a little different

An order is in the database but not in search. The cache shows the old price. The warehouse says yesterday's revenue was $12,000; the payment provider says $13,000. Each copy (index, cache, warehouse, read model, partner system) is there for a good reason, but every copy can disagree with the source, and customers notice. Keeping copies aligned is a discipline with three parts: **propagate** reliably, **apply** safely, **verify** continuously.

2 The trap: dual writes

The tempting code writes to the database, then the search index, then the cache, then publishes to a queue. There is no transaction across those systems, so:

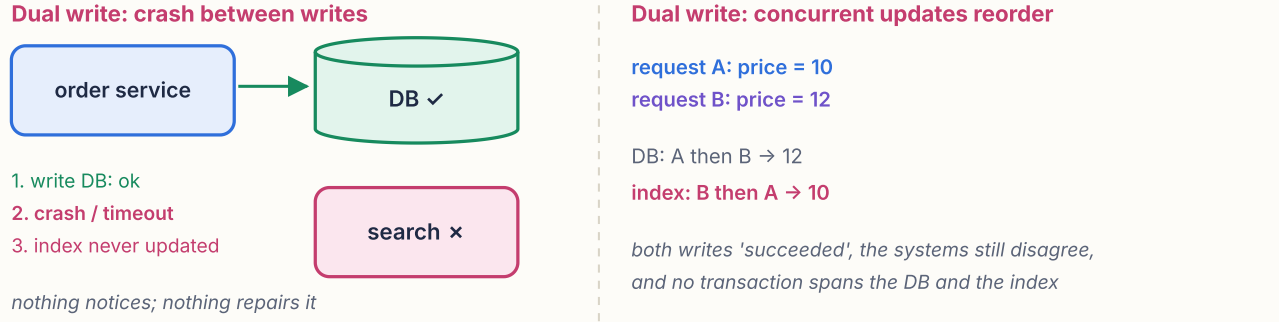


Figure 1. Two ways dual writes go wrong: a partial failure, and a race that reorders updates. Both leave permanent, silent drift.

- A crash, timeout or error between writes leaves some systems updated and others not, and nothing remembers to finish the job.
- Retrying the whole sequence risks duplicates; not retrying risks loss.
- Concurrent requests can apply updates in a different order in each system.
- Swapping the order (queue first, then DB) just moves the problem: now consumers can see an event for a change that was rolled back.

Dual writes are the most common source of drift precisely because they look simple in code.

3 Write once, propagate from the source

Transactional outbox. In the same local transaction as the business change, insert a row into an `outbox` table describing the event. Either both commit or neither. A **relay** then reads unpublished outbox rows (by

polling, or by CDC on the outbox table) and publishes them to the broker, marking or deleting them afterwards. Publishing is **at-least-once**: a relay crash after publish but before marking repeats the event, so consumers must be idempotent.

Change data capture (CDC). Read the database's own change log (PostgreSQL WAL via logical decoding, MySQL binlog, SQL Server CDC, MongoDB change streams) and publish every committed change. Debezium is the common open-source tool; each event carries the row before and after, the operation, and the log position. No application code changes are needed, but events mirror table rows, not business meaning, and they couple consumers to your schema.

	Transactional outbox	CDC on business tables
Event shape	designed business events (`OrderPaid`)	row changes (before / after)
Code change	write the outbox row in each transaction	none in the app
Schema coupling	low: event is a contract	high: consumers see table columns
Captures manual SQL fixes	no	yes, every committed change
Typical use	service-to-service events	replication to search, cache, warehouse

Many teams combine them: the app writes an outbox row, and Debezium's outbox event router streams the outbox table, giving designed events without a polling relay. Operational gotcha: a PostgreSQL replication slot that no consumer reads keeps all WAL on disk, and can fill it. Monitor slot lag.

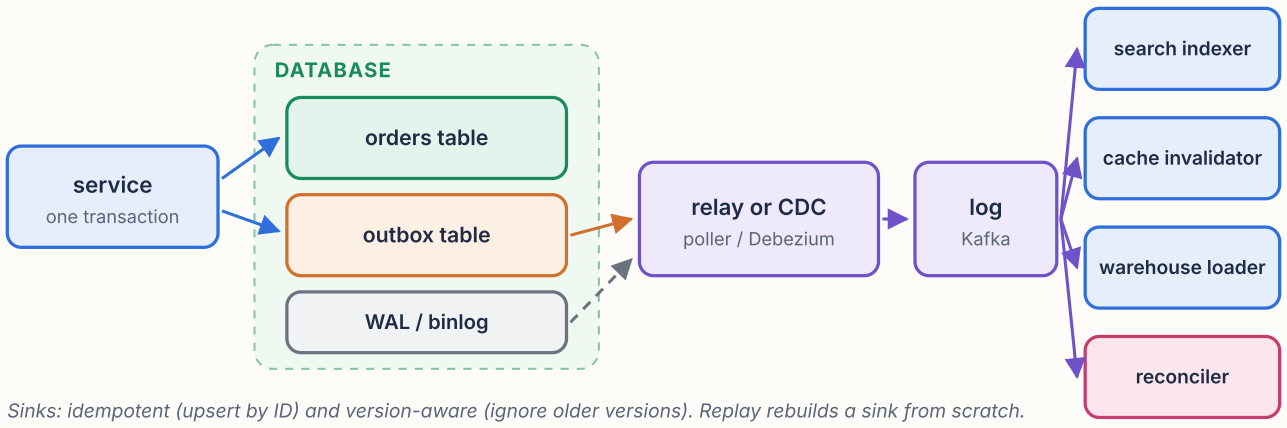


Figure 2. One write, many propagated copies. The log (Kafka) lets each sink consume at its own pace and lets a new sink replay history.

4 Idempotent, version-aware sinks

Events can arrive **twice** (at-least-once delivery) and, across partitions or retries, **out of order**. Each sink must handle both:

- **Idempotent**: key every write by the source record's ID (upsert, not insert), so applying the same event twice gives the same result.
- **Version-aware**: carry a monotonically increasing version (a row version column, or the CDC log position) and apply an update only if it's newer than what the sink holds. Elasticsearch supports this natively with `version_type=external`; SQL sinks use a conditional upsert (below).

- **Ordering by key**: partition the log by record ID so all changes to one record go through one partition in order.
- **Caches**: prefer deleting the key on change over writing the new value, so a late, stale event can't put old data back.

The reward: you can **replay** the log, or re-snapshot the source, to rebuild a sink from scratch at any time, which is also the repair tool of last resort.

```
BEGIN;
  UPDATE orders SET status = 'paid', version = version + 1 WHERE id = 42;
  INSERT INTO outbox (id, aggregate_id, type, payload)
    VALUES (gen_random_uuid(), 42, 'OrderPaid', '{"version": 7, ...}');
COMMIT;   -- both rows or neither

-- sink: version-aware upsert (older events are ignored)
INSERT INTO order_view (id, status, version) VALUES (42, 'paid', 7)
ON CONFLICT (id) DO UPDATE SET status = EXCLUDED.status, version = EXCLUDED.version
  WHERE order_view.version < EXCLUDED.version;
```

Figure 3. The outbox write and a version-aware sink upsert. A replayed or late event with version ≤ 7 changes nothing.

5 Reconciliation: verify, then repair

Even good pipelines drift: a bug in a consumer, a skipped event, a manual fix in one system, a restore of an old backup. A **reconciliation job** compares systems for a period and repairs differences, so drift is short-lived instead of permanent:

- **Cheap first**: compare counts and sums per bucket (per day, per tenant, per ID range).
- **Then drill down**: compute a checksum per bucket (hash of IDs and versions); only mismatching buckets are compared row by row. This is the same anti-entropy idea databases use with Merkle trees.
- **Repair** by re-publishing the source's events for those records, or upserting the target directly, and **record** every repair. A rising repair count is an alert that a pipeline is broken.

6 Reconciling money

For payments, your **ledger** and the provider's **settlement report** must match to the cent. The process is formal: extract both sides, normalise (currency, fees, time zone of the provider's day boundary), match on a shared key (payment or charge ID, amount, currency), and classify the rest as exceptions. Some exceptions are fixed automatically (a lost webhook: fetch the payment from the provider's API and record it); others (amount mismatch, unexplained refunds) go to a person with an SLA. Mature teams add a third leg: the bank statement showing the payout actually arrived.

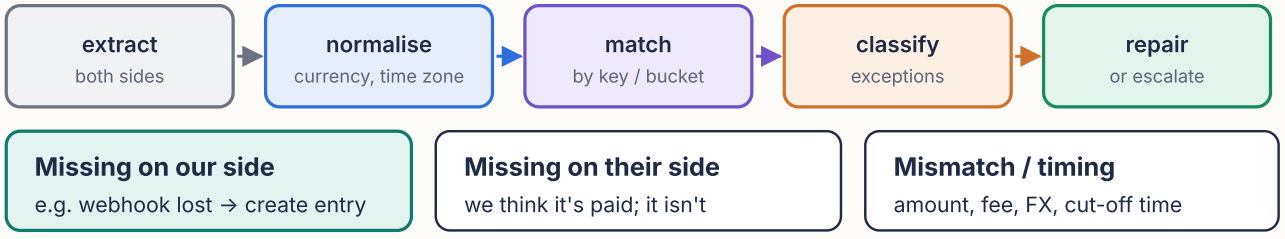


Figure 4. A reconciliation pipeline and its exception classes. Timing differences (a payment at 23:59 in one system's day and 00:01 in the other's) are the most common false alarm.

WORKED EXAMPLE: THE NIGHTLY LEDGER CHECK

At 02:00 the job downloads yesterday's settlement file from the payment provider and sums the internal ledger by **currency and merchant**, using the provider's UTC day boundary. Totals match for every merchant except one, which is short by $412.50. Drilling down by payment ID finds **three captured payments** that exist in the provider's file but not in the ledger: their webhooks never arrived (the endpoint returned errors during a deploy, and retries were exhausted). The job fetches each payment from the provider's API to confirm it, creates the missing ledger entries through the normal service (so the outbox emits events and downstream systems update too), and records the repair with a reason code. A dashboard shows zero open exceptions by 02:15; the lost-webhook count raises a ticket to fix the endpoint's retry handling.

COMMON MISTAKES

- Writing to DB, index and queue in sequence and calling it "eventually consistent".
- Sinks that insert instead of upsert, creating duplicates on replay.
- Last-write-wins by arrival time instead of by source version.
- Repairing a sink by hand, so the source and the log never learn about it.

7 Key terms

Dual write: updating several systems separately for one change

CDC: streaming committed changes from the database log

Version-aware: older updates never overwrite newer ones

Transactional outbox: event row written in the same transaction as the change

Idempotent sink: applying an event twice has the same effect as once

Reconciliation: comparing systems and repairing differences

8 Interview questions

Q1. How do you detect and repair data drift between systems? BASIC

Avoid dual writes; write to one source of truth and propagate changes with an outbox or CDC into idempotent, version-aware sinks; then run reconciliation jobs comparing counts, sums or checksums and repair the differences.

Q2. Why are dual writes dangerous? BASIC

A failure or crash between writes leaves systems inconsistent with nothing to repair it, and concurrent writes can be applied in different orders in different systems.

Q3. Outbox or CDC? BASIC

Outbox for designed business events between services; CDC for replicating table changes to search, caches and warehouses with no app changes; they are often combined.

Q4. Search sometimes shows an old price after updates. Why, and how do you fix it? SCENARIO

Out-of-order or duplicate events overwrite newer data; partition by product ID and make the indexer version-aware (external versioning), so older versions are rejected.

Q5. How would you reconcile two tables with 500 million rows each? SCENARIO

Bucket by ID range or date, compare counts and checksums per bucket, and only compare rows inside mismatching buckets; repair through the normal propagation path.

Q6. Why not publish the event first, then write the database? TRICKY

If the DB write fails or rolls back, consumers have already acted on a change that never happened; the outbox ties both to one transaction.

Q7. Your CDC connector was down for two days and the database disk filled up. Why? TRICKY

Its replication slot kept all WAL since the last confirmed position; monitor slot lag and drop or advance abandoned slots.

Next: SD-V-017 · Privacy Engineering

Privacy Engineering

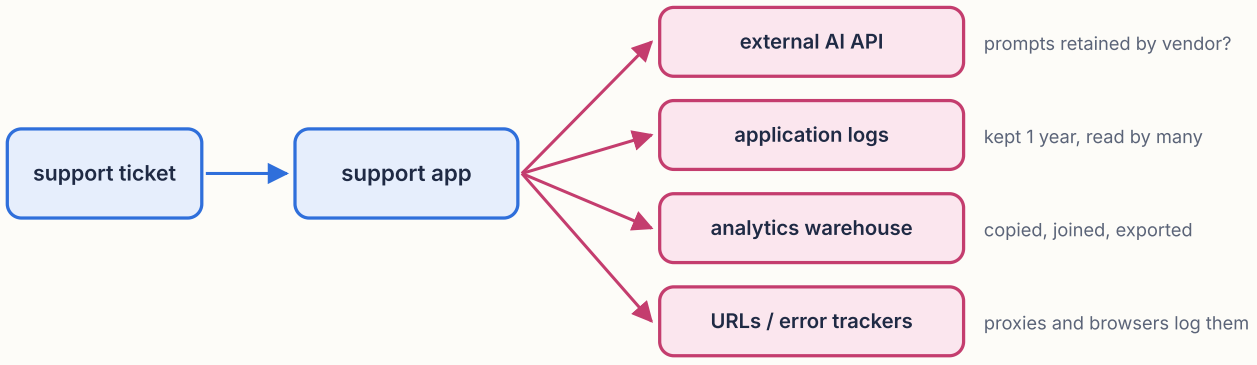
design so personal data can't leak by accident

Watch the 4-min video

In one line: collect and keep only the personal data you need; find and classify it; reduce exposure with masking, tokenisation and pseudonymisation; keep it out of logs, URLs and analytics; enforce consent and deletion across every copy; and review every third-party flow, including AI APIs, like any other data flow.

1 Nobody intended it

A support assistant built on an LLM is a hit. Then an audit finds customer names, phone numbers and card details from tickets were sent to an external AI service, written to application logs, and copied into the analytics warehouse. No one decided to do this; each step was a reasonable default. **Privacy engineering** makes the safe path the default, so leaks don't depend on every developer remembering every rule.



Unintended copies: each one is a place to secure, retain, delete, and report in a breach.

Figure 1. One ticket, four unintended copies. Each copy needs its own access control, retention and deletion, and each is in scope for breach reporting.

2 Privacy by design

GDPR Article 25 calls it **data protection by design and by default**; the principles (Article 5) translate directly into engineering rules:

- **Data minimisation**: collect only fields with a stated purpose. Don't store date of birth if you only need "over 18".
- **Purpose limitation**: data collected for support isn't automatically available for marketing models.
- **Storage limitation**: every table, bucket and log stream has a retention period and a deletion job.
- **Least access**: people and services see only what their task needs.

Every new feature answers three questions in its design review: **what personal data** does it use, **why**, and **where does it flow** (including vendors)? For high-risk processing, the formal version is a Data Protection Impact Assessment (DPIA).

3 Find and classify

You can't protect what you can't find. **PII discovery** scans databases, files and logs using regular expressions (emails, phone formats), checksums (Luhn for card numbers, check digits for national IDs), dictionaries and ML named-entity recognition (names, addresses). Tools include Microsoft Presidio (open source), Amazon Macie, Google Cloud DLP / Sensitive Data Protection and Microsoft Purview. **Classification** then tags each column or field, and those tags drive the controls automatically (lesson 68).



Figure 2. A typical four-level scheme. The tag on a column decides access, masking, encryption and retention, not each team's judgement.

4 Reduce exposure

Technique	What it does	Reversible?	Example use
Masking	hides part of a value for display	no (view only)	phone `•••• ••• 678`; card shows last 4
Tokenisation	replaces a value with a random token; a secured vault maps it back	yes, only via the vault	card numbers: most systems store `tok_8f3…`, shrinking PCI DSS scope
Pseudonymisation	consistent alias for an identifier (keyed hash, e.g. HMAC-SHA-256)	only with the key or mapping	analysts count distinct users without knowing who
Encryption	ciphertext; key holders can decrypt	yes, with the key	field-level encryption of national IDs
Anonymisation / aggregation	removes the link to individuals	no (if done properly)	published statistics

Two traps. A **plain hash** of an email or phone number is not anonymous: the input space is small, so anyone can hash a list of candidates and match. Use a **keyed** hash and protect the key. And under GDPR, **pseudonymised data is still personal data**; only truly anonymised data falls outside it. Treat pseudonymised datasets as sensitive, just less exposed.

5 Differential privacy, consent and deletion

Differential privacy adds calibrated random noise to aggregate results so that the output is almost the same whether or not any one person is in the data. The parameter ε (epsilon) is the privacy budget: smaller ε means more noise and stronger privacy, and every query spends some of the budget. The US Census Bureau used it for the 2020 census; Apple and Google use variants for telemetry. It suits statistics over many people, not individual-level lookups.

Consent management records what each user agreed to (purpose, timestamp, version of the wording) and makes it checkable at the point of use: the marketing pipeline filters on `consent.marketing = true`, and withdrawal propagates within a defined time.

Data subject requests (access, export, deletion) must reach **every copy**: primary DB, search, caches, warehouse, backups, logs and vendors. A data map and lineage (lesson 68) are what make that possible. For data you can't easily rewrite (backups, event logs), **crypto-shredding** helps: encrypt each user's data with a per-user key and delete the key to make every copy unreadable.

6 Where PII leaks most

- **Logs**: request bodies, SQL with parameters, exception messages. Scrub at the source with logging filters and structured logs with an allow-list of fields; set short retention.
- **URLs**: `/reset?email=…` ends up in proxy logs, browser history, analytics and Referer headers. Put sensitive values in request bodies or headers.
- **Analytics and ML**: give pseudonymised, minimised datasets by default; raw access is an exception with approval.
- **Third parties**, especially AI APIs: what is sent, is it retained or used for training, where is it processed (data residency, lesson 46), and is there a data processing agreement?
- **Lower environments**: production copies in dev and test. Use synthetic or masked data.

7 Masking PII before an LLM

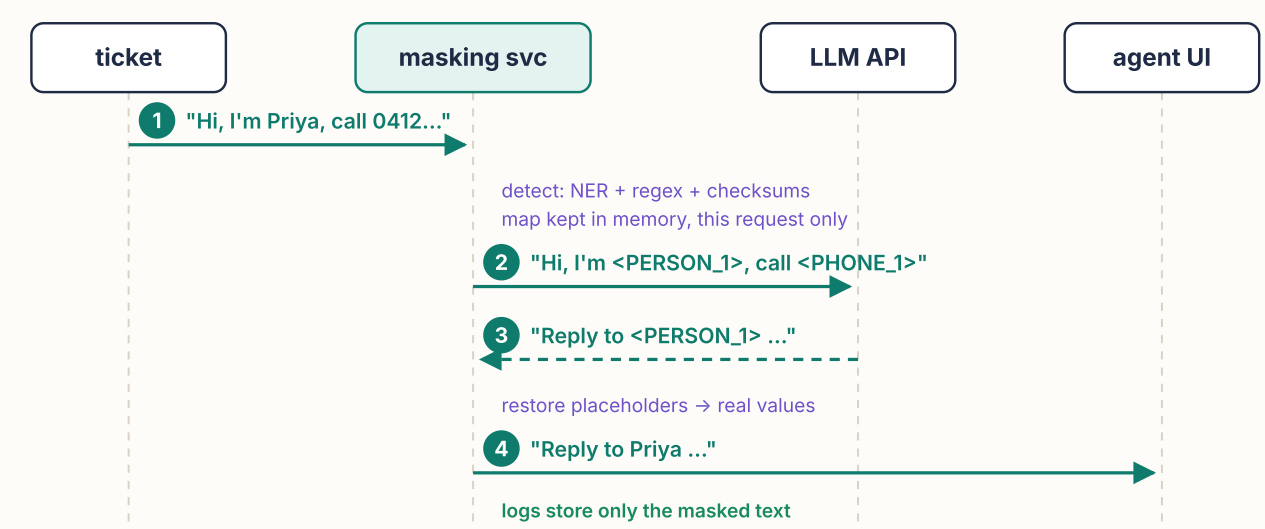


Figure 3. Reversible masking around an LLM call. The model only ever sees placeholders; the mapping never leaves the request's memory.

WORKED EXAMPLE: MASKING PII BEFORE SENDING TEXT TO AN LLM

Before a ticket goes to the model, a PII detection service (for example Presidio's analyzer) finds names, emails, phone numbers and card numbers, combining NER, regular expressions and the Luhn check, and replaces each with a **consistent placeholder**: `<PERSON_1>`, `<PHONE_1>`. Consistency matters: the same person is `<PERSON_1>` everywhere, so the model can still reason about who said what. The placeholder-to-value map is held in memory for this request only. The model's answer uses the placeholders; the service substitutes the real values before showing the draft reply to the support agent. Card numbers are never restored at all: the reply only needs the last four digits. Logs and traces store only the masked text, and the vendor contract specifies no training on inputs and limited retention. Detection is not perfect, so this is one layer of defence, alongside minimising what goes into the prompt in the first place.

COMMON MISTAKES

- Hashing emails without a key and calling the result anonymous.
- Deleting a user from the main DB but not from search, warehouse, logs and vendors.
- Logging full request bodies "temporarily" in production.
- Treating an AI API as a library call instead of a data transfer to a third party.

8 Key terms

PII: information that identifies a person, directly or combined

Tokenisation: random token replaces a value; only a vault maps it back

Differential privacy: calibrated noise so individuals can't be inferred

Data minimisation: collect and keep only what a purpose needs

Pseudonymisation: consistent aliases; still personal data

Crypto-shredding: delete a per-user key to erase all their encrypted copies

9 Interview questions

Q1. How do you design a system to minimise PII exposure? BASIC

Collect and keep only what you need; discover and classify personal data; protect it with masking, tokenisation and pseudonymisation; scrub it from logs, URLs and analytics; enforce consent and deletion across every copy; and review third-party flows like AI APIs.

Q2. What is tokenisation? BASIC

Replacing a sensitive value with a random token that only a secure vault can map back, so most systems never hold the real value.

Q3. Masking vs pseudonymisation? BASIC

Masking hides part of a value for display and is one-way; pseudonymisation replaces an identifier with a consistent alias so records can still be linked and counted.

Q4. A user asks to be deleted. What has to happen? SCENARIO

Find every copy via the data map and lineage (DB, search, caches, warehouse, logs, vendors), delete or anonymise them within the legal deadline, and handle backups by expiry or crypto-shredding.

Q5. Analysts need user-level behaviour data. How do you give it safely? SCENARIO

Provide a pseudonymised dataset with keyed-hash user IDs, minimised columns and no direct identifiers, under access controls, with re-identification keys held separately.

Q6. Is SHA-256 of an email address anonymous? TRICKY

No: emails are guessable, so a dictionary of candidate emails can be hashed and matched; use a keyed hash and treat the result as personal data.

Q7. Why keep the placeholder map only in memory in the LLM example? TRICKY

Persisting it would create a new store linking placeholders to real PII, which then needs its own security, retention and deletion; per-request memory avoids the copy.

Next: SD-V-018 · Data Governance and Lineage backendroadmap.com

Data Governance and Lineage

know what your data means, where it goes, and who depends on it

Watch the 4-min video

In one line: data governance makes data findable, trustworthy and safe: a **catalog** with owners and definitions, **lineage** showing how data flows from sources to tables, dashboards and models, **quality** checks shown where people look, and **tag-based access** with masking. Lineage turns "what breaks if I change this?" from a guess into a query.

1 Who uses this column?

The team plans to drop `customer_phone` from the main database. Someone asks which dashboards, reports and ML models use it. Nobody knows. It gets dropped, and the next morning a revenue dashboard is broken and a model's training job fails. Governance and lineage answer that question **before** the change, in minutes.

2 What governance is (and isn't)

Data governance is the set of practices that make data trustworthy and safe to use: who **owns** each dataset, what it **means**, how **good** it is, and who may **access** it. Without it, a growing company ends up with thousands of tables nobody understands, three definitions of "active customer", and numbers nobody trusts. Done well, it makes data **easier** to use: people find, understand and trust data faster, instead of asking around in chat. Done badly, it's a committee that slows everyone down; the modern approach is automation and clear ownership, not approval forms.

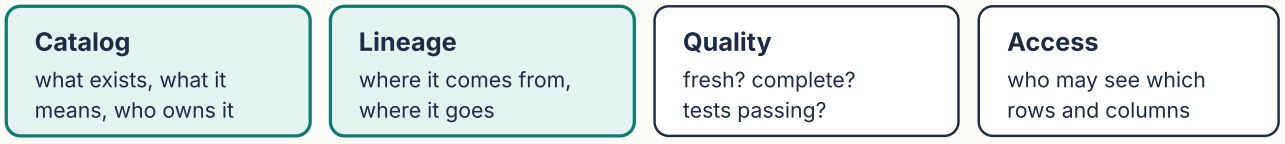


Figure 1. Four pillars. The catalog is where all four are shown to users.

Roles matter: a **data owner** (accountable, usually a business lead) approves access and definitions; a **data steward** maintains descriptions and quality rules; the platform team runs the tooling. A **data contract** between a producing team and its consumers (schema, meaning, freshness, change process) stops upstream changes from breaking downstream users silently.

3 The data catalog

A catalog (Microsoft Purview, Databricks Unity Catalog, DataHub, OpenMetadata, AWS Glue Data Catalog / DataZone, Google Dataplex, Collibra, Alation) lists every dataset with its schema, description, **owner**, tags (`PII`, `finance`, `certified`), freshness, quality status and usage statistics (who queries it, how often). Analysts search it like a search engine; popular, certified tables rise to the top, and deprecated ones are marked. Metadata is harvested automatically from databases, warehouses, pipelines and BI tools; humans add meaning (descriptions, glossary terms like "net revenue").

4 Lineage

Lineage records how data flows: this dashboard reads this gold table, which a dbt model builds from these silver tables, which come from this source column. **Table-level** lineage shows dataset dependencies; **column-level** lineage shows which output columns derive from which input columns, which is what you need for a column drop or PII tracking.

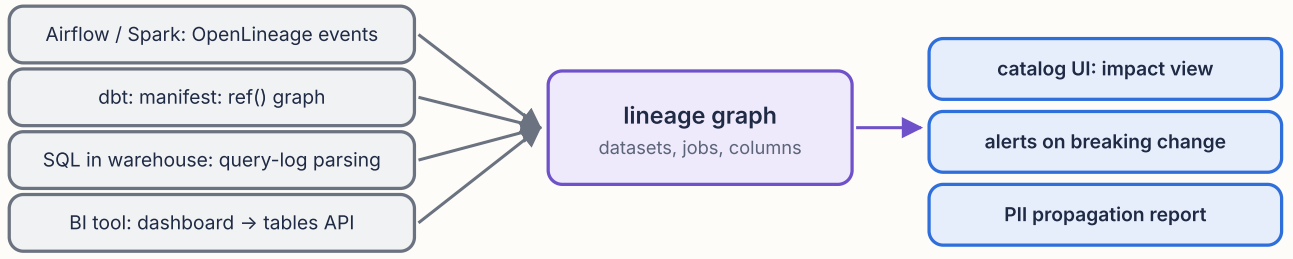


Figure 2. Lineage is captured automatically from where the work happens, then assembled into one graph.

- **OpenLineage** is an open standard: jobs emit run events naming their input and output datasets, with optional facets such as schema and column lineage. Integrations exist for Airflow, Spark, dbt and Flink; Marquez is the reference backend, and several catalogs accept the events.
- **dbt** knows its own graph: every `ref()` is an edge, published in its manifest.
- **SQL parsing** of warehouse query logs derives lineage, including columns, for SQL nobody instrumented.
- **BI connectors** map dashboards and reports to the tables and columns they query.

Gaps are common: Python notebooks, ad-hoc exports and spreadsheets. Treat unknown edges as a risk, not as "no dependencies".

5 Three uses of lineage

Use	Question	Direction in the graph
Impact analysis	What breaks if I change or delete this?	downstream
Root cause analysis	This number looks wrong: where did it come from, what changed?	upstream
Compliance	Where does this personal data end up, so we can protect, delete or report it?	downstream from PII sources

Lineage also propagates **tags**: a column derived from a PII column can inherit the PII tag automatically, so the access rules follow the data. That's how a deletion request or a breach assessment finds every copy (lesson 67).

6 Quality and access

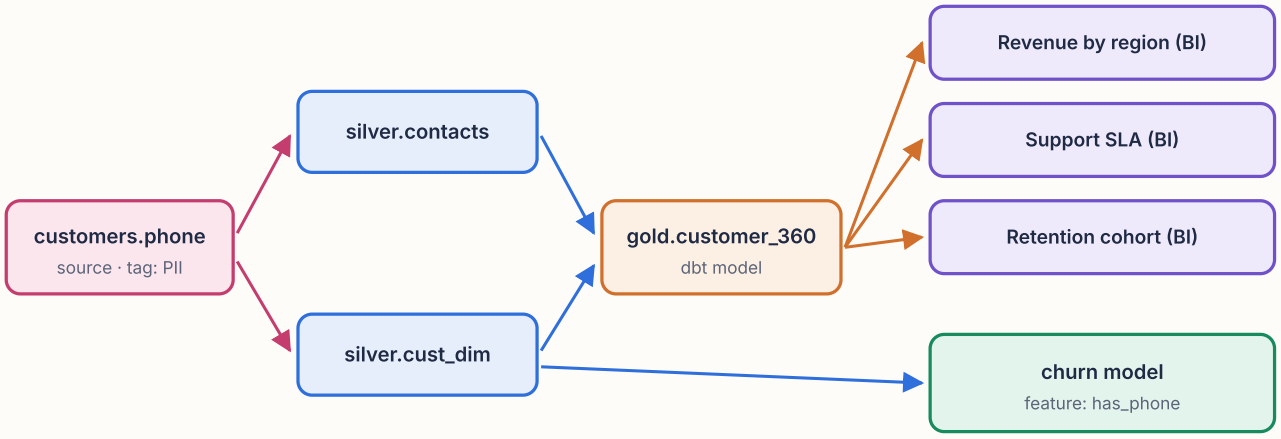
Quality checks run with each pipeline: freshness (updated in the last 2 hours?), volume (row count within the expected range?), schema (columns unchanged?), validity (`not_null`, `unique`, accepted values, referential integrity) and business rules. Tools: dbt tests, Great Expectations, Soda, and observability platforms. Results appear in the catalog, so users see "fresh, 42/42 tests passing" before they trust a table, and a failed check can stop a broken table from being published.

Access policies are best written against tags, not tables (attribute-based access control): "columns tagged PII are masked unless the user is in group `support-pii`"; "rows are filtered to the user's region".

Unity Catalog column masks and row filters, Snowflake tag-based masking policies, and Purview policies implement this. A new table with a PII column is protected the moment it's tagged, with no ticket.

WORKED EXAMPLE: TRACING A PII COLUMN BEFORE REMOVING IT

In the catalog, an engineer opens `customers.phone`, tagged PII. Column-level lineage shows it flows into **two silver tables**, **one gold table** (`customer_360`), **three dashboards** and **one ML feature** (`has_phone`) in the churn model. The catalog lists each asset's owner, and usage stats show one dashboard hasn't been viewed in 90 days. The owners are notified with a removal date. Two dashboards switch to a masked or derived column, the stale one is retired, and the data science team retrains the churn model without the feature (checking that accuracy holds). The column is marked **deprecated** for two weeks; the lineage view is re-checked to confirm no remaining consumers, and only then is it removed. Nothing breaks the next morning.



Impact of dropping customers.phone: 2 silver tables, 1 gold table, 3 dashboards, 1 ML feature, each with a named owner.

Figure 3. Column-level lineage for `customers.phone`: the impact list, with an owner for every node.

COMMON MISTAKES

- A catalog filled once by hand and never updated: metadata must be harvested automatically.
- Datasets without a named owner, so nobody answers questions or approves access.
- Table-level lineage only, when the change or the PII question is about one column.
- Access rules per table instead of per tag, so new copies of PII are unprotected.

7 Key terms

Data catalog: searchable inventory of datasets with owners and meaning

Column-level lineage: which output columns derive from which input columns

Data contract: agreed schema, meaning and change process between producer and consumers

Lineage: record of how data flows between datasets, jobs and reports

Data owner / steward: accountable approver / maintainer of a dataset

Tag-based access: policies that follow classification tags

8 Interview questions

Q1. Why does data lineage matter? BASIC

It shows how data flows from sources to tables, dashboards and models, which enables impact analysis before changes, root cause analysis when numbers look wrong, and compliance for personal data.

Q2. What else is part of data governance? BASIC

A catalog with owners and definitions, data quality checks, and tag-based access policies with masking.

Q3. How is lineage captured? BASIC

Automatically: OpenLineage events from orchestrators and Spark, dbt's dependency graph, parsing of warehouse SQL logs, and BI tool connectors.

Q4. Revenue on the dashboard dropped 20% overnight. How does governance help? SCENARIO

Walk lineage upstream from the dashboard, check freshness and test results at each step, and find the changed or failed source or model.

Q5. How do you make sure new copies of PII are protected? SCENARIO

Classify at the source, propagate tags through column-level lineage, and enforce access with tag-based masking policies rather than per-table grants.

Q6. Lineage shows no consumers for a column. Is it safe to drop? TRICKY

Not yet: notebooks, exports and external tools may be invisible to lineage; deprecate first, check query logs during a grace period, then drop.

Q7. Why do data contracts help when you already have lineage? TRICKY

Lineage tells you who will break; a contract prevents the break by making schema and meaning a versioned agreement with a change process.

Next: SD-V-019 · Edge Computing backendroadmap.com

Edge Computing

run the small, latency-sensitive bit close to the user

Watch the 4-min video

In one line: CDNs now run your code in hundreds of locations near users; use it for lightweight, latency-sensitive logic that doesn't need central, strongly consistent data (routing, token checks, A/B assignment, rewrites, bot filtering, picking cached variants), and keep heavy or transactional work in your region.

1 400 ms in Sydney

Users in Sydney wait about 400 ms for the homepage because every request travels to a server in Virginia and back. The page is the same for everyone except the language and an experiment variant, and that small difference is why it isn't cached. Physics sets the floor: you can't make Virginia closer, so move the logic, and the cache, to Sydney.

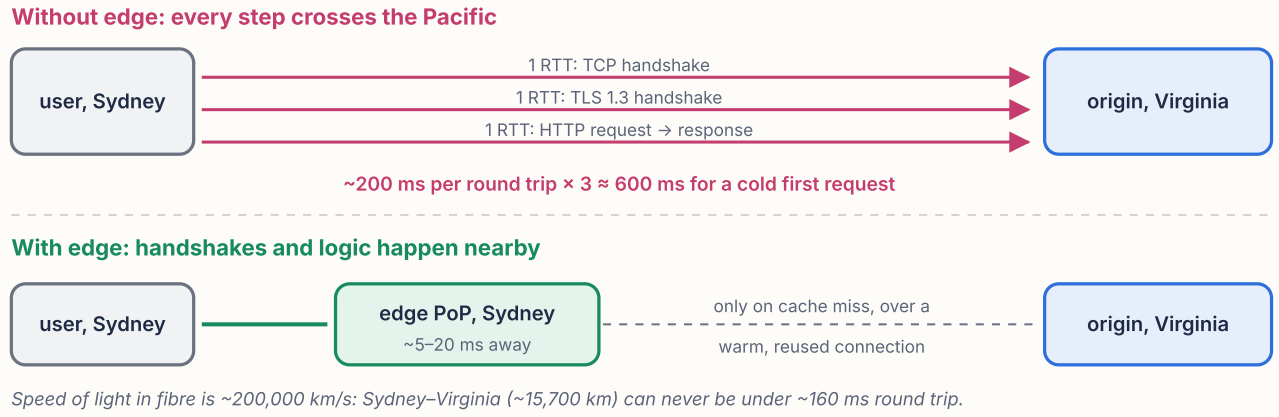


Figure 1. Why distance hurts: new connections need several round trips. An edge location terminates TCP and TLS nearby and reuses warm connections to the origin.

2 Code at the CDN edge

CDNs already cache static files in hundreds of points of presence (PoPs). **Edge compute** lets you run your own code there. The platforms differ a lot:

Platform	Runtime	Shape and limits (check current docs)
Cloudflare Workers	V8 isolates: JS/TS, WebAssembly	runs in every PoP; CPU-time limits per request (10 ms on the free plan, much higher on paid); KV, Durable Objects, D1 for state
CloudFront Functions	lightweight JavaScript	viewer request/response only; sub-millisecond budget, tiny memory, no network calls; very cheap
Lambda@Edge	Node.js, Python	runs in regional edge caches; all four hooks; network calls allowed; 5 s timeout for viewer hooks, 30 s for origin hooks
Fastly Compute	WebAssembly (Rust, Go, JS...)	fast-starting Wasm instances per request; edge KV and config stores
Azure Front Door rules engine	declarative rules, not code	match conditions → rewrite, redirect, header and cache-key actions

Isolates (Workers) start in about a millisecond because many tenants share one process, each in its own sandboxed V8 context; this is why cold starts are a smaller concern than for regional serverless functions, though large bundles and heavy libraries still slow them.

3 Edge functions run on every request

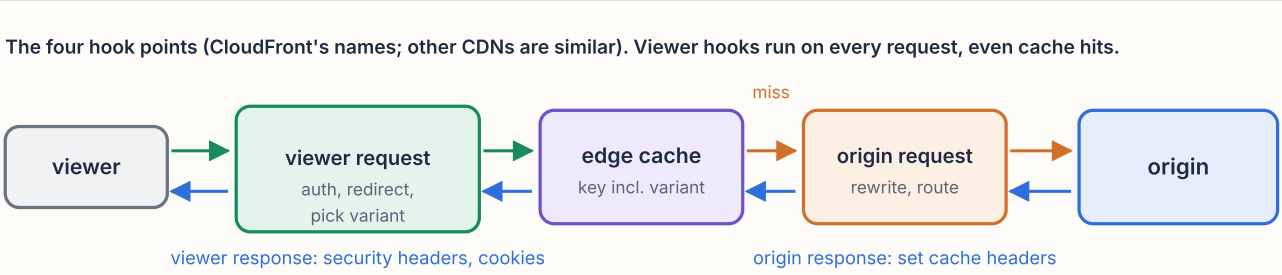


Figure 2. Where edge code hooks into a request. Logic before the cache can change what is looked up; logic after a miss can change what is fetched.

Running **before** the cache, a function can rewrite URLs (`/` → `/en-AU/home`), redirect by country, validate a token and reject with 401, block bots, add headers, and decide **which cached object** to serve. Each of those saves a full trip to the origin, or prevents a bad request from ever reaching it.

4 Personalisation at cache speed

The naive approach to personalisation marks the whole page uncacheable, so every request goes to Virginia. Instead, find the few dimensions that actually vary (language, experiment group, logged-in vs anonymous, device class) and make each combination a **cached variant**: the edge function reads a cookie or header, normalises it to a small set of values, and adds it to the **cache key**. With 3 languages × 2 variants, there are 6 cached copies, all served at edge speed. Keep the number of combinations small, or the hit rate collapses; never put a user ID in the cache key. Per-user fragments (the name in the header) can be filled client-side from a small API call.

5 The edge has limits

- **Tight CPU and memory budgets**, from under a millisecond (CloudFront Functions) to tens of milliseconds or more per request; restricted APIs (no arbitrary sockets or file systems on some platforms).

- **State is eventually consistent**: edge key-value stores replicate writes globally, often taking seconds up to a minute to appear everywhere. Fine for config and feature flags, not for balances. Strongly consistent options exist (Cloudflare Durable Objects put one object in one location), but then requests travel to it.
- **Your database is in one region**. An edge function that queries it on every request just moves the slow round trip, and can make it worse by opening new connections. Edge wins only when the work completes at the edge, or the origin call is a rare cache miss.
- **Operational differences**: debugging and observability are harder, deploys are global within seconds (a bug is global too), and vendor-specific APIs mean lock-in.

6 What suits the edge



Figure 3. Rule of thumb: if it needs only the request plus slowly changing config, it can run at the edge.

Token checks are a good example of the boundary: an edge function can **verify a JWT's signature and expiry** using cached public keys (JWKS) with no network call, rejecting forged or expired tokens instantly. It can't check whether a token was revoked a second ago without asking a central service; keep token lifetimes short or accept that window.

WORKED EXAMPLE: A/B TEST ASSIGNMENT AT THE EDGE

A visitor arrives in Sydney without an `exp` cookie. The edge function takes a visitor ID (from an existing anonymous cookie, or a new random one), computes `hash(visitor_id + experiment_name) mod 100`, and assigns **A** if the result is below 50, else **B**. Hashing makes assignment **deterministic and sticky** without storing anything, and including the experiment name keeps experiments independent. It sets `exp=homepage:B` on the response and looks up the cache with the variant in the key (`/home|en-AU|B`). Both variants are cached at the Sydney PoP, so the page arrives in tens of milliseconds, with **no flicker** (unlike client-side A/B tools that swap content after load) and no trip to Virginia. Exposure events go to analytics asynchronously, and changing the split is a config update pushed to the edge.

COMMON MISTAKES

- Calling the central database from the edge on every request.
- Cache keys with too many variants (or a user ID), so nothing is ever a hit.
- Storing strongly consistent data (stock, balances) in an eventually consistent edge KV.
- Heavy npm dependencies that blow the CPU budget or cold-start time.

7 Key terms

PoP: point of presence: a CDN edge location

Edge function: your code running in CDN locations, per request

Isolate: lightweight sandbox sharing one runtime process; very fast start

Cache key: the values that identify a cached object

Edge KV: globally replicated, eventually consistent key-value store

Origin: your own backend that the CDN fetches from

8 Interview questions

Q1. What workloads suit edge compute? BASIC

Lightweight, latency-sensitive logic that doesn't need central, strongly consistent data: routing and redirects, token checks, A/B assignment, header and URL rewrites, bot filtering, and choosing cached variants for personalisation.

Q2. What are its limits? BASIC

Tight CPU and memory budgets, restricted APIs, only eventually consistent edge storage, and a central database that is still far away.

Q3. How does the edge personalise without losing caching? BASIC

It reduces the request to a few variant values (language, experiment group), adds them to the cache key, and serves the matching cached variant.

Q4. Your edge function queries the regional database on each request and got slower. Why? SCENARIO

Each request still crosses to the region, now possibly on new connections from many PoPs; cache the data at the edge or keep the logic at the origin.

Q5. How would you assign A/B variants consistently without a database? SCENARIO

Hash a stable visitor ID with the experiment name, take it mod 100 against the split, and store the result in a cookie; the same visitor always gets the same variant.

Q6. Can you do authentication at the edge? TRICKY

You can verify signed tokens (signature, expiry, audience) with cached public keys; revocation checks and session lookups still need a central store or short token lifetimes.

Q7. Why do isolates start faster than regional serverless functions? TRICKY

They reuse an already running runtime process and only create a new sandboxed context, rather than booting a container or micro-VM.

Next: SD-V-020 · Platform Engineering backendroadmap.com

Platform Engineering

a product whose customers are your own engineers

Watch the 4-min video

In one line: a platform team builds an **internal developer platform**: golden paths with service templates, self-service infrastructure with guardrails, a portal and catalog, and standard CI, deployment and observability, run **as a product** and judged by adoption, developer satisfaction and faster delivery.

1 Three weeks to launch a service

A new team needs a service. They spend three weeks creating a repository, writing a pipeline, filing a ticket for a database, learning Kubernetes manifests, building dashboards and alerts, and waiting for a security review. The next team does it again, slightly differently, so the company ends up with fifty pipelines, ten ways to deploy and no consistent monitoring. Platform engineering exists to make this take an afternoon, and to make the result consistent.

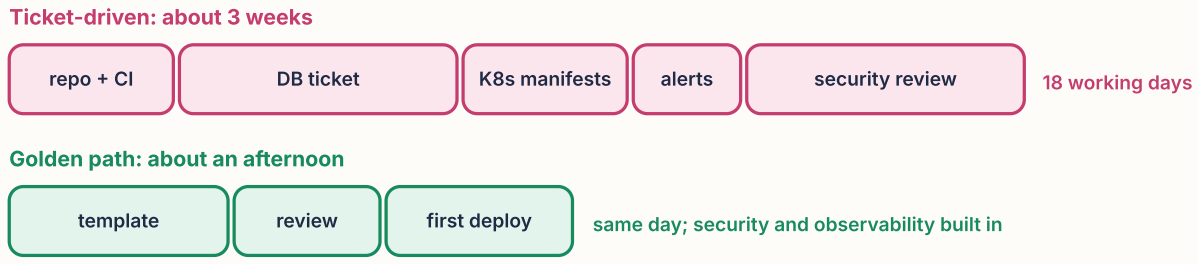


Figure 1. Illustrative timelines. The platform removes waiting on other teams, not just typing.

2 The internal developer platform

An **internal developer platform** (IDP) is the set of self-service tools and paved workflows that let product teams create services, environments and infrastructure on their own, safely, without tickets. The goal is to reduce **cognitive load**: a product team should not need to be expert in Kubernetes, Terraform, network policy and alert tuning to ship a feature. In Team Topologies terms, a **platform team** provides services to **stream-aligned** (product) teams, aiming for the "thinnest viable platform": just enough to remove real friction.

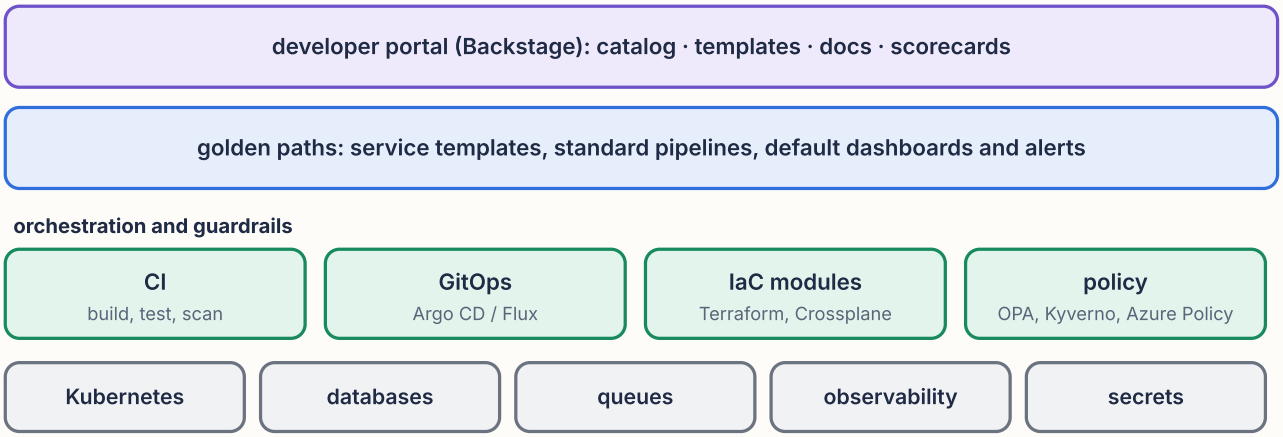


Figure 2. Layers of a typical IDP. Product teams interact with the top two layers; the platform team owns the wiring beneath.

It sits on everything earlier in this phase: Kubernetes (lessons 56–60), IaC modules (lesson 61), GitOps (lesson 62), immutable images (lesson 63) and cost guardrails (lesson 64). The platform's job is to combine them into a few well-lit paths.

3 Golden paths

A **golden path** (Spotify's term; Netflix says **paved road**) is the recommended, supported way to build and run a service: a standard language template, CI pipeline, deployment method, logging, metrics, tracing, security scanning and runbook. It's **opinionated but optional**: teams may leave it when they truly need to, but then they own what they build. Staying on it is simply the easiest choice, because it's faster and comes with support. Mandating instead of attracting tends to produce workarounds.

4 Templates make golden paths real

A **service template** (Backstage Software Templates, or Cookiecutter / Yeoman behind a portal) asks a few questions (name, owner, language, needs a database?) and creates, in one step, a repository with a skeleton app, Dockerfile, CI pipeline, Helm chart or IaC config, dashboards, alert rules, and the catalog entry, all wired together and following company standards. Two subtleties:

- Templates generate code once; updates to the template don't reach existing services by themselves. Keep shared logic in **versioned dependencies** (a shared CI pipeline template, a base Helm chart, base images) so a fix reaches every service by a version bump, possibly automated with Renovate or Dependabot.
- **Scorecards** in the portal show which services meet current standards (owner set, alerts defined, runtime version supported), which is how you migrate the fleet gradually.

5 Self-service infrastructure and the portal

Teams request a database, queue or bucket through the portal or a pull request, using **pre-approved IaC modules** or Kubernetes-style APIs (Crossplane compositions) with guardrails built in: private networking only, encryption on, backups configured, tags and budget set, sizes from an allowed list. **Policy as code** (OPA Gatekeeper, Kyverno, Azure Policy, Sentinel) blocks what modules can't express: no public storage accounts, no privileged containers, mandatory owner labels. Guardrails let teams move fast without accidentally creating a public database or a $50,000 GPU cluster.

A **developer portal** such as **Backstage** (created at Spotify, now a CNCF project) ties it together: a **software catalog** of services, APIs, owners and dependencies (each repository has a `catalog-`

`info.yaml`), the templates, technical documentation (TechDocs) built from Markdown in each repo, and plugins linking to CI runs, deployments, dashboards, costs and on-call. Commercial alternatives (Port, Cortex, OpsLevel) offer the same model.

6 Run the platform as a product

The platform's customers are internal, but the discipline is the same as for any product: user research with developers, a roadmap, documentation, support, deprecation policies and measured outcomes. A platform nobody wants to use has failed, however clever it is.



Figure 3. What to measure. DORA's four keys (deployment frequency, lead time, change failure rate, recovery time) show whether the platform improves delivery.

	Ticket-driven ops	Platform (self-service)
Getting a database	ticket, days of waiting	PR or portal form, minutes, policy-checked
Consistency	depends on who handled the ticket	same modules and templates for everyone
Ops team scaling	grows with the number of requests	grows with the number of capabilities
Security / compliance	manual reviews at the end	guardrails and defaults from the start

WORKED EXAMPLE: A SERVICE TEMPLATE FROM REPO TO FIRST DEPLOY

A developer opens Backstage, picks the **Python API** template, and enters a name (`invoice-api`), an owning team and "needs Postgres: yes". Within minutes the platform creates a repository with a FastAPI skeleton, health endpoints and tests; a CI pipeline that builds, scans and pushes an image; a Helm chart plus an entry in the GitOps config repo, so Argo CD deploys it to **dev** (lesson 62); a Postgres database from the standard Terraform module with credentials in Key Vault; a Grafana dashboard for the four golden signals; and default alerts routed to the team's on-call. The service appears in the catalog with its owner, docs and links. The developer merges a first feature that afternoon, and it's running in dev the same day. Promotion to production follows the same PR flow as every other service.

COMMON MISTAKES

- Building a platform nobody asked for, then mandating it.
- A portal that is just a list of links, without real self-service behind it.
- Templates that fork once and rot, with no versioned shared parts.
- Measuring the platform by features shipped instead of adoption and delivery speed.

7 Key terms

Internal developer platform: self-service tools and paved workflows for engineers

Service template: generator for a fully wired new service

Golden path / paved road: supported, recommended way to build and run a service

Developer portal: UI for catalog, templates, docs (e.g. Backstage)

Guardrails: defaults and policies that prevent unsafe configurations

Cognitive load: what a team must know to deliver; platforms reduce it

8 Interview questions

Q1. What does a platform team provide? BASIC

An internal developer platform: golden paths with service templates, self-service infrastructure with built-in guardrails, a developer portal and catalog, and standard CI, deployment and observability, run as a product for the company's engineers.

Q2. How do you know it works? BASIC

Adoption of the golden path, developer satisfaction, and delivery metrics such as time to first deploy, lead time and deployment frequency.

Q3. What is a golden path? BASIC

The recommended, supported way to build and run a service, bundling template, pipeline, deployment, observability and security; optional but the easiest choice.

Q4. Teams keep creating public storage buckets by accident. What should the platform do? SCENARIO

Provide a storage module that is private by default and enforce a policy-as-code rule that blocks public access, so the safe option is automatic.

Q5. You updated the service template's CI. How do the 200 existing services get it? SCENARIO

Templates don't propagate; keep CI as a versioned shared pipeline referenced by each repo, roll out version bumps automatically, and track compliance with scorecards.

Q6. Isn't a platform team just a renamed ops team? TRICKY

No: ops teams fulfil requests; a platform team builds self-service products so requests aren't needed, and measures success by product teams' outcomes.

Q7. Should you force every team onto the platform? TRICKY

Prefer attraction: make the golden path the easiest choice and support it well; allow exits with ownership, and use mandates only for real guardrails like security policies.

Next: SD-V-021 · Workflow Engines backendroadmap.com

Workflow Engines

durable execution: code that survives crashes, waits and people

Watch the 4-min video

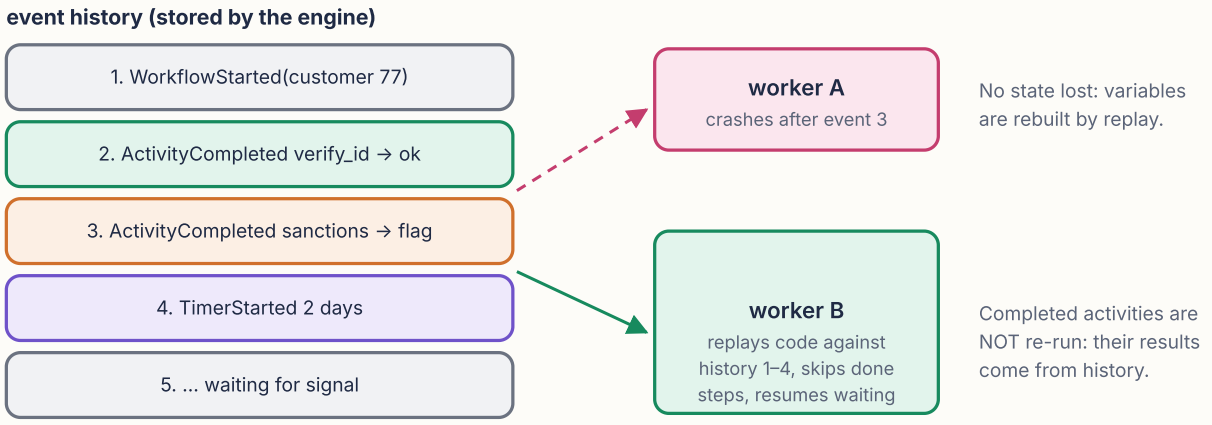
In one line: a workflow engine runs a multi-step business process as ordinary-looking code whose progress is recorded step by step, so it survives crashes, retries steps automatically, can wait for timers or human signals for days, and runs compensations; the price is that workflow code must be deterministic, with side effects in activities.

1 Where were we?

Opening a bank account: verify identity with a third-party service, run a sanctions check, wait for a compliance officer (which can take two days), open the account, send a welcome email. If the service restarts in the middle, where were we? Built from queues and cron jobs, this becomes a status table, a retry loop per step, a timer job that scans for stuck rows, and a dashboard nobody trusts. Workflow engines make the process itself durable.

2 Durable execution

You write the workflow as a function. The engine (Temporal, for example) records an **event history** for each run: started, each activity scheduled and completed with its result, timers, signals. If a worker crashes, another worker picks the run up and **replays** the function from the beginning against that history: completed steps return their recorded results instantly instead of running again, so the code reaches the exact point where it stopped, with all local variables rebuilt. A run can last seconds or months.



That's why workflow code must be deterministic: replay must take the same path to the same commands.

Figure 1. Replay: the history is the source of truth; the code is re-executed to rebuild state, and only new steps actually run.

3 Workflows and activities

Activities are the steps that touch the outside world: call the identity provider, write to a database, send an email. The engine gives each one **timeouts** (for example start-to-close, schedule-to-close, and heartbeats for long tasks) and a **retry policy** with exponential backoff (Temporal's default starts at 1 s, doubles each time, and retries until a timeout or a non-retryable error). Because an activity can run more

than once (a worker can finish the call and crash before reporting it), activities should be **idempotent**, for example by passing an idempotency key to the provider.

Workflow code orchestrates and must be **deterministic**: given the same history it must issue the same commands in the same order. So no direct network or database calls, no system clock (use the engine's `workflow.now()`), no random numbers or UUIDs except through engine APIs, no iterating over unordered maps where order matters. Changing workflow code while runs are in flight needs **versioning** (patch markers), or replay of old runs will diverge.

4 Long waits are first-class

A workflow can `sleep(30 days)`, set a timer, or wait for a **signal** (an external message such as "officer approved") without holding a thread, a process or a connection: the engine stores a timer or waits for an event, and the run is simply resumed when it fires. That makes human-in-the-loop steps, reminders, escalations and deadlines a few lines of code. Runs can also be **queried** for their current state, which is what support staff see in the UI.

```
@workflow.defn
class Onboarding:
    def __init__(self): self.decision = None

    @workflow.signal
    def review(self, decision: str): self.decision = decision

    @workflow.run
    async def run(self, c: Customer):
        await workflow.execute_activity(verify_id, c, start_to_close_timeout=MIN5,
                                        retry_policy=RetryPolicy(maximum_attempts=10))
        risk = await workflow.execute_activity(sanctions, c, start_to_close_timeout=MIN5)
        if risk.flagged:
            await workflow.execute_activity(create_review_task, c, ...)
            try:   # durable wait: no thread is held for these 2 days
                await workflow.wait_condition(lambda: self.decision, timeout=DAYS2)
            except asyncio.TimeoutError:
                await workflow.execute_activity(escalate, c, ...)
                await workflow.wait_condition(lambda: self.decision)
            if self.decision != 'approve': return await reject(c)
```

Figure 2. Temporal-style Python (simplified). Activities do the I/O with timeouts and retries; the workflow just waits for a signal with a 2-day deadline.

5 Engines and when to use one

Engine	Model	Good for
Temporal (and Cadence, its Uber predecessor)	workflows as code in Go, Java, Python, TypeScript, .NET; event history + replay	long-running business processes, sagas, high volume
AWS Step Functions	JSON state machine (Amazon States Language); Standard (up to a year) and Express (up to 5 minutes)	orchestrating AWS services; callbacks with task tokens
Azure Durable Functions	orchestrator functions with replay, durable timers, external events	Azure serverless processes
Apache Airflow (also Dagster, Prefect)	scheduled DAGs of tasks	batch data pipelines, not per-customer request flows
Camunda, Conductor	BPMN diagrams / JSON workflow definitions	processes designed with business analysts; microservice orchestration

Use one when a process has many steps, must survive failures and restarts, includes long waits or people, needs **compensation** when a later step fails (a saga), or must be visible and auditable step by step: support can see exactly which step customer 77 is stuck on, and why.

6 When a queue is enough

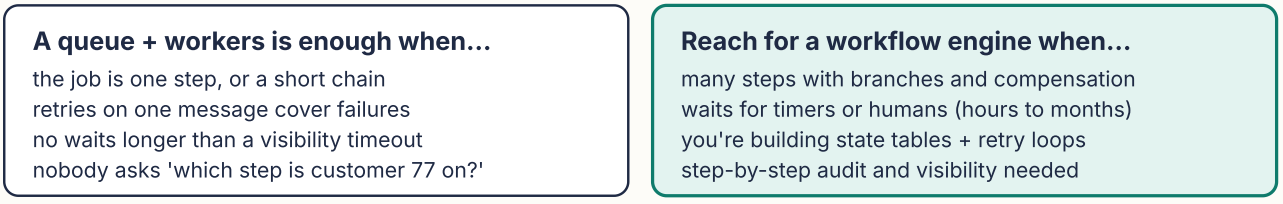


Figure 3. The tell-tale sign: you're writing your own state tables, retry loops and timers around a queue.

Engines add infrastructure to operate (or a managed service to pay for), a programming model to learn (determinism, versioning), and some latency per step (each step is a durable write). For a single background job such as resizing an image or sending one email, a queue with retries and a dead-letter queue is simpler and cheaper.

7 The KYC onboarding workflow

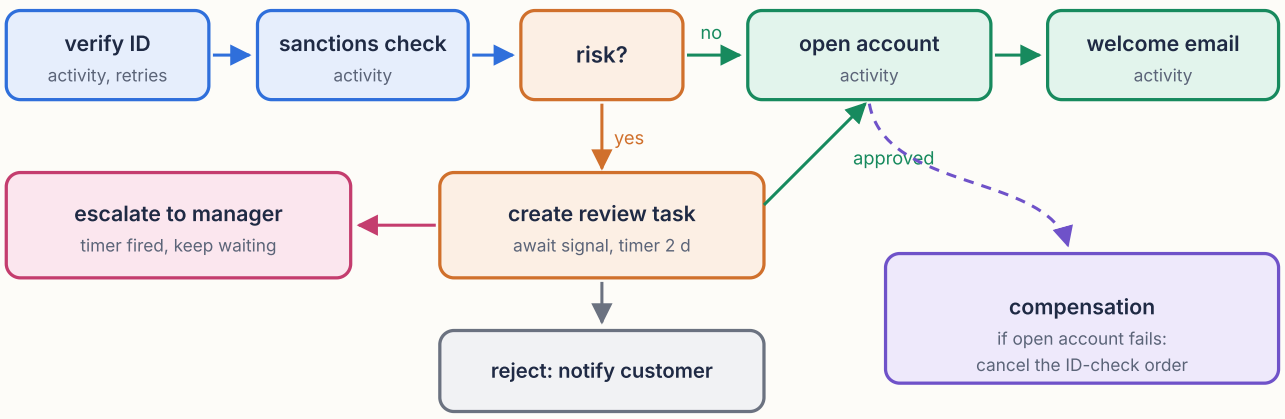


Figure 4. The worked example's flow. Every arrow is a durable step; a crash anywhere resumes at the same point.

WORKED EXAMPLE: KYC ONBOARDING WITH MANUAL REVIEW

The workflow calls the **verify-identity** activity with retries and an idempotency key (so a retried call doesn't create a second paid check), then the **sanctions** activity. If either flags a risk, it creates a review task in the compliance tool and **waits for a signal**, with a **2-day timer**: if the timer fires first, it escalates to a manager and keeps waiting. When the officer clicks approve, the tool sends the signal; the workflow wakes, **opens the account** and sends the welcome email. If opening the account fails permanently, a **compensation** step cancels the identity verification order (and anything else already done, in reverse order). If the worker host is replaced on day one of the wait, nothing happens at all: the run is just a history and a timer until the signal arrives.

COMMON MISTAKES

- Calling APIs or reading the clock directly in workflow code, breaking replay.
- Non-idempotent activities that double-charge or double-send on retry.
- Deploying changed workflow code without versioning while old runs are in flight.
- Using Airflow for per-request business processes, or Temporal for one-step jobs.

8 Key terms

Durable execution: progress persisted so code resumes after any crash

Event history: recorded log of a run's steps and results

Activity: a step with side effects, retried with timeouts

Determinism: same history → same commands on replay

Signal: external message delivered to a running workflow

Compensation: undo action for an earlier step (saga)

9 Interview questions

Q1. When would you use a workflow engine instead of queues? BASIC

When a business process has many steps, must survive crashes and restarts, includes long waits, timers or human approvals, needs compensation, or must be auditable step by step; durable execution handles state, retries and timers for you.

Q2. What's the key rule for workflow code? BASIC

It must be deterministic, with all side effects (I/O, time, randomness) in activities or engine APIs, because the engine replays it from history.

Q3. What is a signal? BASIC

A message sent to a running workflow from outside, such as a human approval, that it can wait on durably.

Q4. A worker crashes after the identity check succeeded. What happens? SCENARIO

Another worker replays the workflow; the identity check's result comes from history, so it isn't called again, and execution continues with the sanctions check.

Q5. How do you implement 'remind the user after 3 days, cancel after 7'? SCENARIO

A workflow with durable timers: sleep 3 days, send the reminder activity unless a completion signal arrived, then wait up to 4 more days before running the cancel activity.

Q6. Can an activity run twice? What do you do about it? TRICKY

Yes: a worker can complete it and crash before the result is recorded, so it's retried; make activities idempotent with idempotency keys or upserts.

Q7. Why can't you just add a new step in the middle of the workflow code and deploy? TRICKY

In-flight runs replay against histories that don't contain that step, so replay diverges; use versioning (patch markers) so old runs follow the old path.

Next: SD-V-022 · Distributed Job Scheduling backendroadmap.com

Distributed Job Scheduling

on time, once (in effect), even when servers die

Watch the 4-min video

In one line: run either one active scheduler chosen by **leader election with leases**, or a **delayed-job store** where many workers **claim due jobs atomically** with lease timeouts; accept that a job may occasionally run twice and make its effect **idempotent**; and decide explicitly how to handle missed runs, time zones and daylight saving.

1 Cron on one server

An app sends a million reminders a day: take your medication at 8, your appointment is tomorrow, your trial ends in three days. One server running cron works until it dies at 7:59, and nobody gets reminded. Add a second server running the same cron, and everyone gets two. Scheduling at scale needs a design for three things: **who fires**, **how work is claimed**, and **what happens on failure**.

2 One active scheduler: leader election

For a modest number of recurring jobs ("rebuild the report at 02:00"), run several scheduler instances but let only the one holding a **lease** fire jobs. The lease is a record with a holder and an expiry in a consistent store: etcd or a Kubernetes `Lease` object, ZooKeeper, Consul, or a database row updated with a conditional write. The leader renews it every few seconds; if it dies, the lease expires and a follower acquires it.

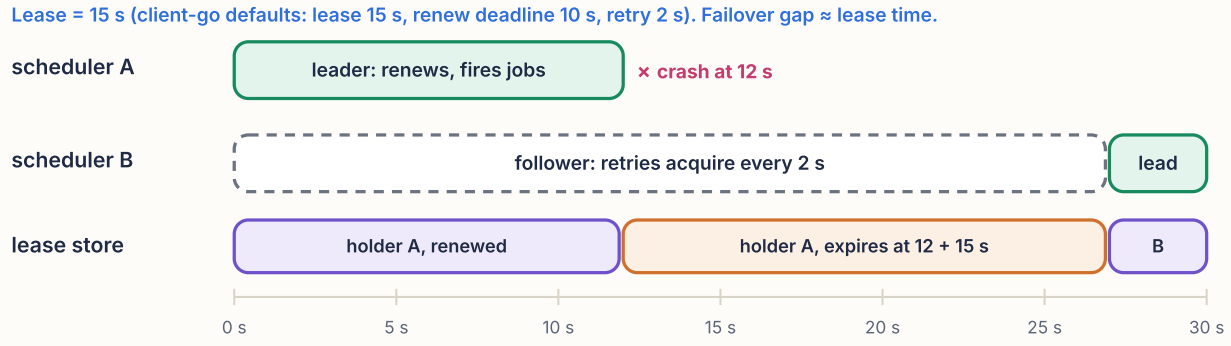


Figure 1. Lease-based leader election. B takes over only after A's lease expires, so there is a gap of up to one lease period with no leader.

Two subtleties. A leader that **pauses** (a long garbage-collection pause, a frozen VM) can wake up after its lease expired and still believe it's the leader, so two schedulers fire briefly. The robust fix is a **fencing token**: each new lease gets a higher number, and the systems the job writes to reject writes with an older token. And the scheduler must track what it has fired durably (a run table), so the new leader knows whether the 02:00 job ran.

Off-the-shelf: Kubernetes **CronJob** (the controller is already leader-elected), Quartz in clustered mode (database row locks), or managed schedulers (EventBridge Scheduler, Cloud Scheduler, Azure Logic Apps / Functions timers). Note that the Kubernetes docs themselves warn a CronJob may create two Jobs, or none, for one schedule in rare cases; it recommends idempotent jobs, and `concurrencyPolicy: Forbid` plus `startingDeadlineSeconds` control overlap and lateness.

3 A million jobs: the delayed-job store

A million individual reminders don't belong in a crontab; they are **data**. Each job is a row with a `due_at` time and a status, in a database table or a Redis **sorted set** scored by timestamp (`ZADD reminders <epoch> <id>`, then `ZRANGEBYSCORE reminders -inf <now>` to find due ones). Workers poll for due jobs every few seconds, claim them, and run them. Recurring schedules are stored as a rule (cron expression or an RFC 5545 RRULE) plus the user's time zone; after each run the next occurrence is computed and inserted. For very high timer volumes in memory, systems such as Kafka and Netty use **hierarchical timing wheels**.

4 Claiming without duplicates

Claiming is where duplicates come from: two workers read the same due row, and both send. The claim must be **atomic**:

- A conditional update: `UPDATE … SET status='running' WHERE id=? AND status='pending'`, and only the worker whose update changed a row owns the job.
- Batch claiming in PostgreSQL or MySQL 8 with `SELECT … FOR UPDATE SKIP LOCKED`: competing workers skip rows another worker has locked, instead of waiting or double-claiming.
- In Redis, a Lua script that pops due members and moves them to an in-flight set in one step (or `ZREM`: whoever removes the member wins).

Every claim carries a **lease** (`lease_until`). If the worker crashes, the lease passes and another worker reclaims the job. Leases should be comfortably longer than the job's normal duration, or a slow worker will be duplicated by a reclaim.

```
-- claim a batch atomically; competing workers skip locked rows
UPDATE reminders SET status = 'running', lease_until = now() + interval '2 minutes',
                     attempts = attempts + 1
WHERE id IN (
  SELECT id FROM reminders
  WHERE (status = 'pending' AND due_at <= now())
     OR (status = 'running' AND lease_until < now())    -- reclaim abandoned jobs
  ORDER BY due_at LIMIT 500
  FOR UPDATE SKIP LOCKED )
RETURNING id, user_id, due_at, payload;

-- index: CREATE INDEX ON reminders (due_at) WHERE status IN ('pending','running');
```

Figure 2. One statement claims up to 500 due or abandoned jobs. The partial index keeps the scan cheap even with millions of finished rows.

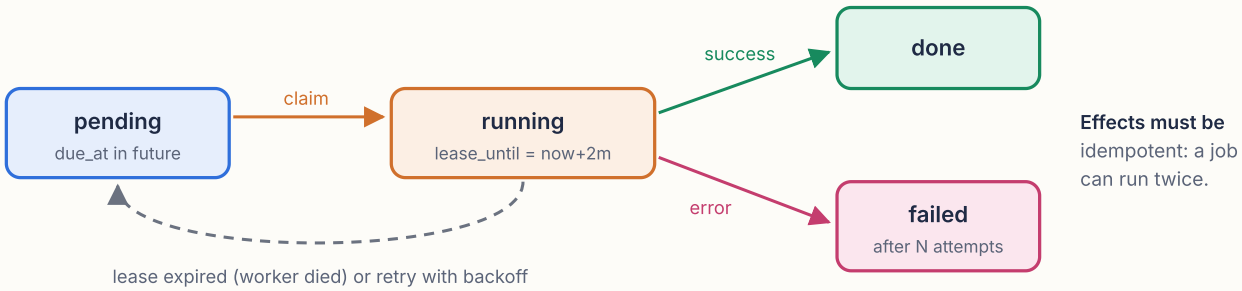


Figure 3. The job lifecycle. The dashed path is the at-least-once part: a job whose worker vanished goes back to be claimed again.

5 At-least-once + idempotent

A job can still run twice: a worker sends the reminder, then crashes before marking it done; the lease expires and another worker sends it again. **Exactly-once firing** across a crash would need the side effect and the status change in one transaction, which is impossible when the side effect is an external call. What is achievable is **at-least-once execution with idempotent effects**: pass an **idempotency key** such as `reminder_id + due_at` to the notification service, which records keys it has seen (with a TTL) and drops repeats. Including `due_at` keeps tomorrow's run of the same recurring reminder distinct.

6 Missed runs, load spikes and time zones

- **Misfire policy**, per job type, after an outage: *catch up* (run every missed occurrence; right for billing), *run once* (right for a report), or *skip if too late* (right for "take your 8:00 pill" at 13:00). Quartz calls these misfire instructions.
- **Thundering herds**: a million reminders a day average about 12 per second, but if a third are set for 8:00 local time, each time zone's 8:00 brings a burst. Pre-claim slightly early, scale workers ahead of known peaks, and add a few seconds of jitter where exact timing doesn't matter.
- **Time zones**: store the user's IANA zone (`Europe/London`), not a fixed UTC offset, because offsets change with daylight saving and with law. Compute the next run in local time, convert it to UTC for `due_at`, and recompute after each run.

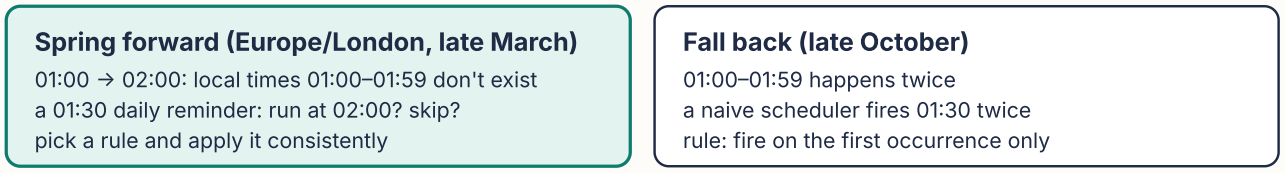


Figure 4. Daylight-saving edge cases. Both need an explicit rule, and a test.

WORKED EXAMPLE: A MILLION SCHEDULED REMINDERS PER DAY

Reminders are rows in a PostgreSQL table with `due_at` (UTC), `status`, `lease_until`, `attempts`, and a partial index on due time for unfinished rows. **Twenty workers** poll every few seconds, each claiming up to 500 due reminders with `SKIP LOCKED` and a **two-minute lease** (the query above). Each send calls the notification service with `reminder_id:due_at` as the **idempotency key**; then the row is marked done, and for recurring reminders the next occurrence is computed in the user's zone and inserted. Twenty workers × 500 per claim every few seconds gives thousands per second of headroom over the 12/s average, enough for the 8:00 bursts. After an outage, overdue reminders are sent if they're **less than an hour late** and marked skipped otherwise, with a metric for each, so a medication reminder never arrives at dinner time.

COMMON MISTAKES

- The same cron on every replica: every job fires N times.
- Read-then-update claiming without a lock or condition, so two workers take one job.
- Storing a UTC offset instead of an IANA time zone, breaking at the next DST change.
- No misfire policy: after an outage, a flood of stale jobs fires at once.

7 Key terms

Leader election: choosing one active instance via a renewable lease

Delayed-job store: jobs as rows or sorted-set members keyed by due time

Misfire policy: what to do with runs missed during an outage

Fencing token: increasing number that lets systems reject a stale leader

SKIP LOCKED: row-lock option letting workers claim different rows concurrently

Idempotency key: identifier that makes a repeated effect a no-op

8 Interview questions

Q1. How do you make sure a scheduled job runs once? BASIC

Use one active scheduler through leader election with leases, or a delayed-job store where workers claim due jobs atomically with lease timeouts, and make the job's effect idempotent so a rare duplicate run is harmless.

Q2. How do you handle time zones? BASIC

Store the user's IANA time zone, compute the next run in local time and store it as UTC, and handle daylight-saving gaps and overlaps with explicit rules.

Q3. Why not run cron on every replica? BASIC

Every replica fires every job, so effects multiply; only a leader should fire, or jobs must be claimed atomically.

Q4. Workers keep sending some reminders twice. What do you check? SCENARIO

Whether claims are atomic, whether leases are shorter than job duration so slow jobs get reclaimed, and whether the send uses an idempotency key.

Q5. The scheduler was down 3 hours. What runs when it returns? SCENARIO

Depends on each job type's misfire policy: billing catches up every run, reports run once, and time-sensitive reminders older than a threshold are skipped and recorded.

Q6. Why isn't leader election alone enough to prevent double execution? TRICKY

A paused leader can resume after its lease expired while a new leader acts; use fencing tokens and idempotent effects.

Q7. Why is exactly-once firing so hard? TRICKY

The external side effect and recording that it happened can't be one atomic step, so a crash between them forces either a possible repeat or a possible loss; at-least-once with idempotency is the practical answer.

Next: SD-V-023 · Green Software backendroadmap.com

Green Software

same work, less energy, cleaner energy

Watch the 4-min video

In one line: measure emissions per unit of work; then cut energy (raise utilisation, remove idle resources, right-size, write efficient software), use cleaner energy (lower-carbon regions, carbon-aware timing for flexible work), and get more years and more work out of each piece of hardware. Most of these also cut cost.

1 A few percent of the world's electricity

Data centres used roughly **1.5%** of the world's electricity in 2024 (about 415 TWh, by the IEA's estimate), and AI is growing that fast: the IEA projects demand could roughly double by 2030. Your company has committed to halving its emissions, and the cloud bill is one of the largest sources. **Green software** means designing systems that do the same work with **less energy**, with **cleaner energy**, and with **less hardware**.

2 Measure first

Cloud providers publish carbon footprint tools (AWS Customer Carbon Footprint Tool, Azure Emissions Impact Dashboard, Google Cloud Carbon Footprint) that estimate emissions per service and region, usually monthly. Open-source tools such as Cloud Carbon Footprint estimate from billing data. For engineering decisions you want a **rate**, not a total, so improvements stay visible as the business grows. The Green Software Foundation's **Software Carbon Intensity** (SCI, published as ISO/IEC 21031:2024) is exactly that:

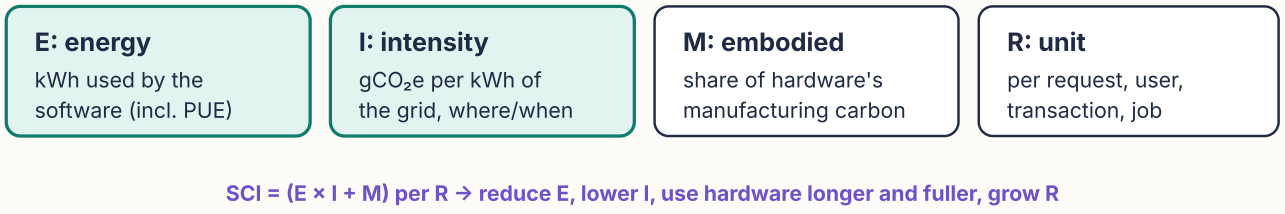


Figure 1. The SCI score. Each term is a lever: energy, the carbon intensity of that energy, embodied carbon of the hardware, and the unit you divide by.

Two accounting details worth knowing. Data-centre overhead is captured by **PUE** (power usage effectiveness: total facility power ÷ IT power); large cloud data centres run around 1.1, while the industry average is about 1.5, which is one reason consolidating into the cloud often helps. And cloud emissions appear in your own reporting as **Scope 3** (purchased services), computed **location-based** (actual grid mix) or **market-based** (after renewable energy purchases); the two can differ a lot, so say which one you mean.

3 Efficiency: the biggest lever

Servers are not energy-proportional: an idle server still draws a large share of its peak power (around half for many older servers, less for modern ones). So **energy per unit of work** falls steeply as utilisation rises.

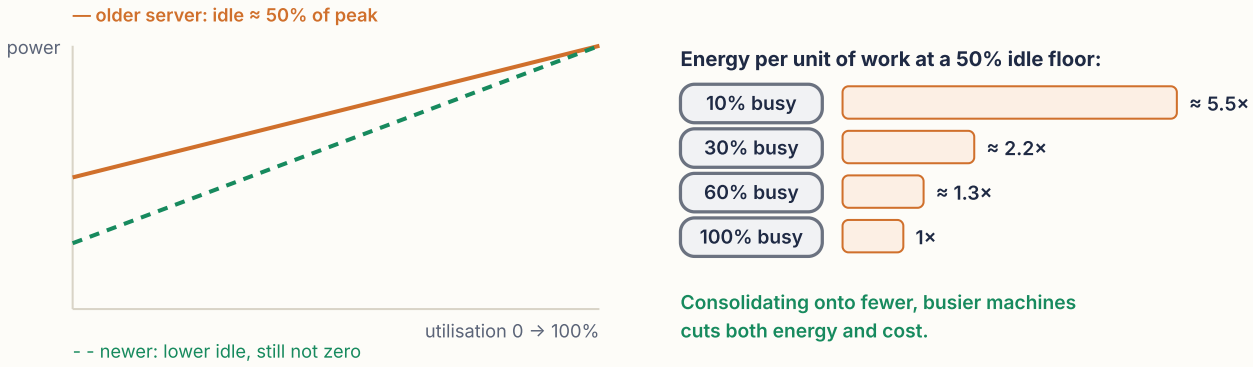


Figure 2. Why utilisation matters. With a 50% idle floor, a machine at 10% load spends about 5.5× the energy per unit of work of a fully used one.

- **Right-size and autoscale** (lessons 58 and 64): fewer, fuller machines.
- **Remove zombies**: idle environments, forgotten clusters, unused replicas. Turning dev/test off outside working hours removes about 64% of their hours.
- **Consolidate** small services onto shared clusters with good bin-packing, and prefer managed or serverless services whose provider runs hardware at high utilisation.
- **Efficient hardware**: newer generations and ARM instances often do more work per watt; for AI, the right accelerator and high batch utilisation matter most.
- **Embodied carbon**: manufacturing hardware is a large part of its lifetime footprint, so running fewer servers for longer, at higher utilisation, beats buying more.

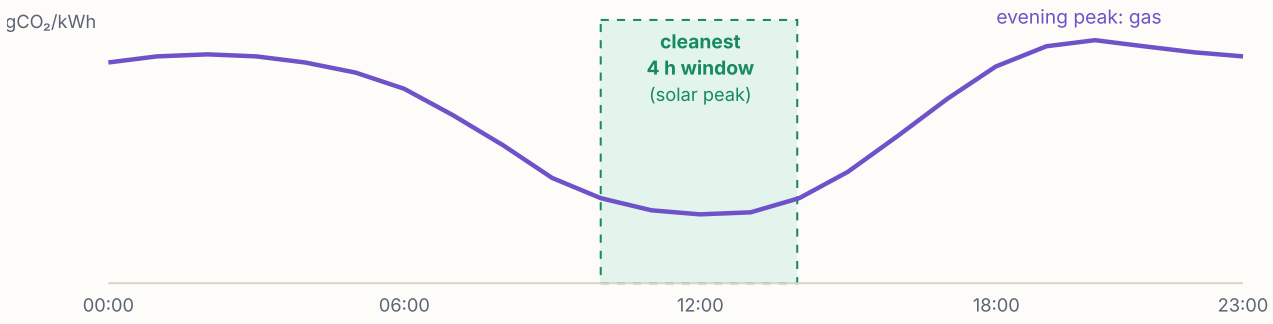
4 Carbon intensity by region

Carbon intensity is grams of CO_2-equivalent per kWh of electricity. It varies hugely by place: grids dominated by hydro, nuclear or wind (Norway, Sweden, Quebec, France) are often under about 50 g/kWh, while coal-heavy grids can exceed 600 g/kWh. The same job can therefore emit **ten times more** depending on the region. When latency, data residency (lesson 46) and cost allow, choose lower-carbon regions for new workloads, especially big batch and training jobs. Providers publish per-region carbon data or "low carbon" indicators to help.

Grid type (illustrative)	Typical intensity	Same 500 kWh monthly job
Mostly hydro / nuclear / wind	~20–60 g/kWh	~10–30 kg CO_2e
Mixed grid with gas and renewables	~200–400 g/kWh	~100–200 kg CO_2e
Coal-heavy	~600–800 g/kWh	~300–400 kg CO_2e

5 Carbon-aware scheduling

Intensity also varies **by hour**: solar lowers it at midday, wind when it's windy, and evening peaks are often met with gas. Flexible work (batch analytics, model training, backups, video transcoding, CI for non-urgent branches) can **shift in time** to the cleanest window before its deadline, or **shift in place** to a region that is cleaner right now. Carbon-intensity APIs (Electricity Maps, WattTime; the Green Software Foundation's Carbon Aware SDK wraps them) provide current values and forecasts that a scheduler can query: "start this 4-hour job in the lowest-intensity window before 06:00 tomorrow". A related idea is **demand shaping**: degrade gracefully when the grid is dirty (lower video quality, defer optional work).



Illustrative day on a grid with lots of solar. Forecasts come from carbon-intensity APIs (e.g. Electricity Maps, WattTime).

Figure 3. Time shifting: the same 4-hour job emits far less in the midday window than starting at the evening peak.

One nuance: **average** intensity says how clean the grid's mix is; **marginal** intensity says which power plant responds when you add load. They can differ, and providers of the data argue for different signals, so pick one consistently and state it.

6 Software choices

- **Do less work**: caching (the greenest request is the one you don't compute), avoiding repeated polling, incremental instead of full recomputation.
- **Efficient code and formats**: better algorithms, compiled or optimised hot paths, compact formats (Parquet over CSV, compression).
- **Smaller AI models** where they are good enough, quantisation, batching, and caching repeated prompts: these change GPU hours by multiples.
- **Store and move less**: retention policies and tiering (lesson 12), fewer copies, less cross-region transfer.
- **Lighter clients**: smaller pages and fewer scripts save energy on billions of user devices, and extend those devices' useful life.

Almost every item also saves money, which makes green work much easier to fund: present it alongside cost (lessons 64–65).

WORKED EXAMPLE: BATCH JOBS IN LOW-CARBON REGIONS

Nightly analytics jobs use about **500 kWh a month** in a coal-heavy region at about 700 g/kWh: roughly **350 kg CO_2e**. The data can legally be processed in another region powered mostly by hydro and wind (residency checked), and the jobs are not latency-sensitive, so they move there. At about 50 g/kWh the same energy is roughly **25 kg**. Model retraining is scheduled with a carbon-intensity forecast to start in the cleanest four-hour window each day, with a deadline so it always finishes before the morning. Emissions for these workloads fall by far more than half; cost barely changes (prices differ by region, and cross-region data transfer was minimised by moving the input data once and keeping it there). The team tracks SCI per job run, so the improvement stays visible as data grows.

COMMON MISTAKES

- Reporting total emissions only, so growth hides efficiency gains.
- Moving workloads to a "green" region without checking residency, latency or transfer cost.
- Ignoring embodied carbon: buying new hardware to save a little energy.
- Treating green as separate from cost and reliability work, instead of the same levers.

7 Key terms

Carbon intensity: gCO_2e emitted per kWh of electricity, by place and time

PUE: facility power ÷ IT power; 1.0 is ideal

Carbon-aware scheduling: shifting flexible work to cleaner times or places

SCI: Software Carbon Intensity: (E × I + M) per unit R

Embodied carbon: emissions from manufacturing and disposing of hardware

Energy proportionality: power use scaling with load; real servers fall short

8 Interview questions

Q1. How can architecture reduce energy use? BASIC

Raise utilisation and remove idle resources, right-size and autoscale, choose lower-carbon regions, shift flexible batch work to cleaner times and places with carbon-aware scheduling, and write efficient software that caches, stores less and avoids unnecessary work.

Q2. How do you track progress? BASIC

Measure emissions and efficiency per unit of work, for example SCI per request or per job, using provider carbon tools and energy estimates.

Q3. What is carbon intensity? BASIC

Grams of CO_2-equivalent per kWh of electricity, which varies by region and by hour with the grid's mix of sources.

Q4. Which workloads would you make carbon-aware first? SCENARIO

Large, flexible, deadline-tolerant ones: batch analytics, training, backups, transcoding; not user-facing latency-critical paths.

Q5. A team wants to move all services to the cleanest region. What do you check? SCENARIO

Data residency and compliance, latency to users, cross-region transfer cost, service availability in that region, and whether batch-only moves get most of the benefit.

Q6. Why can running servers hotter be greener even though it uses more power per server? TRICKY

Idle power is a large floor, so energy per unit of work drops as utilisation rises; fewer, busier servers also need less embodied carbon.

Q7. Why doesn't a renewable-energy purchase make your midday and midnight emissions equal? TRICKY

Market-based accounting nets out purchases annually, but the physical grid's intensity still varies hourly; location-based or hourly signals reveal the difference that carbon-aware scheduling exploits.